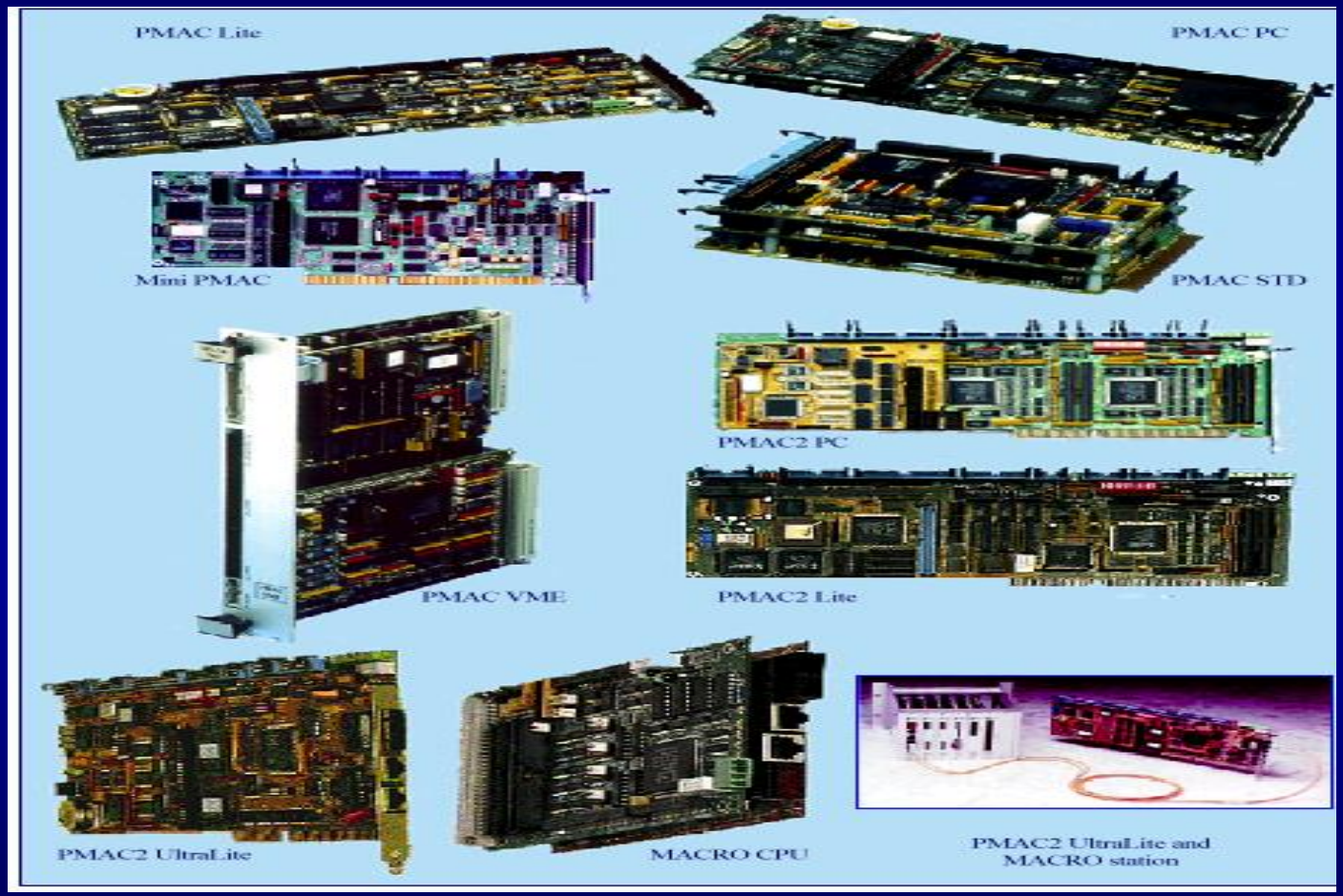
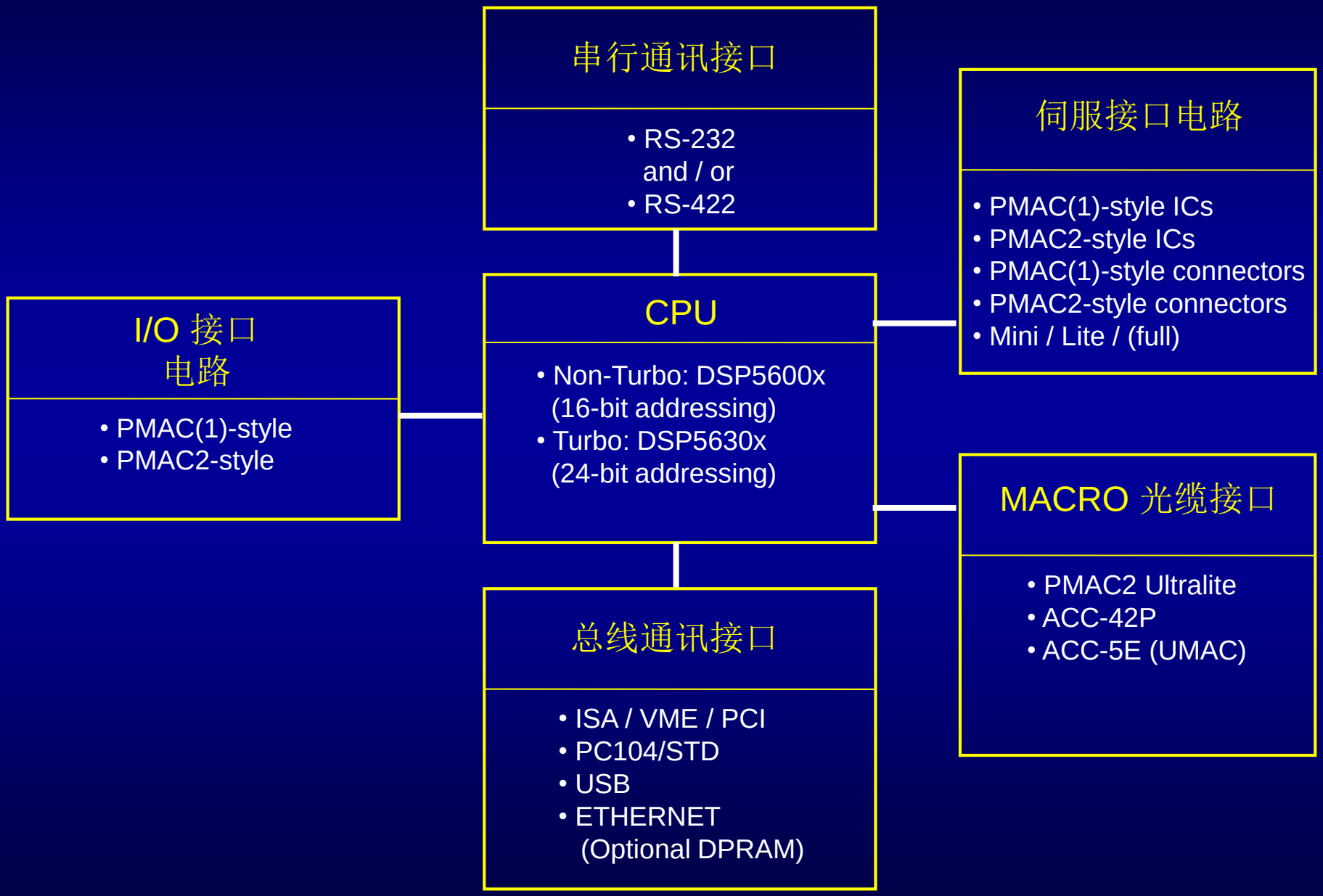
E-MOTION

PMAC多轴运动控制卡（高级培训）

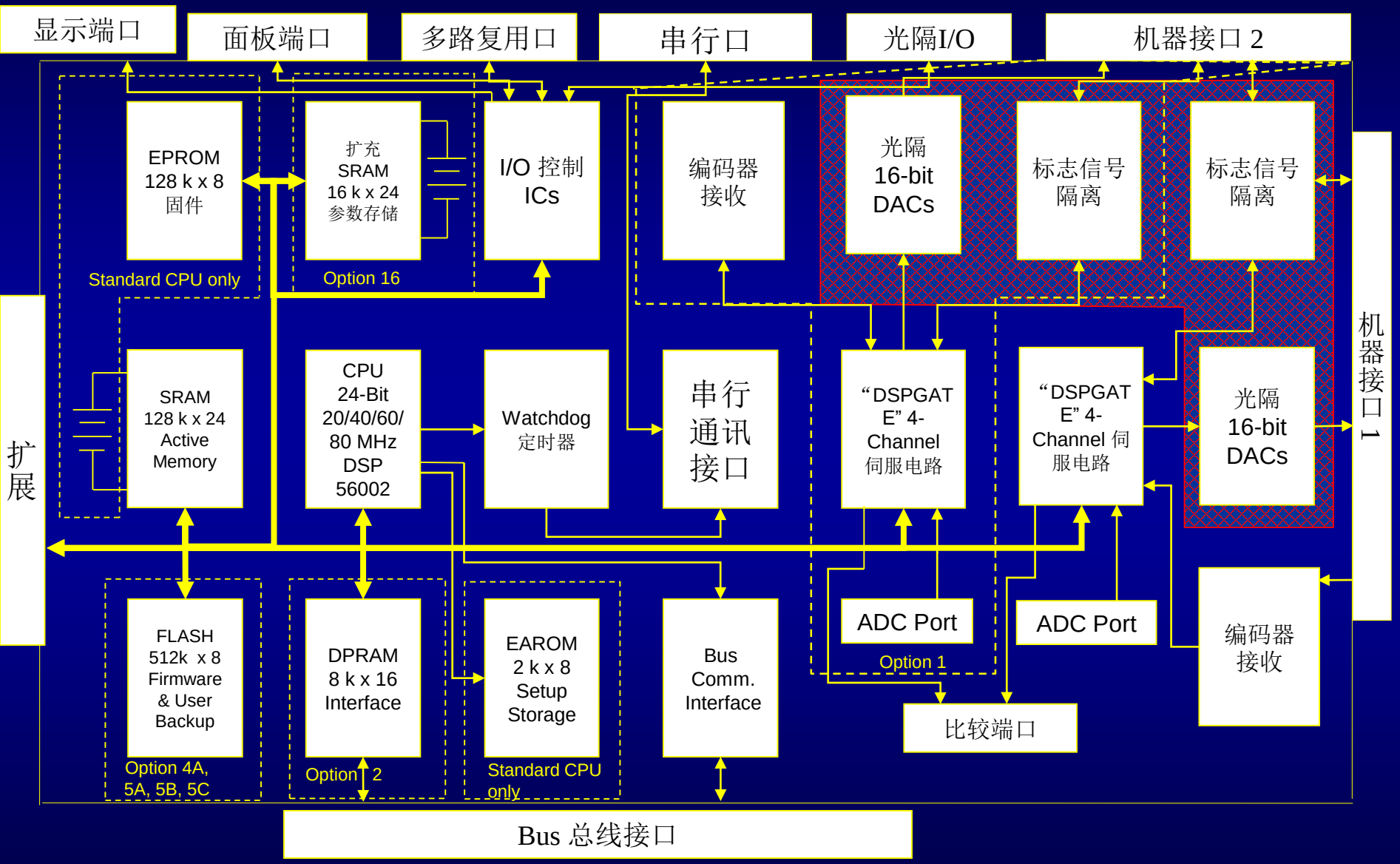


NEW IDEAS IN MOTION

PMAC 卡主体结构


串行通讯接口
• RS-232
and / or
• RS-422
伺服接口电路
• PMAC(1)-style ICs
• PMAC2-style ICs
• PMAC(1)-style connectors
• PMAC2-style connectors
• Mini / Lite / (full)
I/O 接口
电路
• PMAC(1)-style
• PMAC2-style
CPU
• Non-Turbo: DSP5600x
(16-bit addressing)
• Turbo: DSP5630x
(24-bit addressing)
MACRO 光缆接口
• PMAC2 Ultralite
• ACC-42P
• ACC-5E (UMAC)
总线通讯接口
• ISA / VME / PCI
• PC104/STD
• USB
• ETHERNET
(Optional DPRAM)

PMAC1 结构框图


显示端口
面板端口
多路复用口
串行口
光隔I/O
机器接口 2
EPROM
128 k x 8
固件
扩充
SRAM
16 k x 24
参数存储
I/O 控制
ICs
编码器
接收
光隔
16-bit
DACs
标志信号
隔离
标志信号
隔离
Standard CPU only
Option 16
扩展
SRAM
128 k x 24
Active
Memory
CPU
24-Bit
20/40/60/
80 MHz
DSP
56002
Watchdog
定时器
串行
通讯
接口
“DSPGATE” 4-Channel
伺服电路
“DSPGATE” 4-Channel 伺服电路
光隔
16-bit
DACs
机器接口 1
FLASH
512k x 8
Firmware
& User
Backup
DPRAM
8 k x 16
Interface
EAROM
2 k x 8
Setup
Storage
Bus
Comm.
Interface
ADC Port
ADC Port
Option 1
编码器
接收
比较端口
Option 4A,
5A, 5B, 5C
Option 2
Standard CPU
only
Bus 总线接口

Channels, Motors, and Axes on PMAC

“Channels”——“通道”为 硬件定义
由放大器,编码器,信号接口组成

“Motors”——“电机”为 软件定义
非Turbo CPUs PMAC支持8个电机
Turbo CPUs PMAC支持32个电机

“Axes”——“轴”：为 软件定义
在运动程序中有效，每个坐标系最多可到 9个轴



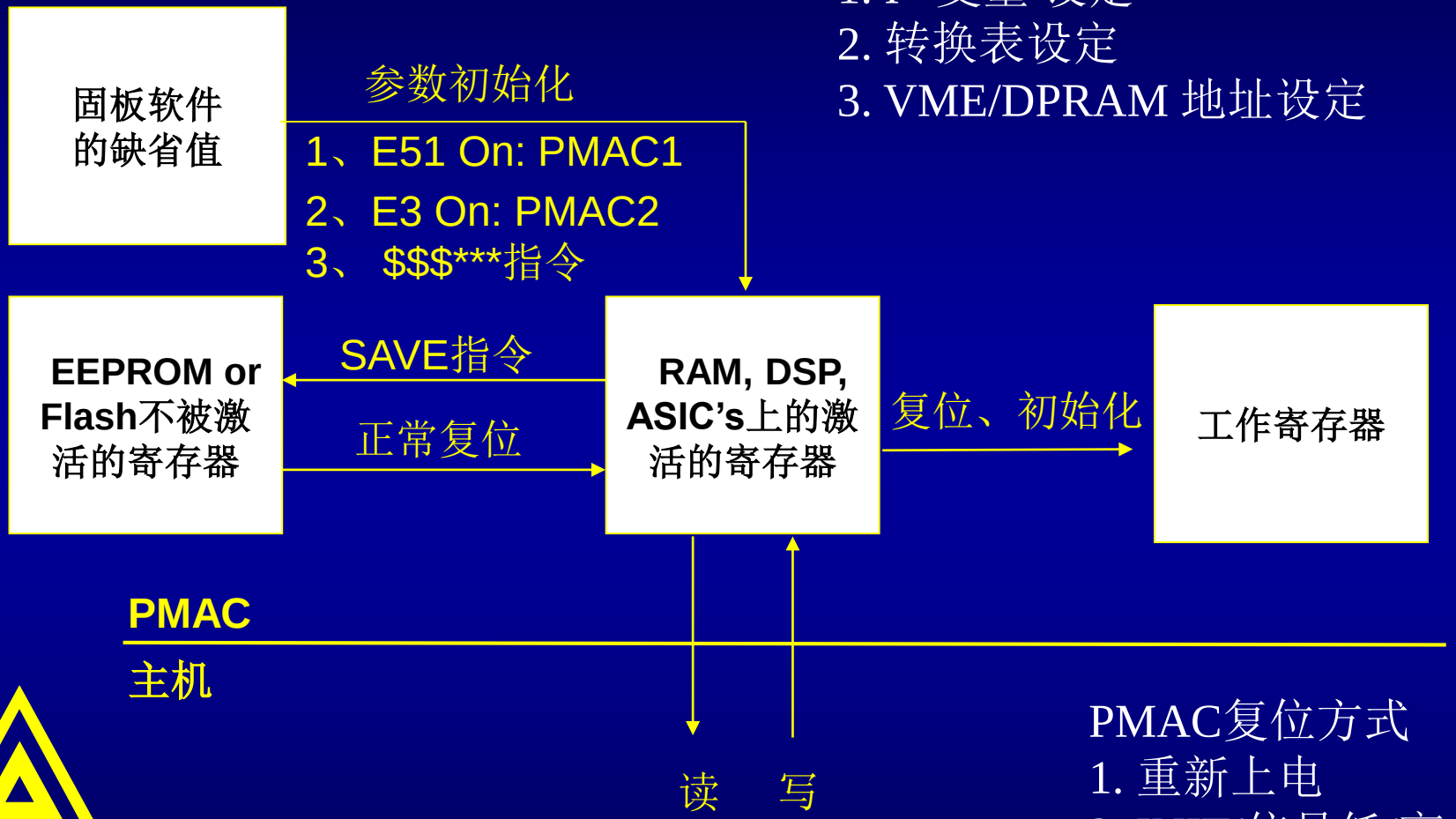
Non-Turbo PMAC/PMAC2 CPU 版本

PMAC1 Version	PMAC2 Version	CPU Frequency	Internal Memory	RAM Wait States	Memory Backup
Standard*	? ? ?	20 MHz	1k x 24	1	Battery
Opt 4A	? ? ?	20 MHz	1k x 24	0	Flash
Opt 5A	Standard	40 MHz	1k x 24	0	Flash
Opt 5B*	Opt 5B	60 MHz	1k x 24	0	Flash
Opt 5C	Opt 5C*	80 MHz	1k x 24	0	Flash
Opt 5CF	Opt 5CF	80 MHz	128k x 24	0	Flash
Opt 5EF	Opt 5EF	160 MHz	128k x 24	0	Flash

* Legacy product for existing users only

PMAC参数存储器设定

1. I –变量 设定
2. 转换表设定
3. VME/DPRAM 地址设定

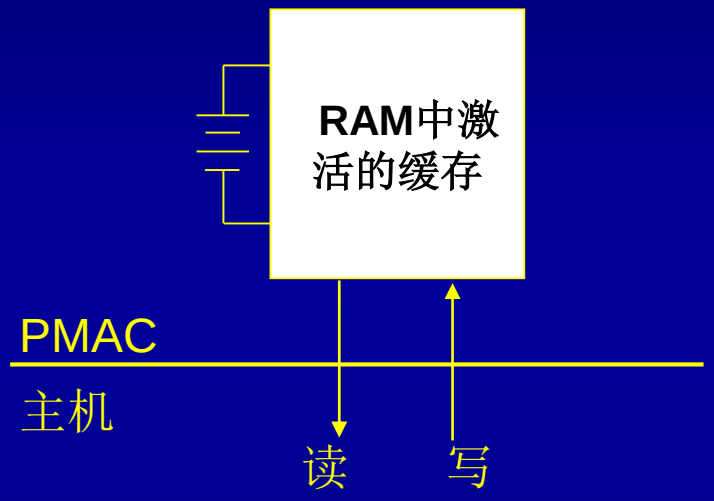


PMAC复位方式
1. 重新上电
2. INIT/信号低/高
3. $$$ 指令

NEW IDEAS IN MOTION

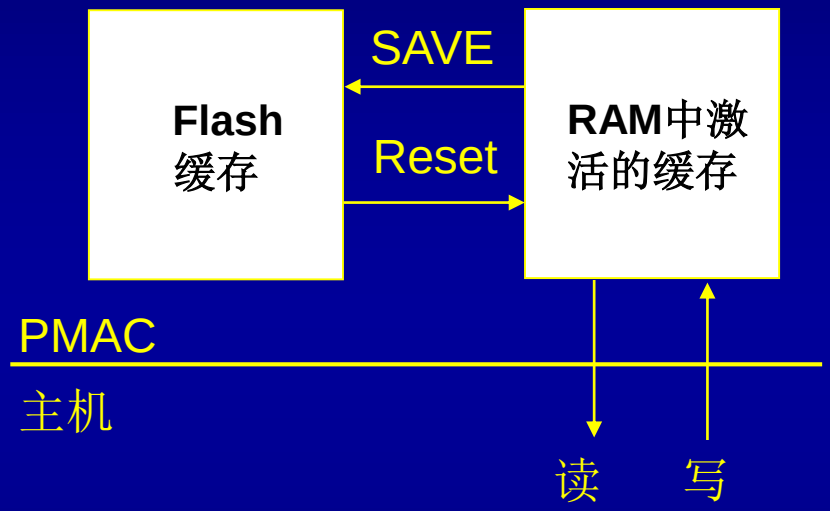
PMAC 程序和缓冲区存储

带后备电池 主存储器 [PMAC1]



- Programs 和buffers 自动被电池存储

Flash主存储器 [PMAC1.5, PMAC2]



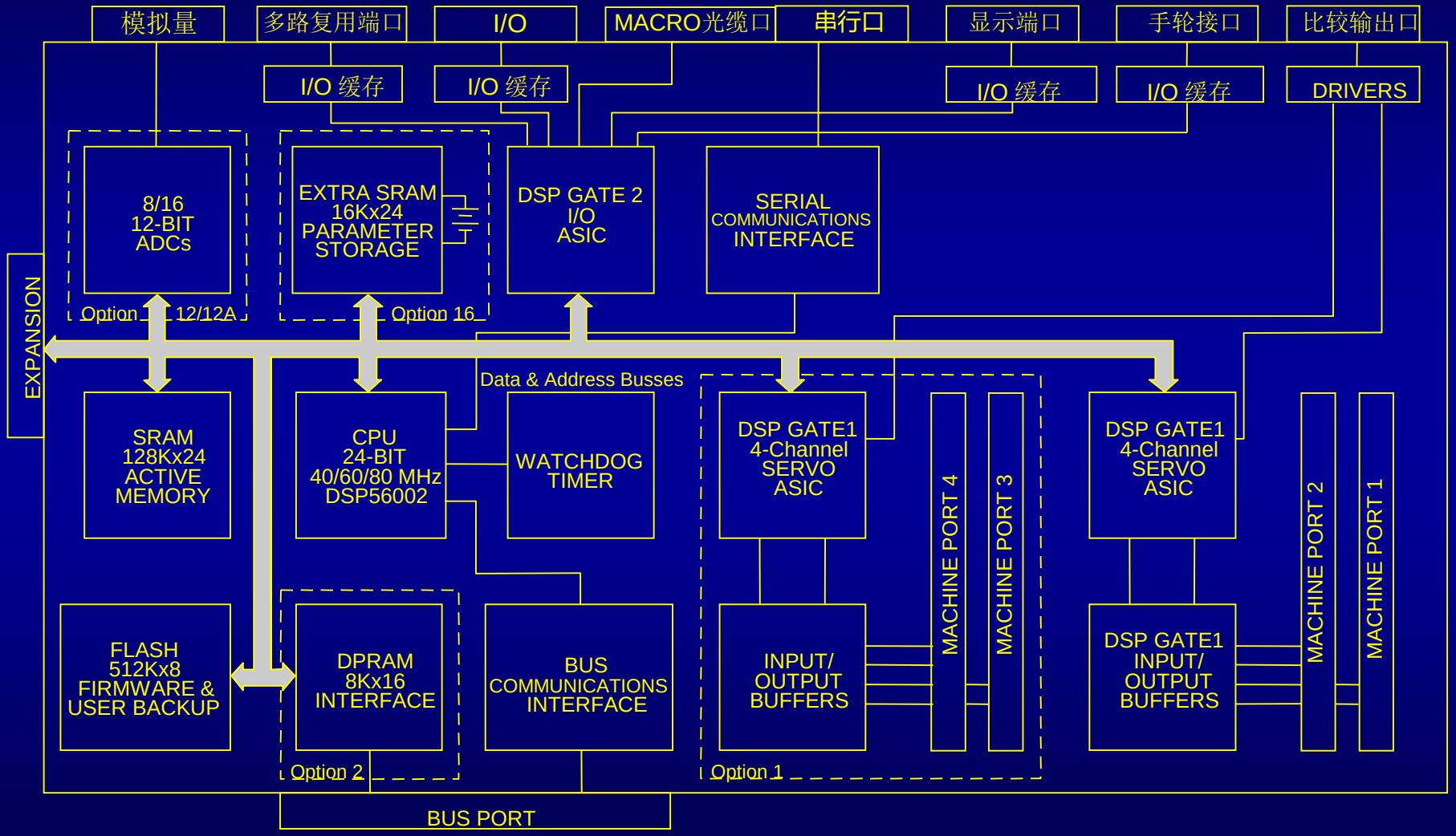
- Programs and buffers必须用SAVE 指令进行存储到FLASH中

PMAC(1) 系列板卡

板卡型号	通道数	总线	插槽	串口	DPRAM	注释
PMAC-PC*	4/8	ISA	1.5	RS422	Off-Board	
PMAC-Lite	4	ISA	1	RS232/ 422	On-Board	
Mini-PMAC	2+	ISA	0.7	RS232	On-Board	No JPAN, Interrupts
PMAC-VME*	4/8	VME	2	RS422	On-Board	
PMAC-PCI*	4/8	PCI	1.5	RS232 422	On-Board	

* Available with Turbo CPU

PMAC2 结构框图


模拟量
多路复用端口
I/O
MACRO光缆口
串行口
显示端口
手轮接口
比较输出口
I/O 缓存
I/O 缓存
I/O 缓存
I/O 缓存
DRIVERS
8/16
12-BIT
ADCs
EXTRA SRAM
16Kx24
PARAMETER
STORAGE
DSP GATE 2
I/O
ASIC
SERIAL
COMMUNICATIONS
INTERFACE
EXPANSION
Option 12/12A
Option 16
Data & Address Busses
SRAM
128Kx24
ACTIVE
MEMORY
CPU
24-BIT
40/60/80 MHz
DSP56002
WATCHDOG
TIMER
DSP GATE1
4-Channel
SERVO
ASIC
MACHINE PORT 4
MACHINE PORT 3
DSP GATE1
4-Channel
SERVO
ASIC
MACHINE PORT 2
MACHINE PORT 1
FLASH
512Kx8
FIRMWARE &
USER BACKUP
DPRAM
8Kx16
INTERFACE
BUS
COMMUNICATIONS
INTERFACE
INPUT/
OUTPUT
BUFFERS
DSP GATE1
INPUT/
OUTPUT
BUFFERS
Option 2
Option 1
BUS PORT

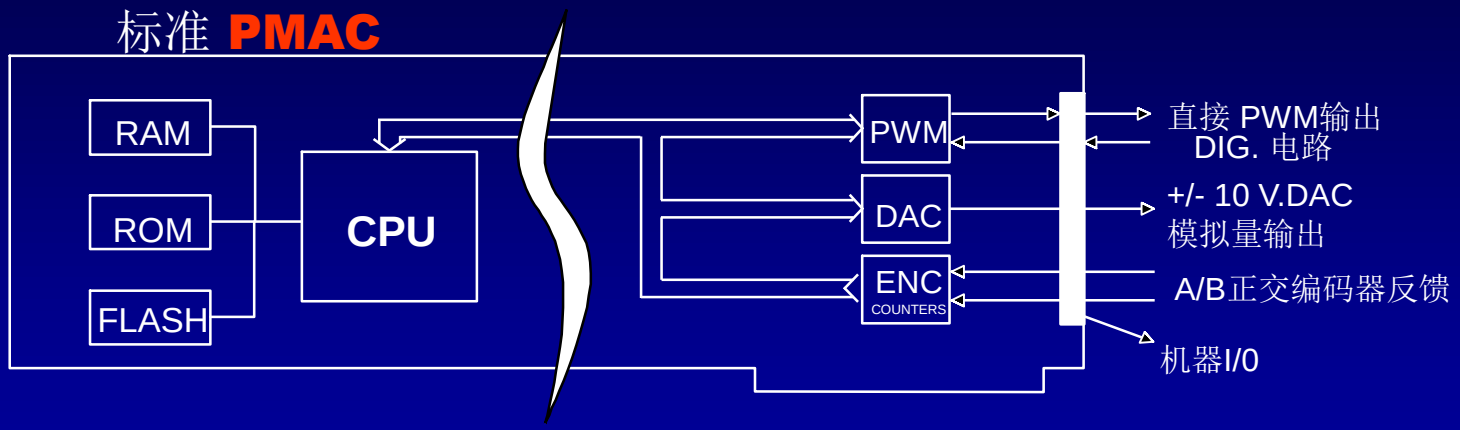
PMAC2 板卡

板卡型号	通道数*	总线	插槽	串口	MACRO
PMAC2-PC***	4/8	ISA	1.5	RS232**	Off-Board
PMAC2-Lite	4	ISA	1	RS232/422	Off-Board
Mini-PMAC2	2	ISA	0.7	RS232	Off-Board
PMAC2-PCI***	4/8	PCI	1.5	RS232/422	Off-Board
PMAC2-VME***	4/8	VME	2	RS232/422	Off-Board
PMAC2-PC Ultralite***	0	ISA	1	RS232**	On-Board
PMAC2-VME Ultralite	0	VME	1	RS232/422	On-Board

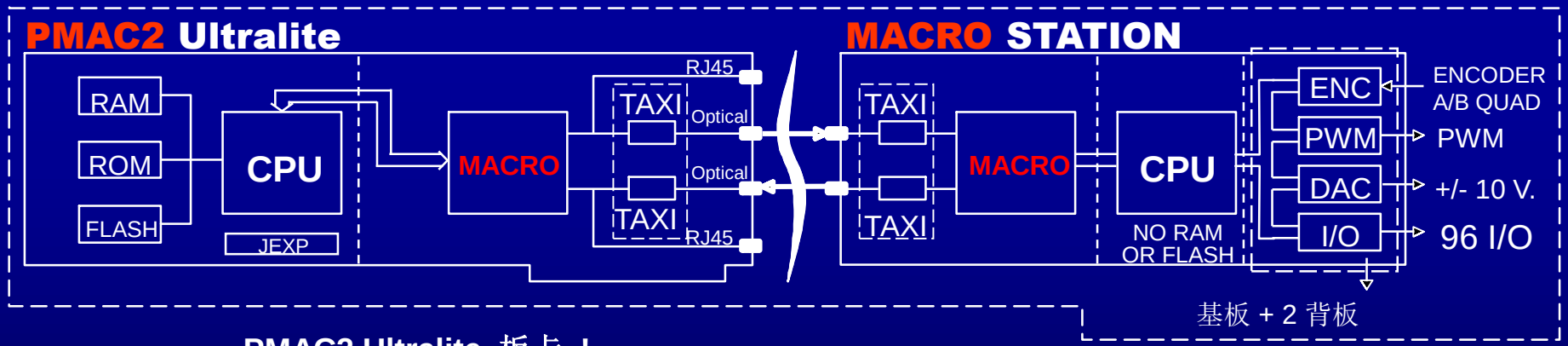


* 所有型号均有2路附加通道
** RS422 with Option 9L
*** Available with Turbo CPU

MACRO STATIONS



PMAC ＋（ MACRO ）



PMAC2 Ultralite 板卡 ！

1) 板卡接口只有 MACRO & I/O 接口
2) 保留了 PMAC2 的基板功能，用MACRO STATION控制8轴。
3) PC & VME 格式
4) TURBO 版本的 PMAC ULTRALITE 能控制 32 轴

Delta Tau 3U 矩阵结构

	MACRO	Turbo
Stack	MACRO Stack	Turbo Stack
PACK	UMAC MACRO	UMAC Turbo



PMAC vs. Turbo PMAC

功能	PMAC	Turbo PMAC
最大控制轴数	8	32
最大伺服通道接口数	16 (8 off-board)	40 (32 off-board)
最大 MACRO 光缆通讯接口数 Ultralite)	16	64 (Turbo
最大坐标系数	8	16
CPU 频率 (MHz)	20, 40, 60, 80	80, 100, 160
CPU 时钟周期（uS)	0.5	0.75
有效 CPU 频率 (MHz)	20, 40, 60, 80	120, 150, 240
I,P,Q & M-变量数	1024 each	8192 each
直接数组调用 {e.g. P(P1)=2}?	No	Yes
用户 PID 扩展伺服算法功能?	No	Yes

PMAC vs. Turbo PMAC (cont.)

功能	PMAC	Turbo PMAC
固件存储器容量	32k x 24	64k x 24
编译 PLC存储器容量	15k x 24	48k x 24, 432k x 24
用户缓冲存储器容量	34k x 48	26k x 48, 212k x 48
DPRAM 容量	8k x 16	8k x 16, 32k x 16
追加后备电池存储器容量	7k x 48	16k x 48, 64k x 48
Lookahead 功能?	选项	标准配置
串行口数量	1	2 (2nd optional)
同时多端口通讯?	No	Yes
最大同时数据采集源	24	48
最大转换表行数	32	192
单运动最大同时分配M变量	3	63 (1 个坐标系.)
反向运动学功能?	No	Yes
快速运动中改变目的位置功能?	No	Yes

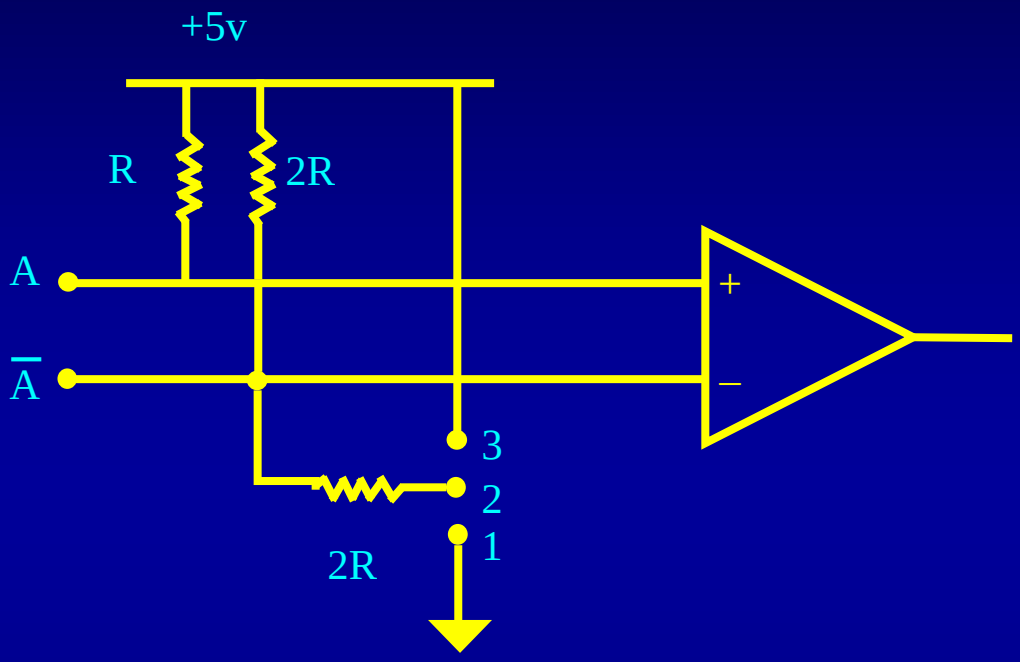
Turbo PMAC CPU 版本

Option	CPU Speed	CPU Internal Memory	External "P" Program Memory (Code)	External "X/Y" Data Memory
5C0*	80 MHz	8k x 24	128k x 24	128k x 24
5C3	80 MHz	8k x 24	512k x 24	512k x 24
5D0	100 MHz	34k x 24	128k x 24	128k x 24
5D3	100 MHz	34k x 24	512k x 24	512k x 24
5E0	160 MHz	128k x 24	128k x 24	128k x 24
5E3	160 MHz	128k x 24	512k x 24	512k x 24

PMAC Watchdog 看门狗计时器

- 要求电压 > **4.75 V**
- 要求输入频率 > **20 Hz**
  - 计数 > **0**实时中断
  - 触发器计数定时
  - 后台管理重置计数器最大值 **(=512)**
  - $$Frequency = \frac{Servo\ Frequency}{2 \bullet (I8 + 1)}$$
- 在要求电压或输入频率之下时
  - 强制**0**值输出
  - 放大器禁能
  - **“FEFCO” 输出 (PMAC1)**
  - **“WDO” 输出锁存 (PMAC2)**

PMAC 编码器输入电路

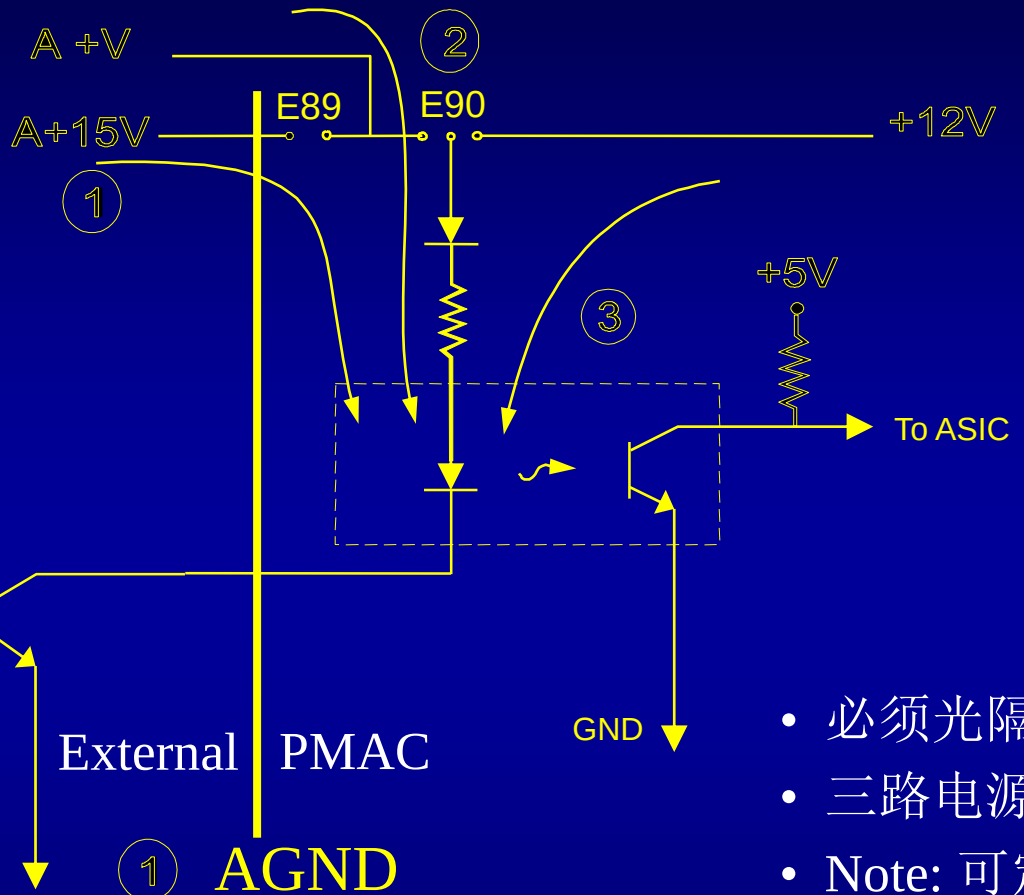


- 无编码器或单端编码器输入：
  pin 1 to 2 差分 +2.5 V 输入

- 开集电极编码器输入：
  pin 2 to 3 差分 +5 V 输入
  能支持编码器电路外部XOR丢失

•差分编码器输入方式直接接卡

PMAC(1) 标志信号输入

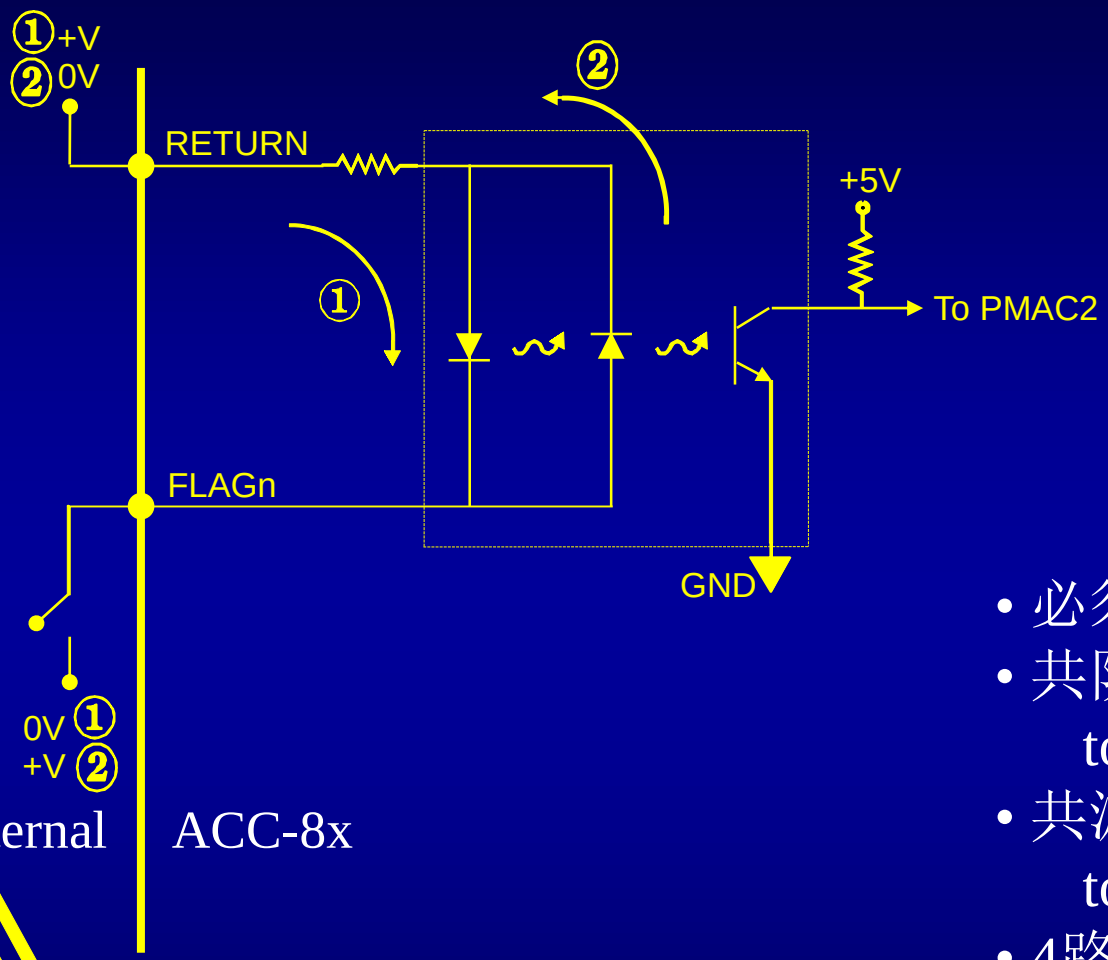


FLAG INPUTS

FAULTn
+LIMn
-LIMn
HMFLn

- 必须光隔输入
- 三路电源线回路
- Note: 可定义共源极或共阴极输入

PMAC2 ACC-8x标志信号输入



FLAG INPUTS

USERn

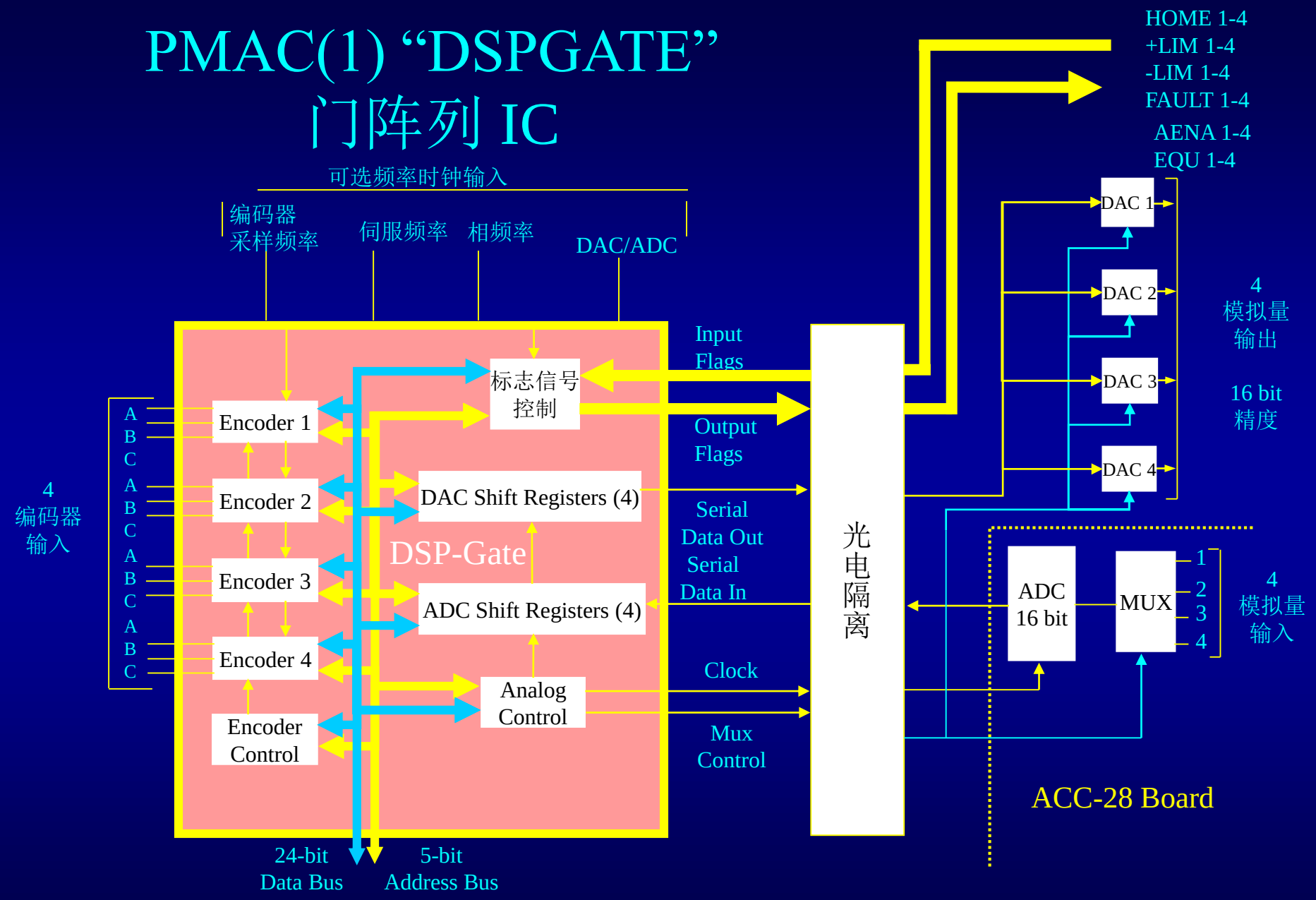
PLIMn

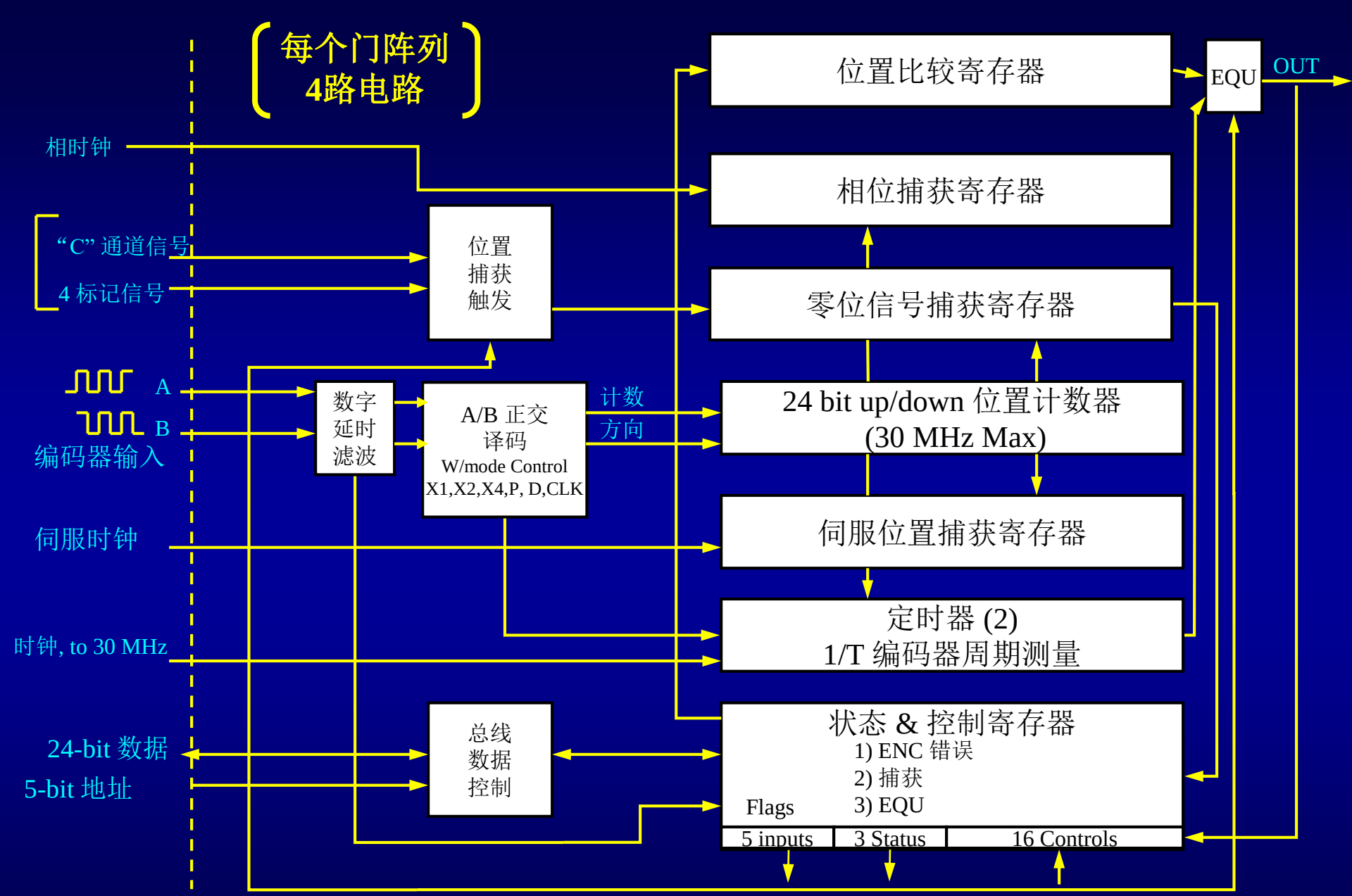
MLIMn

HOMEn

- 必须光隔输入
- 共阴极输出, 连接 RETURN to +24V
- 共源极输出, 连接 RETURN to 0V
- 4路标志信号共用1个 RETURN

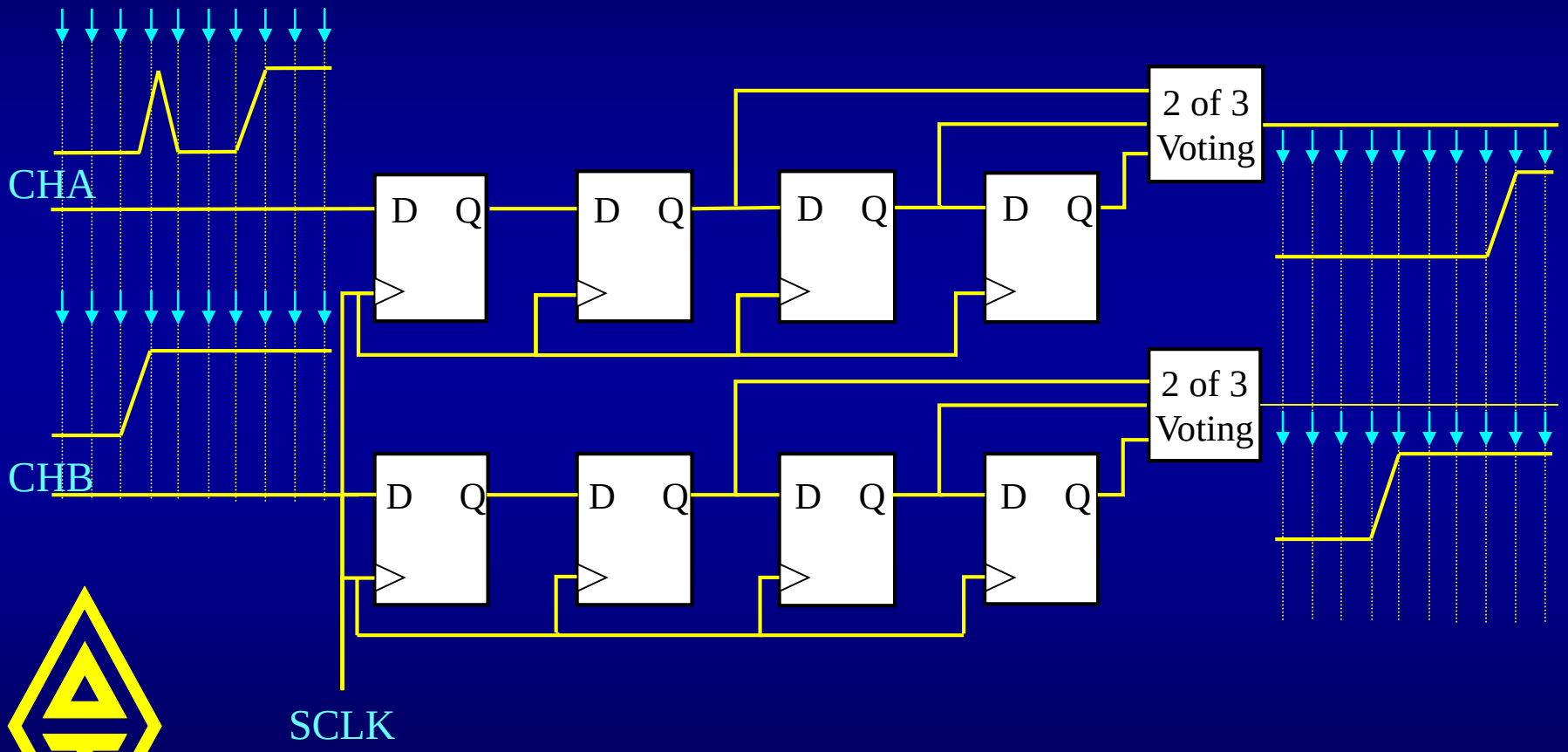
PMAC(1) “DSPGATE” 门阵列 IC



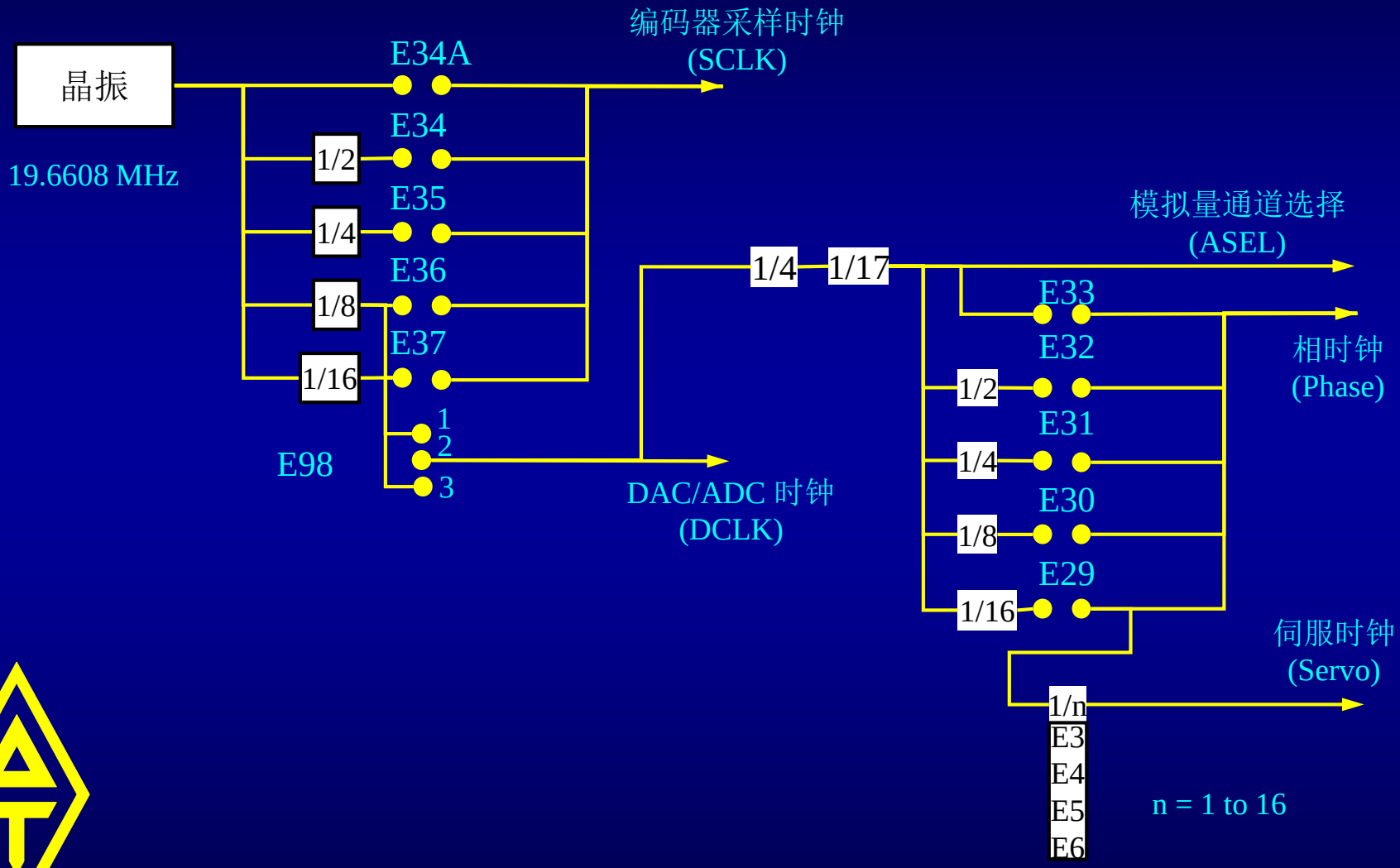


编码器功能: DSP Encoder/Flag 门电路 (1 of 4)

编码器数字延时滤波

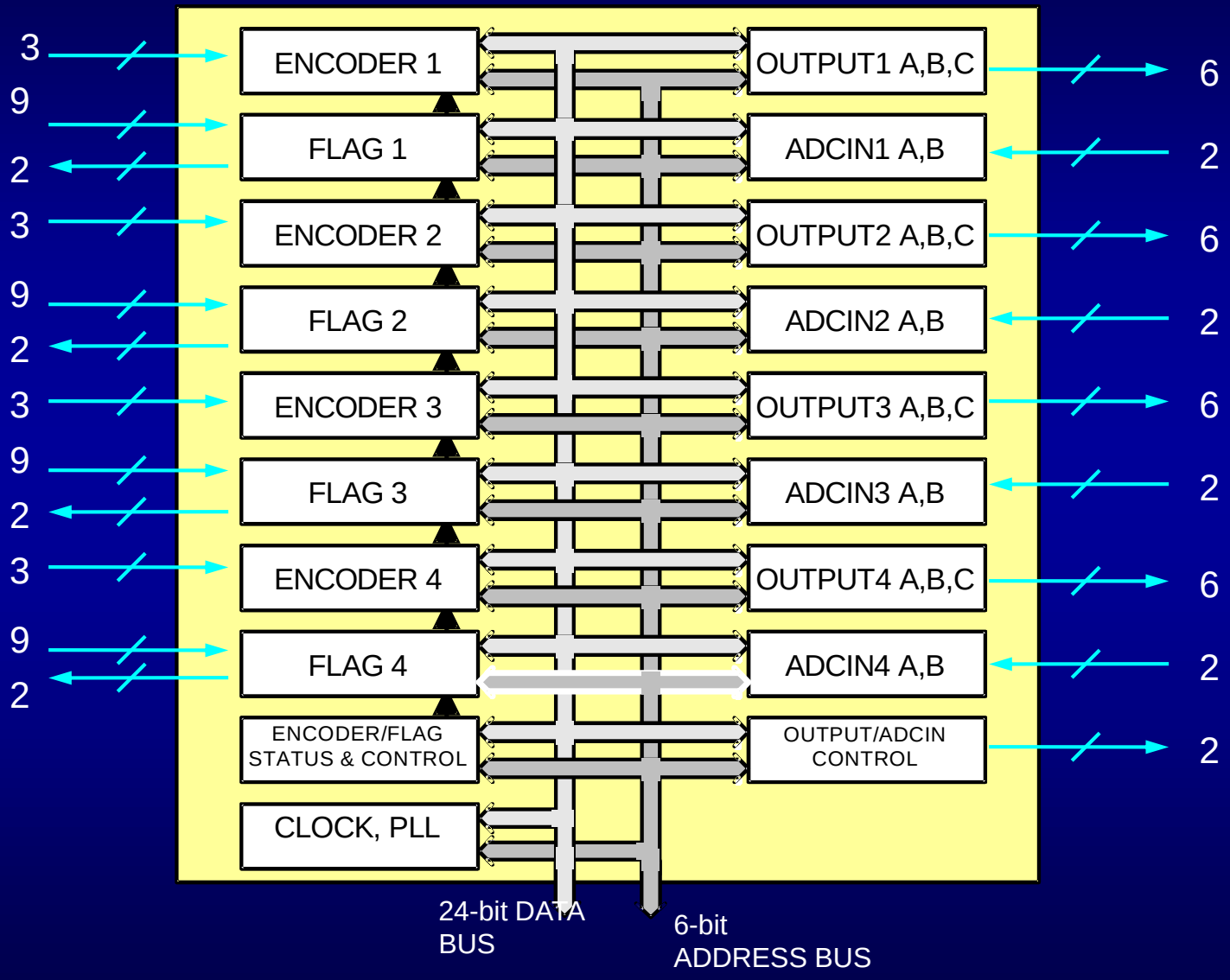


PMAC(1) 时钟分配电路

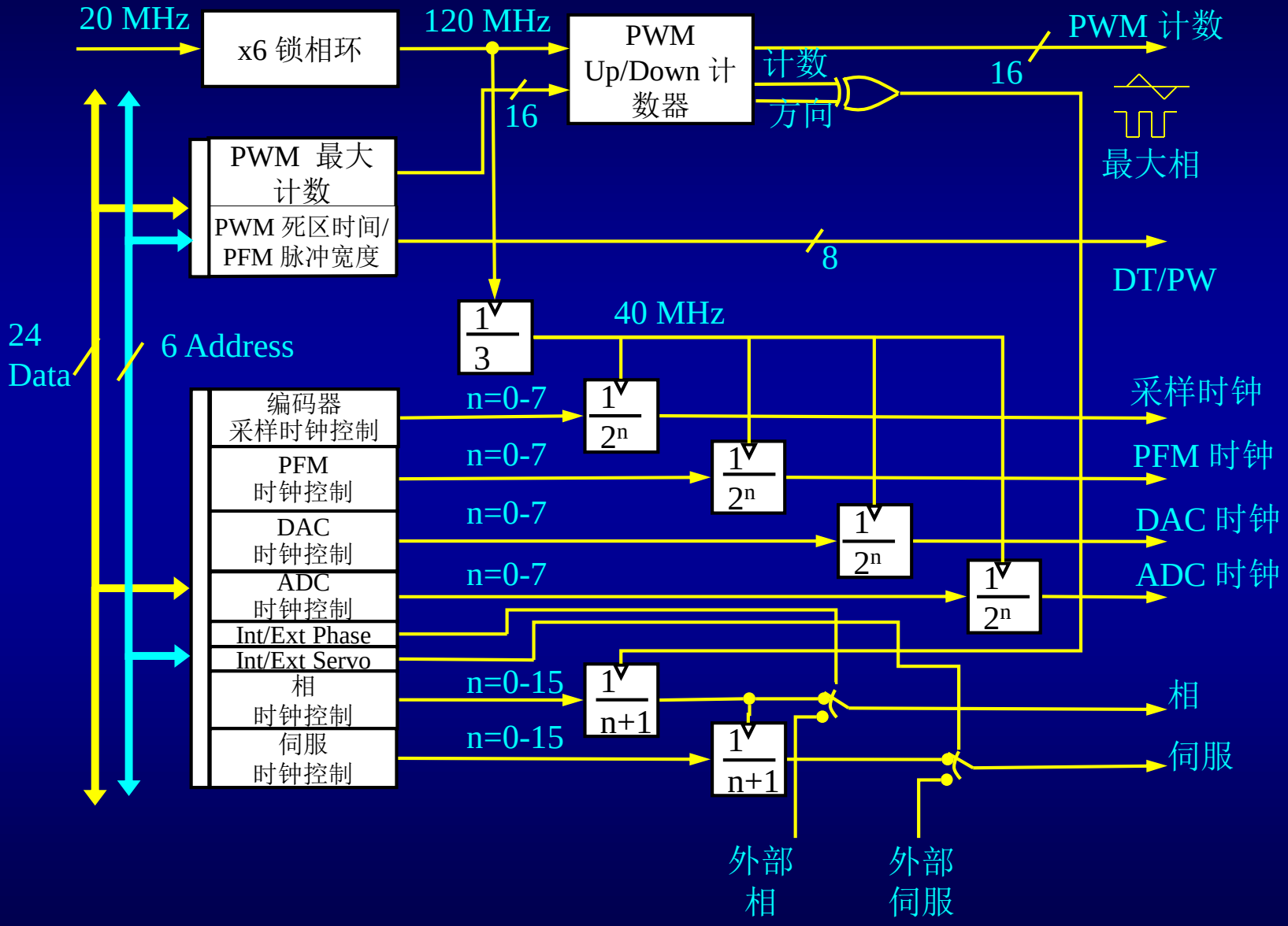


NEW IDEAS IN MOTION

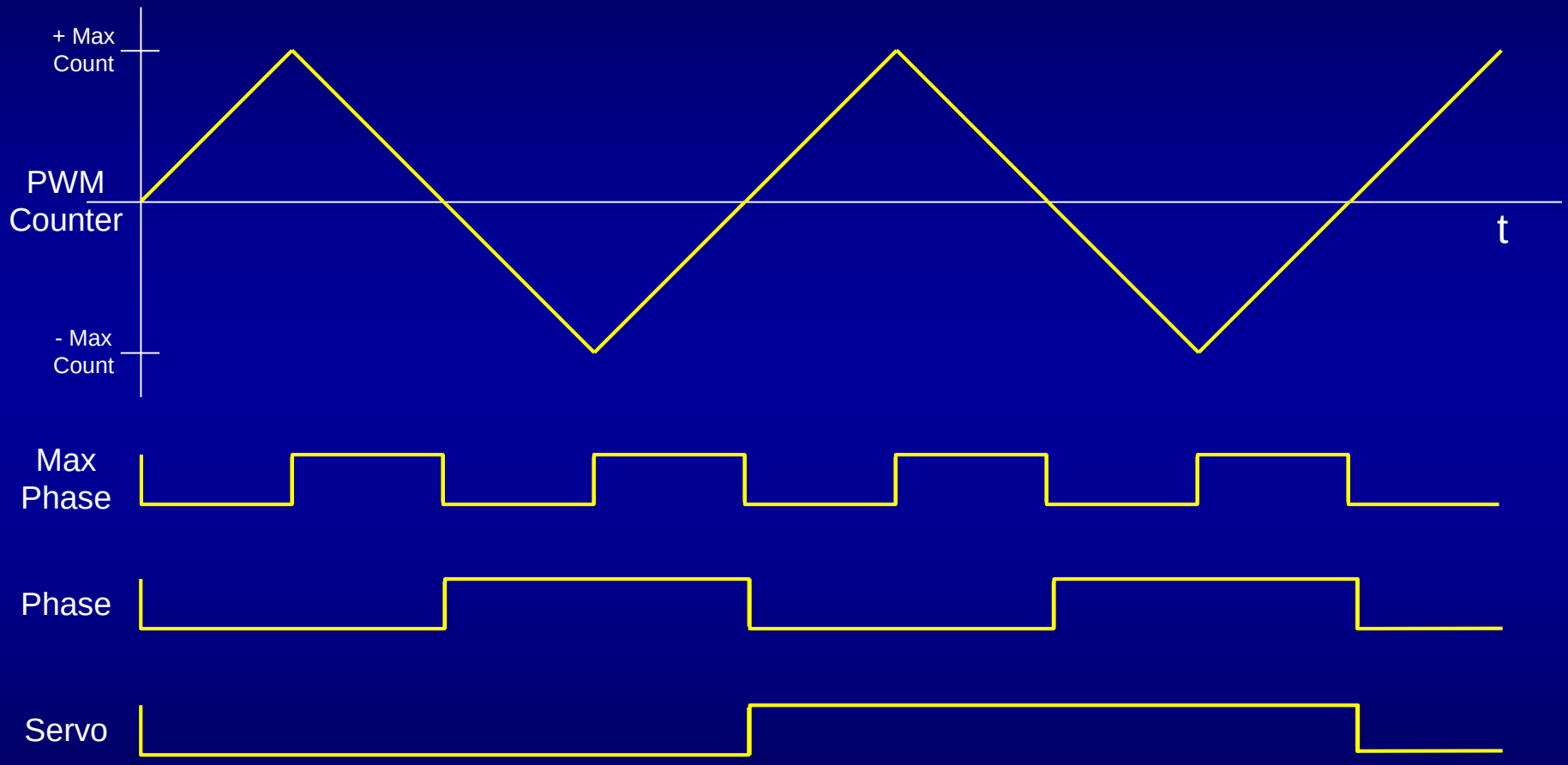
PMAC2 门阵列 IC “DSPGATE1”



PMAC2 门阵列 IC 时钟控制

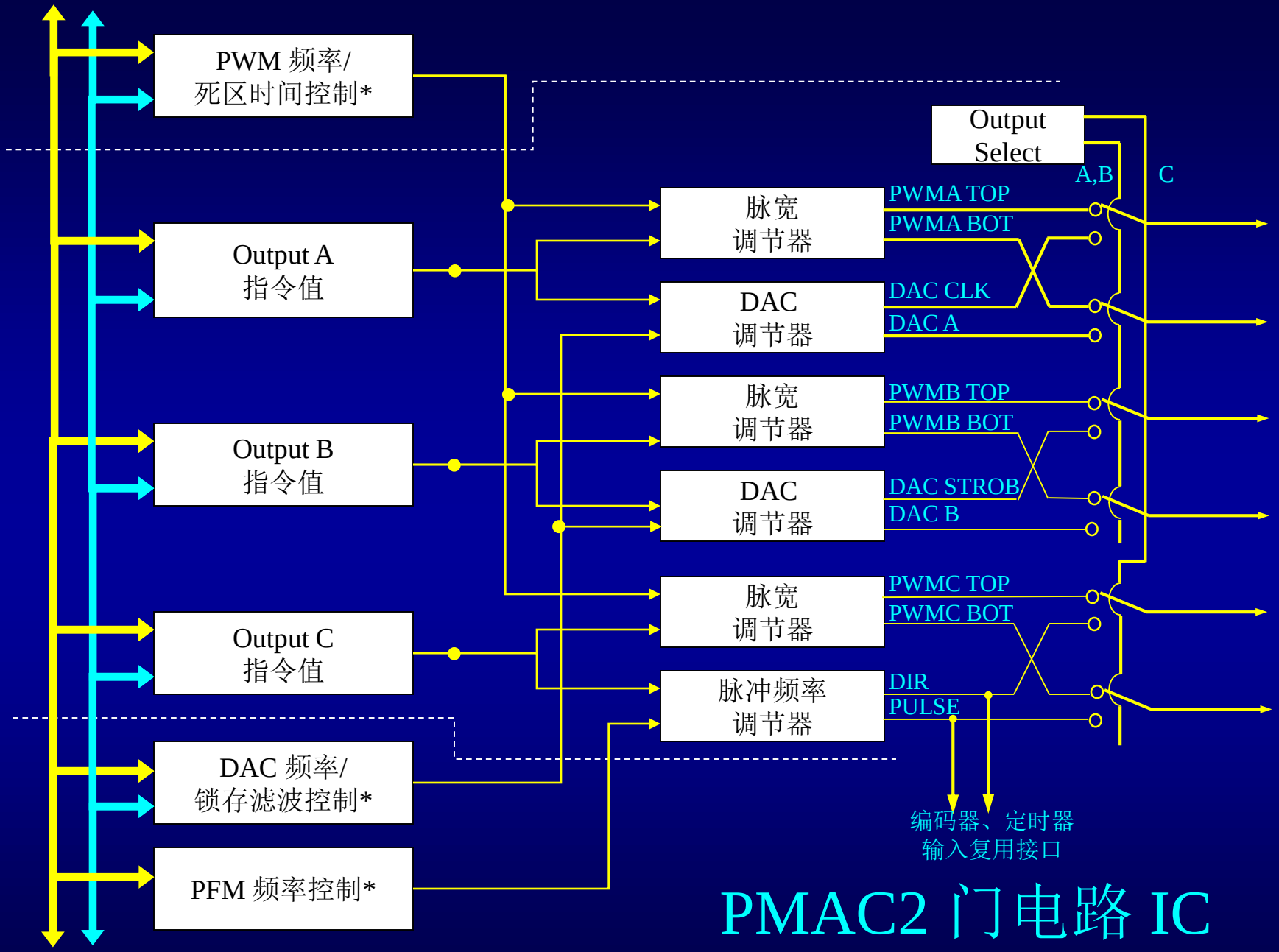


PMAC2 时钟信号时序



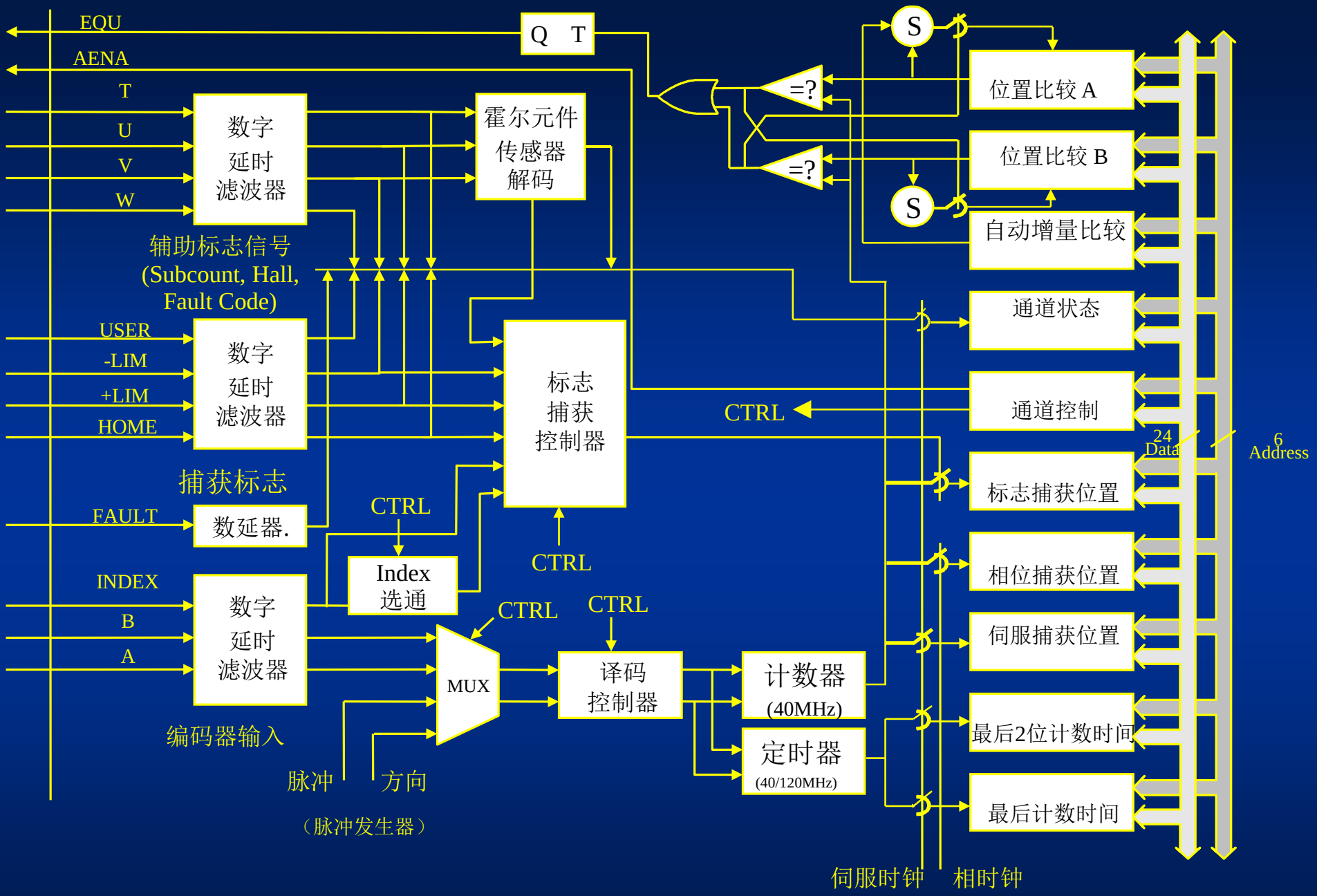
PMAC/PMAC2 时钟跳线控制

Clock	PMAC(1)	PMAC2	PMAC2 Ultralite
"Max Phase"	E98 (DAC, ADC Clock)	I900 [I7m00]	I992 [I6800]
Phase	E98, E29-E33	I900, I901 [I7m00,I7m01]	I992, I997 [I6800, I6801]
Servo	E98, E29-E33, E3-E6	I900, I901 , I902 [I7m00,01,02]	I992, I997, I998 [I6800,01,02]
Encoder Sample	E34-E38	I903, I907 [I7m03]	MI903, MI907
DAC	E98	I903, I907 [I7m03]	MI903, MI907
ADC	E98	I903, I907 [I7m03]	MI903, MI907
PFM	--	I903, I907 [I7m03]	MI903, MI907
PWM	--	I900, I906 [I7m00]	MI900, MI906

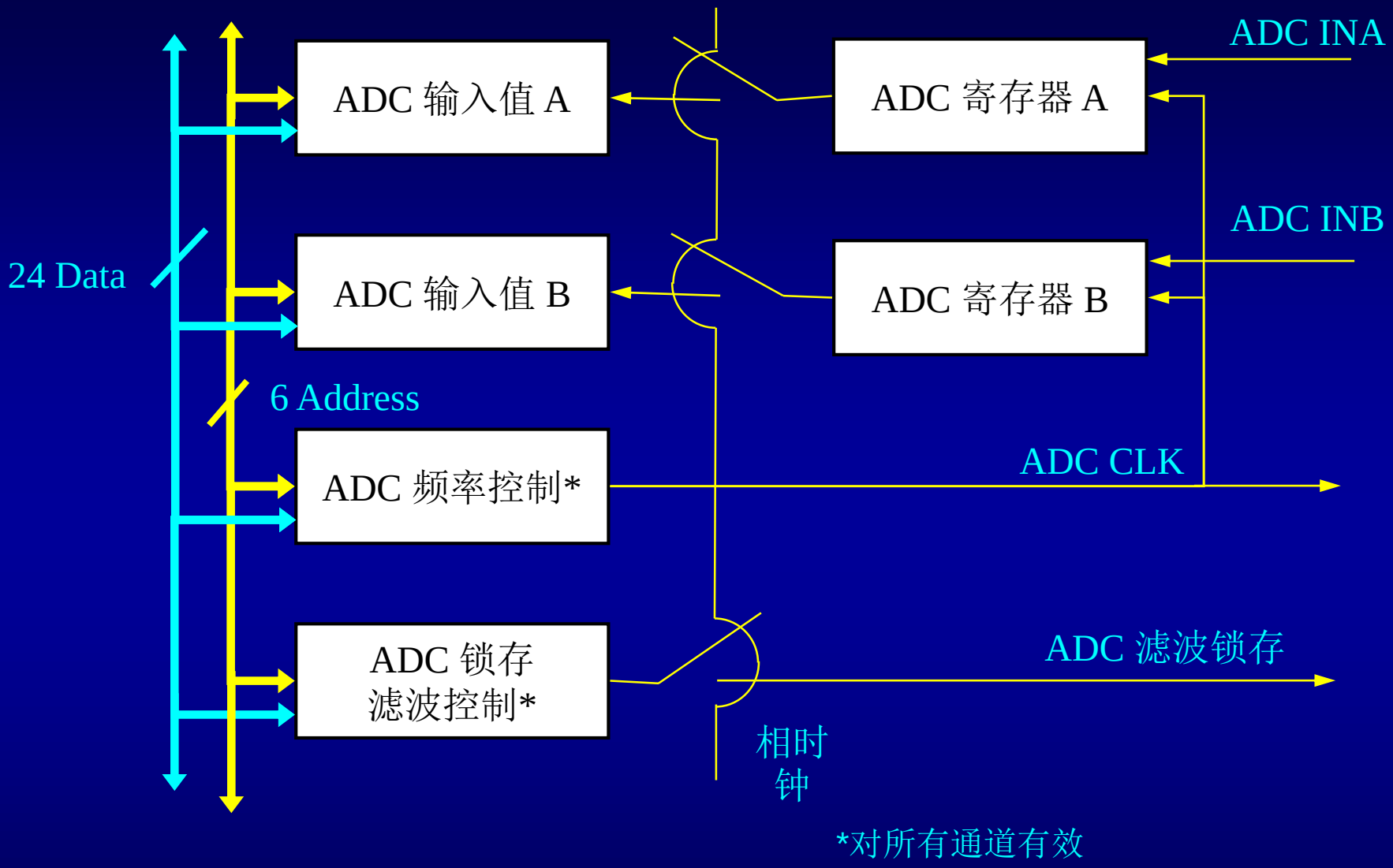


*对所有通道有效

PMAC2 门电路 IC 输出通道 (1 of 4)

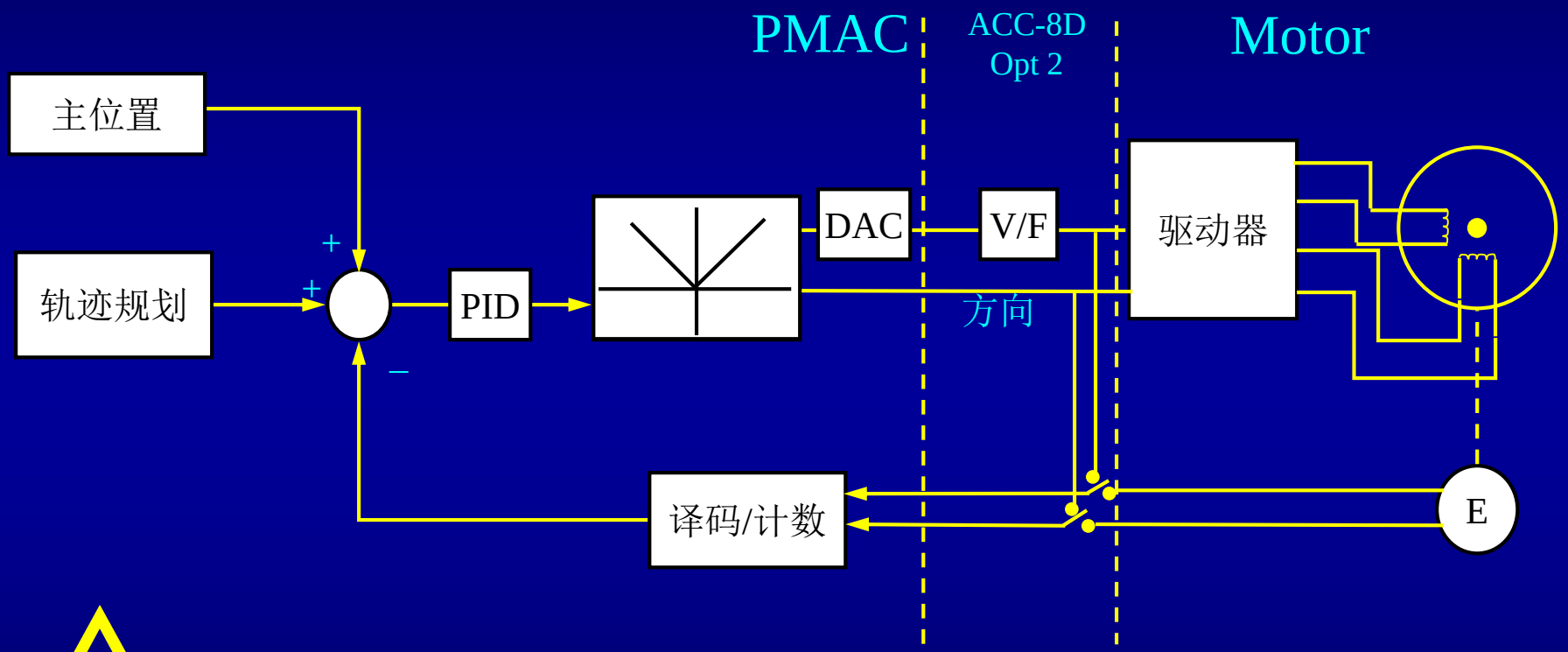


PMAC2 门阵列 IC 编码器/标志信号通道 (1 of 4)

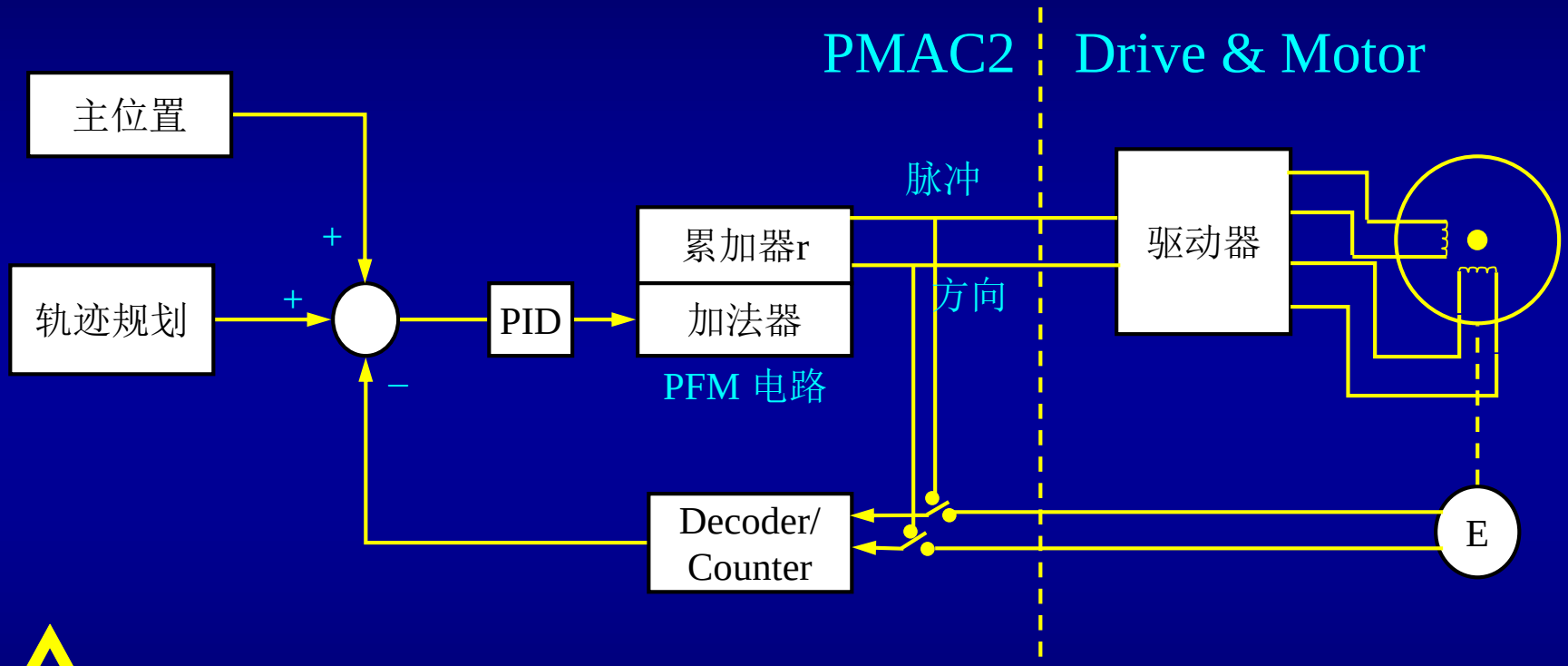


PMAC2 门电路 IC ADC 输入通道(1 of 4)

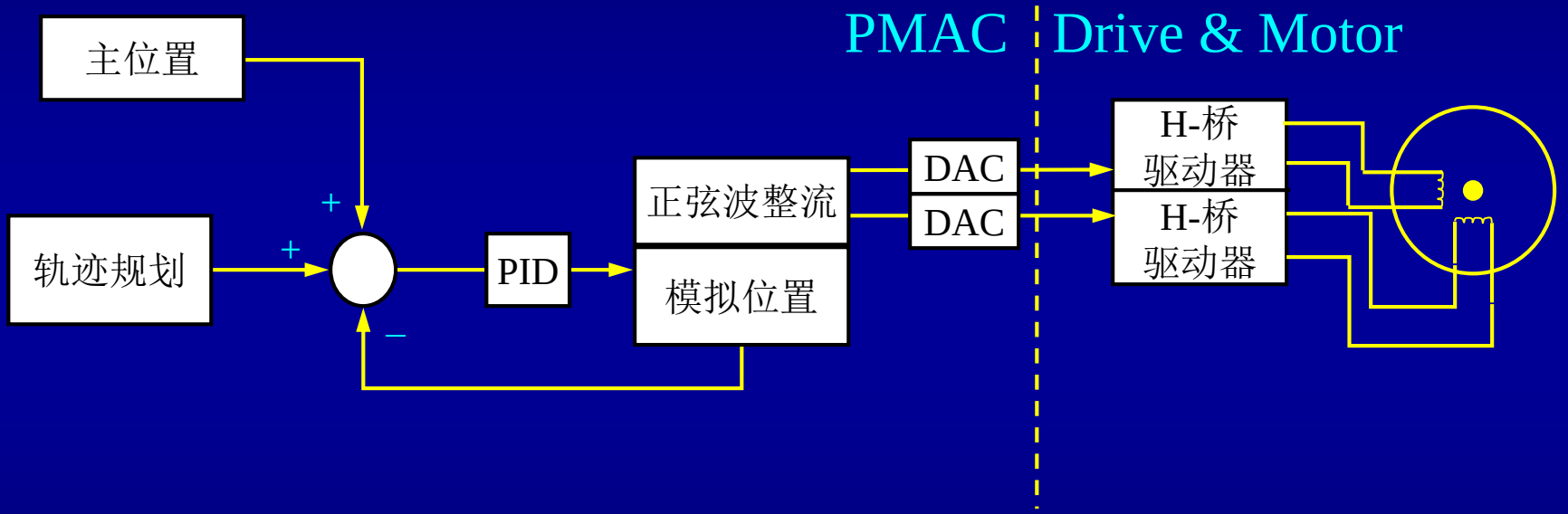
PMAC(1) 脉冲加方向控制



PMAC2 PMAC(1) 脉冲加方向控制

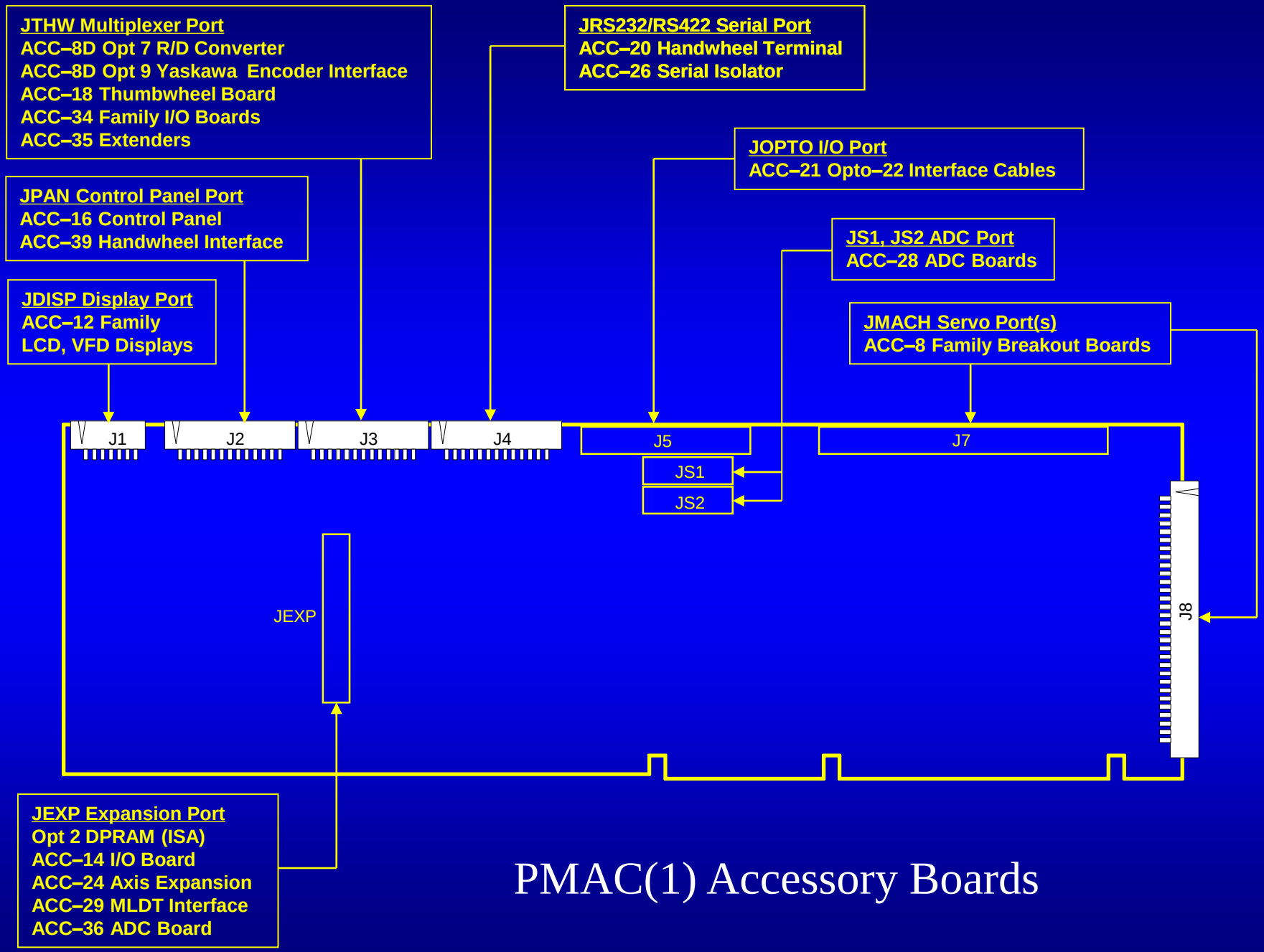


PMAC/PMAC2 直驱微步控制

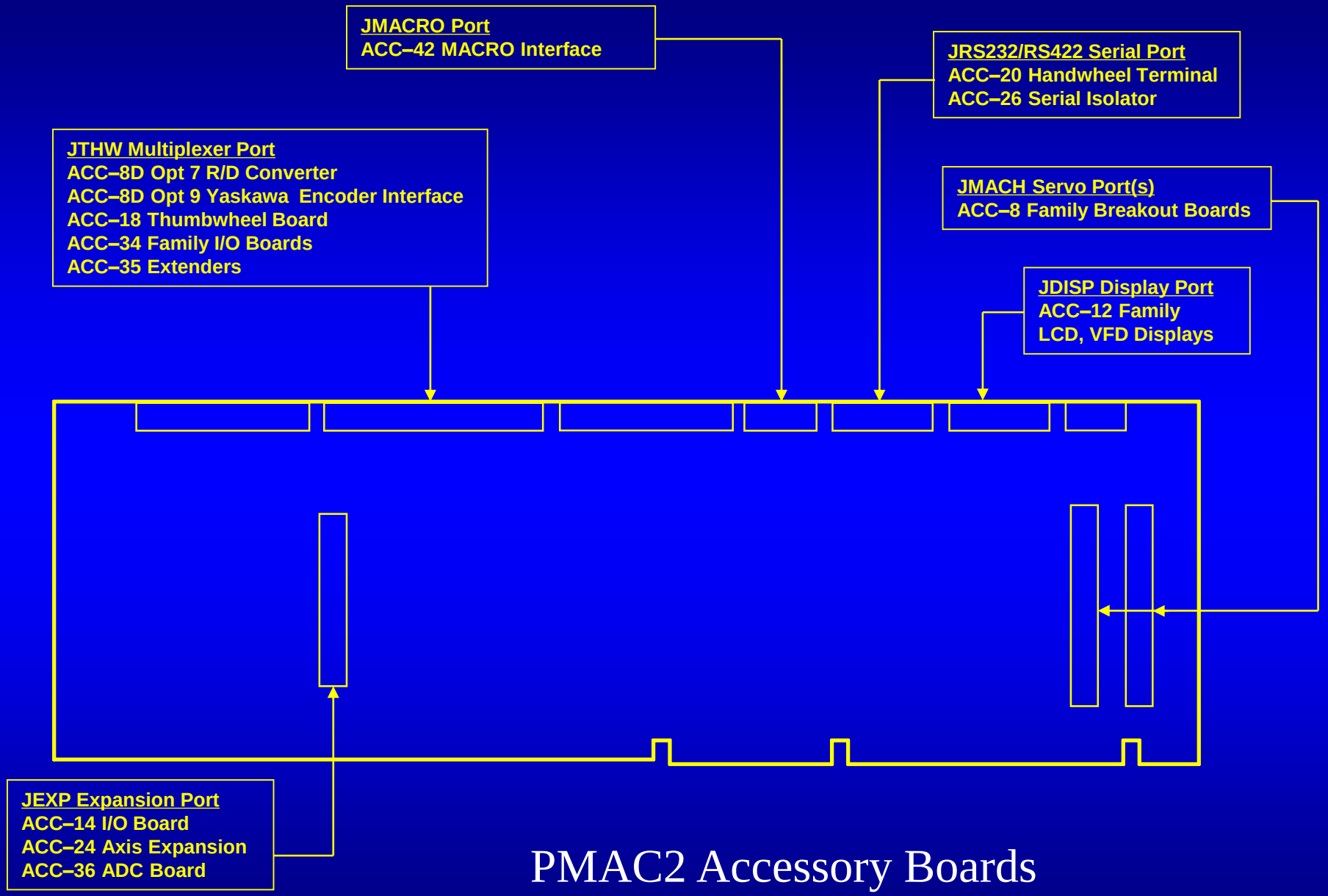




NEW IDEAS IN MOTION

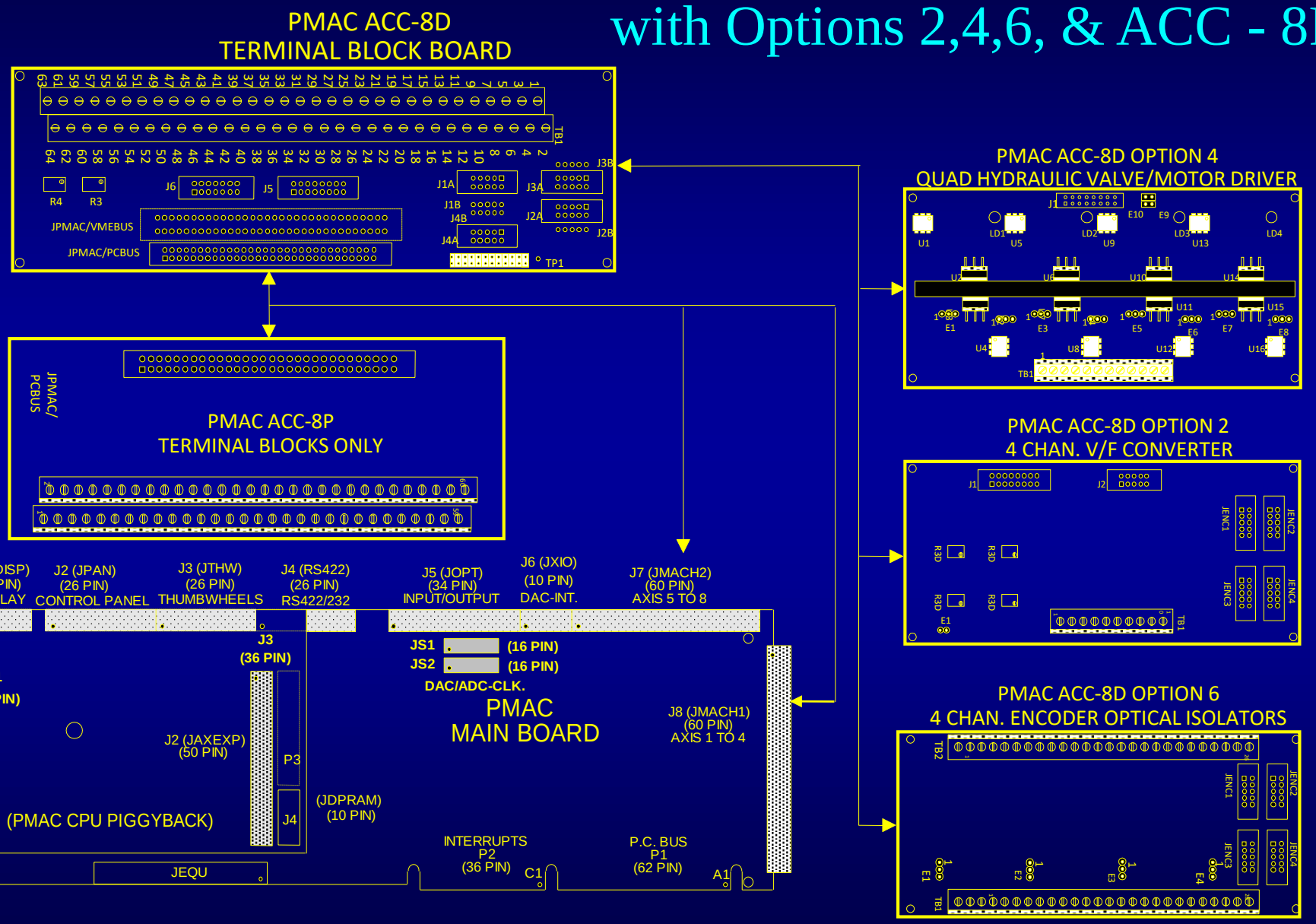


PMAC(1) Accessory Boards

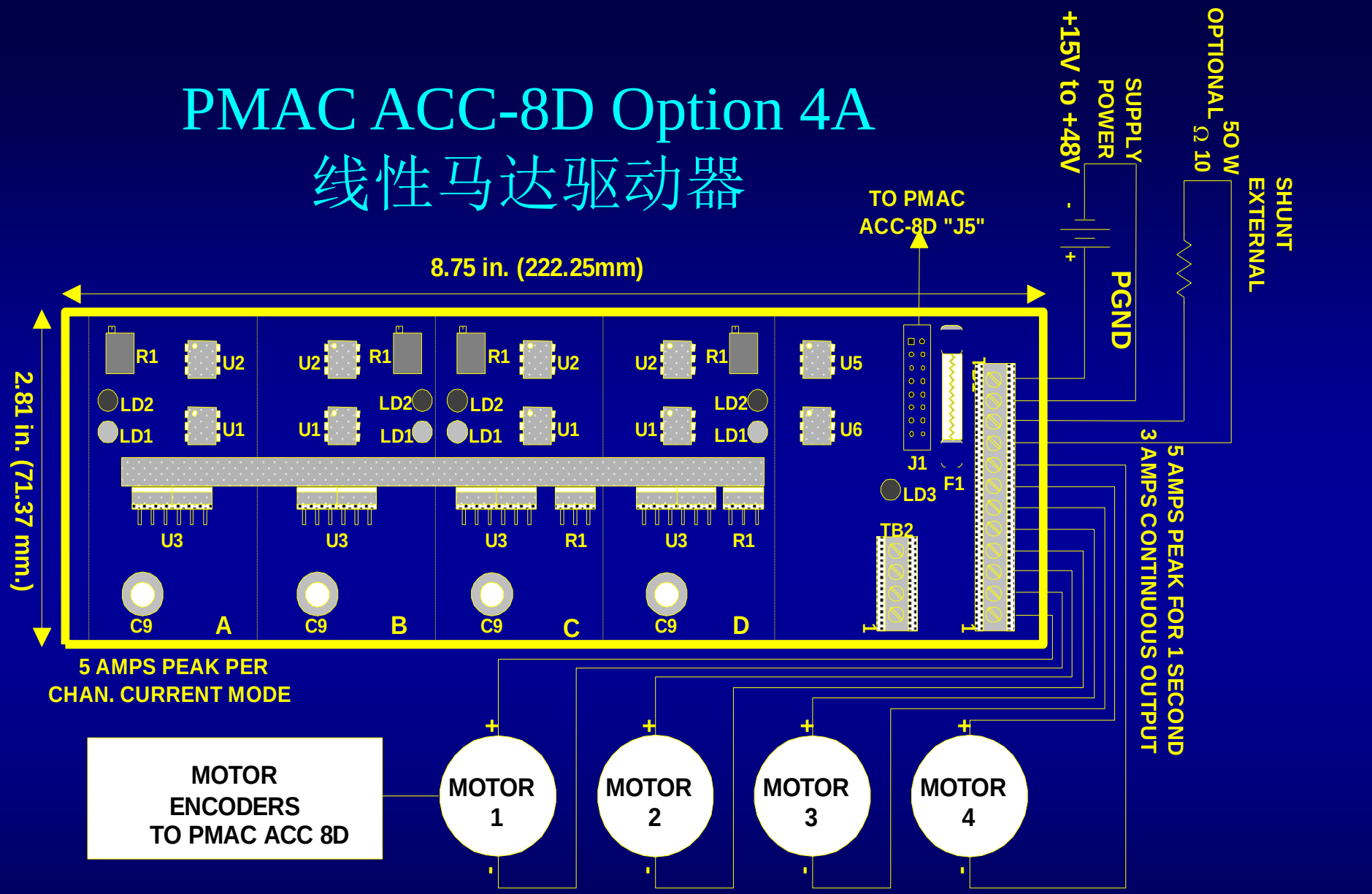


PMAC2 Accessory Boards

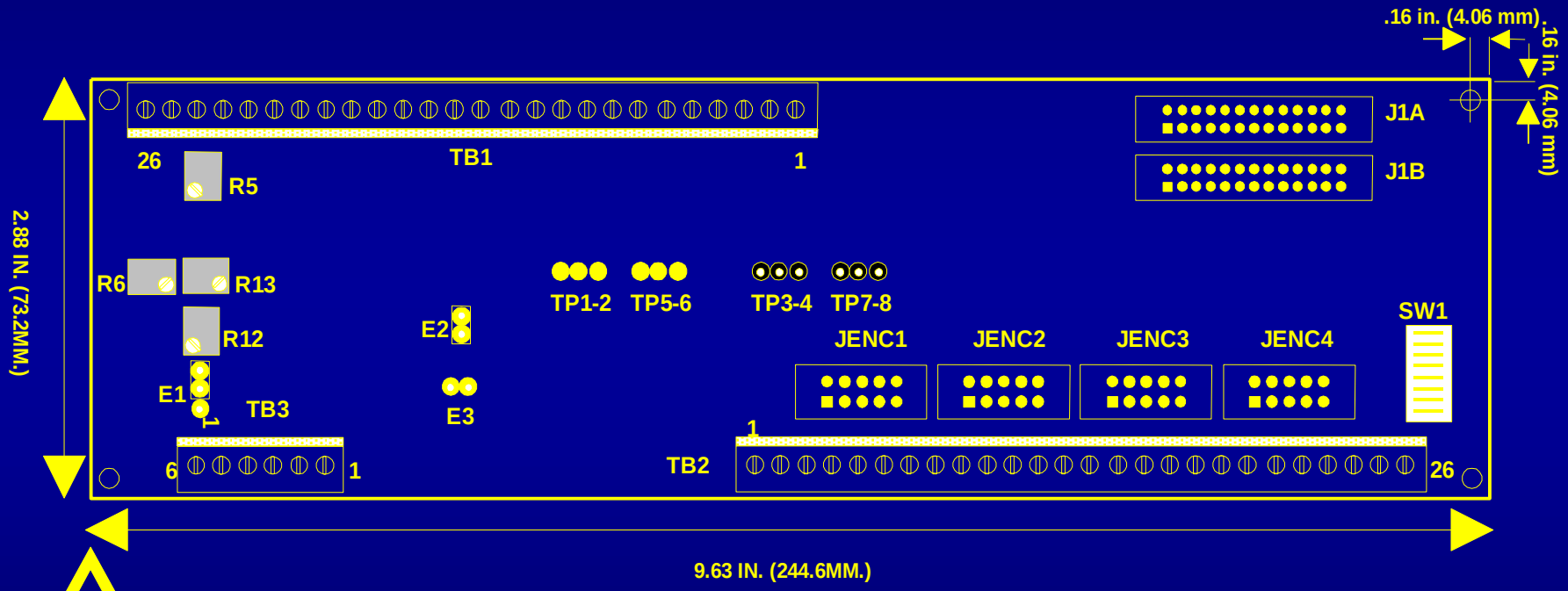
PMAC Accessory ACC - 8D with Options 2,4,6, & ACC - 8P



PMAC ACC-8D Option 4A
线性马达驱动器



ACC-8D Option 7
旋转变压器—数字转换器



NEW IDEAS IN MOTION

ACC-8D Option 7
旋转变压器—数字转换器

- 基板提供 2 极点 RDC’s
- 可选option A , 提供 4 极点 RDC’s
- 12-bit 精度: 4096 计数 (1024 “线”) /每电子周期
- 能接收 2.5 kHz, 5 kHz, 10 kHz 激励
- 单独激励增益调节
- 2.5 V_{pp} - 10 V_{pp} sin/cos 反馈信号
- 提供感应同步器接口

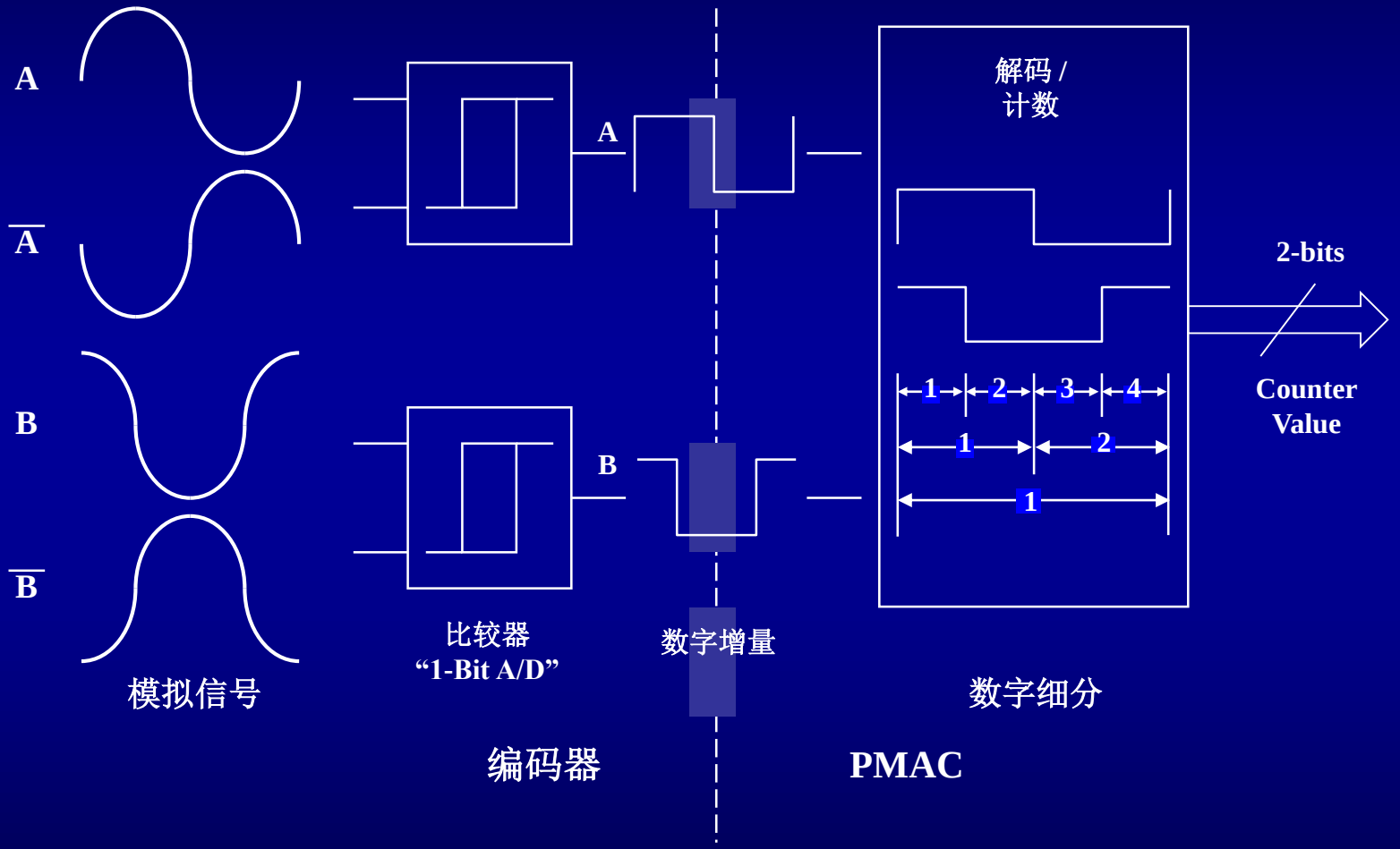
ACC-8D Option 7
旋转变压器—数字转换器(续)

- 通过PMAC多路复用口读入绝对位置
  - 从Ix81读入上电相位置
  - 从Ix10读入上电伺服位置
  - 从 I9x, I8x读入上电旋变齿轮比
  - 用户使用M变量读入 TWR 格式
- 通过PMAC通用编码器口读入增量位置
  - 类似于编码器输入进行伺服更新
  - 只有旋变齿轮设定需要这样做

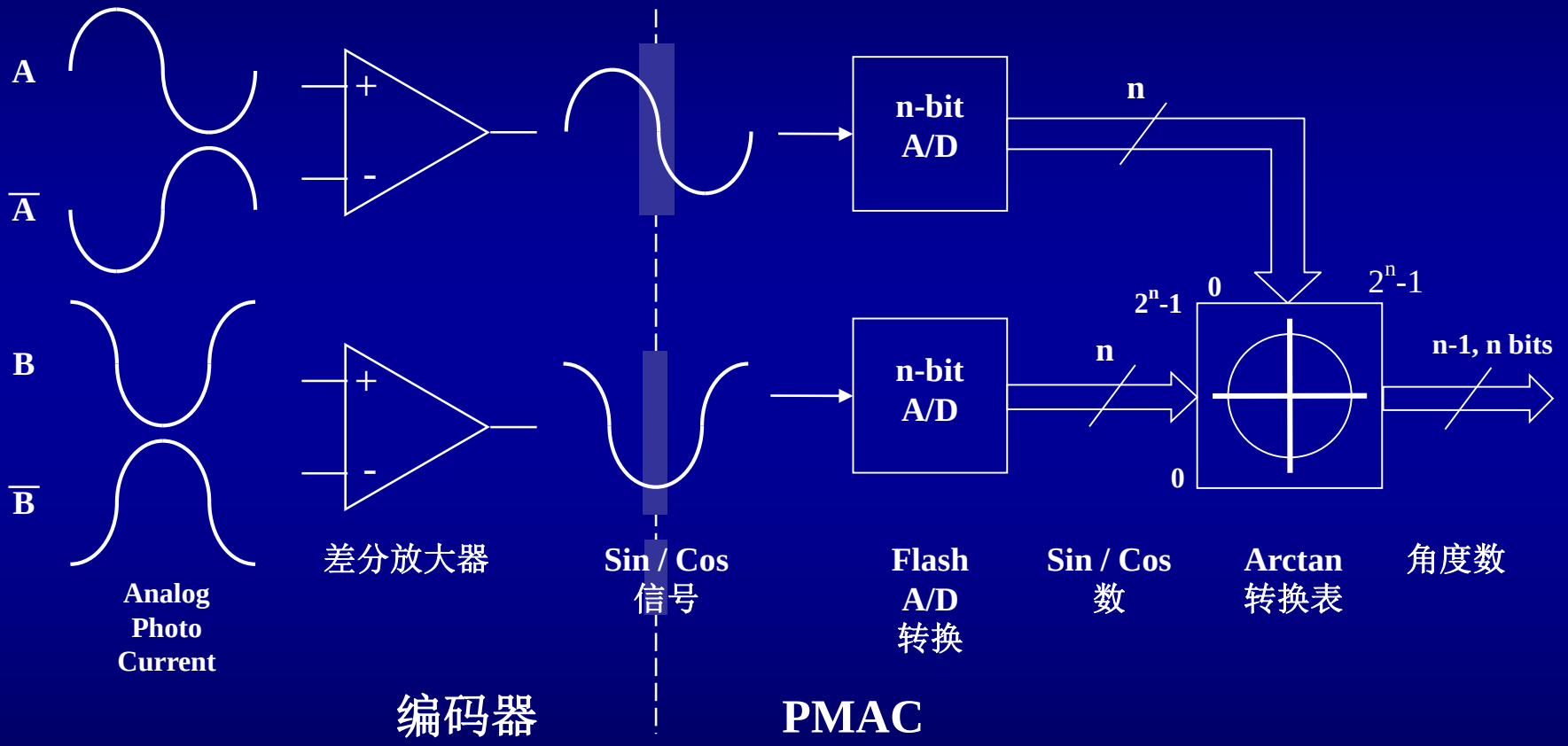
(Only fine resolver of geared set needs this)



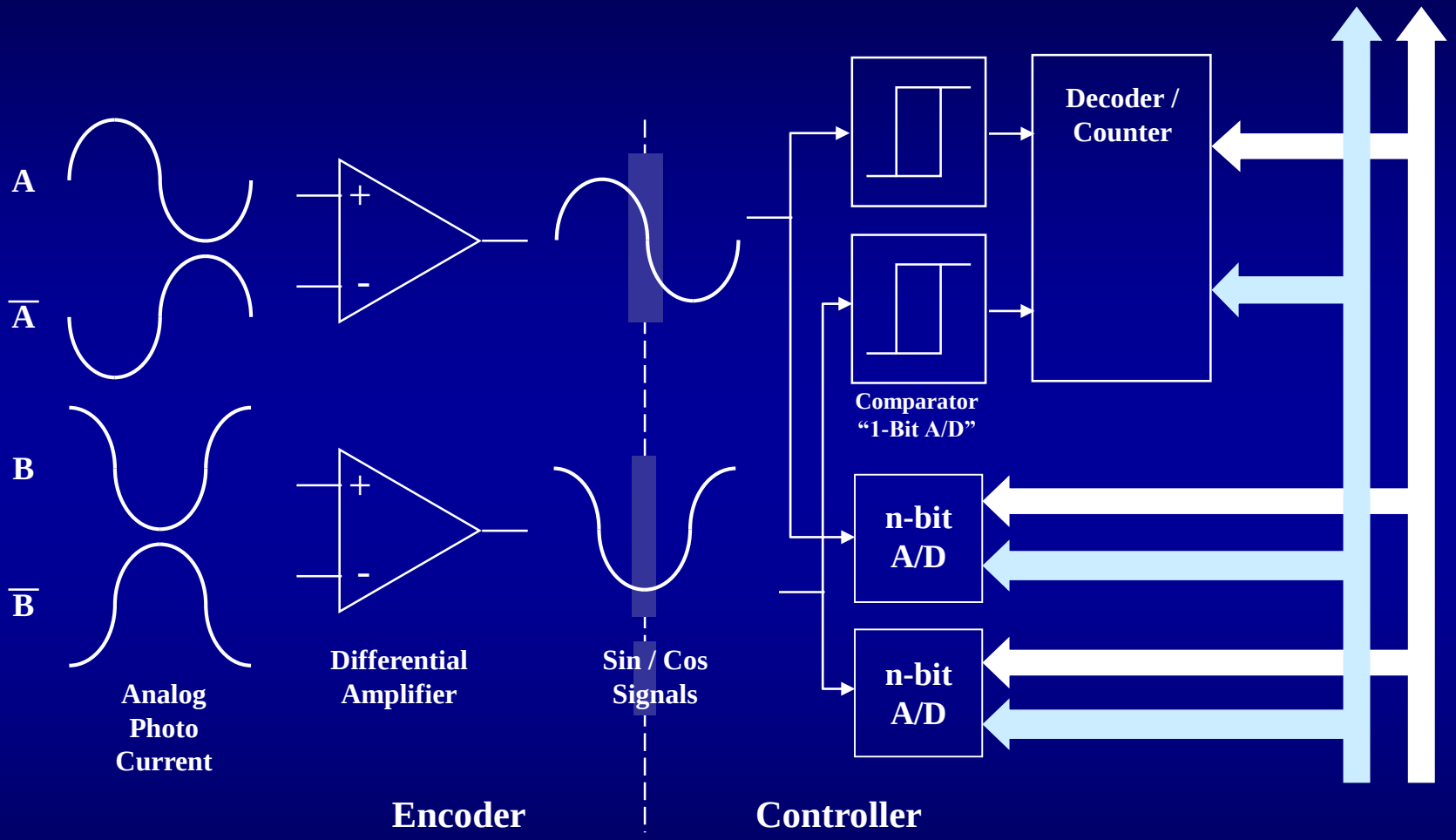
编码器细分: 数字增量编码器



编码器细分: 模拟量“Sin / Cos”编码器



编码器细分: 高分辨率模拟量“Sin / Cos”编码器



256x 低精度细分器

- ACC-8A, ACC-8D-Opt8, ACC-DE, ACC-8FE
- 128 or 256 细分数/线 (LSB=1/32 software count)
- 增量计数频率 1x or 2x 倍
- 最大线输入频率 150 kHz



ACC-51高精度细分器

- ACC-51P 用于 PMAC PC/ISA ，ACC-51E 用于 UMAC
- 应用于 sin/cos 增量编码器 或干涉仪
- 支持 2—4 路（选项）编码器输入
- 转换表 ($F 方式) 计算细分插值后的位置
- 4096 细分数/线 (LSB=1/32 software count)
- 最大线输入频率 1MHz

绝对编码器接口

	Product (s)	Power On Interface	Ongoing Interface
Yaskawa Sigma I	ACC-8D Opt9 ACC-57E OptY	Multiplexer Port UBUS Expansion Port	Encoder Counter Encoder Counter
SSI	ACC-53E	UBUS Expansion Port	UBUS Expansion Port
Stegman Hiperface	ACC-51P Opt2 ACC-51C Opt2	JEXP Expansion Port UBUS Expansion Port	Encoder Interpolator Encoder Interpolator
Heidenhain EnDat	ACC-51C Opt3	C-UBUS Expansion Port	Encoder Interpolator
Mitsubishi	ACC-57E OptM	UBUS Expansion Port	UBUS Expansion Port
Sanyo	ACC-49P	JEXP Expansion Port	JEXP Expansion Port

PMAC2 信号转接板

Board	Drive Interface Signals	Breakout Style	Breakout Connector	Notes
ACC – 8A	Single/Dual Analog	DIN - Rail	D-Sub	- 1 or 2 16 Bit DAC's per axis - Opt. 256x Interpolators
ACC – 8E	Dual Analog	DIN - Rail	Terminal Block 12-point	- 2 18-bit DAC's per axis - Relays for amp enable
ACC – 8F	Direct PWM & Digital Current	DIN - Rail	Mini - D 36-pin	- Loss-of-encoder circuit - Opt. Yaskawa Hall Decode
ACC – 8S	Pulse & Direction	DIN - Rail	Term Block 10-point	Optional encoder interface; "Pulse swallower" for slow drives
ACC – 8FP	Direct PWM & Digital Current	Panel	Mini - D 36-pin	- Used in PC-Pack - Loss-of-encoder circuit - Opt. Yaskawa Hall decode
ACC – 8FE	Direct PWM & Digital Current	3U - Rack	Mini - D 36-pin	- Used in MACRO Station - Opt. 256x interpolator

ACC-12 系列显示器件

- 应用于独立的操作界面
- 应用于软件调试
- 液晶屏 (LCD)
- 高亮、宽视角真空管 (VFD):
- 显示文本 “{message}”
- 显示数值 {variable}
- 直连距离 3m (10ft)
- 最大距离200m附加 ACC-12D

ACC - 12系列

Display	Rows	Characters Per Row	Character Height	Type
ACC – 12	2	24	5 mm	液晶LCD
ACC – 12A	2	40	5 mm	LCD
ACC – 12C1	2	40	6 mm	真空管

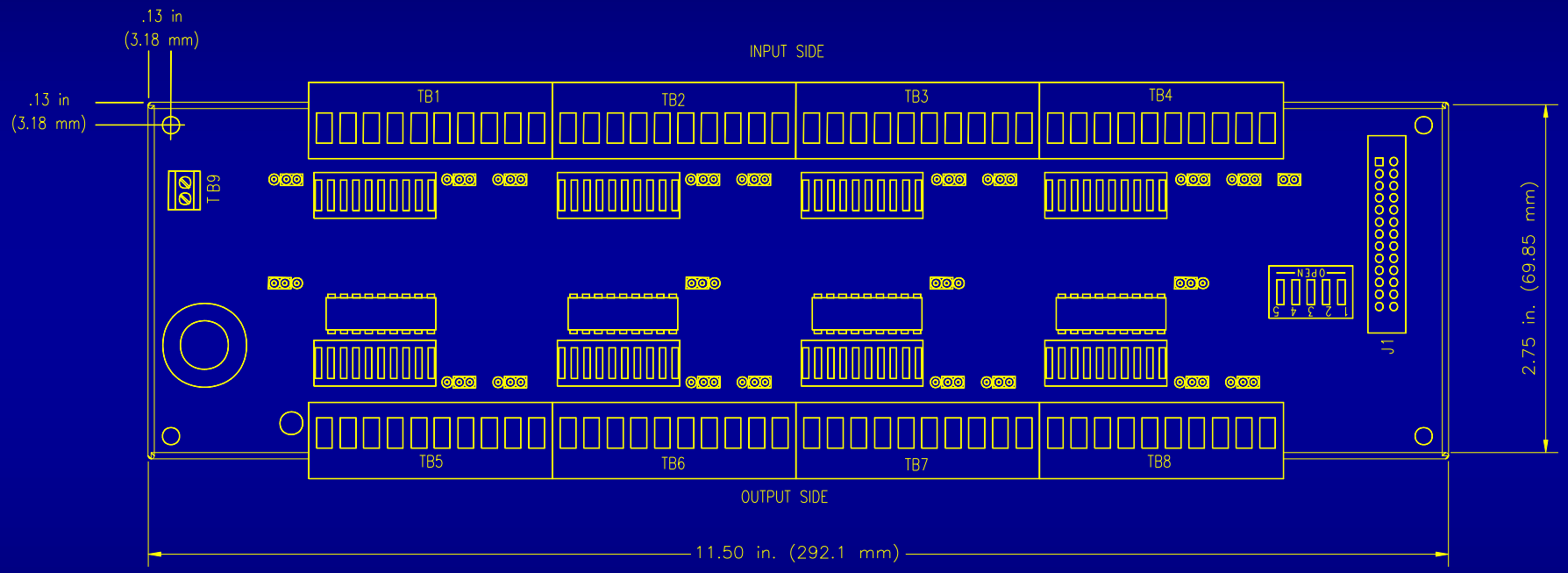
ACC - 34 系列I/O板

Board	Mounting	Voltage Rating	Output Current	Individual Isolation	Input RC Filter	Serial Parity
ACC – 34A	DIN – Rail	24 V	100 mA	No	No	No
ACC – 34AA	DIN – Rail	24 V	100 mA	Yes	Yes	Yes**
ACC – 34B	DIN – Rail	5 V	20 mA	No*	No*	No
ACC – 34D	Panel	5 V	20 mA	No*	No*	Yes**

* This function usually provided by mating Opto - 22 or compatible board.

** Requires V1.16 or newer firmware.

ACC-34A 64 点光电隔离 I/O 板



= Note: Shaded LED's are always lit when power is supplied to pin #10 of the respective terminal block. Other LED's connected to individual I/O pins.

U17–20 – UDN2981 for sourcing outputs
U17–20 – ULN2803 for sinking outputs

PMAC/PMAC2 I/O 点选择

Type	Direction	Access	Typical Latency	Notes
Position Capture Flags	Input	Hardware	50-500 nsec	Latch counted actual position
Position Compare Flags	Output	Hardware	50-500 nsec	Based on counted actual position
Onboard I/O Ports	In/Out	Foreground/ Background Software	200 μsec - 2 msec	Speed limited by software rates
ACC-14 Family I/O	In/Out	Foreground/ Background Software	200 μsec - 2 msec	Opt. 3 inputs can be latched by servo clock
ACC-34 Family I/O	In/Out	Background Software	5 - 20 msec	Serial communications to PMAC

Working With ACC-34 Family I/O

- Only background access permitted (PLC 1-31, on-line)
- Foreground access disabled (PLC0, motion program)
- 100 microseconds to copy 32-bit word to/from ACC-34 (20MHz)
- Suggested approach is like real PLC's:
  - At beginning of PLC scan, copy all input words into fixed-point registers in internal memory
  - Work with individual bits of internal registers (Use single-bit M-variables)
  - Prepare output words in fixed-point registers in internal memory
  - At end of PLC scan, copy all output words from internal registers



ACC-34 Family M-Variable Definition

n = Base address of board on multiplexer port
as set by 5-point DIP switch on board
(n=0 - 248, divisible by 8)

Input Port (A): Mx -> TWS:{n + 1}
Output Port (B): Mx -> TWS:{n + 6}

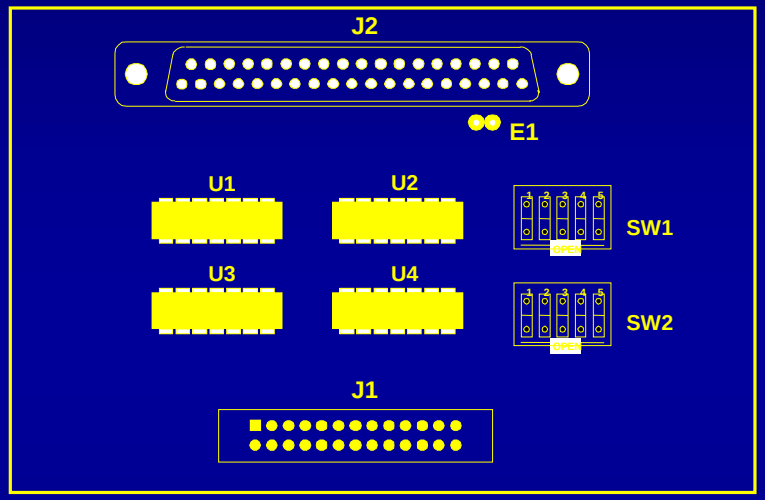
Example: n=0
M901->TWS:1
M902->TWS:6

M1001=M901 ; copy inputs into image
• • •
M902=M1002 ; copy image to outputs

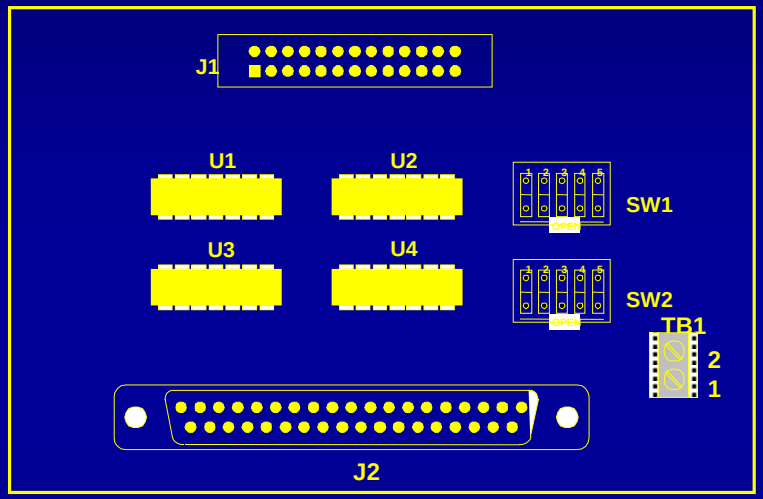


ACC-35 Multiplexer Port Extenders

ACC-35A Option 1
The DB37S Connector Option



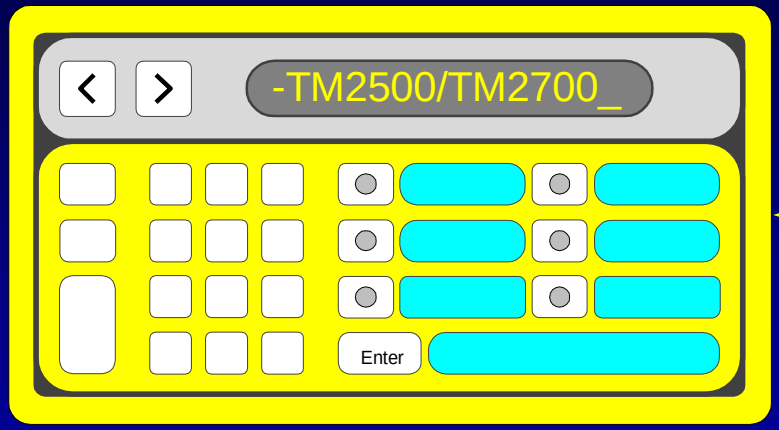
ACC-35B Option 1
The DB37S Connector Option



For use with:

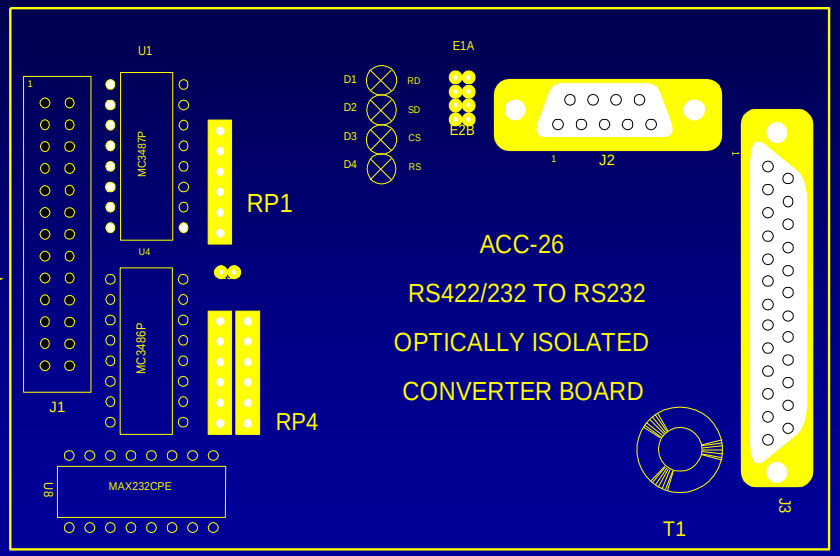
- ACC-34 family of I/O input
- ACC-18 thumbwheel input
- ACC-8D Opt 7 R/D converter
- ACC-8D Opt 9 Yaskawa absolute encoder board

ACC-20

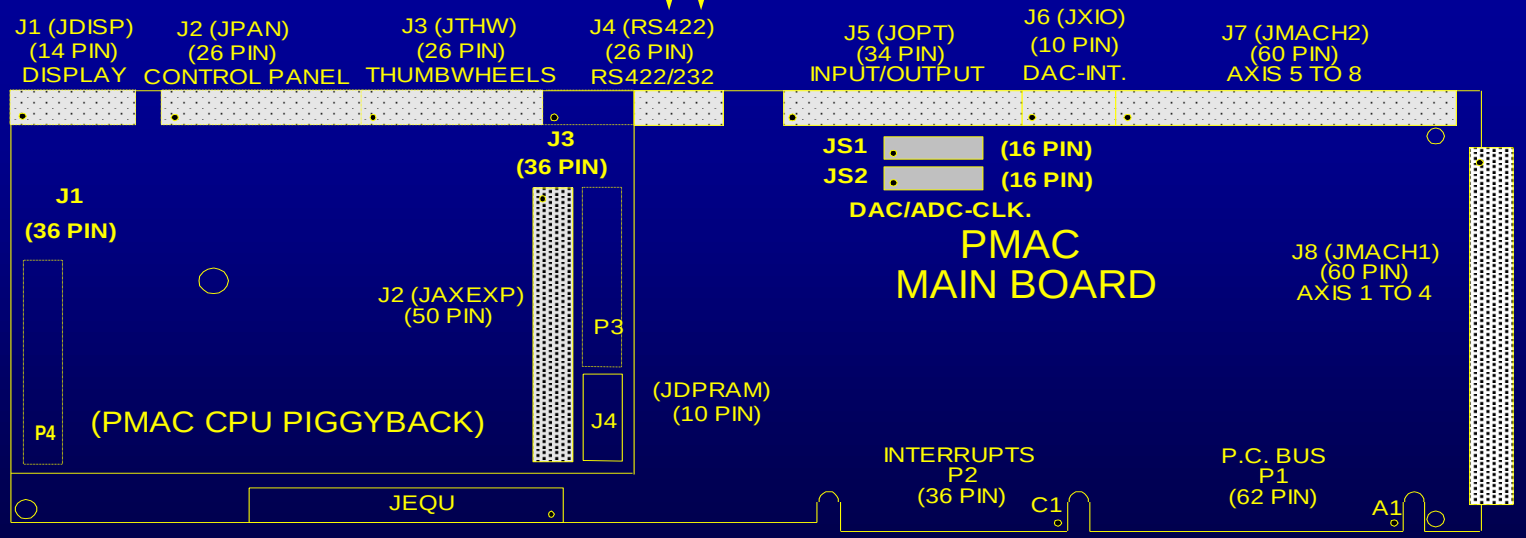


Control Panel
Numeric Input
24 Char. Display
6 Function Keys

ACC-26

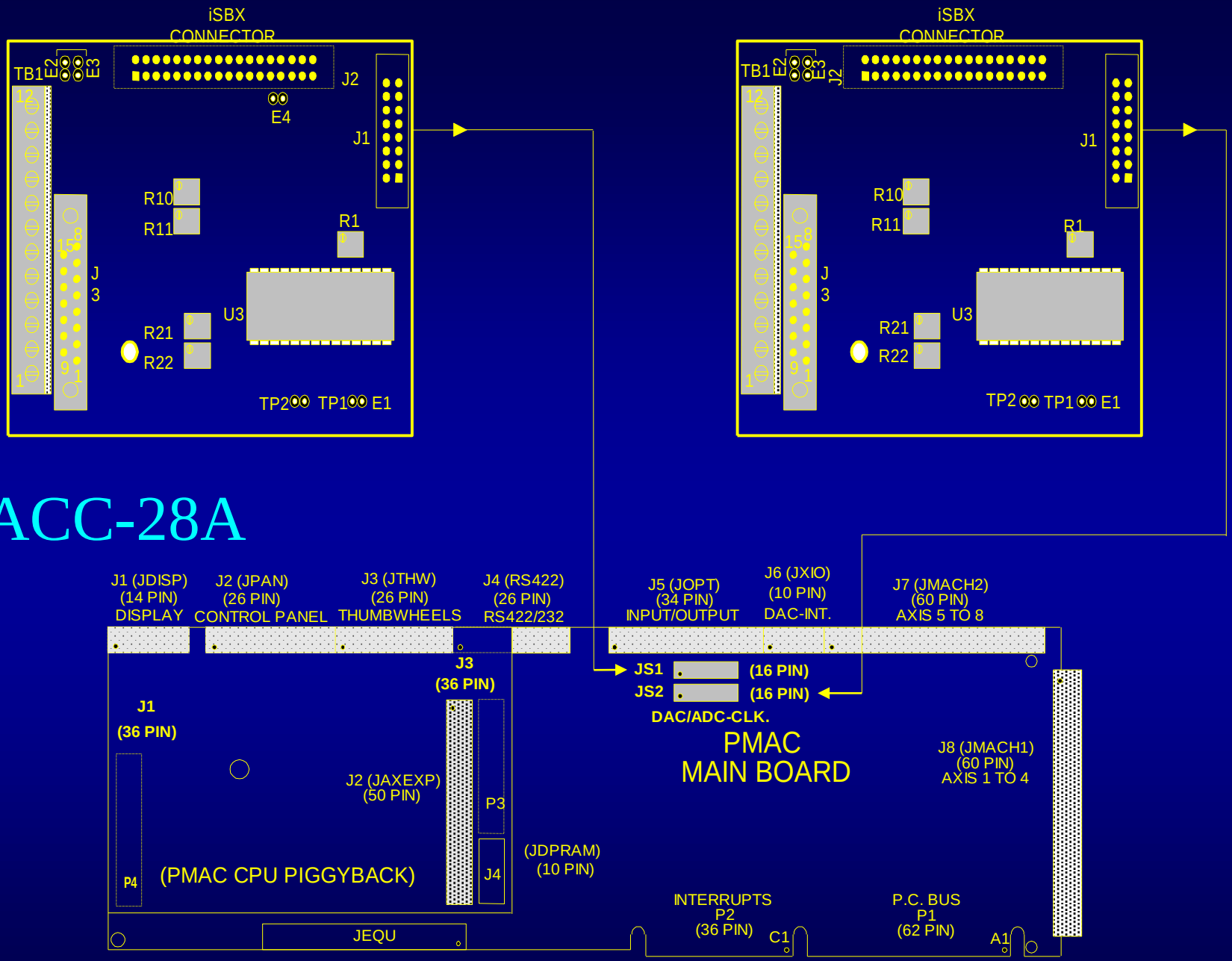


Provides Optical Isolation for RS422/232



PMAC/PMAC2 A/D Converter Choices

Type	# of Channels	Resolution (bits)	Repeatability (bits)	Voltage Range	Mounting	Optical Isolation?	PMAC Family Compatibility
ACC – 28A	4	15	12	±10V	DIN Rail	Yes	1
ACC – 28B	2, 4	16	15	±10V	DIN Rail	Yes	1, 2
ACC – 36P	16	12	11	0-10V, ±5V	ISA Bus	No	1, 2
ACC – 36V	16	12	11	0-10V, ±5V	VME Bus	No	1, 2
Opt. 12, 12A	8/16	12	11	0-5V, ±2.5V	Onboard Option	No	2
Opt. 15 (V/F)	1 (2 on Mini)	11	10	0-10V (±10V on Lite)	Onboard Option	No (Yes on Mini)	1


iSBX
CONNECTOR
TB1
J2
E4
J1
R10
R11
R1
J
3
U3
R21
R22
TP2
TP1
E1
iSBX
CONNECTOR
TB1
J2
J1
R10
R11
R1
J
3
U3
R21
R22
TP2
TP1
E1
ACC-28A
J1 (JDISP)
(14 PIN)
DISPLAY
J2 (JPAN)
(26 PIN)
CONTROL PANEL
J3 (JTHW)
(26 PIN)
THUMBWHEELS
J4 (RS422)
(26 PIN)
RS422/232
J5 (JOPT)
(34 PIN)
INPUT/OUTPUT
J6 (JXIO)
(10 PIN)
DAC-INT.
J7 (JMACH2)
(60 PIN)
AXIS 5 TO 8
J3
(36 PIN)
JS1
(16 PIN)
JS2
(16 PIN)
DAC/ADC-CLK.
J1
(36 PIN)
PMAC
MAIN BOARD
J8 (JMACH1)
(60 PIN)
AXIS 1 TO 4
J2 (JAXEXP)
(50 PIN)
P3
P4
(PMAC CPU PIGGYBACK)
J4
(JDPRAM)
(10 PIN)
JEQU
INTERRUPTS
P2
(36 PIN)
C1
P.C. BUS
P1
(62 PIN)
A1

3U Stack Boards

- ACC-1E 2-Axis Servo Interface Board*
- ACC-2E 4-Axis Servo Interface Board
- ACC-3E 48/96/144 TTL I/O Board
- ACC-4E 24-In/24-Out Isolated I/O Board
- ACC-6E 16-Channel 12-Bit ADC Board

* For use with MACRO Interface / CPU Board only

UMAC Servo IC Boards

	Output	Feedback	Flags
ACC-24E2	3 Phase PWM	AB Quad	Isolated Inputs Diff Enable/Fault
ACC-24E2A	Dual Analog Pulse & Direction	AB Quad	Isolated Inputs Relay Amp Enable Isolated Amp Fault
ACC-24E2S	Pulse & Direction	AB Quad	Isolated Inputs Diff Enable/Fault
ACC-51E	None	Sine/Cosine Optional Hiperface	One Trigger Flag

UMAC Digital I/O Boards

	# of Inputs	Input Voltage	# of Outputs	Output Voltage	Output Current
ACC-9E	48	12-24VDC	0	---------	---------
ACC-10E	0	----------	48	5-24VDC	+100mA (sourcing) -100mA (sinking)
ACC-11E	24	12-24VDC	24	5-24VDC	+100mA (sourcing) -100mA (sinking)
ACC-12E	24	12-24VDC	24	5-60VDC <=240VAC	+/-1A
ACC-14E	0-48	5V	0-48	5V	10mA

UMAC Analog I/O IC Boards

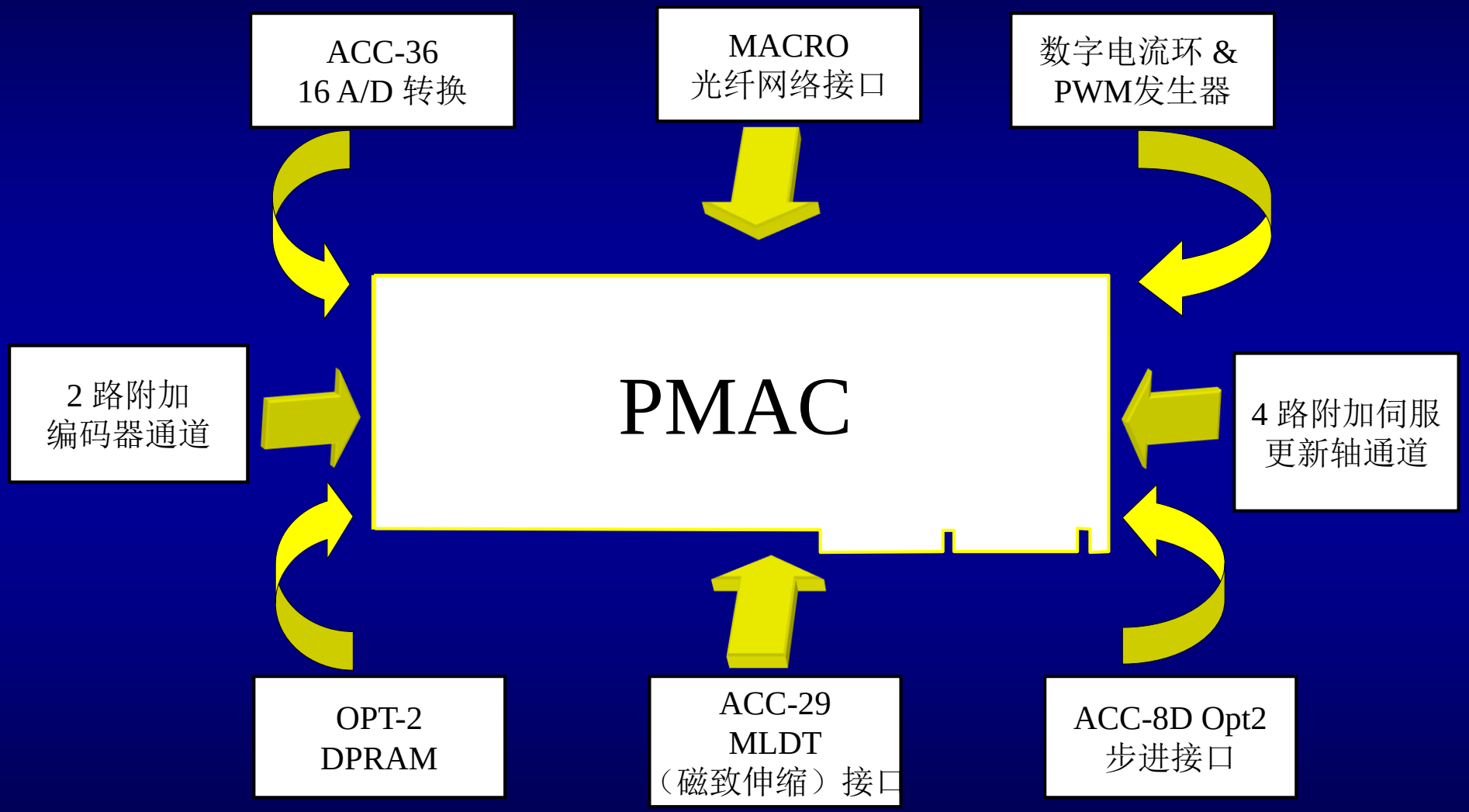
	# of ADCs	Resolution	Input Range	# of DACs	Resolution	Output Range
ACC-28E	2/4	16-bit	+/-10V	0	---------	---------
ACC-36E	16	12-bit	0-10V +/-5V	0	---------	---------
ACC-59E	8	12-bit	0-10V +/-5V	8	12-bit	+/-10V

"JEXP" Expansion Port Boards

Board	Function	Form Factor
ACC-14D	48 digital I/O	ISA
ACC-14V	48 digital I/O	VME
ACC-24P	4 or 8 PMAC(1) servo channels	ISA
ACC-24P2	4 or 8 PMAC2 servo channels	ISA
ACC-24V	4 or 8 PMAC(1) servo channels	VME
ACC-36P	16 12-bit ADCs	ISA
ACC-36V	16 12-bit ADCs	VME
ACC-42P2	MACRO ring interface	PCI
ACC-51P	2 or 4 4096x encoder interpolators	PCI
OPT 2	PMAC(1)-PC dual-ported RAM	ISA



PMAC2 Integration



Why Use PMAC2 over PMAC(1)?

- Ability to do direct PWM control
- Ability to do on-board MACRO interface
- More compact for sinewave control
- More compact for analog encoder interpolation
- Superior pulse-and-direction stepper control
- Superior MLDT interface
- Enhanced position compare output function
- Increased DAC resolution (18-bit vs. 16-bit)
- Many accessory boards come on-board:
  - DPRAM, ADCs, MLDT interface, supplemental encoder interface, stepper interface



Why Use PMAC(1) over PMAC2?

- Existing PMAC customers don't have to change
- More cost-effective for traditional analog drive interface:
  - Velocity-mode drives
  - Torque-mode drives
- Onboard 24V I/O (JOPTO port)
- Automatic control panel port



PMAC Command Set

- On-line commands

  (executed immediately, not stored)

  – Global

  – Coordinate system specific (&n…)

  – Motor specific (#n…)

  (Note: Motor-specific action commands locked out when motor is in a coordinate system running a motion program)

  – Mode setting, move commands, queries, variable setting, buffer control



PMAC Command Set (continued)

- Buffered commands (into OPEN buffer)

  (stored for future execution)

  – Motion Program

    Moves, modes, math, logic, commands, messages

  – PLC Program

    Math, Logic, commands, messages



PMAC Variables

- I-Variables (1024) [8192]
  - **Initialization and setup variables**
  - **Pre-defined meanings**
  - **Some affect whole card**
  - **Some are motor specific**
  - **Some are coordinate system specific**
  - **Some are hardware-channel specific**
- P-Variables (1024) [8192]
  - **General purpose user variables**
  - **48-bit floating point format**
  - **Global (whole card) access**



PMAC Variables (continued)

- Q-Variables (1024) [8192]
  - **General Purpose user variables**
  - **48-bit floating point format**
  - **Specific to a coordinate system**
- M-Variables (1024) [8192]
  - **Provide user access to memory and I/O**
  - **User defines address, offset, and bit width**
  - **Unsigned, 2's comp, BCD, floating point formats available**



P & Q Variable Array Capabilities

- Reading:

```
P0=P(P100+3)
Q99=Q(Q0+P10)
```

- Writing (non-Turbo) :

Setup:

```
M0-> L:$1000              ;Points to P0
M10-> Y:$BC00, 0, 12;Points to M0 definition
```

Action:

```
M10=P100+3                ;select P(P100+3)
M0=P0                     ;Same as P(P100+3)=P0
```



P & Q Variable Array Capabilities (continued)

Setup:

```
M1-> L:$1400              ;Points to C.S.1 Q0
M11-> Y:$BC01, 0, 12;Points to M1 definition
```

Action:

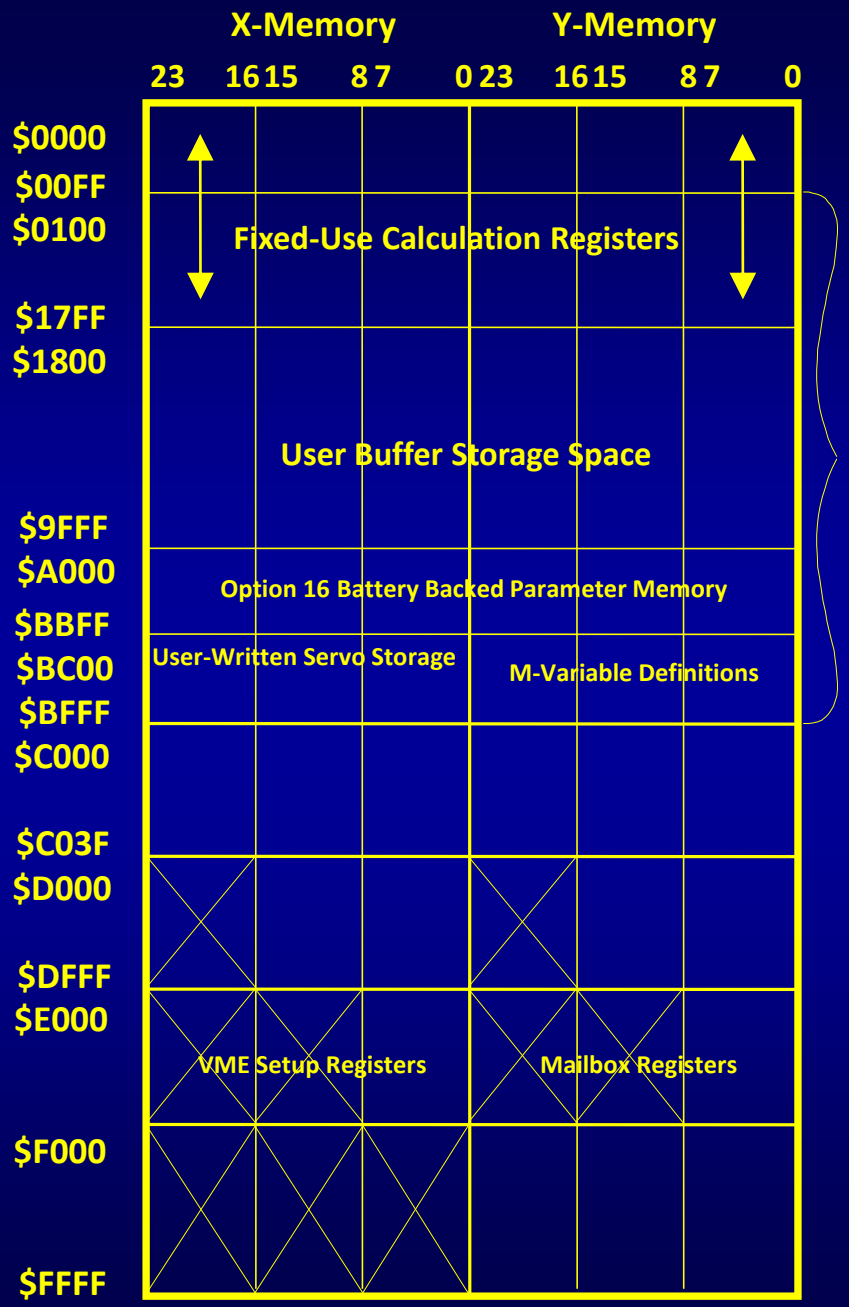
```
M11=1024+Q0+P10  ;select C.S.1 Q(Q0+P10)
M1=Q99                    ;Same as Q(Q0+P10) =Q99
```

Writing (Turbo):

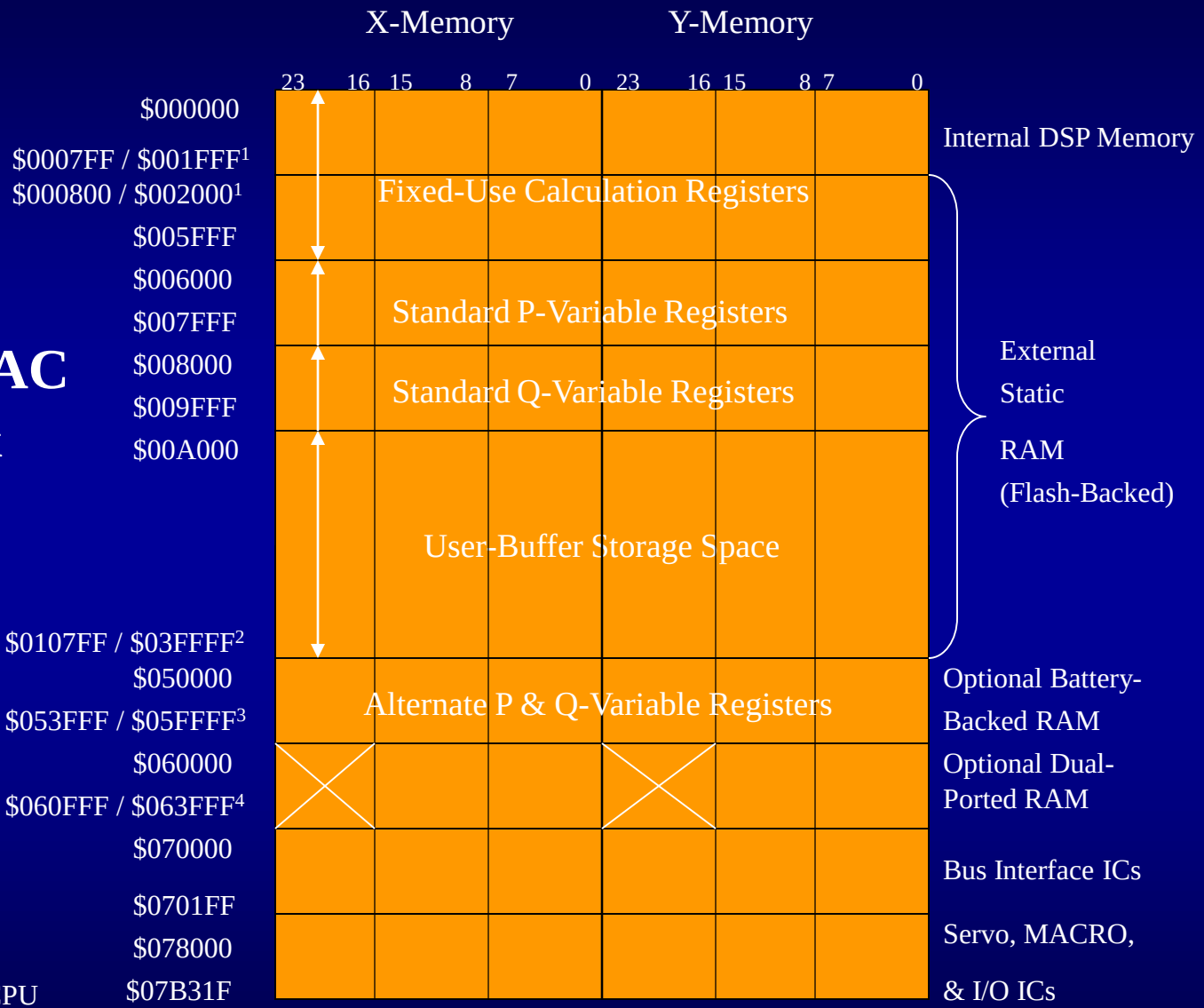
```
P(P100+3)=P0
Q(Q0 + P10)=Q99
```



PMAC Memory & I/O Map



Turbo PMAC Memory & I/O Map



Notes:
1. 56303 CPU / 56309 CPU
2. Standard / Extended Data Memory
3. Standard / Extended BBRAM Option
4. Standard / Extended DPRAM Option

Accessing PMAC Registers

- **Servo and Commutation Use**
  - **I-Variables & Conversion Table**
  - **Ix02, Ix03, Ix04, Ix05, Ix10, Ix25 - servo addresses**
  - **Ix81, Ix82, Ix83 - Commutation addresses**
  - **Ix93 - Time base addresses**
  - **I21 through I44 - Data gathering addresses**
  - **Conversion table setup word - Preprocessing**
- **User Program Use (Motion, PLC, & Host)**
  - **M-Variables**
  - **Assigned to registers first**
  - **Used as normal variables in PMAC programs, on-line commands from host**
  - **Direct Memory read/write (Host use only)**
  - **W{address}, {constant}**
  - **R[H] {address}**

PMAC I-Variable Groups

•I0 - I99	General card setup
•I100 - I186	Motor 1 setup
•I187 - I199	Coordinate System 1 setup
•I200 - I286	Motor 2 setup
•I287 - I299	Coordinate System 2 setup
...	…
•I800 - I886	Motor 8 setup
•I887 - I899	Coordinate System 8 setup
•I900 - I999	Hardware setup
•I1000 - I1023	MACRO & Misc. setup



Turbo PMAC I-Variable Groups

- I0 - I99 General Card Setup
- I100 - I199 Motor 1 Setup
- …
- I3200 - I3299 Motor 32 Setup
- I3300 - I3349 Motor 1 Supplemental Setup
- …
- I4850 - I4899 Motor 32 Supplemental Setup
- I4900 – I4999 System Configuration Status
- I5000 - I5099 Data Gathering & ADC Demux Control
- I5100 - I5199 Coordinate System 1 Setup
- …
- I6600 - I6699 Coordinate System 16 Setup



Turbo PMAC I-Variable Groups (cont.)

- I6800 - I6849 MACRO IC 0 Hardware Setup
- …
- I6950 - I6999 MACRO IC 3 Hardware Setup
- I7000 - I7099 Servo IC 0 Hardware Setup
- …
- I7900 - I7999 Servo IC 9 Hardware Setup
- I8000 - I8191 Encoder Conversion Table Setup



PMAC(1) Encoder I-Variables

- **I900, I905, ….. I975 – Encoder Decoding Control [I7mn0]**

0: Pulse and Direction CW
1: X1 Quadrature CW
2: X2 Quadrature CW
3: X4 Quadrature CW
4: Pulse and Direction CCW
5: X1 Quadrature CCW
6: X2 Quadrature CCW
7: X4 Quadrature CCW
8 – 15: Timer up/down

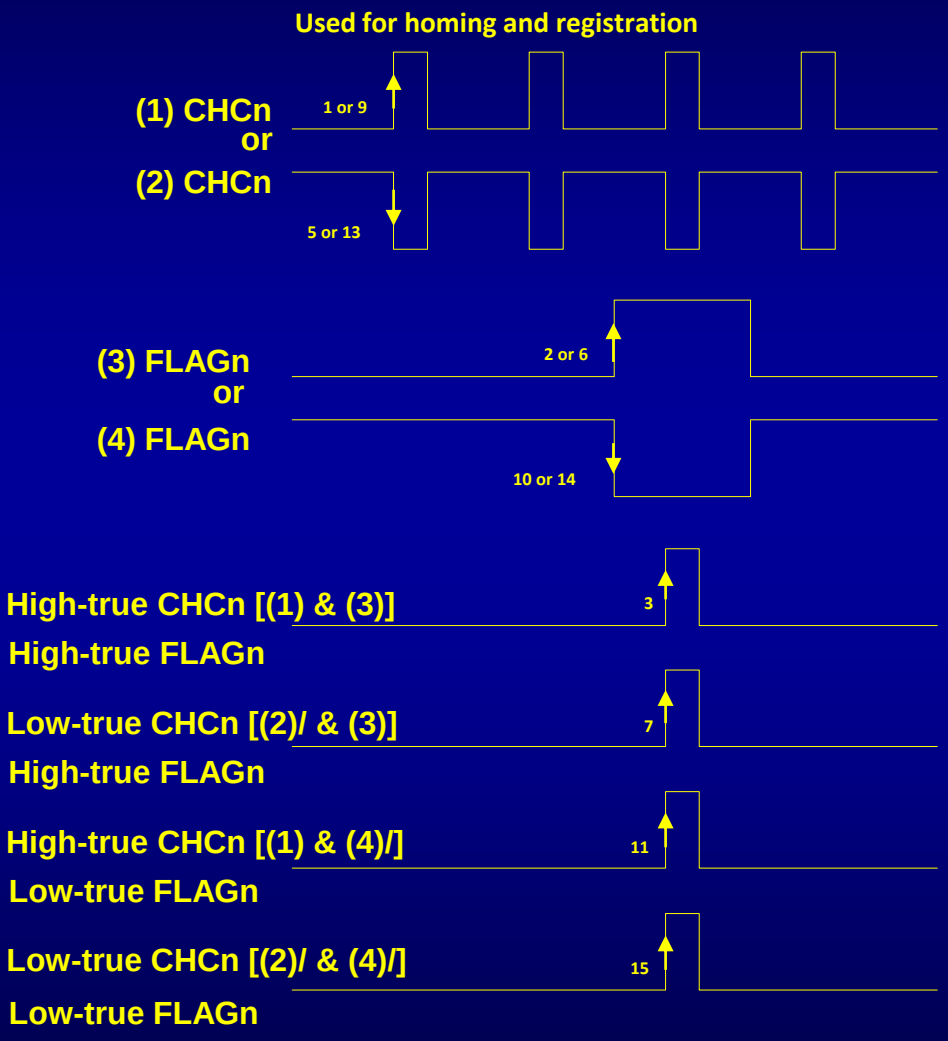
- **I901, I906, ….. I976 – Encoder Delay Filter Disable [I7mn1]**

0: Filter enabled, flags transparent
1: Filter disabled, flags latched

Disable for parallel subcount interpolation on both encoder used and encoder whose flags are used for sub-count data

PMAC(1) Encoder I-Variables (continued)

I902, I907, ..I977: Encoder Position Capture Control [I7mn2]



PMAC(1) Encoder I-Variable (continued)

•I903, I908, ….. I978
[I7mn3] Flag Select Control

Determines which FLAGn is used for position capture

0: HMFLn
1: -LIMn*
2: +LIMn*
3: FAULTn*

***must disable normal function with Ix25 [Ixx24]**



PMAC2 Global Hardware Control I-Variables

I900: **Max Phase and PWM 1-4 Frequency Control**
[I7m00]
Max Phase (kHz)=117,968/(2*I900+3)
PWM (kHz)=117,968/(4*I900+6)

I901: **Phase Clock Frequency Control**
[I7m01]
Phase(kHz)=Max Phase/(I901+1)

I902: **Servo Clock Frequency Control**
[I7m02]
Servo (kHz)=Phase/(I902+1)

I903: **Hardware Clocks 1-4 Frequency Control Encoder**
[I7m03]
SCLK, DAC_CLK, ADC_CLK, PFM_CLK

PMAC2 Global Hardware Control I-Variables (cont.)

I904: **PWM 1-4 Deadtime/PFM 1-4 Pulse Width Control**
[I7m04] **Dead Time (μsec)=0.135*I904**
Pulse Width (μsec)=PFM_CLK period*I904

I905: **DAC 1-4 Strobe word (=$7FFFC0 for ACC-8E)**
[I7m00]

I906: **PWM 5-8 Frequency Control (see I900)**
[I7m03]

I907: **Hardware Clocks 5-8 Frequency Control (see I903)**

I908: **PWM 5-8 Deadtime/PFM 5-8 Pulse Width Control**
[I7m04] **(see I904)**

I909: **DAC 5-8 Strobe Word (see I905)**
[I7m05]

PMAC2 Channel n Hardware Control I-Variables

(Non-Turbo: n = 1 to 8)

[Turbo: m = 0 to 9, n = 1 to 4]

I9n0: **[I7mn0]** **Encoder n Decode Control**

0: Pulse & Direction CW	4: Pulse & Direction CCW
1: x1 Quadrature CW	5: x1 Quadrature CCW
2: x2 Quadrature CW	6: x2 Quadrature CCW
3: x4 Quadrature CW	7: x4 Quadrature CCW
8:Internal Pulse & Dir.	12: MLDT Pulse Timing
11: x6 Hall CW	15: x6 Hall CCW

I9n1: **[I7mn1]** **Position Compare n Channel Select**

0: Use Encoder n

1: Use 1st encoder on IC

I9n2: **[I7mn2]** **Encoder n Capture Control**

As on PMAC(1)

PMAC2 Channel n Hardware Control I-Variables (cont.)

I9n3: [I7mn3] Encoder n Capture Flag Select

0: HMFLn
1: PLIMn
2: MLIMn
3: USERn

I9n4: [I7mn4] Encoder n Gated Index Select

0: Use ungated index for capture
1: Use gated index (one quad state wide) for capture

I9n5: [I7mn5] Encoder n Index Gate State

0: Gate index with "High-High" quad state
1: Gate index with "Low-Low" quad state



PMAC2 Channel n Hardware Control I-Variables (cont.)

I9n6: [I7mn6] Output n Mode Select

0: A&B - PWM; C - PWM
1: A&B - DAC; C - PWM
2: A&B - PWM; C - PFM
3: A&B - DAC; C - PFM

I9n7: [I7mn7] Output n Invert Control

0: A&B - Not Inverted; C - Not Inverted
1: A&B - Inverted; C - Not Inverted
2: A&B - Not Inverted; C - Inverted
3: A&B - Inverted; C - Inverted

I9n8: [I7mn8] Output n PFM Direction Control

0: Direction output not inverted
1: Direction output inverted

PMAC Motor I-Variable Groups

- **Ix00 - Ix10 Motor x Definition (also Ix25)**
- **Ix11 - Ix19 Motor x Safety variables**
- **Ix20 - Ix29 Motor x Motion variables**
- **Ix30 - Ix35 Motor x PID Gains**
- **Ix36 - Ix39 Motor x Notch Filter**
- **Ix40 - Ix59 Motor x Pole Placement (Option 6 required)**
- **Ix60 - Ix69 Motor x Servo loop Modifiers**
- **Ix70 - Ix84 Motor x Commutation Parameters (only used if Ix01=1)**
- **Ix85 - Ix86 Motor x Backlash**

Basic Motor Definition I-Variables

Ix00 Motor x Activation

Ix00=0 Motor x Deactivated
No servo calculations done
No position reported

Ix00=1 Motor x Activated
Servo calculations can be performed
Not necessarily enabled



Basic Motor Definition I-Variables (continued)

Ix01 Motor x Commutation

Ix01=0 Not commutated by PMAC
No phasing calculations done
One output for the motor

Ix01=1 Commutation performed by PMAC
Phasing calculations done each phase update.
2 analog or 3 PWM outputs used
Ix70-Ix84 must be used



PMAC Address I-Variables (Non-Turbo)

- Low 16 bits (4 hex digits) specify the address
- When high 8 bits are zero, address is used in normal mode
- When one or more high bit=1, address used in different mode
- I9=2 or 3: PMAC reports variable value in hex

Hex ($)	0				1				C				0				0				3			
Binary	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1

- Applies to
  I21-I44
  Ix02,Ix03,Ix04,Ix05,Ix10,Ix25,
  Ix81, Ix82, Ix83, Ix93

Motor Address I-Variables
PMAC vs. Turbo PMAC

- Ix02 Command Output Address & Mode
- Ix03 Position-Loop Feedback Address & Mode
- Ix05 Master Position Address & Mode
- Ix06 Master Following Enable
- Ix10 Absolute Servo Position Address & Mode
- Ix25 Flag Address & Modes
- Ix81 Absolute Phase Position Address & Mode

- Ixx02 Command Output Address
  Ixx96 Command Output Mode
- Ixx03 Position Feedback Address
  Ixx97 Position Capture Mode
- Ixx05 Master Position Address
- Ixx06 Master Following Enable & Mode
- Ixx10 Absolute Position Address
  Ixx95 Absolute Position Format
- Ixx25 Flag Address
  Ixx24 Flag Modes
- Ixx81 Absolute Phase Position Address
  Ixx91 Absolute Phase Position Format

Motor Address I-Variables

Ix02 Command Output Address

Bits 0-15 specify address of first command register (of one, two or three) usually DAC, PWM, PFM, or MACRO output registers

Bit 16=1 specifies unipolar output, with sign on AENA/DIRn (if Ix01=0)

Ixx96 Bit 0

Bit 16=1 specifies direct microstepping control (if Ix01=1)



Motor Address I-Variables (continued)

Ix03 Position Loop Feedback Address

Bits 0-15 specify (X) register for position loop feedback (usually in conversion table)

Bit 16=1 specifies software position capture on trigger (not hardware capture) [Ixx97 bit 0]

Bit 17=1 specifies trigger on warning following error (not capture trigger) [Ixx97 bit 1]



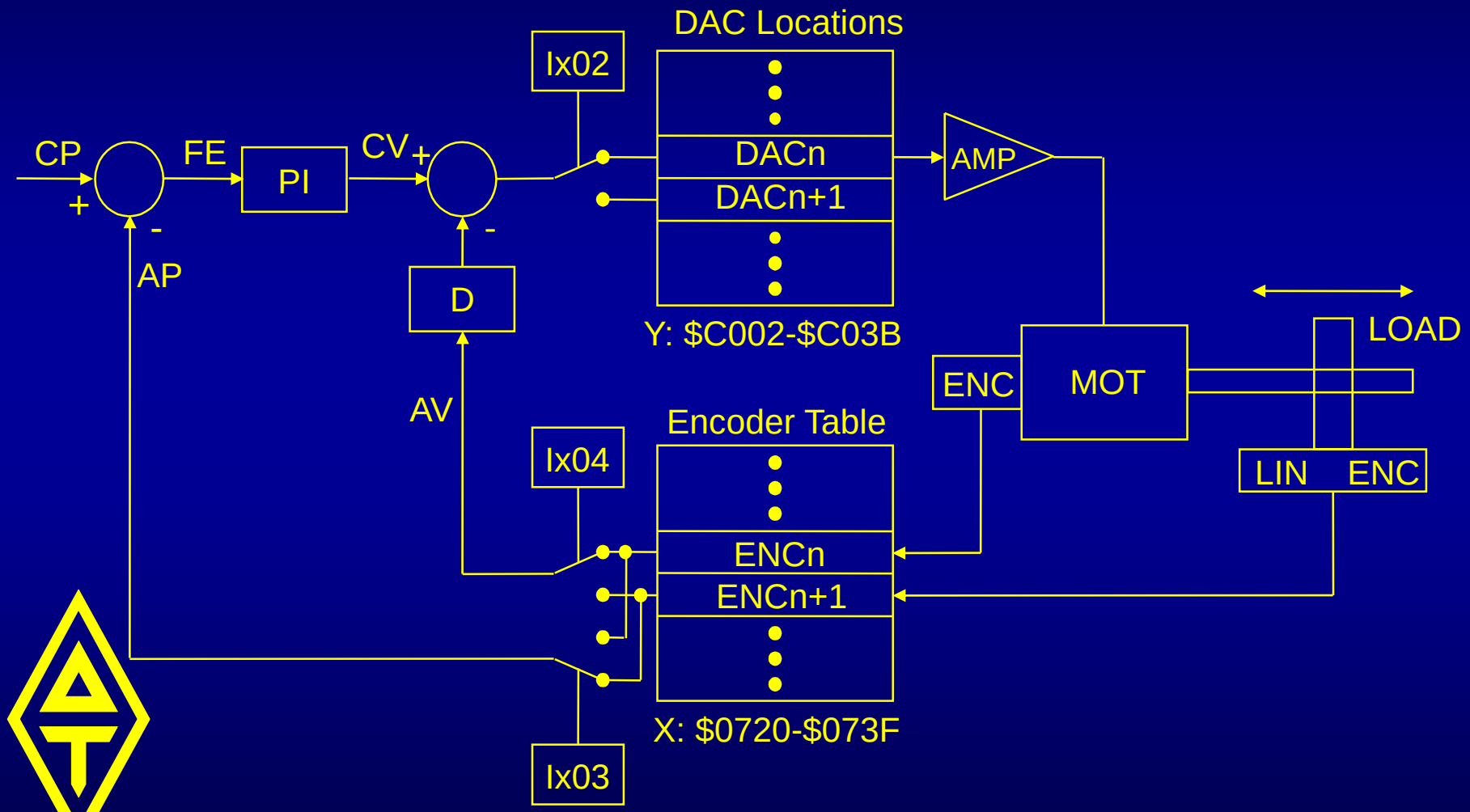
Address I-Variables (continued)

Ix04 Velocity Loop Feedback Address

Bits 0-15 specify (X) register for velocity loop feedback
(usually in conversion table, register must contain position data)



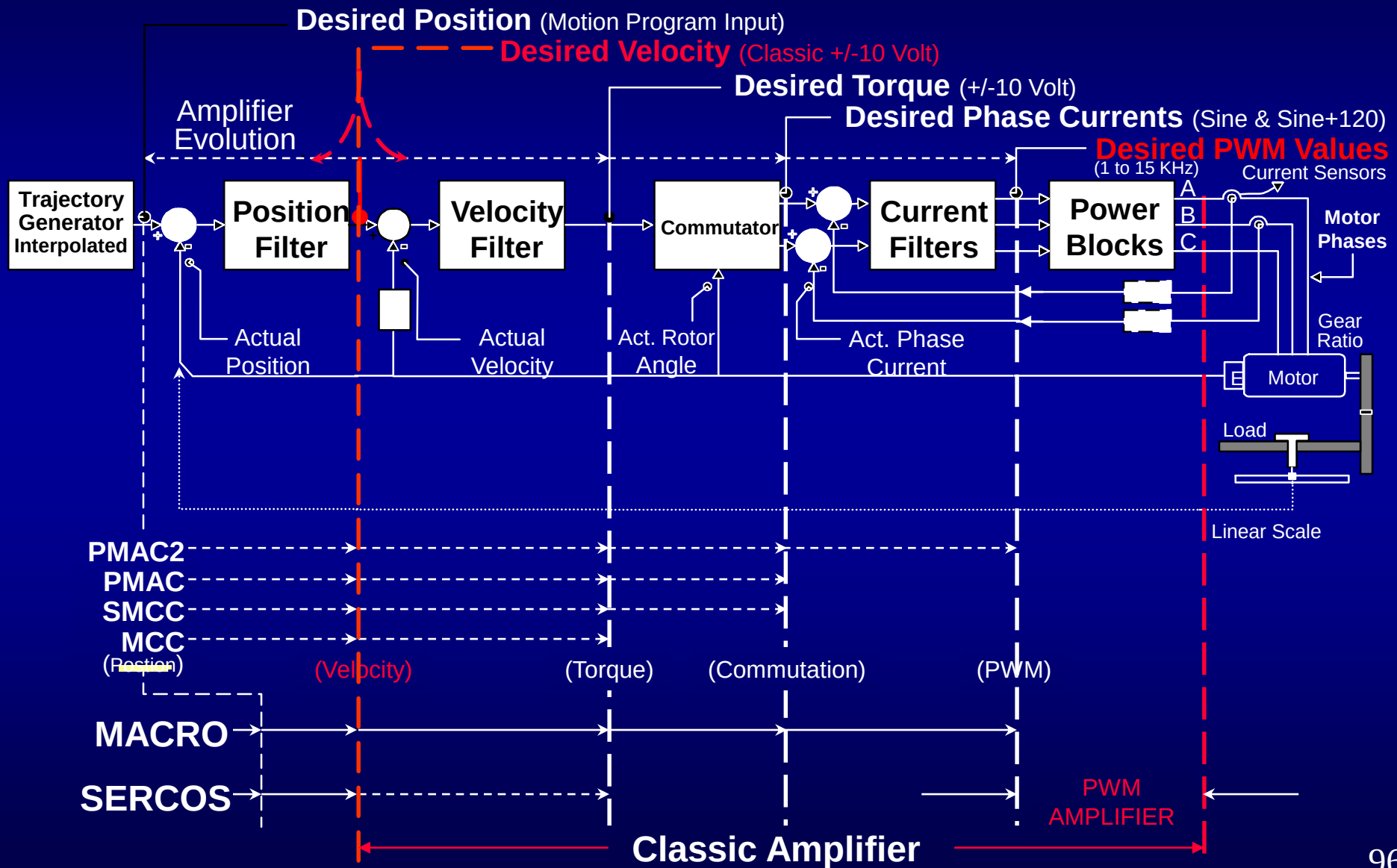
Ix02 COMMAND output address
Ix03 POSITION loop feedback address
Ix04 VELOCITY loop feedback address



DELTA TAU
Data Systems, Inc.

CLOSED LOOP SERVO

(Evolution of the Controller Amplifier Interface)



速度模式放大器

- 放大器上闭合速度环和电流环
- 由电机换向 (brush DC)或放大器换相(brushless)
- 速度环参数单独设置
  - 必须由机器制造人员设定
- 替代模拟速度环的原因在于
  - 模拟速度环的增益不能由数字量控制
  - 模拟速度环的增益不能轻易改变



转矩模式放大器

- 放大器上闭合电流环
- 电机换向 (brush DC)或放大器换相(brushless)
- 速度环在PMAC上闭合
  - 由数字量控制
- Amplifier tasks not load-dependent
  - Can be set by amp manufacturer



什么情况下要用到转速计?

- 非常低速平滑运动时
  (当数字量速度估算不充分时)
- 克服扰动 （e.g. 转矩不平衡, 摩擦）
  (当数字速度环增益值不够时)
- 更高精度
- 当负载上的编码器与负载连接不好时



Address I-Variables (continued)

Ix05 Master (Position) Address

- For position following (electronic gearing)

  Bits 0-15 specify (X) register for master

  Bit 16=1 specifies offset mode following

  Ixx06 bit 1 (reported position does not change with following; can superimpose on programmed moves)

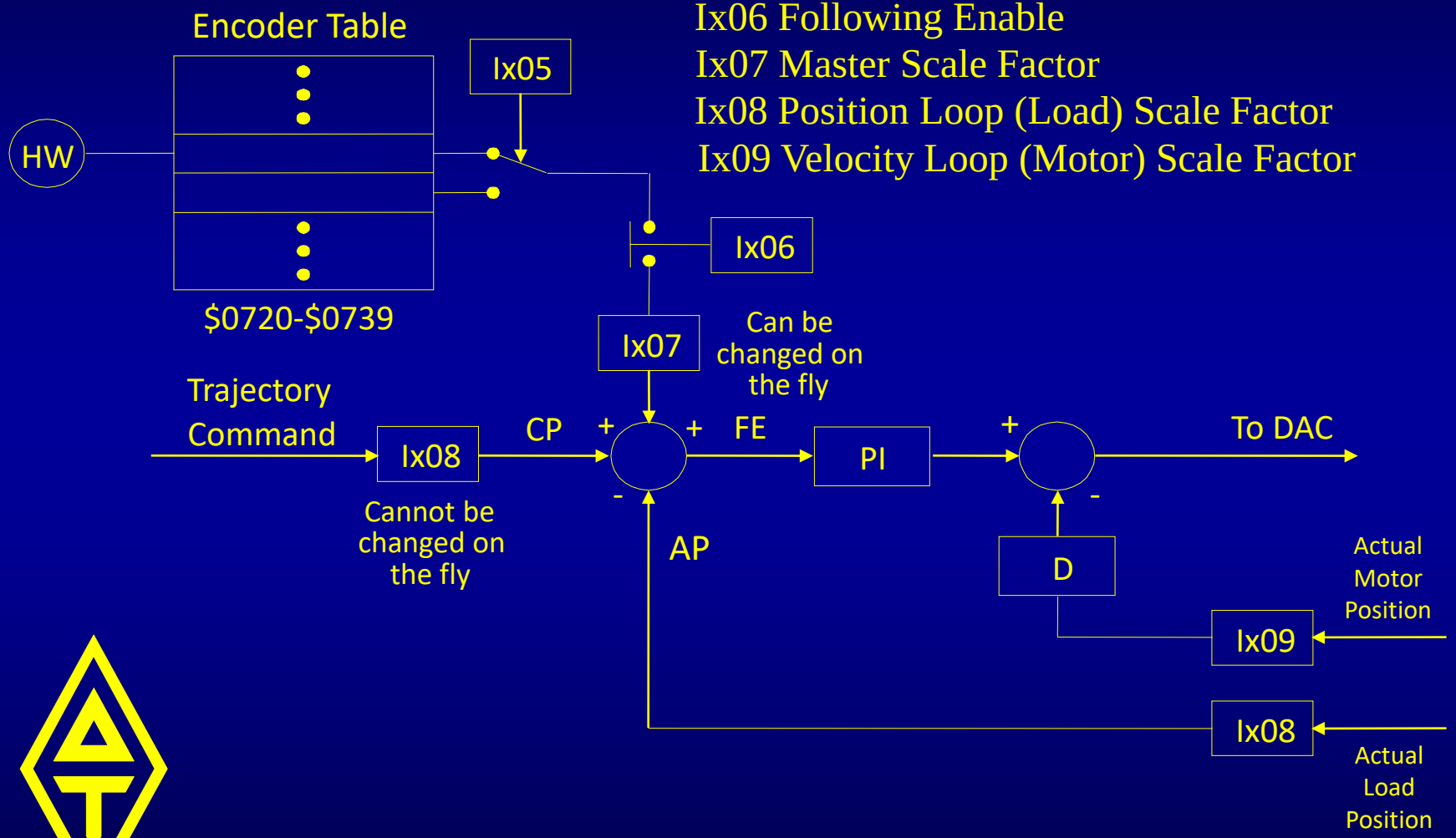
  Bit 16=0 (default) specifies normal mode

  Ixx06 bit 1 (reported position changes with following; next programmed move cancels effect of following)

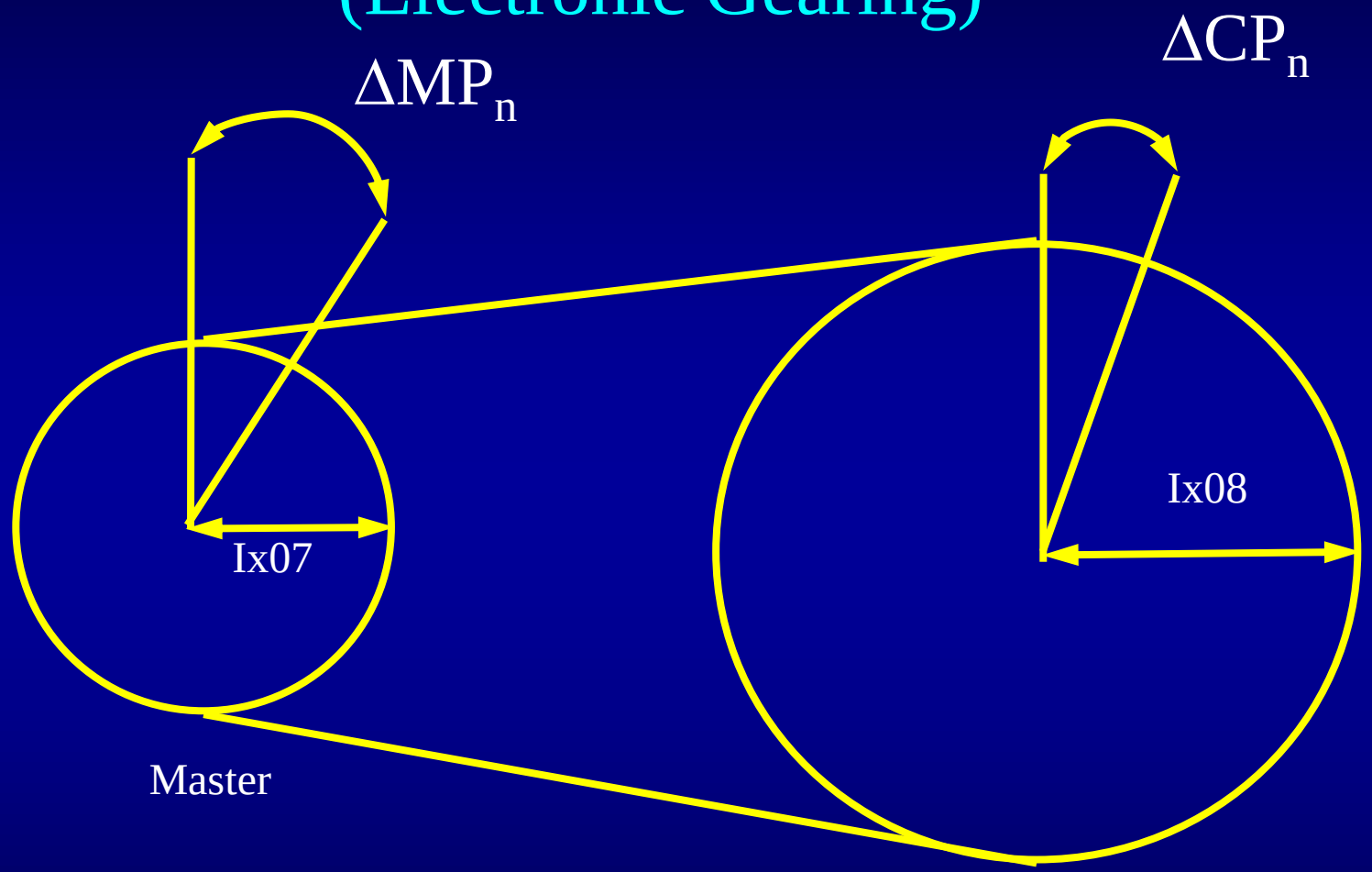


Position Following Parameters

Ix05 Master (HW) Position Address & Mode
Ix06 Following Enable
Ix07 Master Scale Factor
Ix08 Position Loop (Load) Scale Factor
Ix09 Velocity Loop (Motor) Scale Factor



PMAC Position Following (Electronic Gearing)



Mechanical Analogy

PMAC Position Following

Motor Commanded Position } follows { Master Position

$$\Delta CP_n = \frac{Ix07}{Ix08} * \Delta MP_n$$

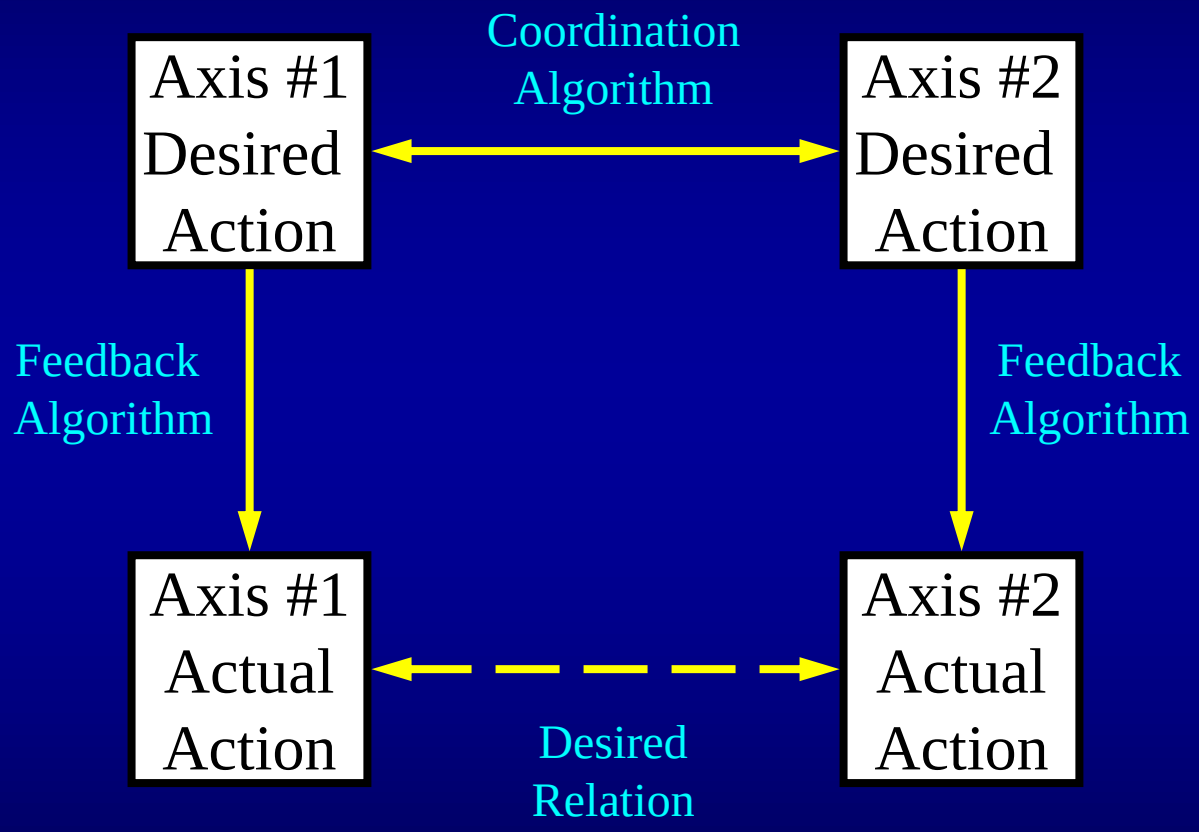
$$Ix08 * \Delta CP_n = Ix07 * \Delta MP_n$$

With 1/T:

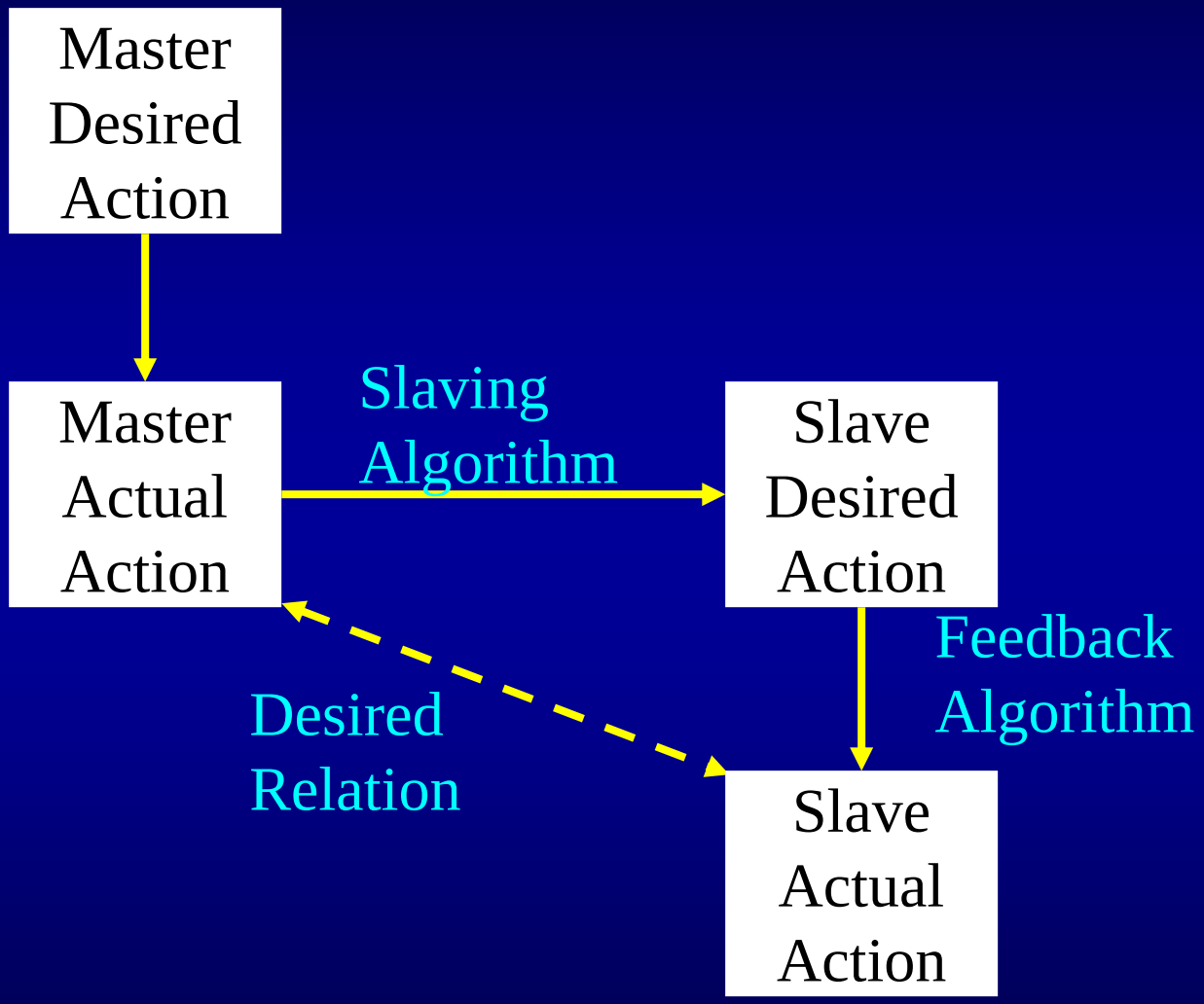
$$32 * Ix08 * \Delta CP_n = 32 * Ix07 * \Delta MP_n$$



Coordination Concept



Master - Slave Concept



Use Coordination when:

- Have good control of all axes
- No axis is predominant in function
  - If you cannot decide which should be master, coordinate!
- Easy to describe simultaneous motion
  - i.e. start, stop, turn together



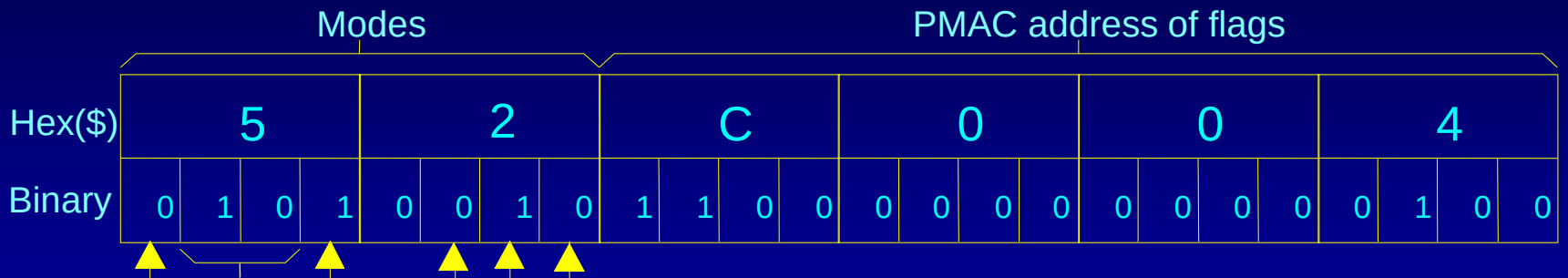
Use Master-Slave when:

- Too expensive for tight control of an axis
  - e.g. high power spindle or conveyor motor
- Too difficult for tight control of an axis
- Writing a master/slave motion program is substantially easier (slaving to commanded position recommended here)
  - Examples: coil winding, thread cutting



Ix25 - Motor x Flag Address and Modes

[Ixx25 Flag Address / Ixx24 Flag Modes]



=0 Use amplifier enable function
=1 Do not use amplifier enable function

=0 Enable hardware position limits
=1 Disable hardware position limits

=0 Direct Flag Interface
=1 Flag interface thru MACRO

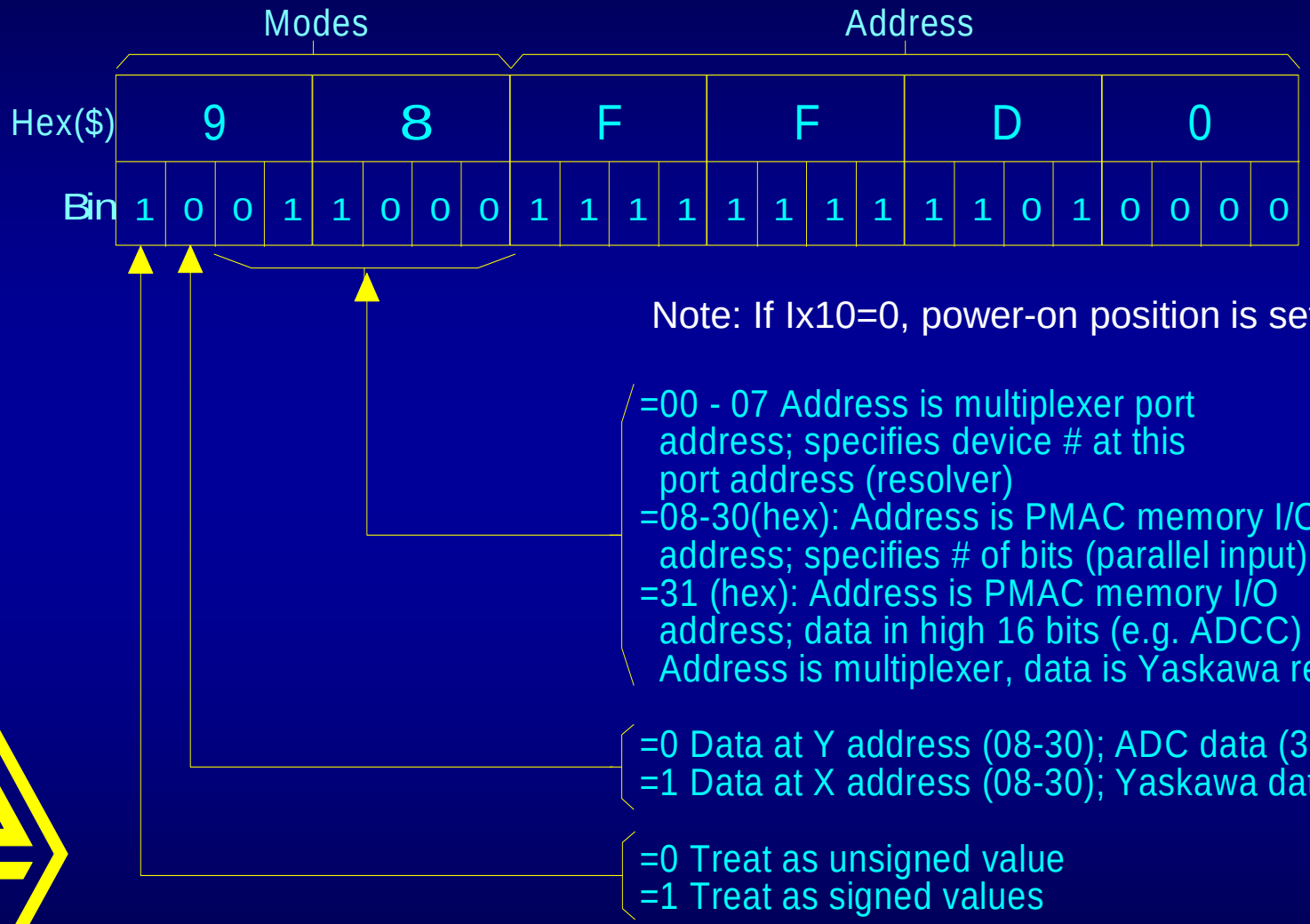
=0 Enable amplifier fault input
=1 Disable amplifier fault input

=00 Kill all PMAC motors on fault or F.E.
=01 Kill all C.S. motors on fault or F.E.
=1x Kill this motor only on fault or F.E.

=0 Low true fault input
=1 High true fault input

Ix10 - Motor Absolute Power-On Position Address

[Ixx10 Address / Ixx95 Format]

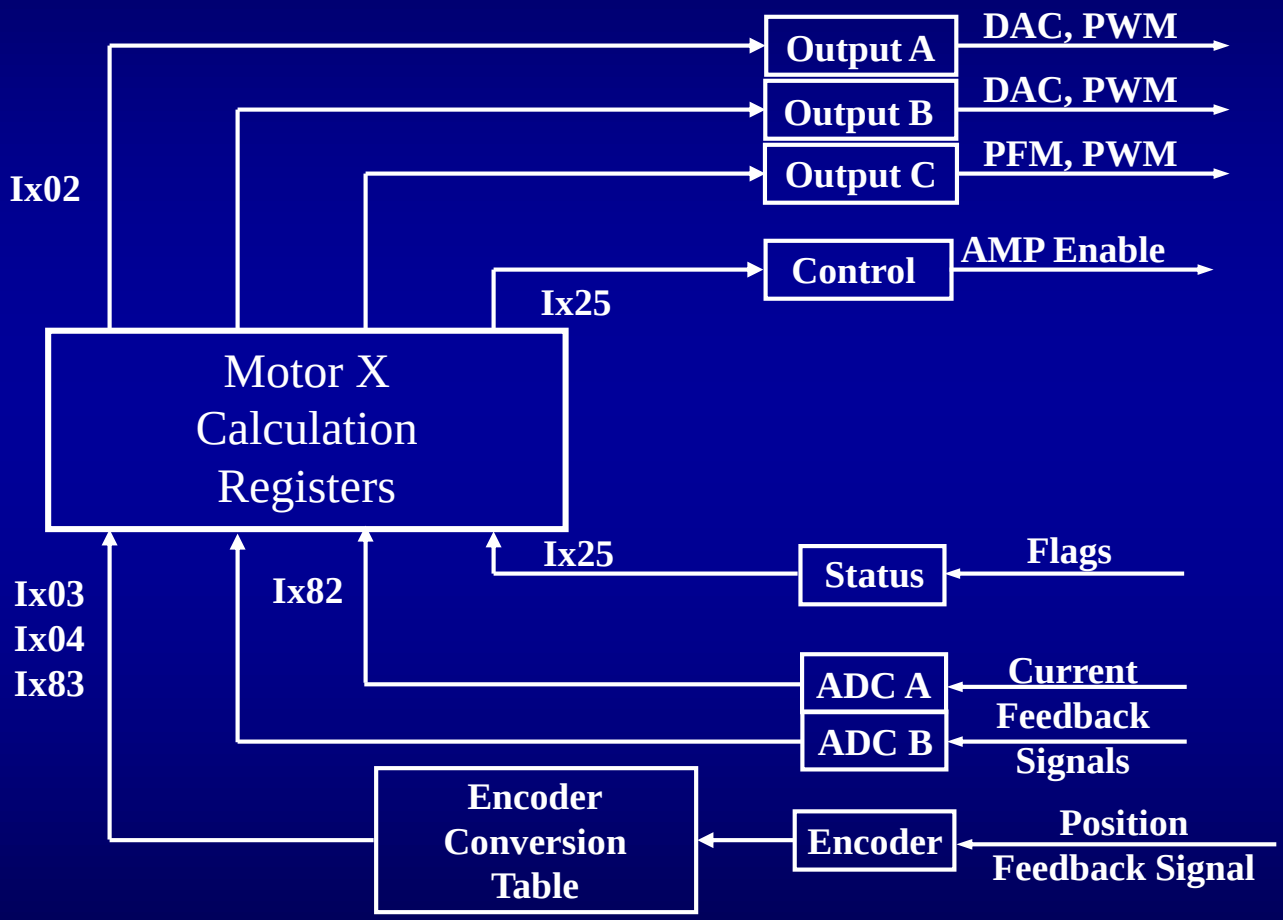


Absolute Position Read on Power-Up

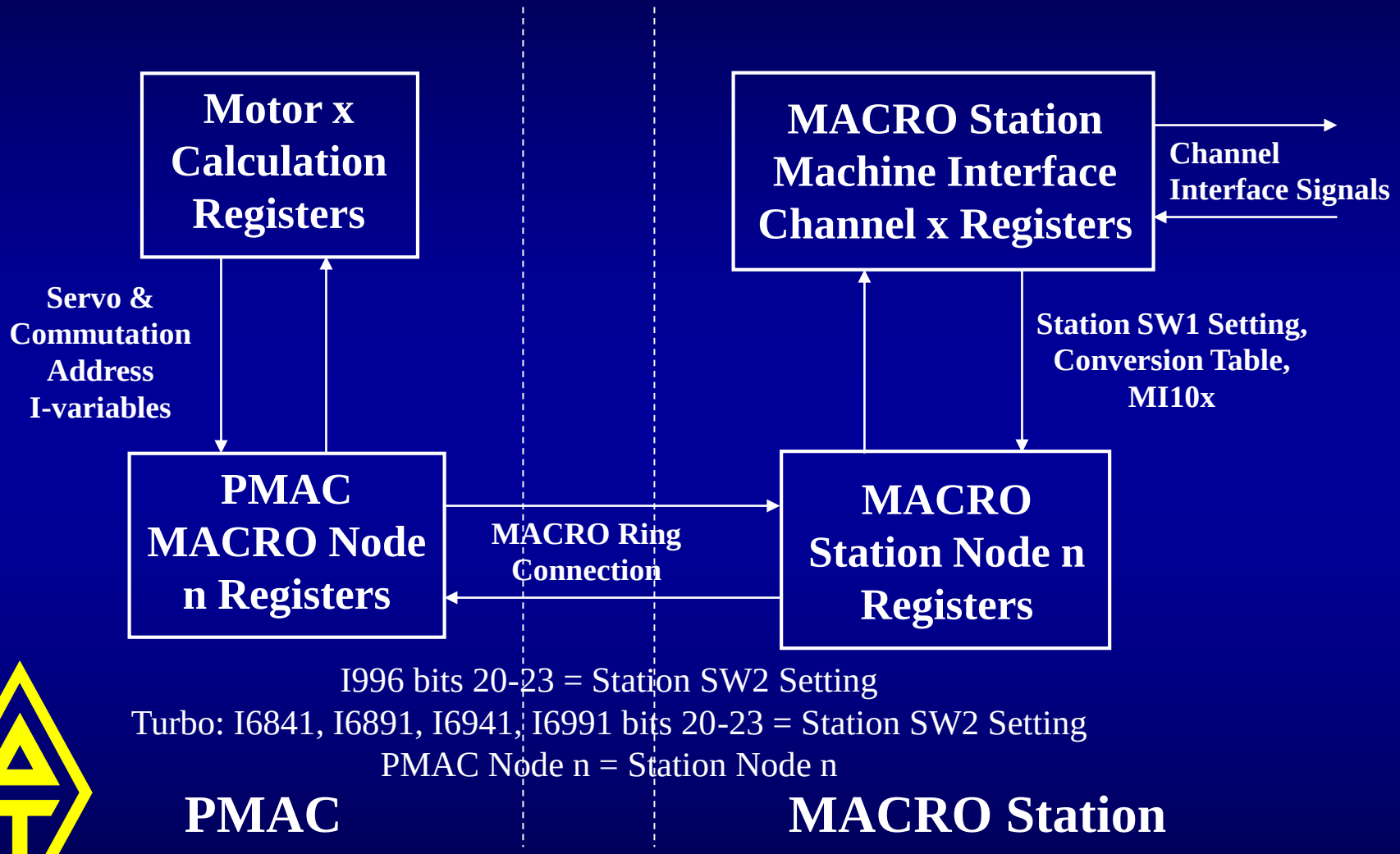
- Power-up servo position read with Ix10
  - Resolvers
  - Absolute encoders
  - Analog input devices
- Geared resolver support with I9x and I8x
  - I9x [Ixx99] is gear ratio of second to first resolver
  - I8x [Ixx98] is gear ratio of third to second resolver
- Power-up phase position read with Ix81
  - Resolvers
  - Absolute encoders
  - Ix75 holds position offset



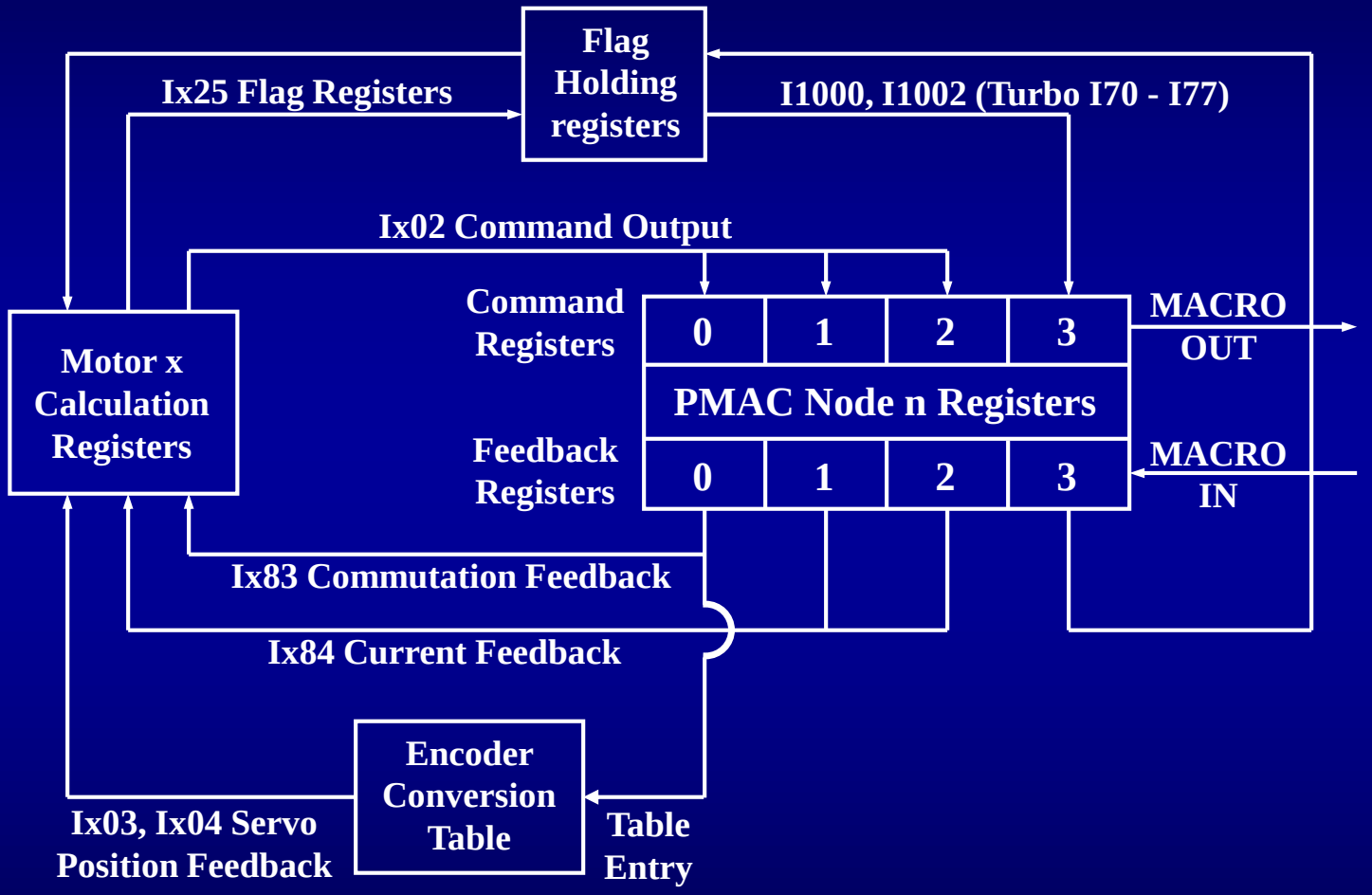
PMAC Direct Register Mapping



PMAC/MACRO Station Register Mapping

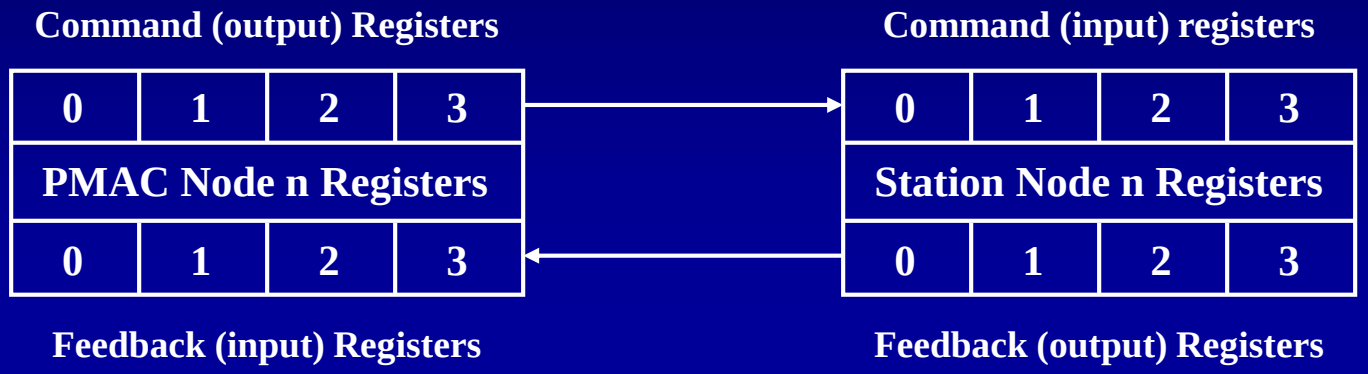


PMAC MACRO Servo and Commutation Addressing



PMAC/MACRO Station Node Matching

MACRO hardware automatically copies every phase cycle



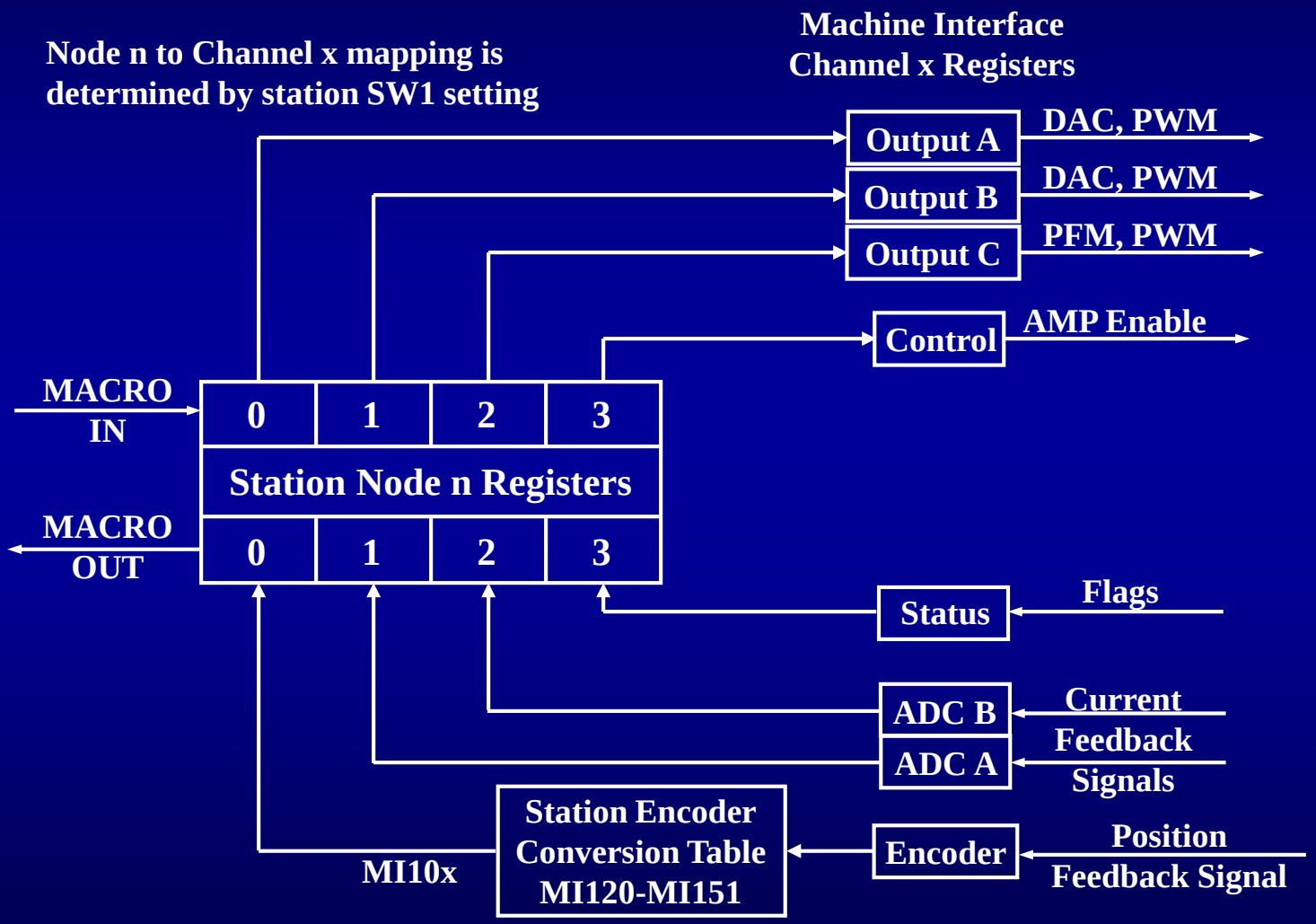
PMAC Node Master Number set by I996 bits 20-23

Turbo PMAC Node Master Number set by I6841, I6891, I6941, I6991 bits 20-23

Station Node Master Number set by SW2

Node Slave Number n automatically matches between PMAC and MACRO Station

MACRO Station Node-to-Channel Mapping



Motor x Safety Variables

Ix11: Fatal Following Error Limit

(Units: 1/16 ct)

=0 disables

When exceeded:

- Coordinate system program aborted
- Motor(s) killed as set by Ix25 [Ixx24] bits 22 and 21
  - (1x): This motor only
  - (01): All motors in C.S.
  - (00): All PMAC motors



Motor x Safety Variables (continued)

Ix12: Warning Following Error Limit
(Units: 1/16 ct)
=0 disables
When exceeded:
- Sets motor and C.S. status bits
- Can set output and interrupt
- Can be used for trigger condition



Motor x Safety Variables (continued)

Ix13: Software Positive Position Limit

Ix14: Software Negative Position Limit

(Units: counts, referenced to motor zero)

=0 disables

When exceeded:

- Aborts motion program or motor move
- Decelerates motor(s) at Ix15 rate

Ix15: Deceleration Rate on Abort or Limit

(Units: cts/msecs^2; floating point)



DO NOT SET TO ZERO!
(MOTOR WOULD NOT DECELERATE)

Motor x Safety Variables (continued)

Ix16: Maximum Permitted Program Velocity
(Units: cts/msec; floating point)

- For linear blended program moves only (with I13=0)
- Limit scales with feedrate override (% value)
- Also serves as RAPID velocity if I50=1



Motor x Safety Variables (continued).

Ix17: Maximum Permitted Program Acceleration

(Units: cts/msec^2; floating point)

- For linear blended program moves only (with I13=0)
- Limit scales with feedrate override (% value)
- Of limited effectiveness (without special lookahead) when acceleration/deceleration requires more than one programmed move



Motor x Safety Variables (continued)

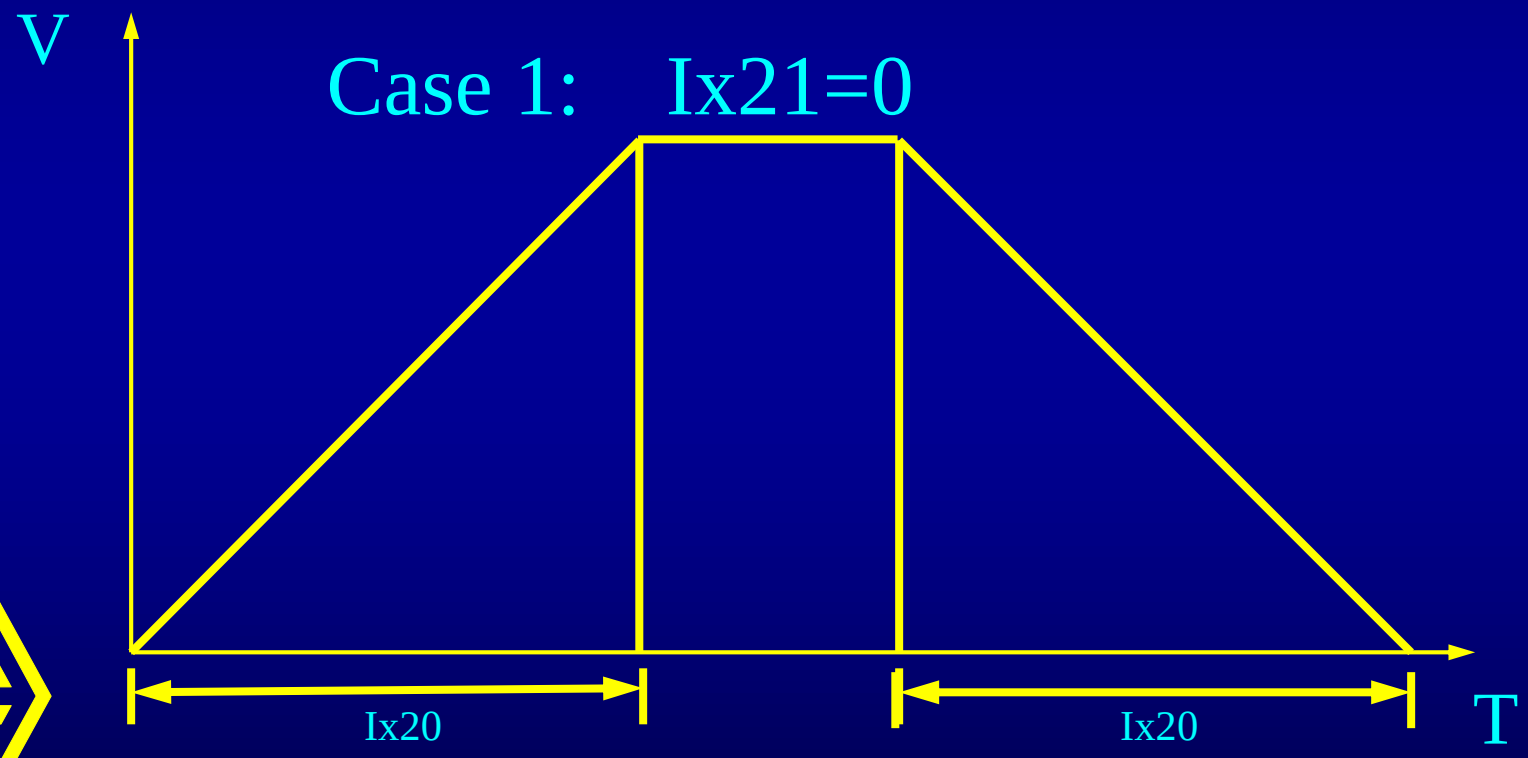
Ix19: Maximum Permitted Jog/Home/Rapid Acceleration
(Units: cts/msec^2; floating point)

- Can override TA (Ix20) and TS (Ix21)
- Used always if Ix20≅0 and Ix21≅0



Motor x Motion Variables

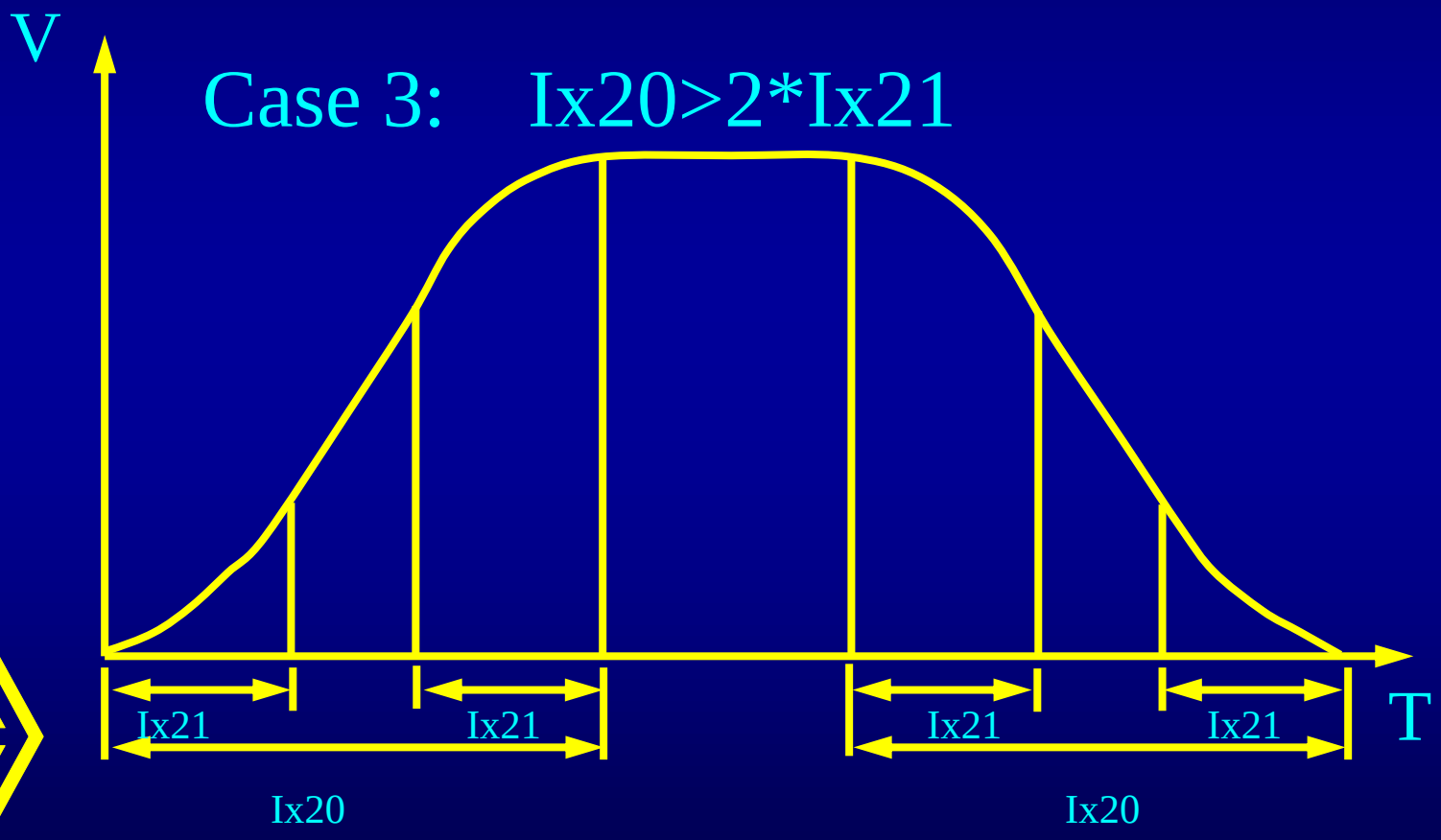
Ix20 Acceleration Time (Jog, Home)
Ix21 S-Curve Time (Jog, Home)



Motor x Motion Variables

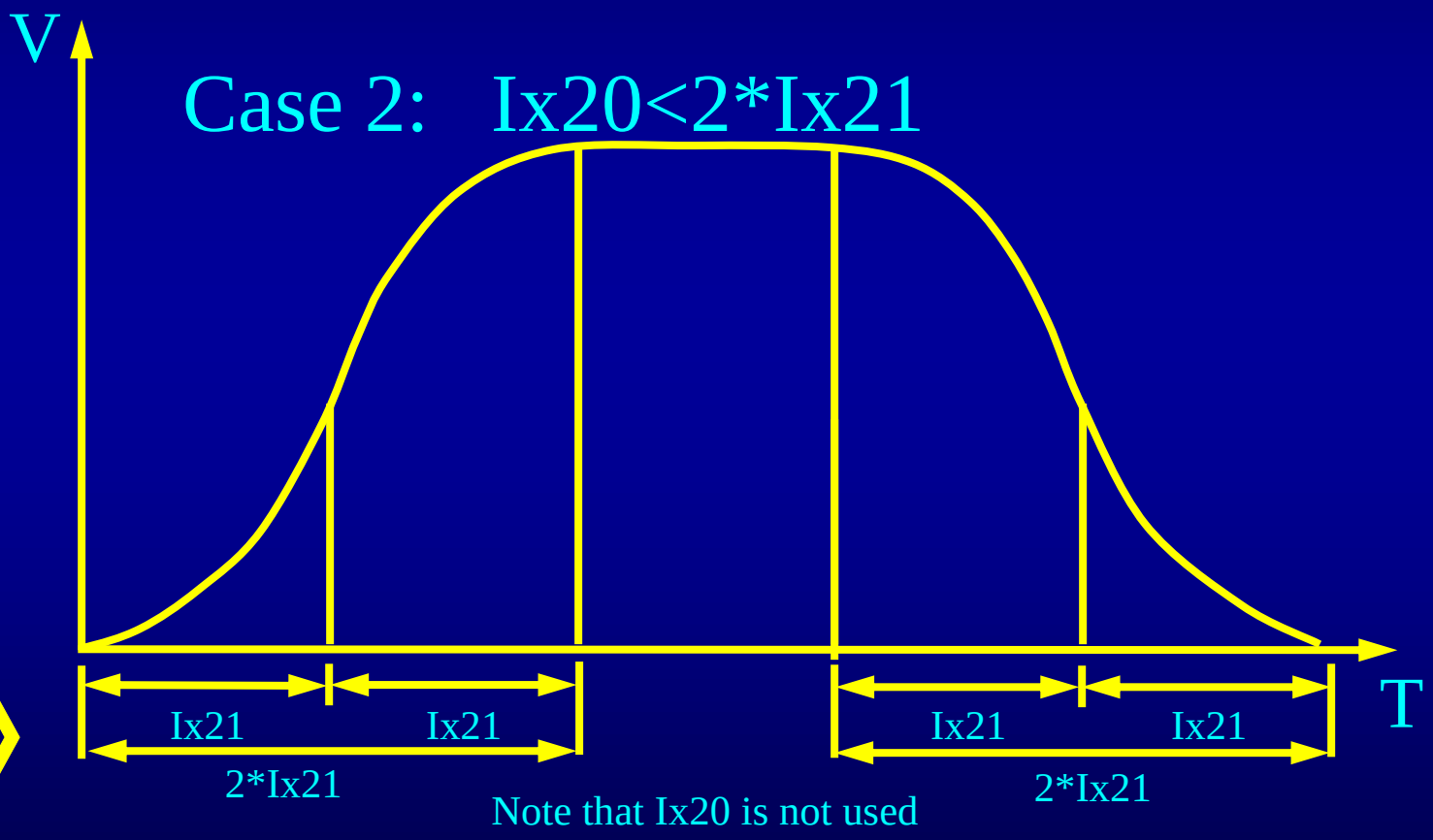
Ix20 Acceleration Time (Jog, Home)

Ix21 S-Curve Time (Jog, Home)

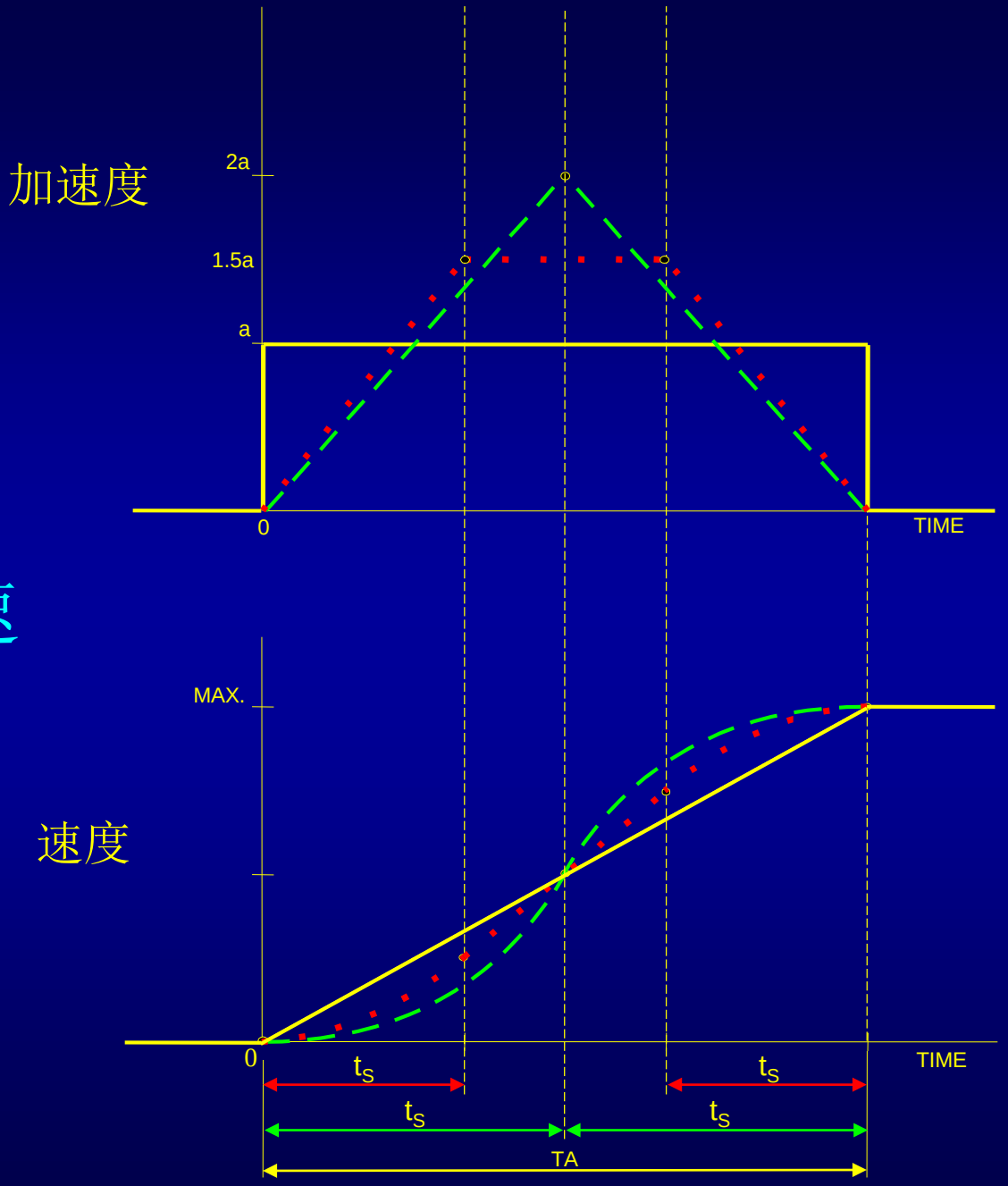


Motor x Motion Variables

Ix20 Acceleration Time (Jog, Home)
Ix21 S-Curve Time (Jog, Home)



PMAC
“S” 曲线加速



Motor x Motion Variables (continued)

Ix22: Jog Speed (Absolute value)
(Units: cts/msec; floating point)

Ix23: Home Speed (Sign is direction)
(Units: cts/msec; floating point)

Ix25: LIMIT/HOME/FAULT/AENA
[Ixx25, Ixx24] Flag Address and Modes
(See Separate Slide)

Ix26: Home Offset (Units: 1/16 ct)
Distance from home trigger (or absolute sensor zero) to motor zero position



Basic Jog Commands

J+ - Jog positive indefinitely

J- - Jog negative indefinitely

J/ - Jog stop (or close loop)

J= - Jog back to prejog (last programmed) position

J={constant} - Jog to specified position (in counts)
J^{constant} - Jog to specified distance from present actual position

J:{constant} - Jog specified distance from present commanded position

Variables Ix19 through Ix22 control the jog. These may be changed on the fly, but do not take effect until the next jog command



Variable Jog Commands

Variables Ix19 through Ix22 control the jog. These may be changed on the fly, but do not take effect until the next jog command

J=* - Jog to position (in counts) specified in variable jog register

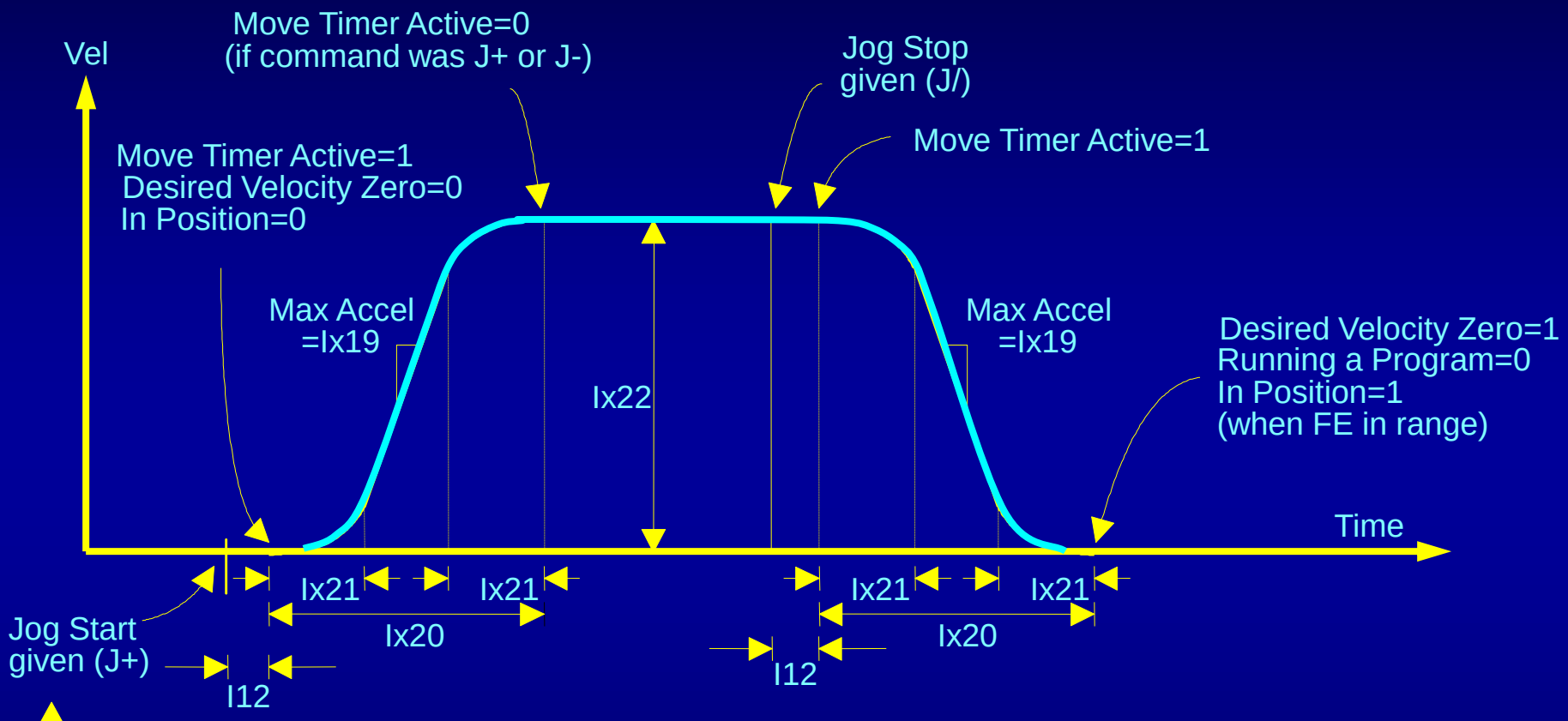
J^* - Jog distance (from present actual) specified in variable jog register

J:* - Jog distance (from present commanded) specified in variable jog register

Mx72 is suggested M-variable for variable jog register
Example:

```
M172 = P1+500
#1J=*
```

Jog Move Trajectory



Note: If the jog command is not of indefinite length (not J+ or J-) then the "Running a Program" status bit will be high for the entire jog move

Note: Rate of acceleration limited by Ix19 - can override Ix20 and Ix21

PMAC Triggered Moves

- **Move relative to trigger-captured position**

- **Three types of triggered moves:**
  - **-Homing search move**
  - **-On-line jog-until trigger**
  - **-Program move-until-trigger (RAPID mode)**

- **Two types of trigger:**
  - **-Input trigger: Index and/or flag**
  - **-Error trigger: Warning following error exceeded**

- **Two methods of position capture:**
  - **-Hardware capture: Immediate; encoder counter only**
  - **-Software capture: Up to 1 background cycle delay**



Triggered Moves: Input Trigger vs. Error Trigger

- **Input Trigger (flag and/or index edge)**
  - **-Ix03 bit 17 [Ixx97 bit 1] = 0 (default) selects**
  - **-Ix25 specifies index/flag-set address**
  - **-Enc/flag I-variable 2 [I7mn2] selects use/polarity of flag and/or index**
  - **-Enc/flag I-variable 3 [I7mn3] selects which flag used**
  - **-Flag "capture" status bit = 1 is trigger**

- **Error Trigger (warning following error-WFE)**
  - **-Ix03 bit 17 [Ixx97 bit 1] = 1 selects**
  - **-Ix12 sets magnitude of WFE limit**
  - **-Must select "software capture" (Ix03 bit 16 [Ixx97 bit 0] = 1)**
  - **-Motor WFE status bit = 1 is trigger**

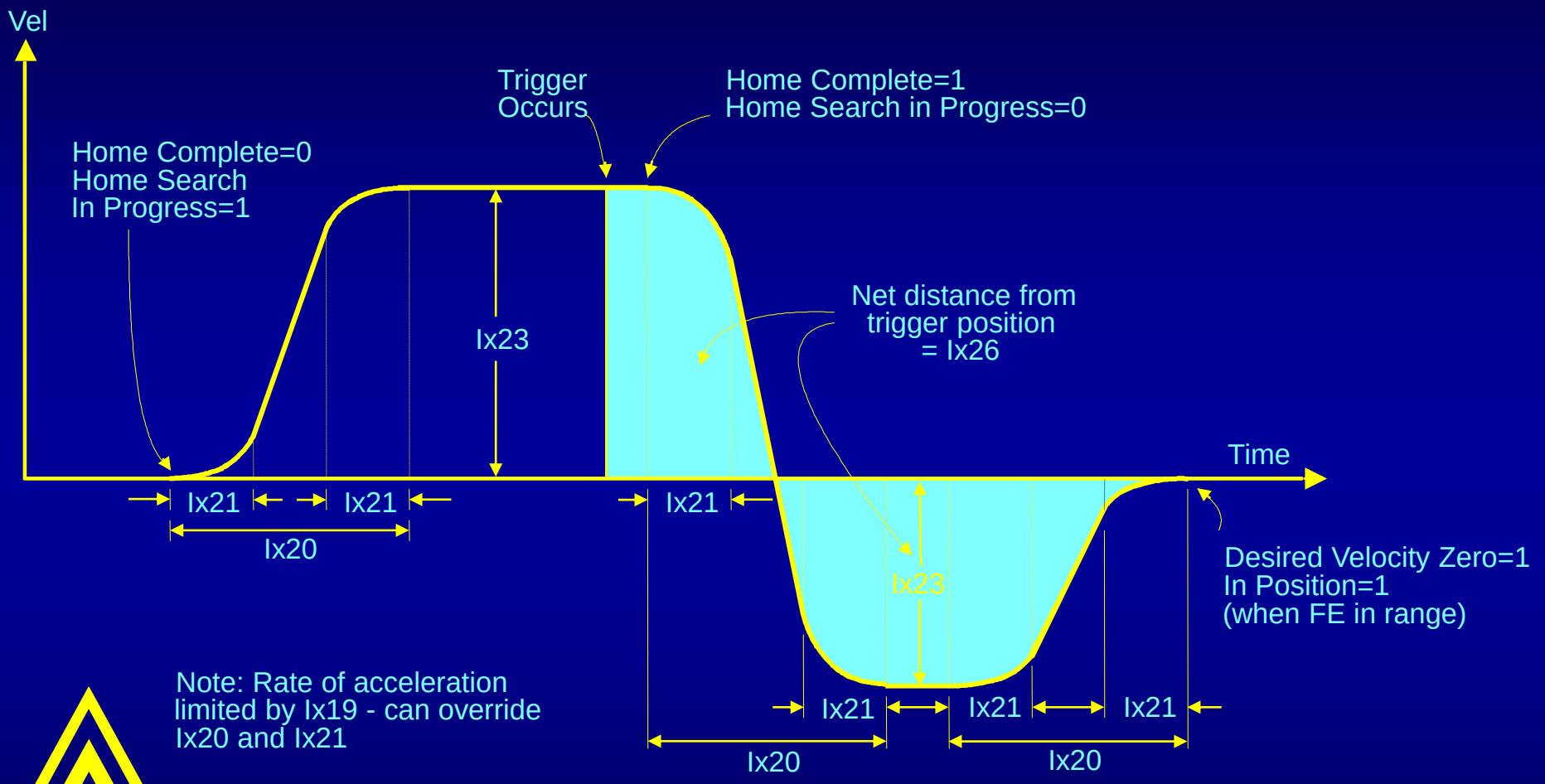
Triggered Moves: Hardware Capture vs. Software Capture

- **Hardware Capture**
  - **-Ix03 bit 16 [Ixx97 bit 0] = 0 (default) selects**
  - **-Requires input trigger, not error trigger**
  - **-Requires position feedback through encoder counter**
  - **-Requires position and flags from same hardware channel**
  - **-Captures exact count at any speed**

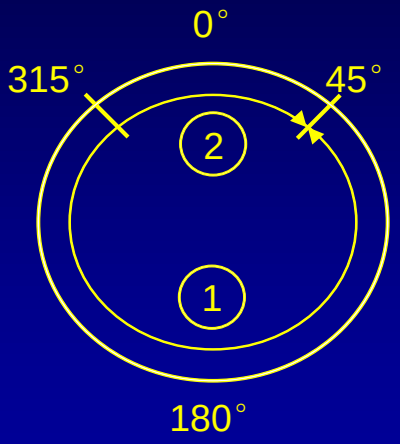
- **Software Capture**
  - **-Ix03 bit 16 [Ixx97 bit 0] = 1 selects**
  - **-Permits input trigger or error trigger**
  - **-Any type of position feedback**
  - **-Potential capture error proportional to speed**



Homing Search Move Trajectory



Motor x Motion Variables (continued)



ABS
....
A315
A45
① Without rollover
② With rollover

Ix27: Position Rollover Range
(Units: cts)
For rotary axes (A,B,C) only

If Ix27>0, axis will make short move within range for ABS moves
(<=Ix27/2 counts)

Motor x Motion Variables (continued)

Ix28: In Position Band (Units: 1/16 ct)

For indicator purposes only

A motor is “in-position” if:

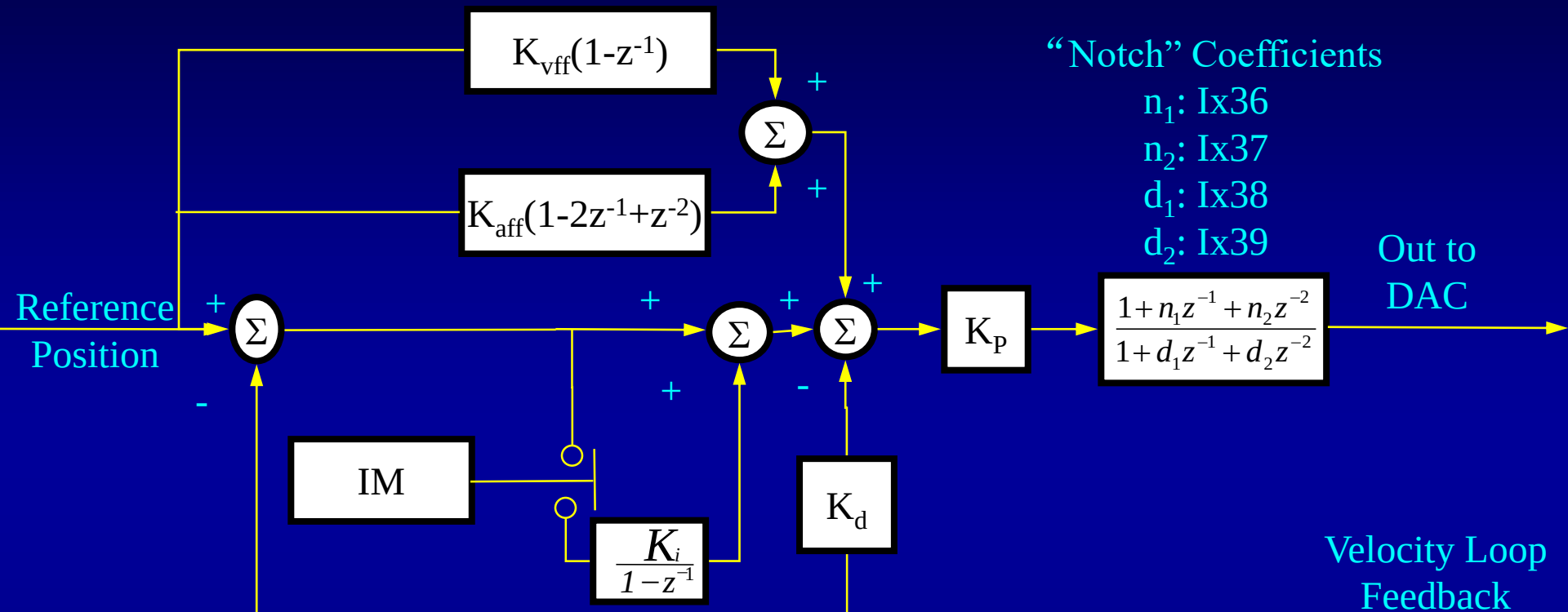
- Closed loop
- Desired vel =0
- Prog timer off
- |FE|<Ix28
- Above conditions true for (I7+1) [Ixx88 +1] consecutive background scans

Example use in program:

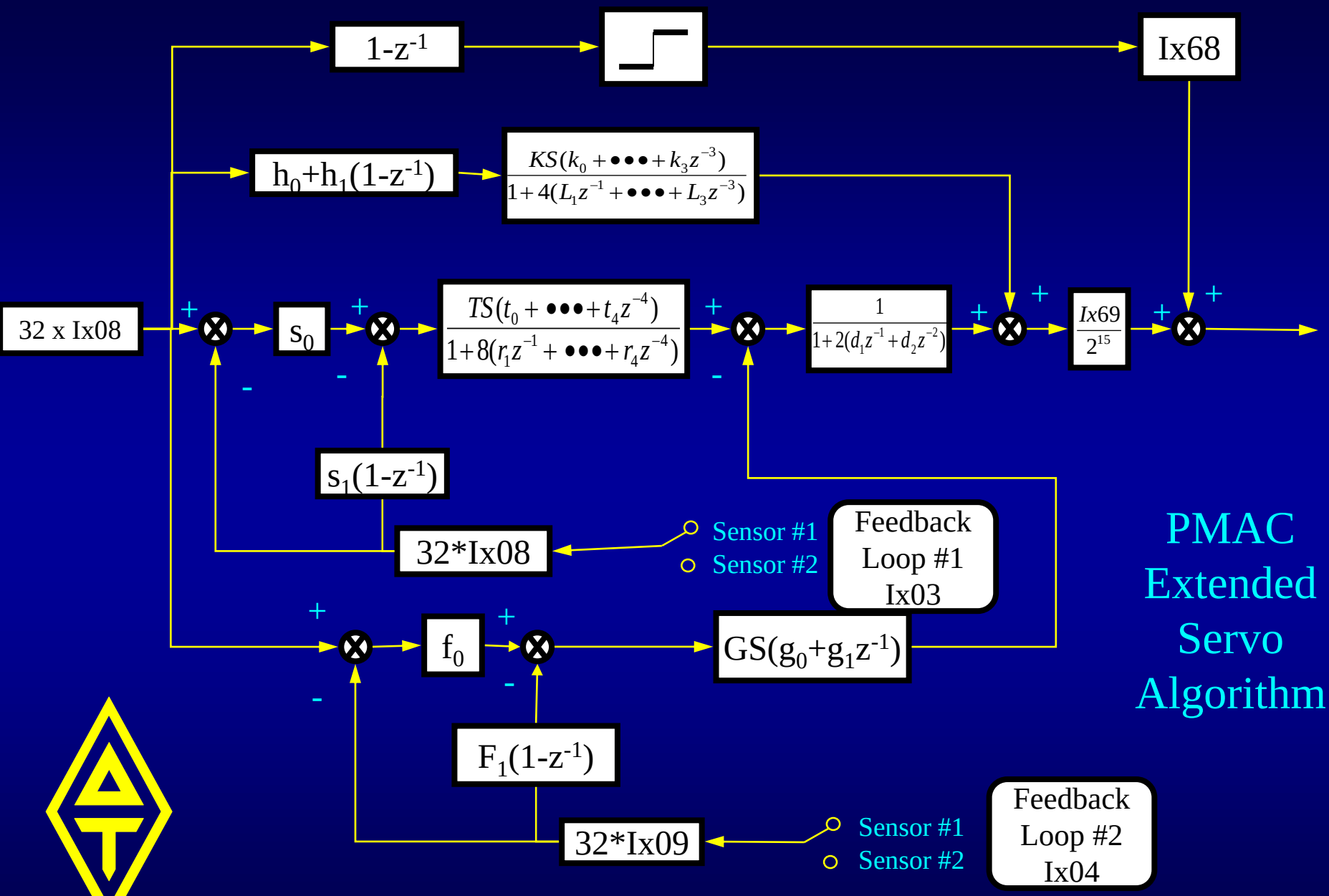
```
X10              ;move
DWELL0     ;stop lookahead
WHILE(M140=0)WAIT  ;while not in position
M1=1             ;action on in-position
```



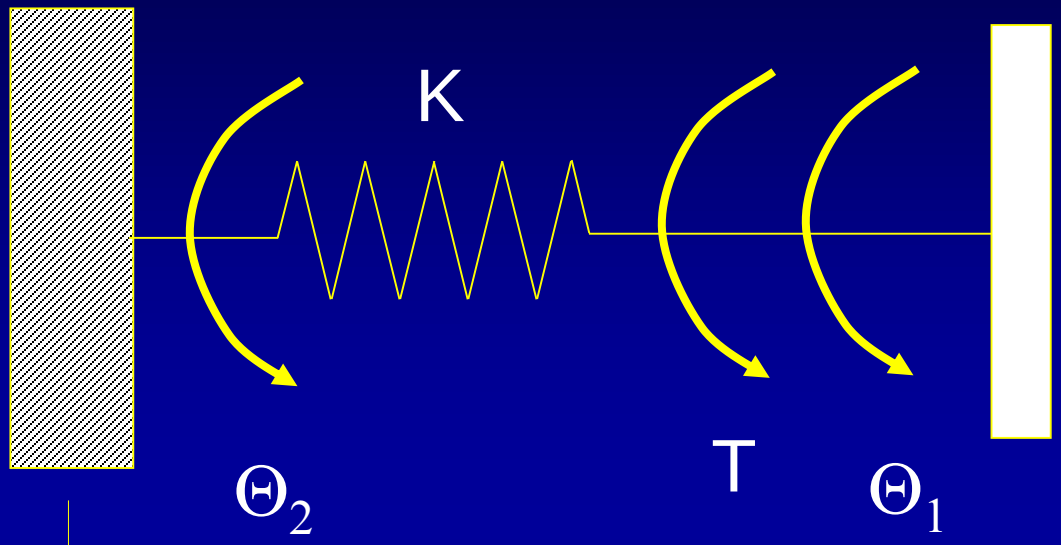
PMAC PID + Notch Servo Filter

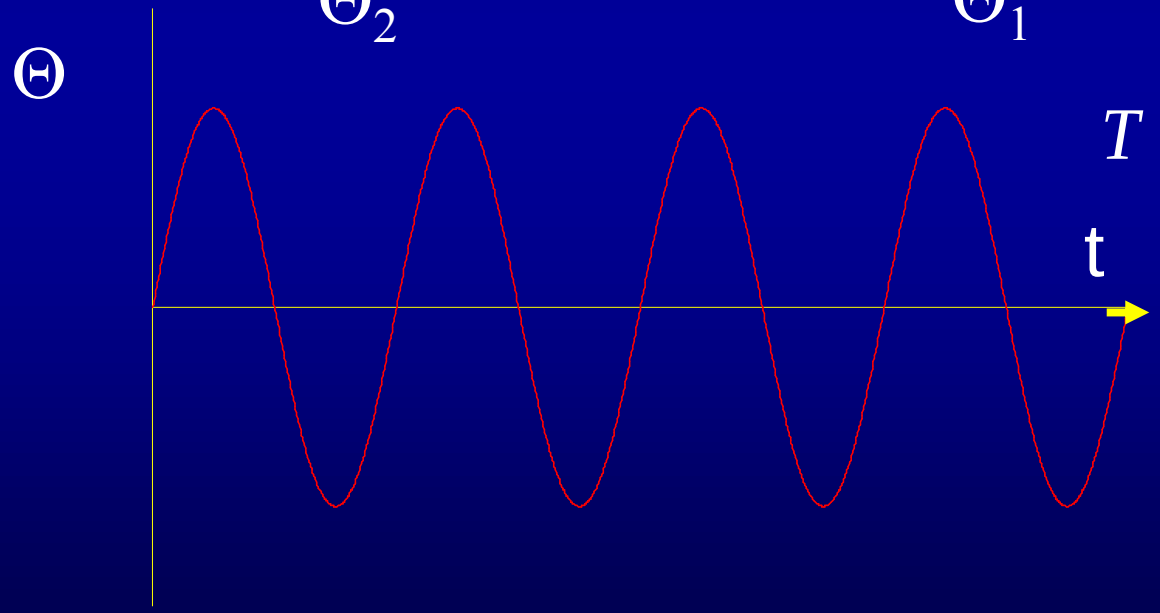


K_P: Proportional Gain (Ix30)
K_d: Derivative Gain (Ix31)
K_{vff}: Vel Feedforward Gain(Ix32)
K_i: Integral Gain(Ix33)
IM: Integration Mode (Ix34)
K_{aff}: Acceleration Feedforward Gain(Ix35)


1-z-1
Ix68
h0+h1(1-z-1)
KS(k0 + ••• + k3z-3) / 1 + 4(L1z-1 + ••• + L3z-3)
32 x Ix08
s0
TS(t0 + ••• + t4z-4) / 1 + 8(r1z-1 + ••• + r4z-4)
1 / 1 + 2(d1z-1 + d2z-2)
Ix69 / 215
s1(1-z-1)
32*Ix08
Sensor #1
Sensor #2
Feedback Loop #1 Ix03
f0
GS(g0+g1z-1)
F1(1-z-1)
32*Ix09
Sensor #1
Sensor #2
Feedback Loop #2 Ix04
PMAC Extended Servo Algorithm

Proportional Control





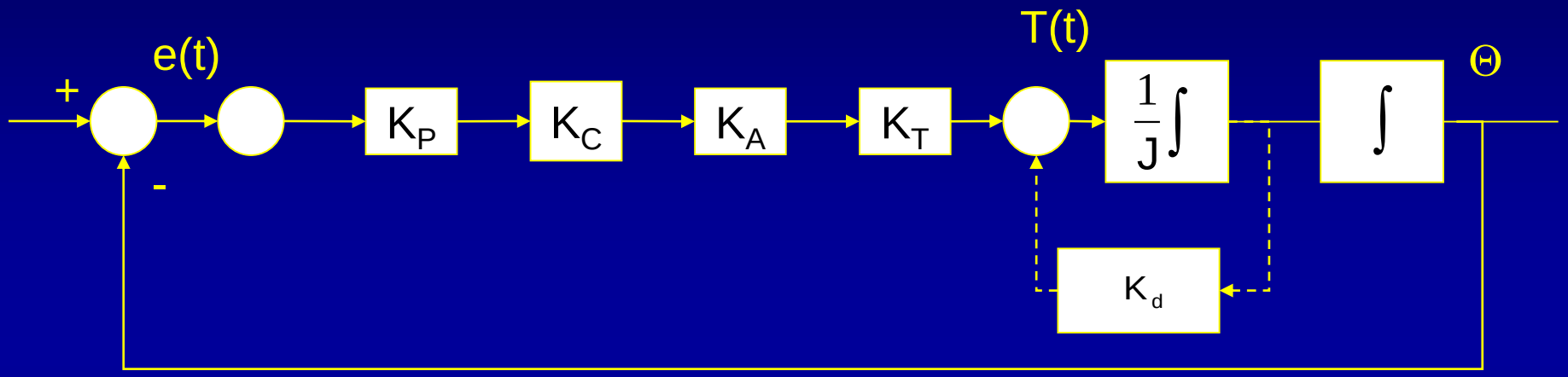
$$T = K \bullet \Theta = -J_1 \frac{d^2\Theta}{dt^2}$$

SHO, $\omega_n = \sqrt{\frac{K}{J_1}}$

$\zeta = 0$

SHO: Simple Harmonic Oscillator

Proportional Control



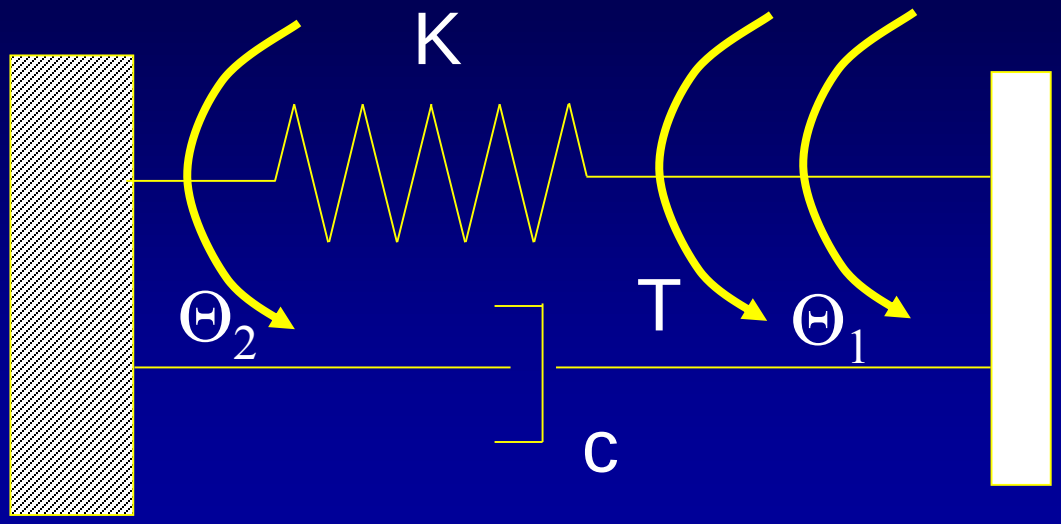
$$T(t) = K_p \bullet K_C \bullet K_A \bullet K_T \bullet e(t) = -K_p \bullet K_C \bullet K_A \bullet K_T \bullet \Theta(t) = J_1 \bullet \frac{d^2\Theta}{dt^2}$$

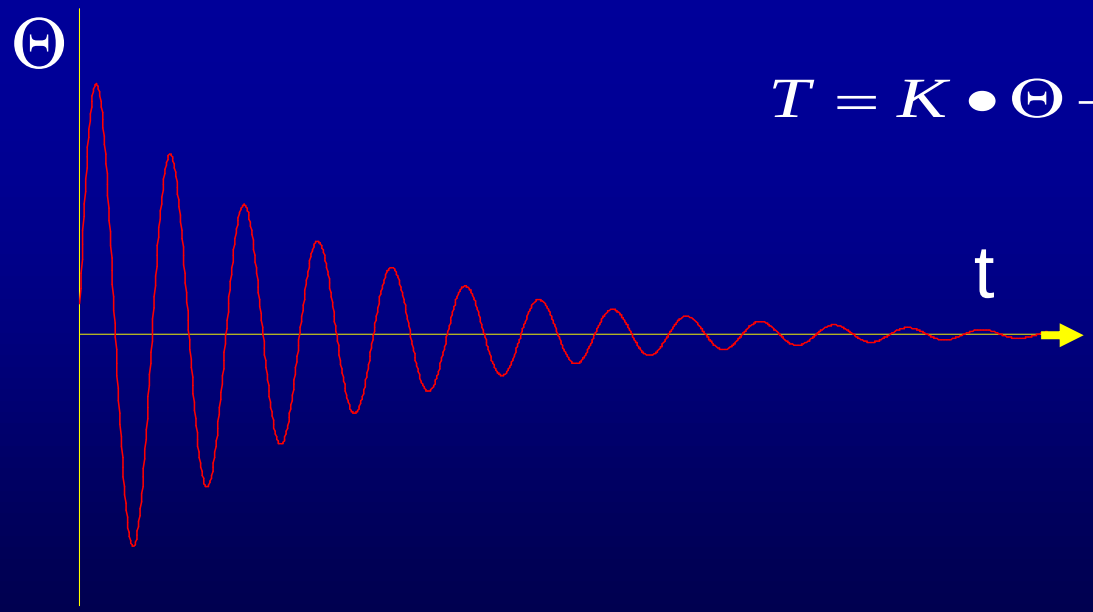
This is an <u>undamped</u> SHO

$$\omega_n = \sqrt{\frac{K_p \bullet K_C \bullet K_A \bullet K_T}{J_1}} \qquad \zeta = 0$$

- Thus the proportional gain $K_P \rightarrow$ spring stiffness
- Higher $K_P \rightarrow$ higher stiffness

Derivative Control



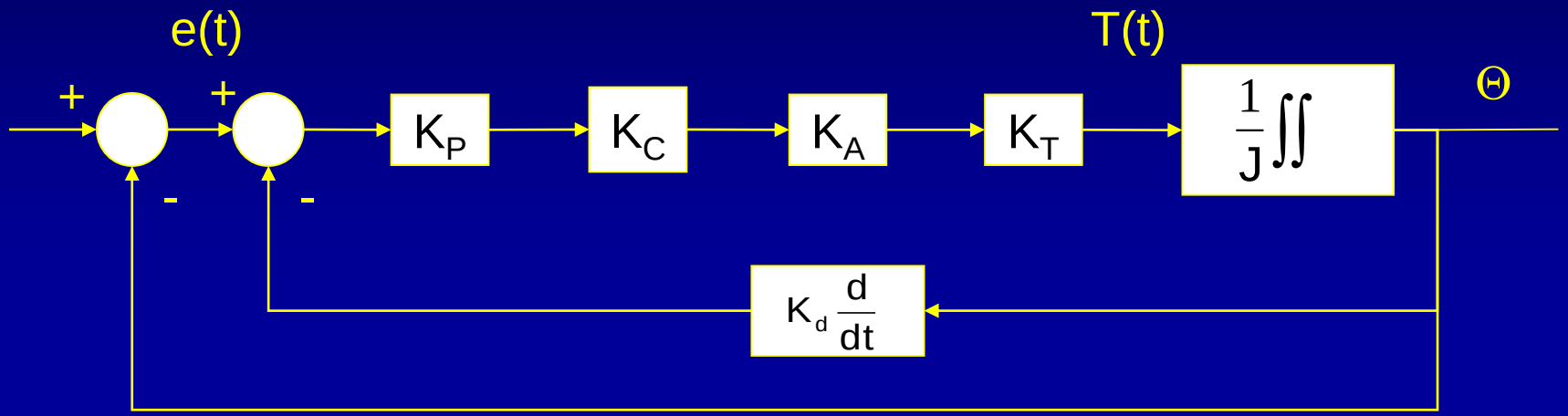


$$T = K \bullet \Theta + c \bullet \frac{d\Theta}{dt} = -J_1 \bullet \frac{d^2\Theta}{dt^2}$$

$$\omega_n = \sqrt{\frac{K}{J_1}}$$

$$\zeta = \frac{c}{2}\sqrt{KJ_1}$$

Derivative Control



$$T(t) = -K_p \bullet K_C \bullet K_A \bullet K_T \bullet \Theta(t) - K_p \bullet K_C \bullet K_A \bullet K_d \bullet \frac{d\Theta}{dt} = J_1 \bullet \frac{d^2\Theta}{dt^2}$$

This is a damped SHO

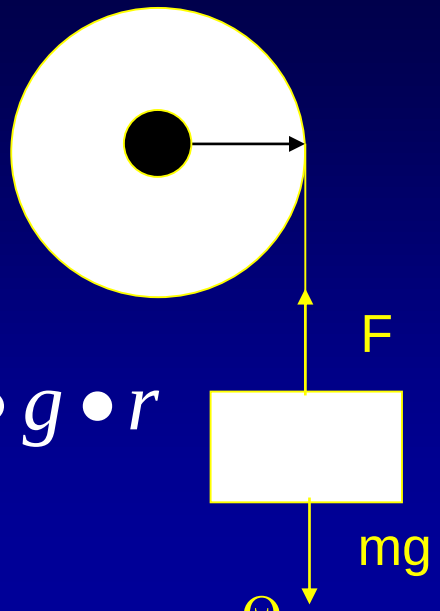
$$\omega_n = \sqrt{\frac{K_p \bullet K_C \bullet K_A \bullet K_T}{J_1}}$$

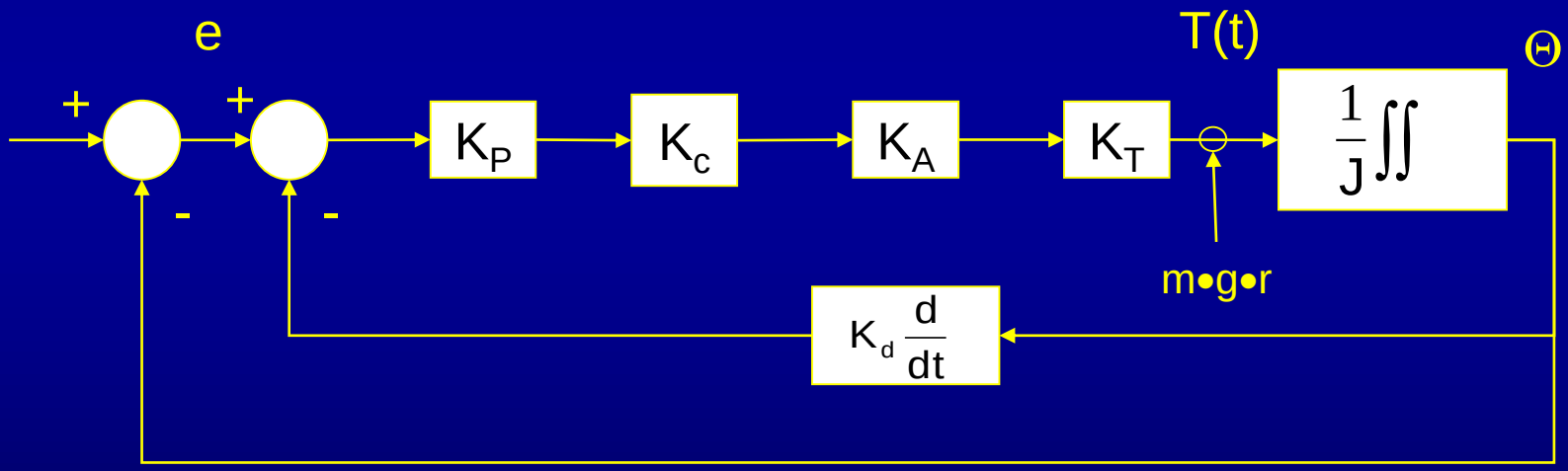
$$\zeta = \frac{K_d}{2}\sqrt{\frac{K_p \bullet K_C \bullet K_A \bullet K_T}{J_1}}$$

Integral Term

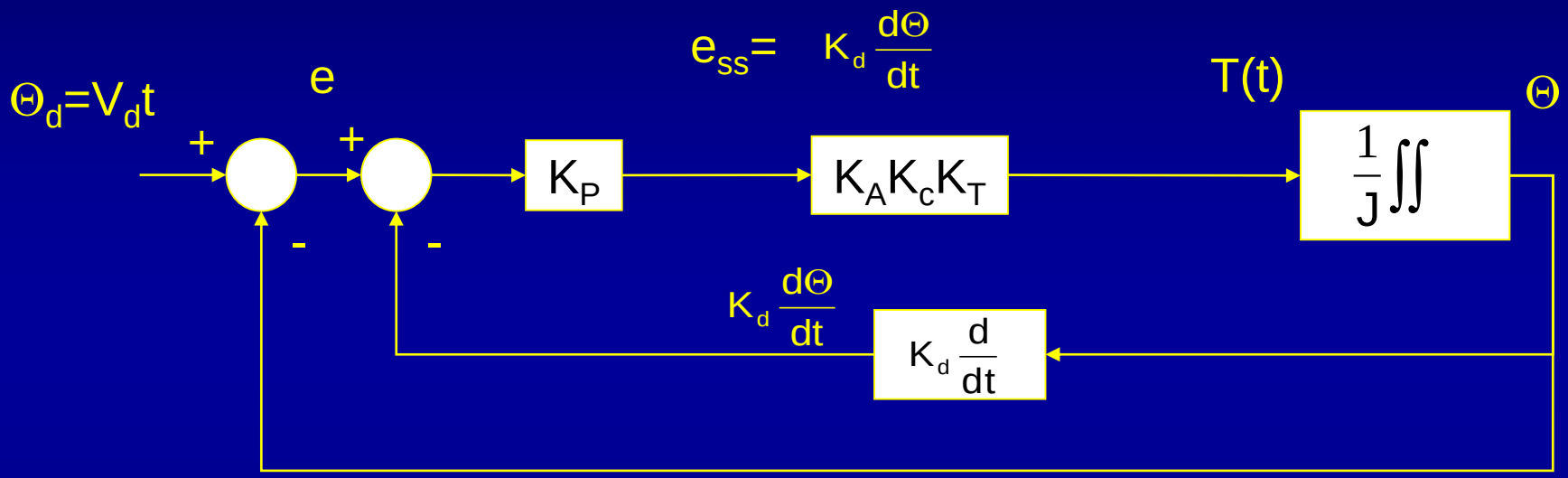
Without Integral:

$$T(t) = (K_P \bullet K_C \bullet K_A \bullet K_T) \bullet e(t) = m \bullet g \bullet r$$

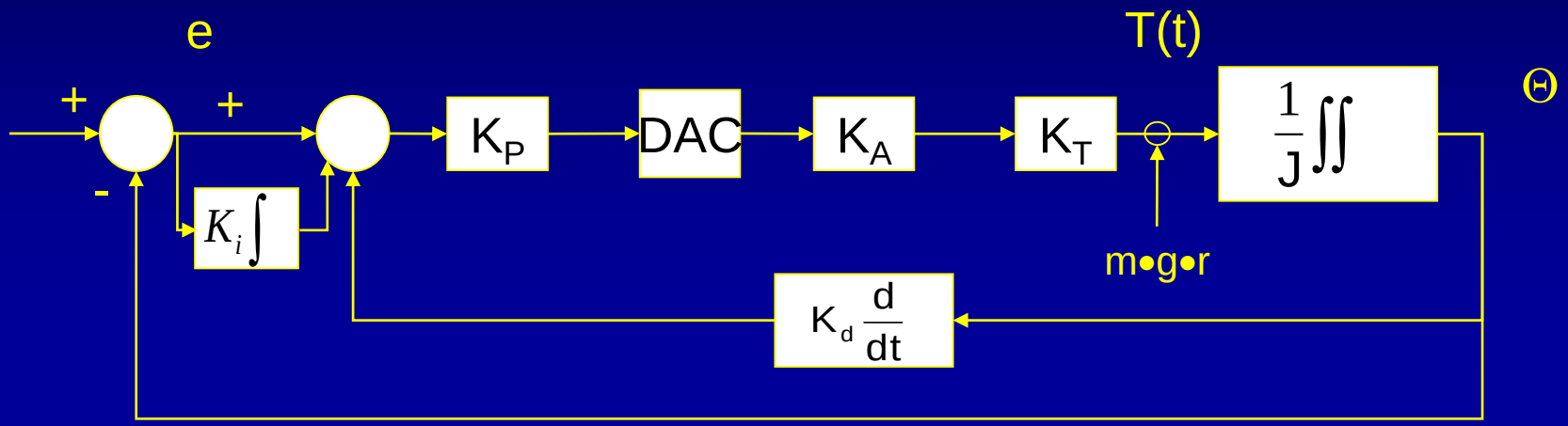




Errors at Steady State Due to Constant Speed Trajectory Tracking



Integral Term



$$T(t) = (K_P \bullet K_C \bullet K_A \bullet K_T) \bullet [K_i \int e(t)dt + d(t)] = m \bullet g \bullet r$$

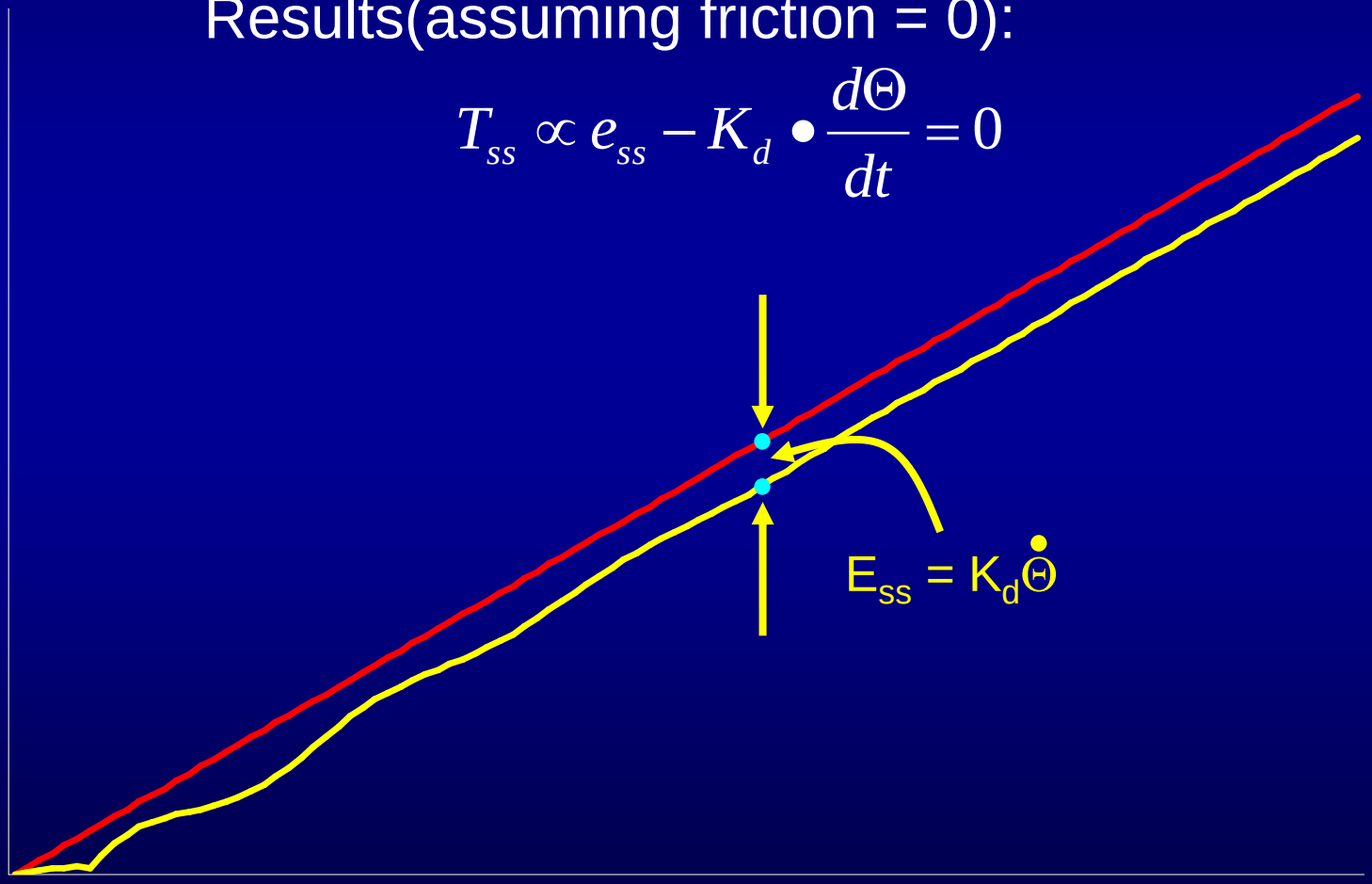
Therefore as $t \rightarrow \infty$, $e(t) \rightarrow 0$

Errors at Steady State Due to Constant Speed Trajectory Tracking

Ramp Input: $\Theta_d = V_d \bullet t, \quad \frac{d\Theta}{dt} = V_d, \quad \frac{d^2\Theta}{dt^2} = 0$

Results(assuming friction = 0):

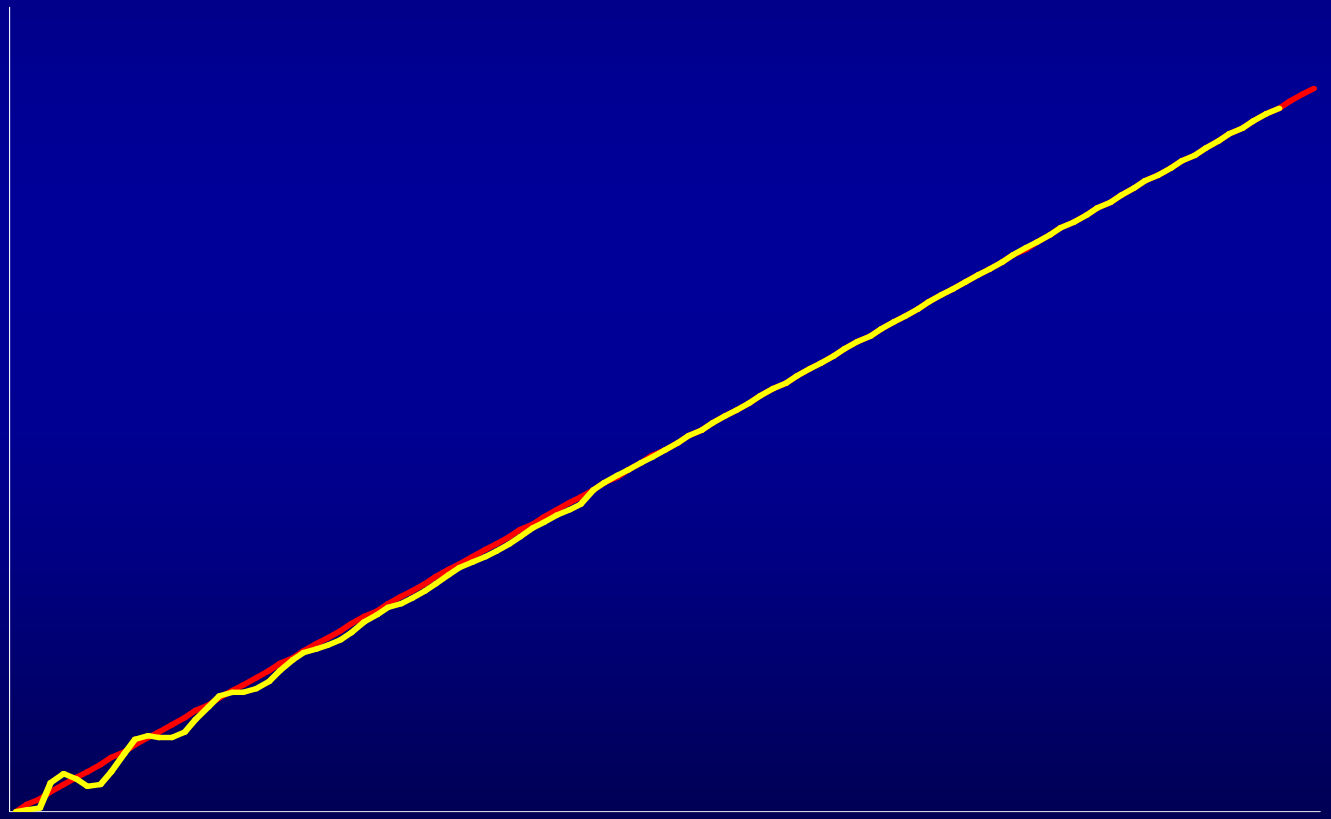
$$T_{ss} \propto e_{ss} - K_d \bullet \frac{d\Theta}{dt} = 0$$



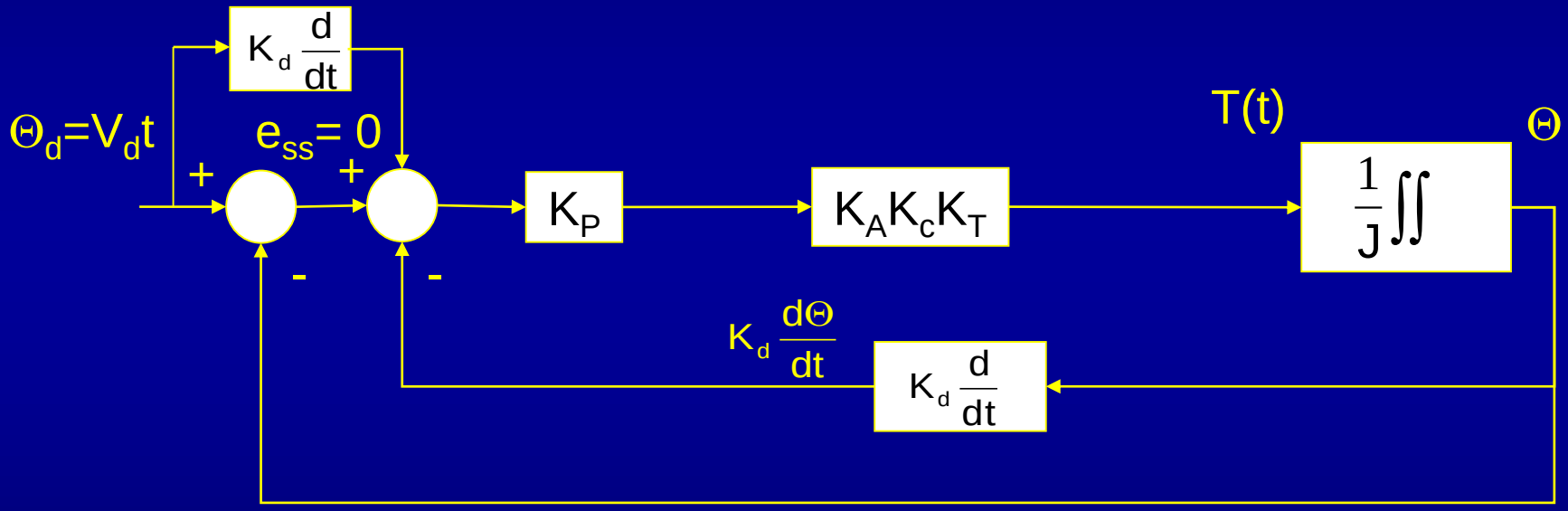
速度前馈的作用

$K_{vff} = K_d \quad \rightarrow \quad e_{ss}=0$

由于恒速跟踪引起的不稳定误差



速度前馈


$K_d \frac{d}{dt}$
$\Theta_d=V_dt$
$e_{ss}= 0$
+
+
-
-
K_P
$K_AK_cK_T$
T(t)
$\frac{1}{J}\iint$
Θ
$K_d \frac{d\Theta}{dt}$
$K_d \frac{d}{dt}$

加速度前馈的作用

In general, the trajectories contain higher order time functions:

e.g.: Constant jerk trajectory $\Theta_d(t) = c_0 + c_1 \bullet t + c_2 \bullet t^2$

By choosing,

$$K_{aff} = \frac{J_1}{K_P \bullet K_C \bullet K_A \bullet K_T} \quad \rightarrow \quad e(t) = 0$$

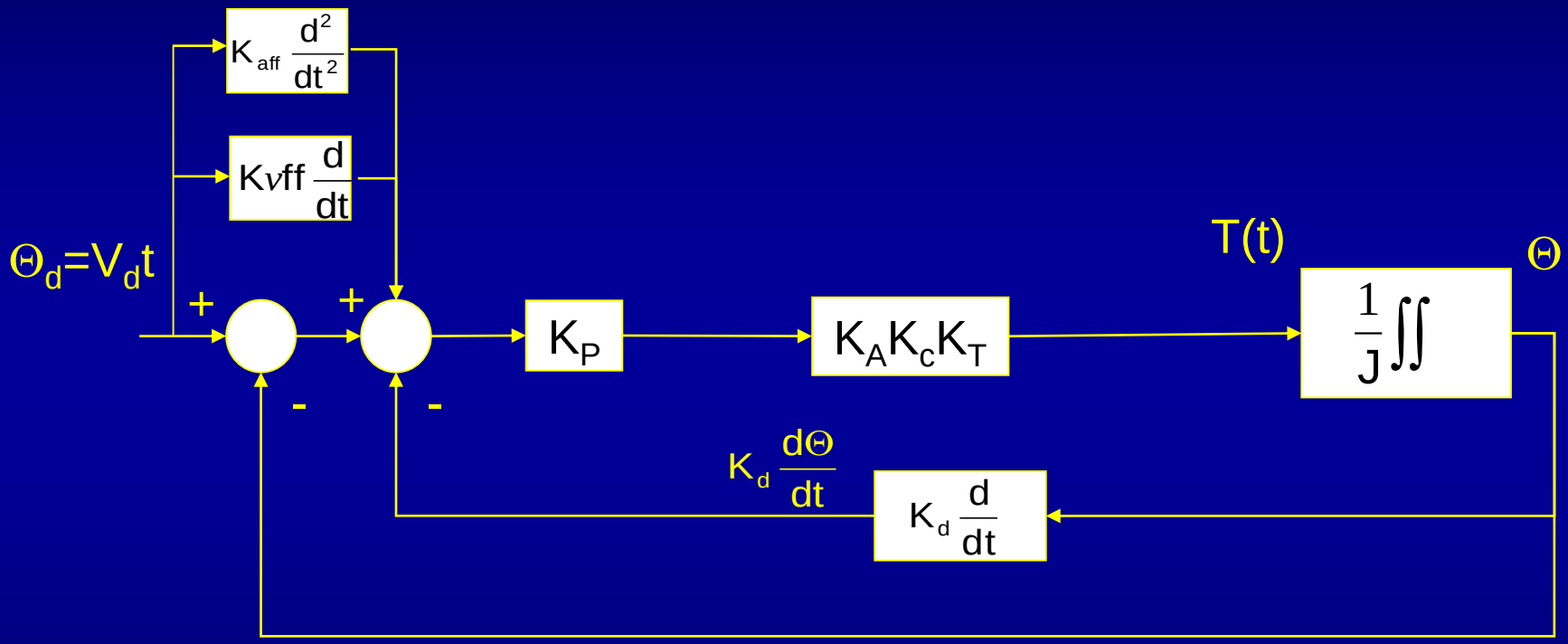
This results in no tracking error for the *Ideal* system.

Since,

$$T(t) = \left(\frac{J_1}{K_P \bullet K_C \bullet K_A \bullet K_T}\right) \frac{d^2\Theta_d}{dt^2} (K_P \bullet K_C \bullet K_A \bullet K_T) = \frac{J_1 \bullet d^2\Theta_d}{dt^2}$$

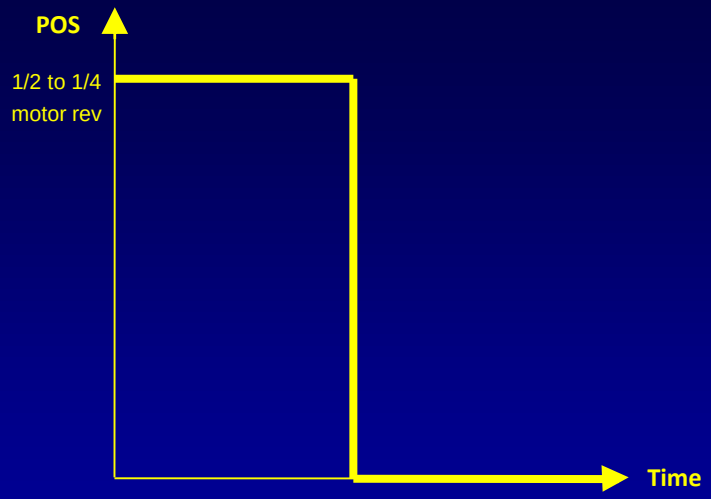
$$\frac{d\Theta_d}{dt^2} = \frac{d^2\Theta}{dt^2} \quad \rightarrow \quad \Theta_d = \Theta \quad \rightarrow \text{ No Tracking errors}$$

Acceleration Feedforward

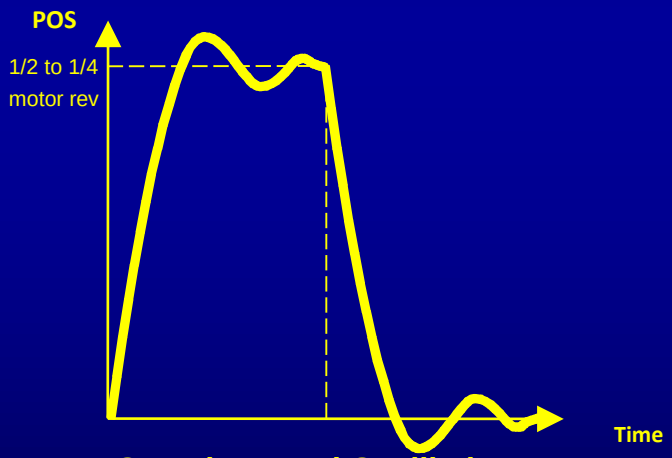


Step Move Tuning

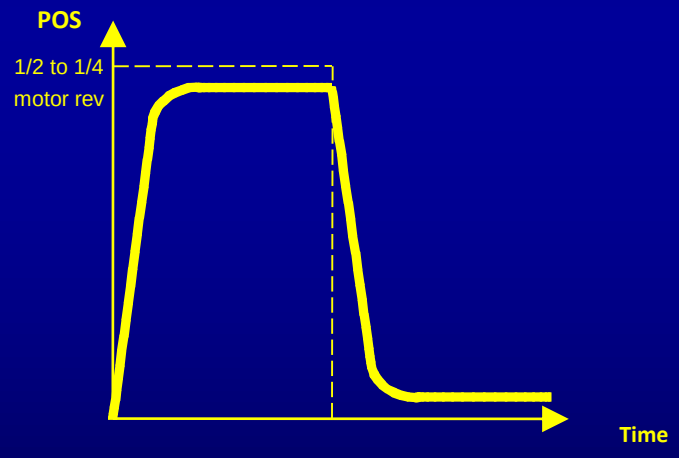




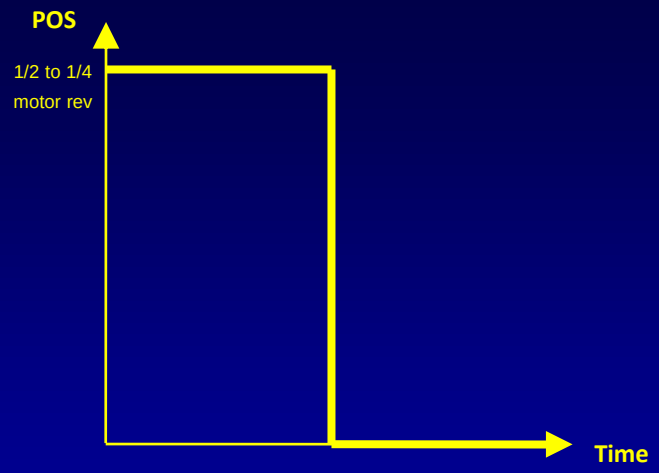
Commanded Position



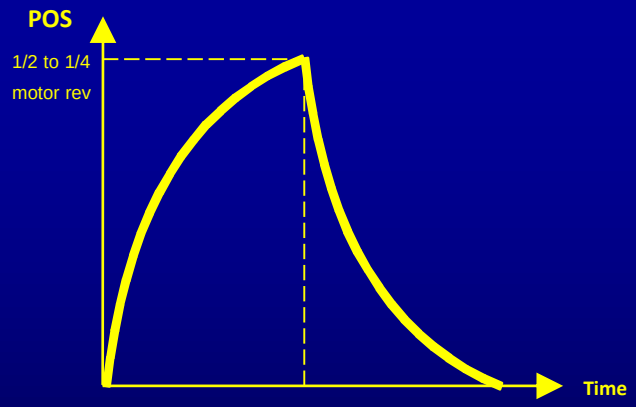
Overshoot and Oscillation
Cause: too little Damping
or too much proportional gain
Fix: Increase K_P (Ix31)
Decrease K_d (Ix30)



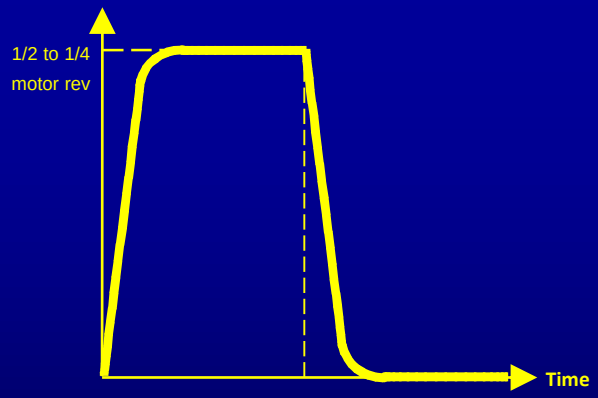
Position Offset
Cause: Friction or Constant Force
Fix: Increase K_i (Ix33)



Commanded Position

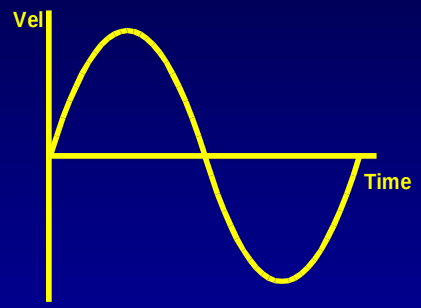


Sluggish Response
Cause: Too much Damping or too little Proportional
Fix: Increase K_P (Ix30) or Decrease K_d (Ix31)

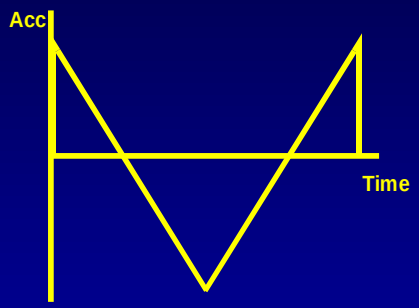


Physical System Limitation
Cause: Limit of the Motor/Amplifier/Load and gain combination
Fix: Evaluate Performance and maybe use more K_i (Ix30)

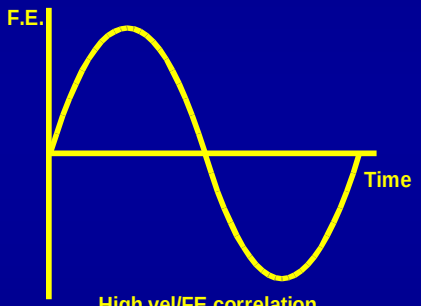
Parabolic Move Tuning



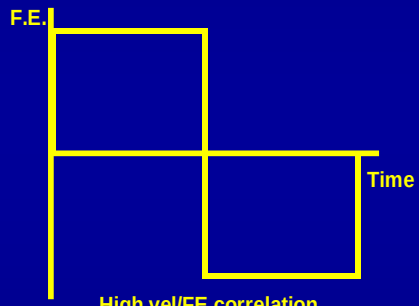
Velocity Profile



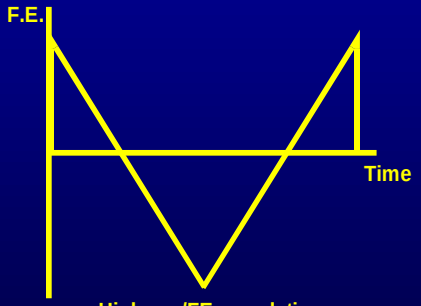
Acceleration Profile



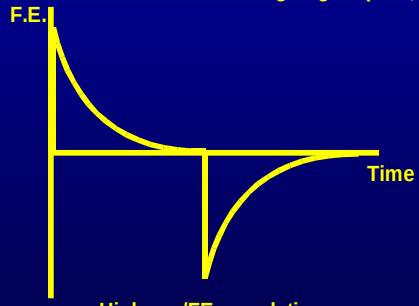
High vel/FE correlation
Cause: Damping
Fix: Increase K_{vff}(Ix32)



High vel/FE correlation
Cause: Friction
Fix: Add "Friction Feedforward" (Ix68)
and/or turn on integral gain (Ix33, Ix34)

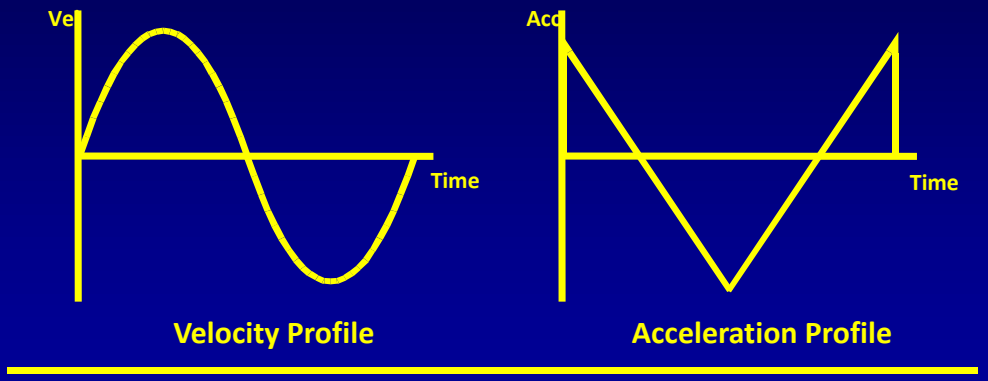


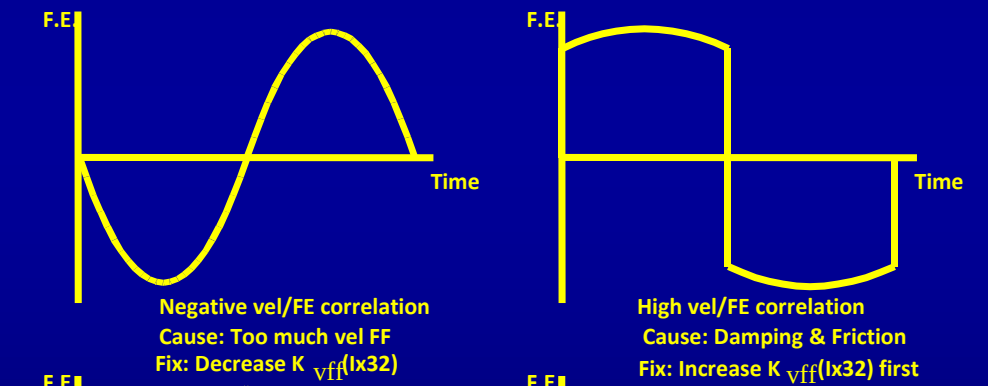
High acc/FE correlation
Cause: Inertial lag
Fix: Increase K_{aff}(Ix35)

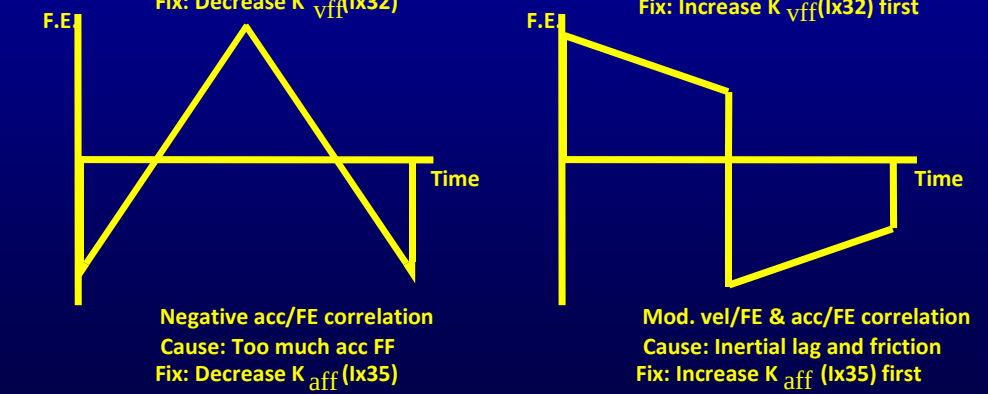


High acc/FE correlation
Cause: Physical system limitations
Fix: Use less sudden accel, or add
"Friction Feedforward" (Ix68)

Parabolic Move Tuning (continued)







PMAC Current Limit Parameters

Ix69: Instantaneous Current Limit
- Saturation limit, no error
- Works as velocity limit for velocity-mode drives

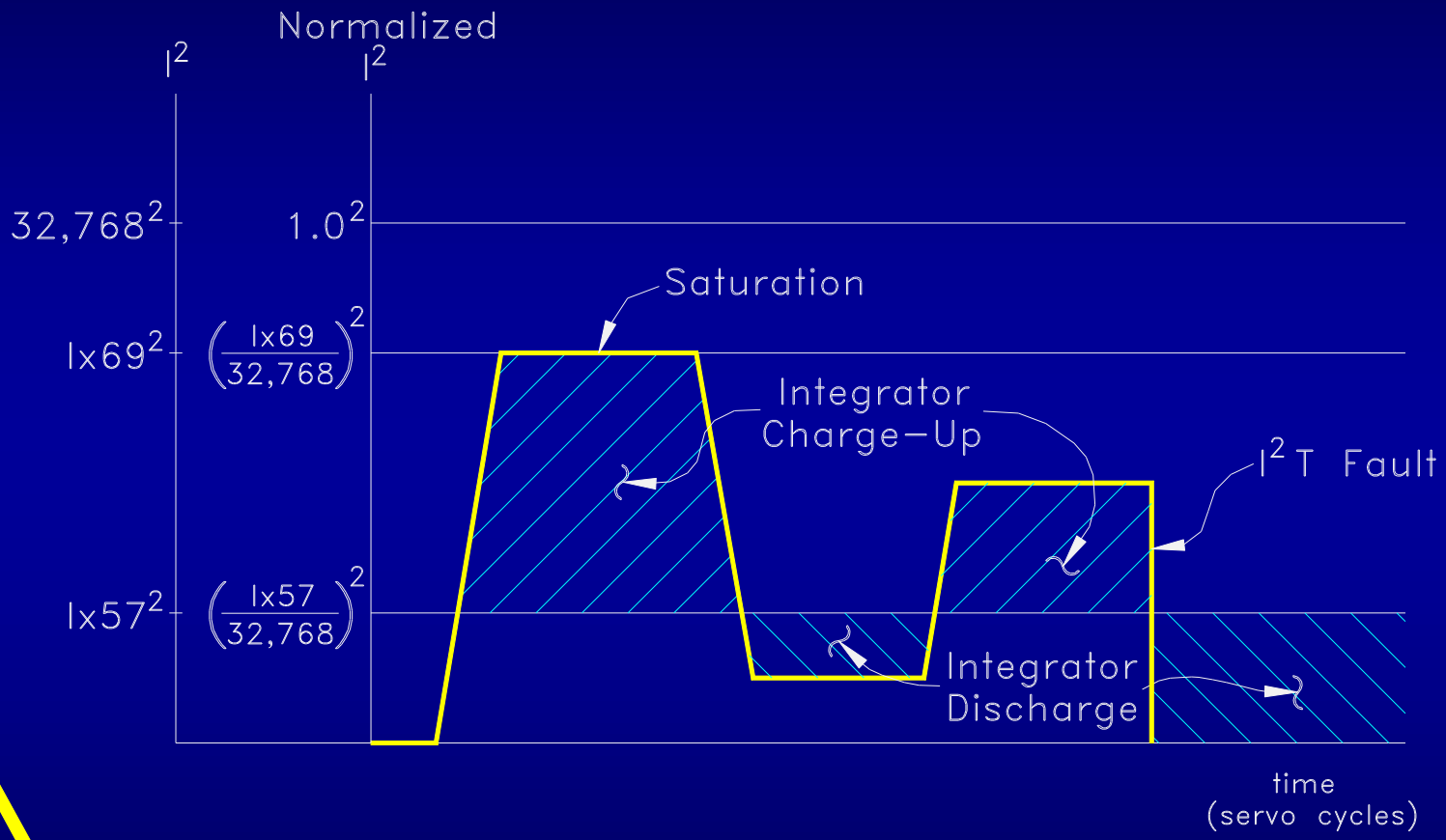
Ix57: Continuous Current Limit
- Current values over Ix57 increase current integrator
- Current values under Ix57 decrease current integrator
- Measured current used in direct PWM mode
- Commanded current used in other modes

Ix58: Current Integrator Limit

When current integrator exceeds Ix58:
- Motor killed as amplifier fault error
- "Integrated current fault" bit set also



PMAC I²T Protection Feature



Servo Loop Modifier Variables

Ix59: User-Written Servo/Phase Enable

=0: Use standard PID phase algorithms

=1: Use custom servo, standard phase algorithms

=2: Use standard PID, custom phase algorithms

=3: Use custom servo, phase algorithms

Ix60: Servo Cycle Extension Period

Loop closed every (Ix60+1) servo interrupts

Useful for slow, low-resolution axes

Useful for process control “axes”



Servo Loop Modifier Variables (continued)

Ix63: Integration Limit
Maximum integrated error accumulated
When negative, killed on saturation

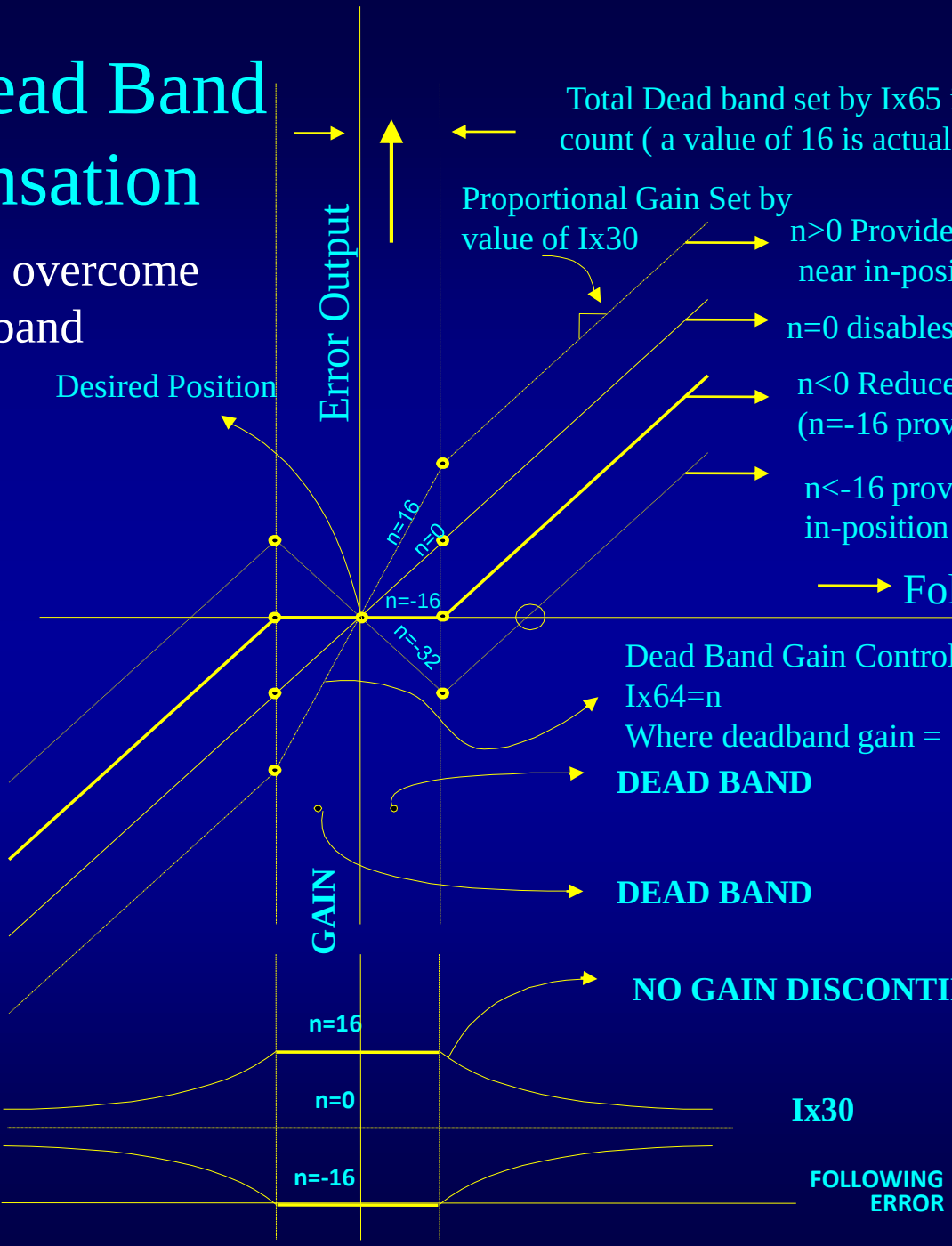
Ix64: “Deadband Gain”
Region of modified gain

Ix65: Deadband Size
Region of modified gain



PMAC Dead Band Compensation

To create or overcome dead band



Servo Loop Modifier Variables (continued)

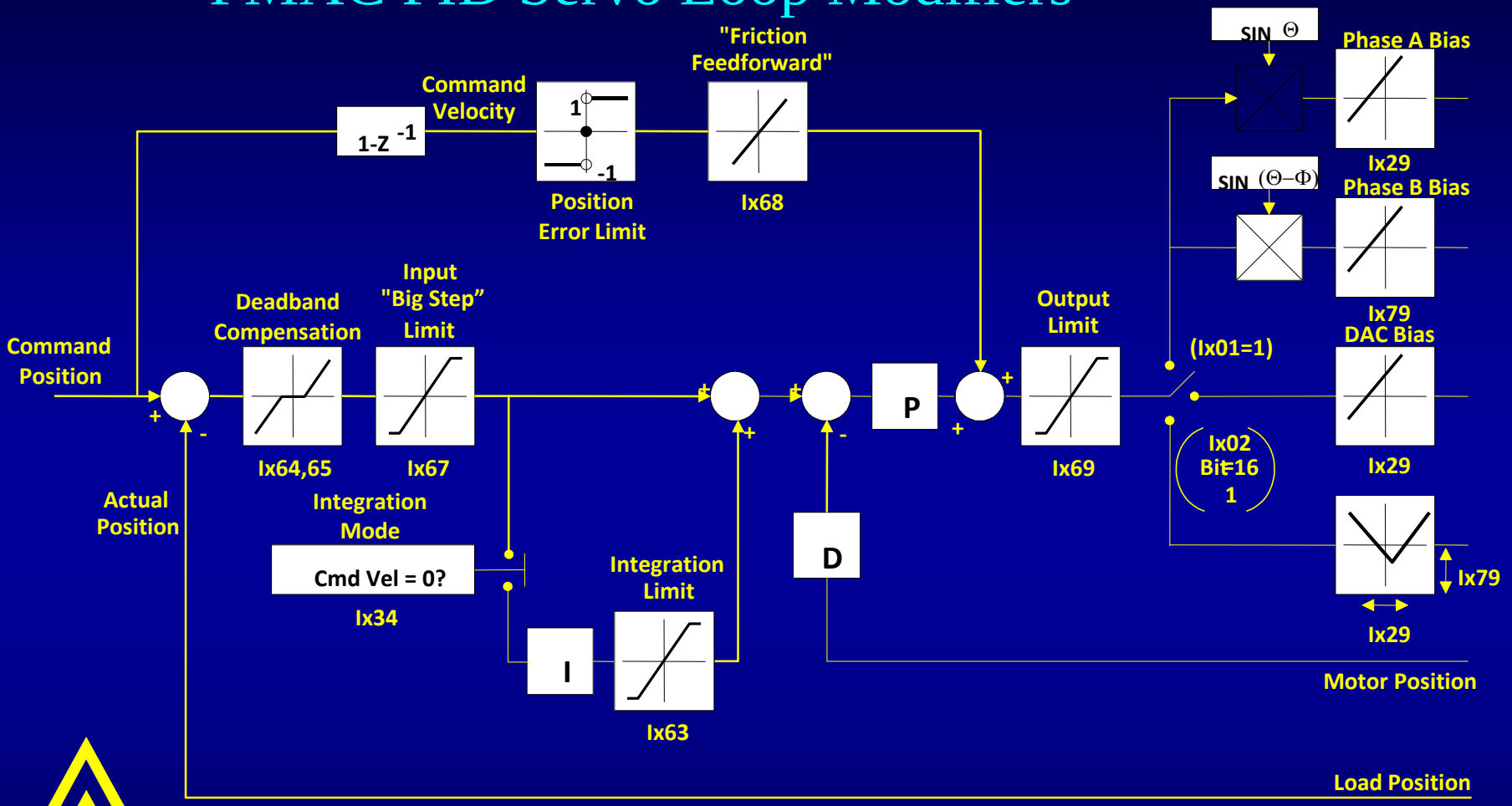
Ix67: Position Error Limit
Limits error that filter "sees"

Ix68: Direction-Sensitive Bias
"Friction Feedforward"

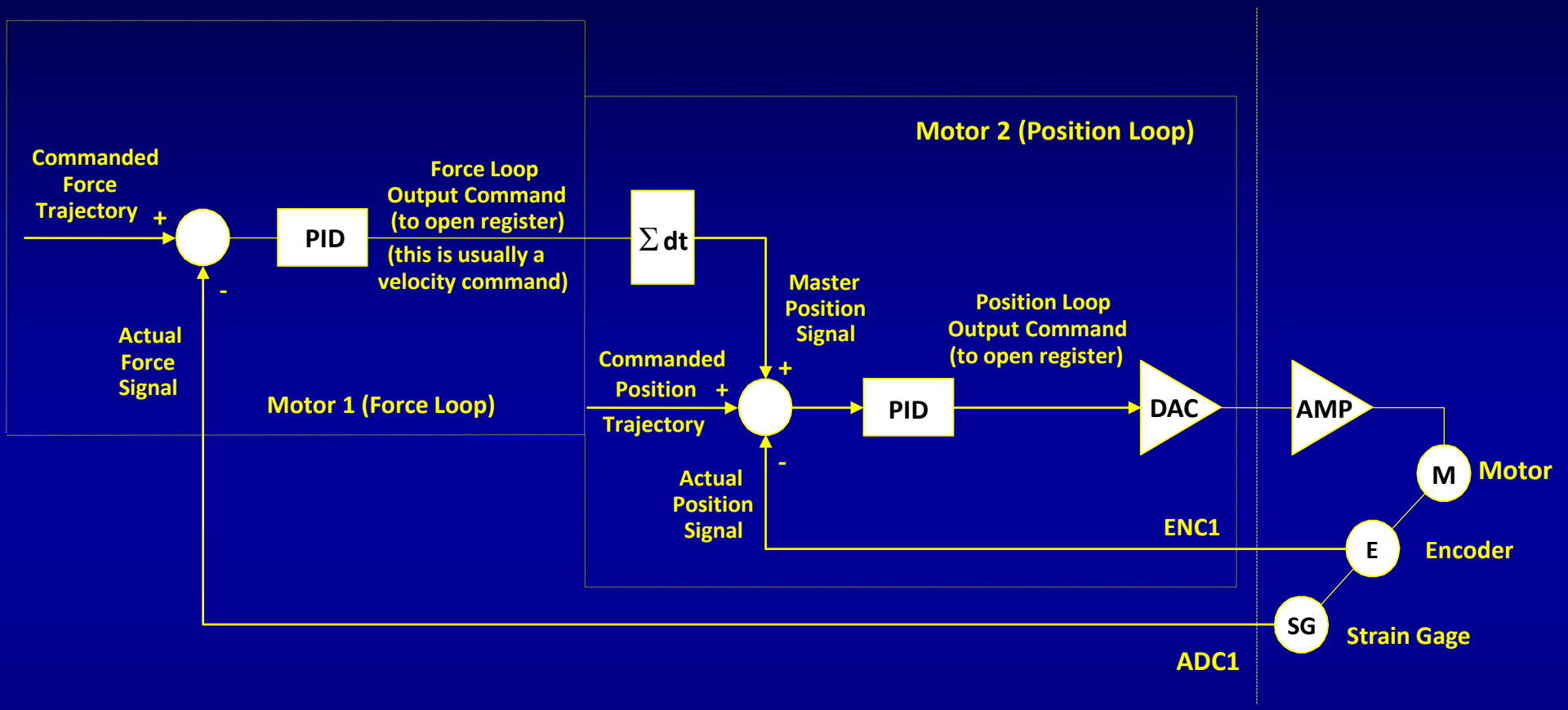
Ix69: Output Command (DAC) Limit
Limits output of filter
Acts as torque (current) limit if commutating



PMAC PID Servo Loop Modifiers



Closing a Force Loop Around a Position Loop



I102 = $07F0 (Open Y register $07F0)
I103, I104 = $072A (Converted ADC1)
I105 = $072B (Converted ADC2)

I202 = $C003 (DAC1)
I203, I204 = $0720 (ENC1)
I205 = $1072D to use integrated force command superimposed on position trajectory

Conversion Table to do integration Example
Set up: WY$072C, $5007F0,$0
Result in X:$072D

Conversion Table must process all encoders and ADCs that are used

为什么需要换向整流控制？

- **电机需要AC交流电流定相**
- **DC直流控制电机**
- **Commutation switches direction of current in motor phases to maintain desired torque direction**
- **Optionally, it also controls magnitude of current in motor phases to maintain desired torque level**
- **有刷电机机械换向**
- **无刷电机电子换向**



PMAC的整流控制?

- 不需要将位置反馈到驱动器上
- 驱动器更简单经济
- 高效能的正弦整流算法
- 同步电动机使用增量编码器信号整流
- 感应电机由伺服驱动器做矢量控制
- 反馈丢失或反向会引起转矩减小,而不会引起飞车



电机的电枢和励磁

- 电枢
  - 大电流（快速变化）
  - 在有刷电机的转子上
  - 在无刷电机的定子上
- 励磁绕组
  - 小电流（ 慢速变化）
  - 在有刷电机的定子上
  - 在无刷电机的转子上



整流: 转子励磁控制

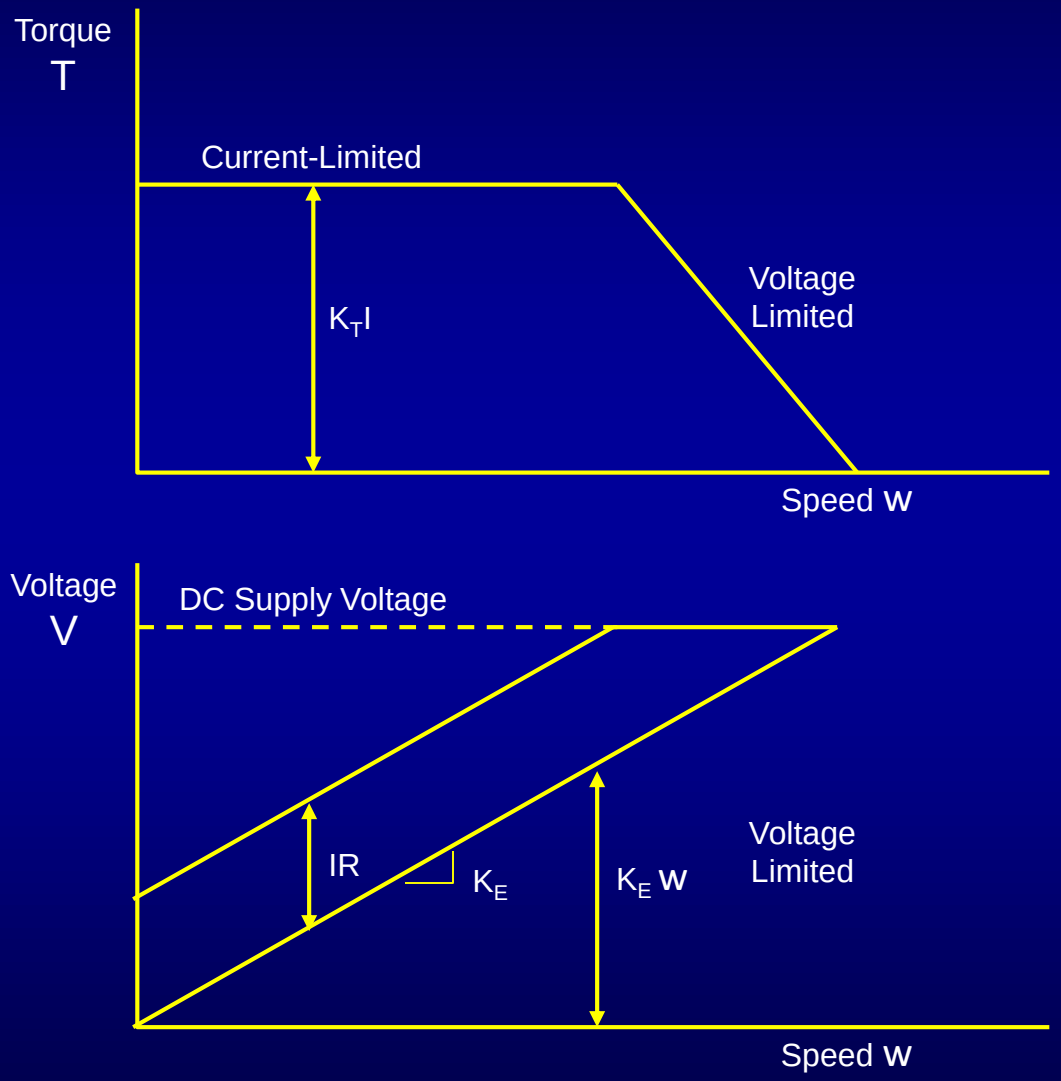
- 电机励磁有两种方式:
  - 永磁体: aligned electron “spin”
  - Electromagnet: circulating electric current
- Rotor magnetic field strength (λ_R) determines:
  - Torque constant K_T : ($T=K_T I_A$)
  - Back EMF constant K_E : ($E=K_E \omega$)

(Note: $EI=T\omega$, so $K_T=K_E$)

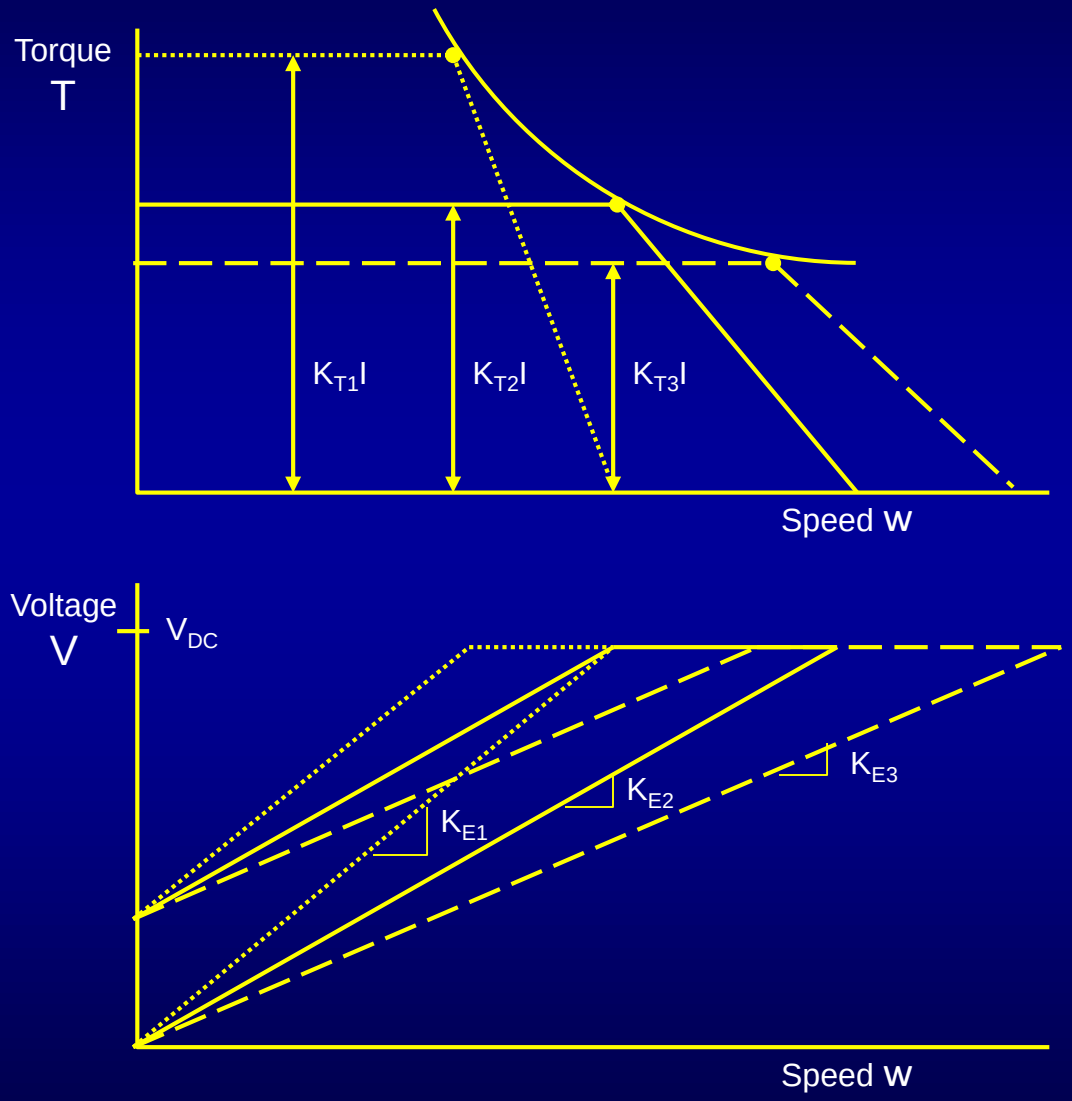
- Electromagnetic rotor field strength can be varied dynamically
  - Strong field for high-torque at low speed
  - Weak field for low back EMF at high speed
  - Technique is called “field weakening”
  - Creates “software transmission”



Motor Torque Capabilities – Constant Field



Motor Torque Capabilities – Varying Field

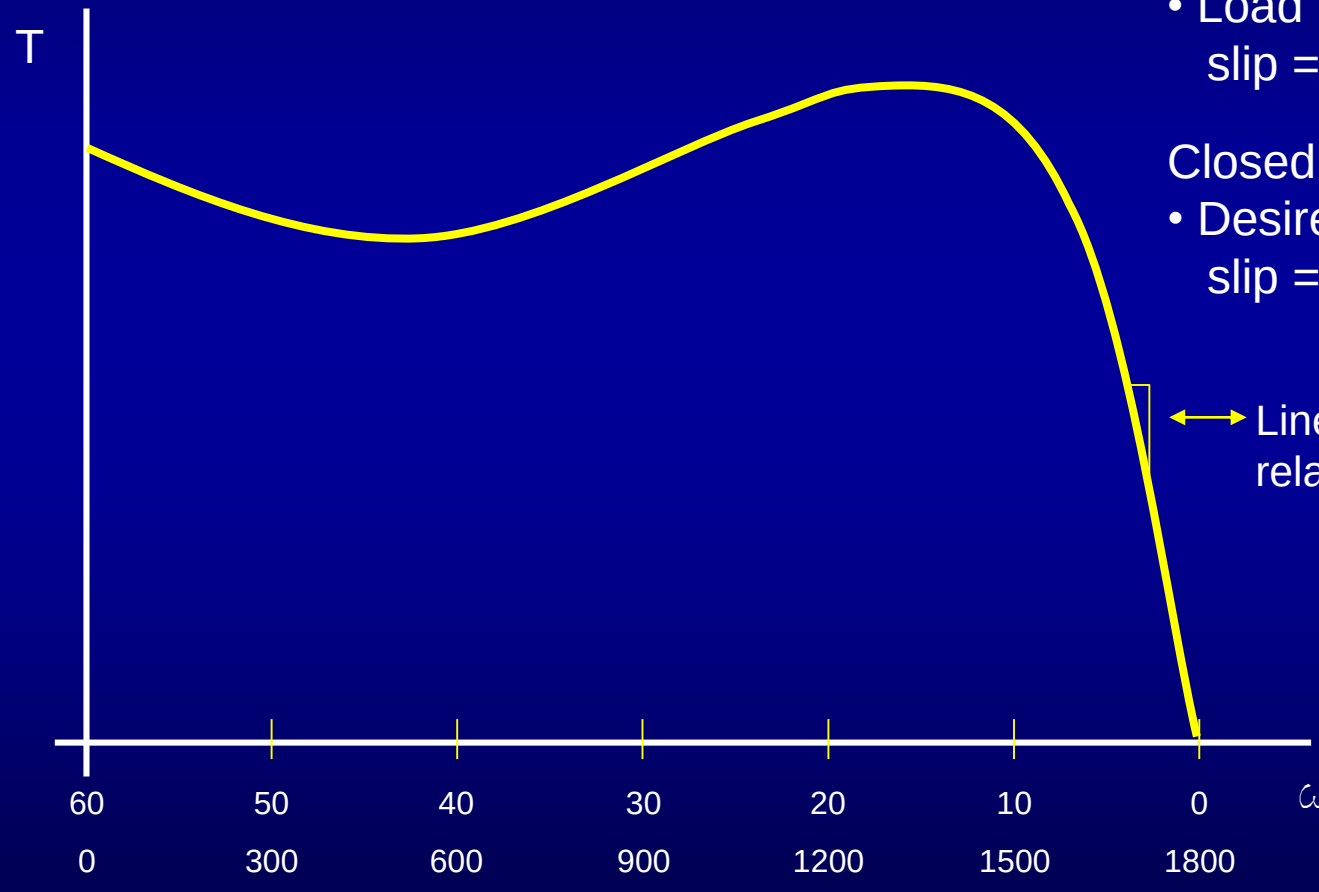


What Is Vector Control?

- **Commutation algorithm applied to induction motor provides positioning servo motor capability**
- **AKA "flux vector control", "field oriented control"**
- **Two issues not present in permanent magnet brushless motors:**
  - **Must induce rotor current for electromagnetic field**
  - **Must create "slip" between rotor field and rotor mechanics**
- **Rotor field strength dynamically controllable by "magnetization current" parameter**
- **"Slip gain" parameter must be set to match motor dynamics**



Induction Motor Principle of Operation



Open loop (ω_e=const):
- Load torque => decel => slip => motor torque

Closed loop:
- Desired Motor torque => slip => motor torque

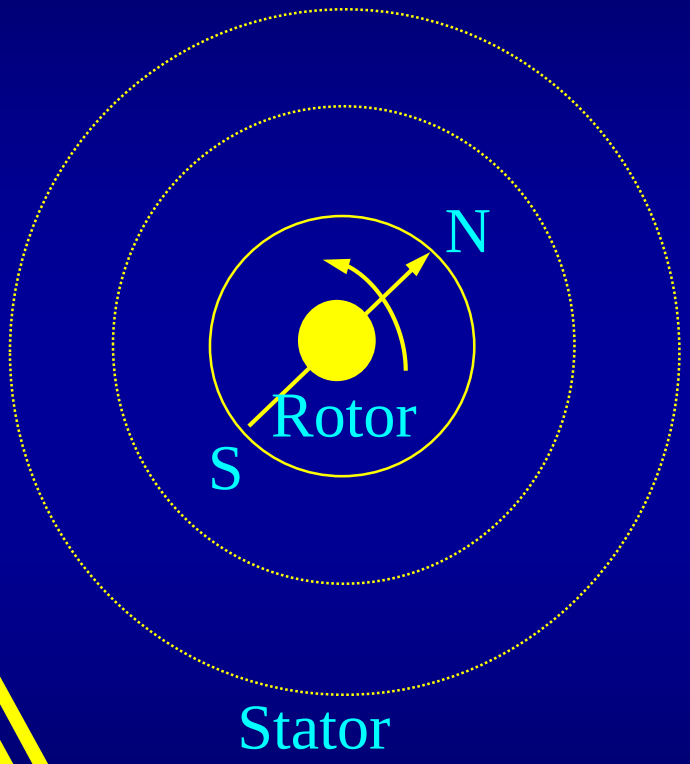
↔ Linear torque/slip relationship

为什么要用矢量控制?

- **Induction motors are a substantial cost benefit, especially in large sizes**
- **Induction motors are available in very large sizes**
- **Induction motors are very rugged**
- **Induction motors cannot be demagnetized**
- **Technique eliminates most possible runaway conditions**
- **Field strength control permits flexible torque/speed range**



PMAC Commutation Algorithm



I. Compute rotor field orientation

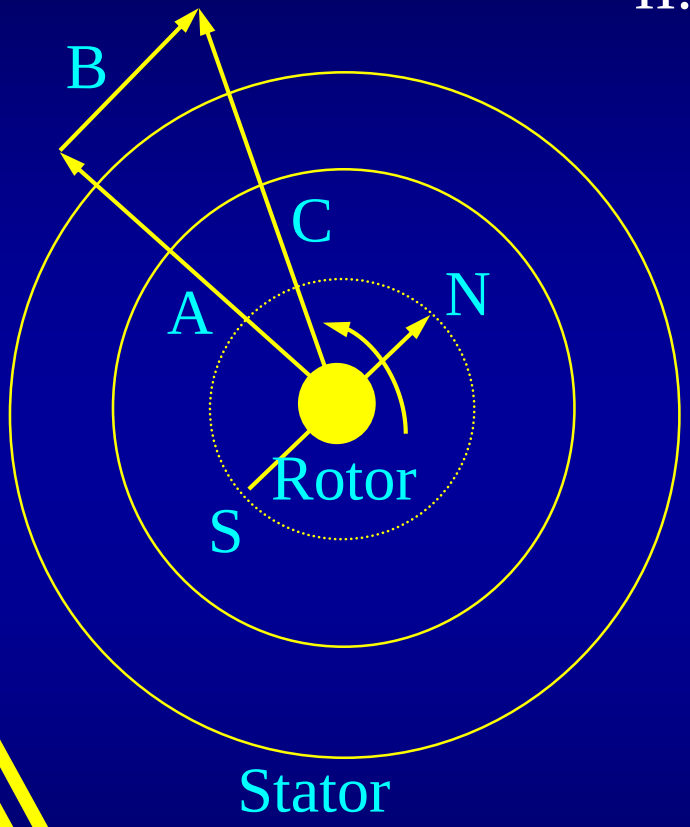
A. Synchronous motor

1. Read encoder position
2. Compare to reference position (power-on or absolute)

B. Asynchronous motor

1. Read encoder position
2. Compute slip frequency
3. Add slip advance to rotor position

PMAC Commutation Algorithm



II. 分配定子电流

A. Torque command (“quadrature”) current
Magnitude from servo filter or open loop
Aligned perpendicular to rotor field

B. Magnetization (“direct”) current
Used mainly for induction motors
Aligned parallel to rotor field

C. Components added vectorially

PMAC Commutation Algorithm



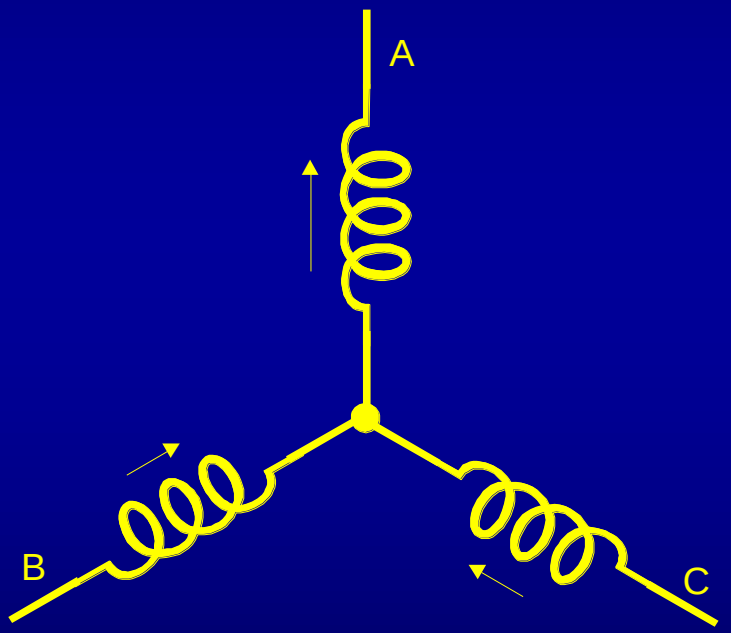
III. Align Current to Phases

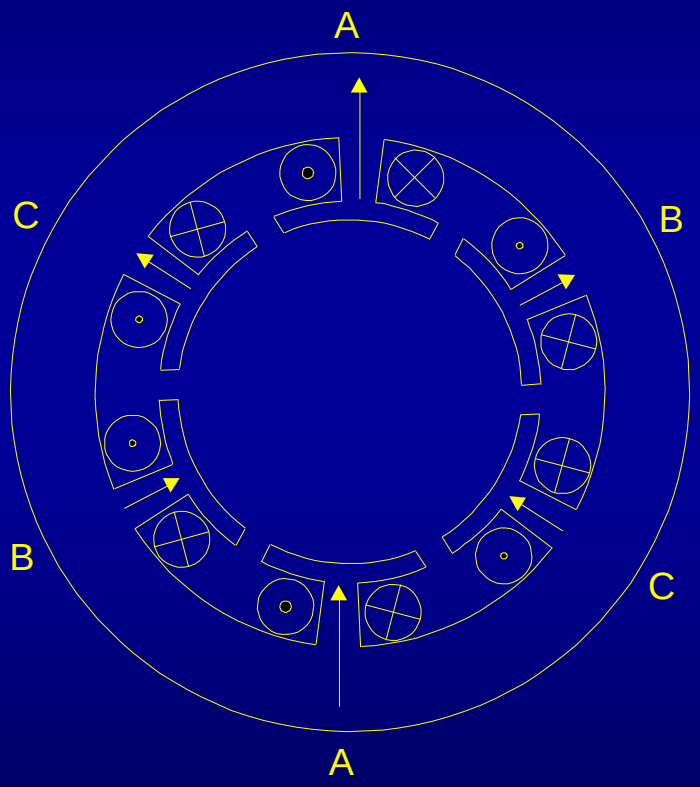
A. Graphically, drop perpendicular to each phase line from net current vector

B. Mathematically, take dot product of net current vector and each unit phase vector

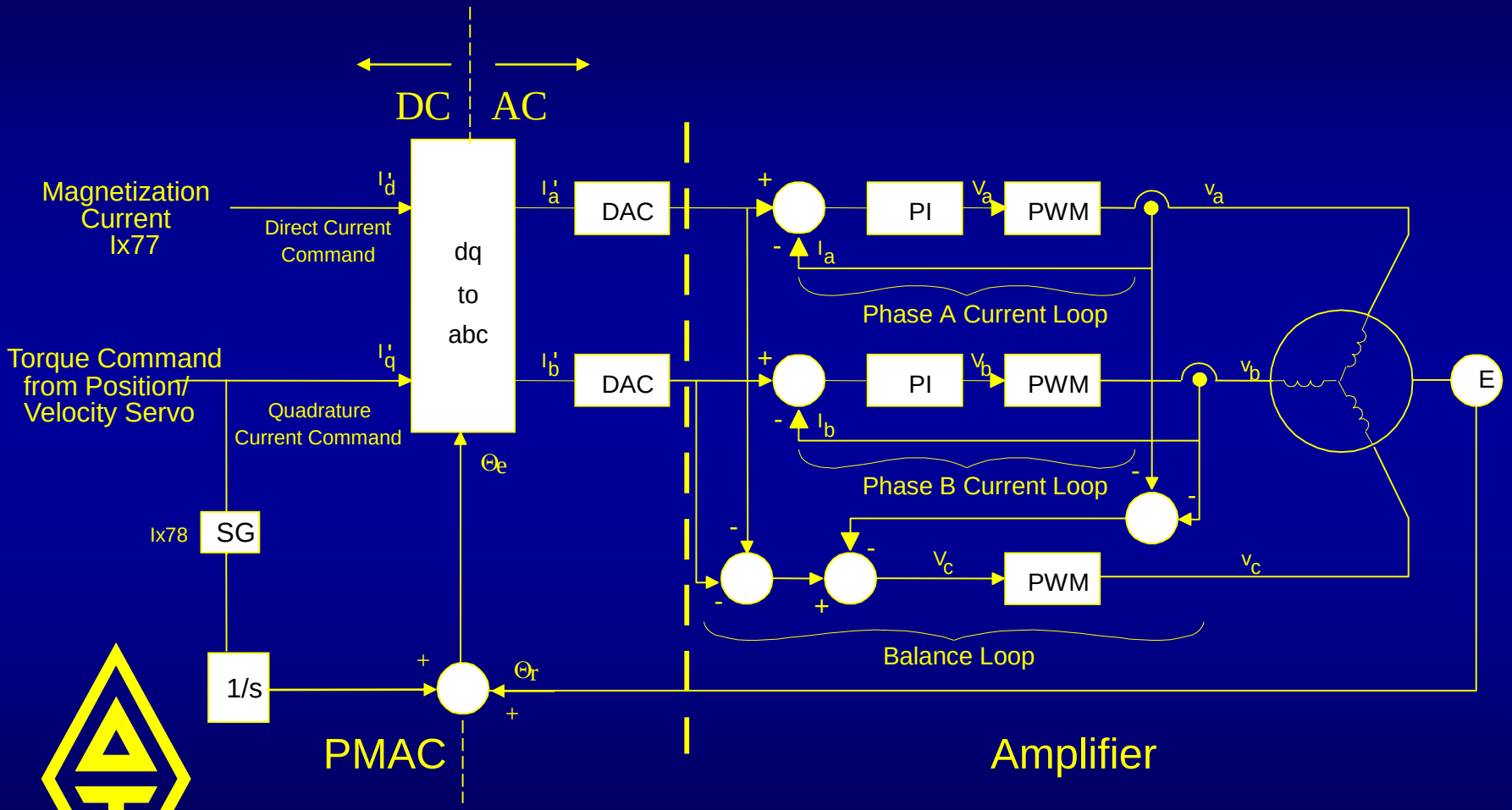
C. Only compute 2 phases; 3rd (&4th) phase(s) can be generated by balance loop

Relationship of Winding Diagram to Actual Motor Winding

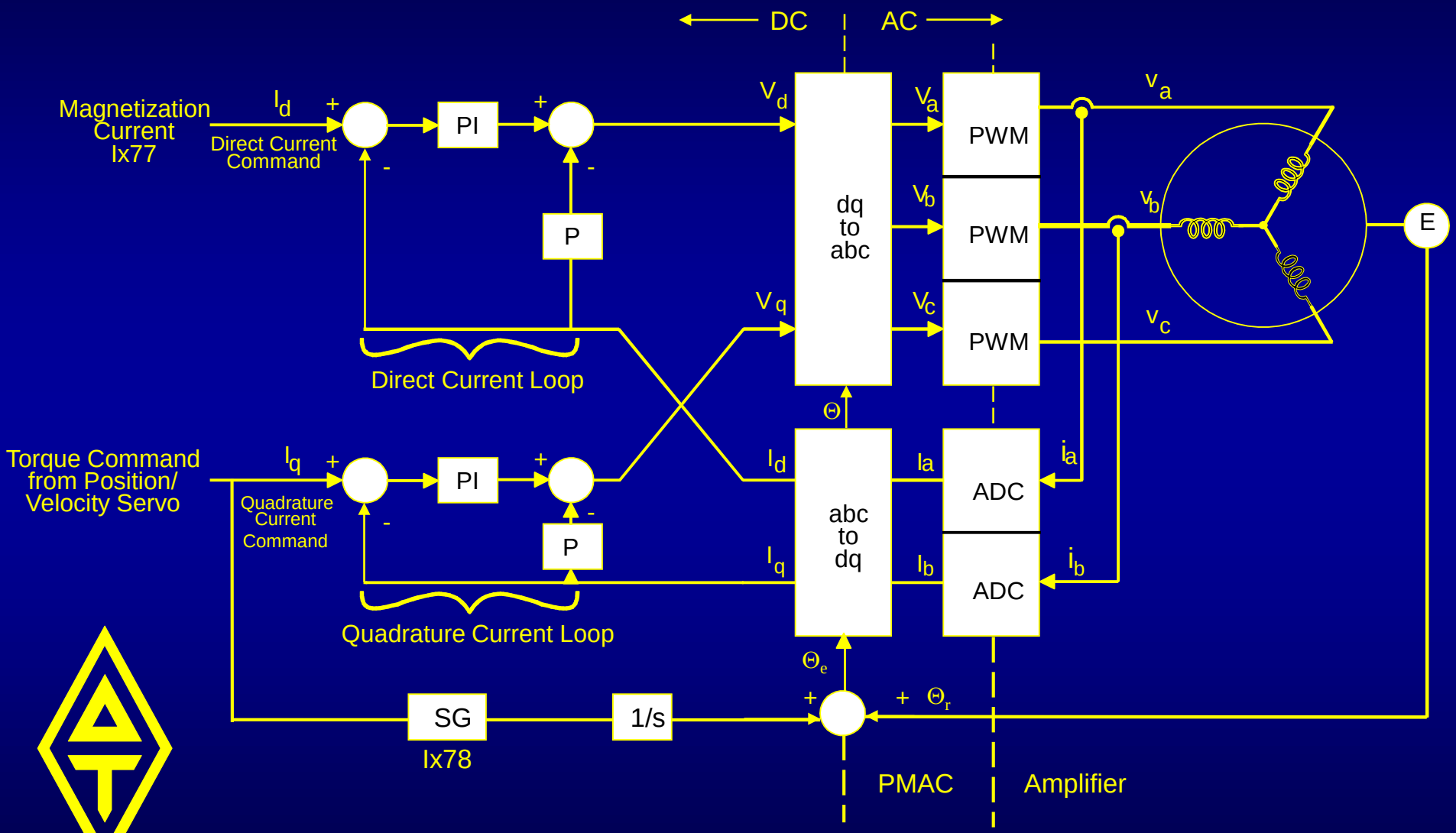




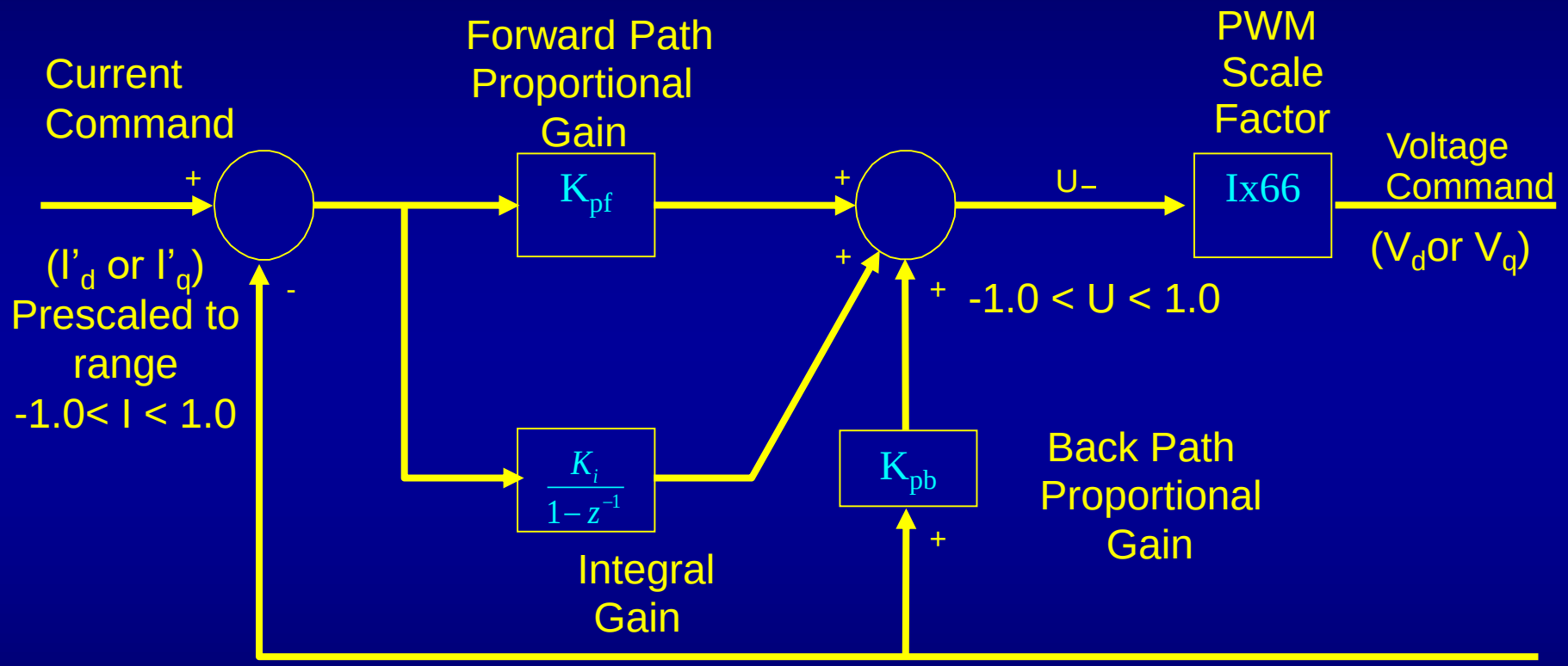
PMAC/PMAC2 Commutation with Analog Current Loop



PMAC2 Commutation with Digital Current Loop



PMAC2 数字电流环

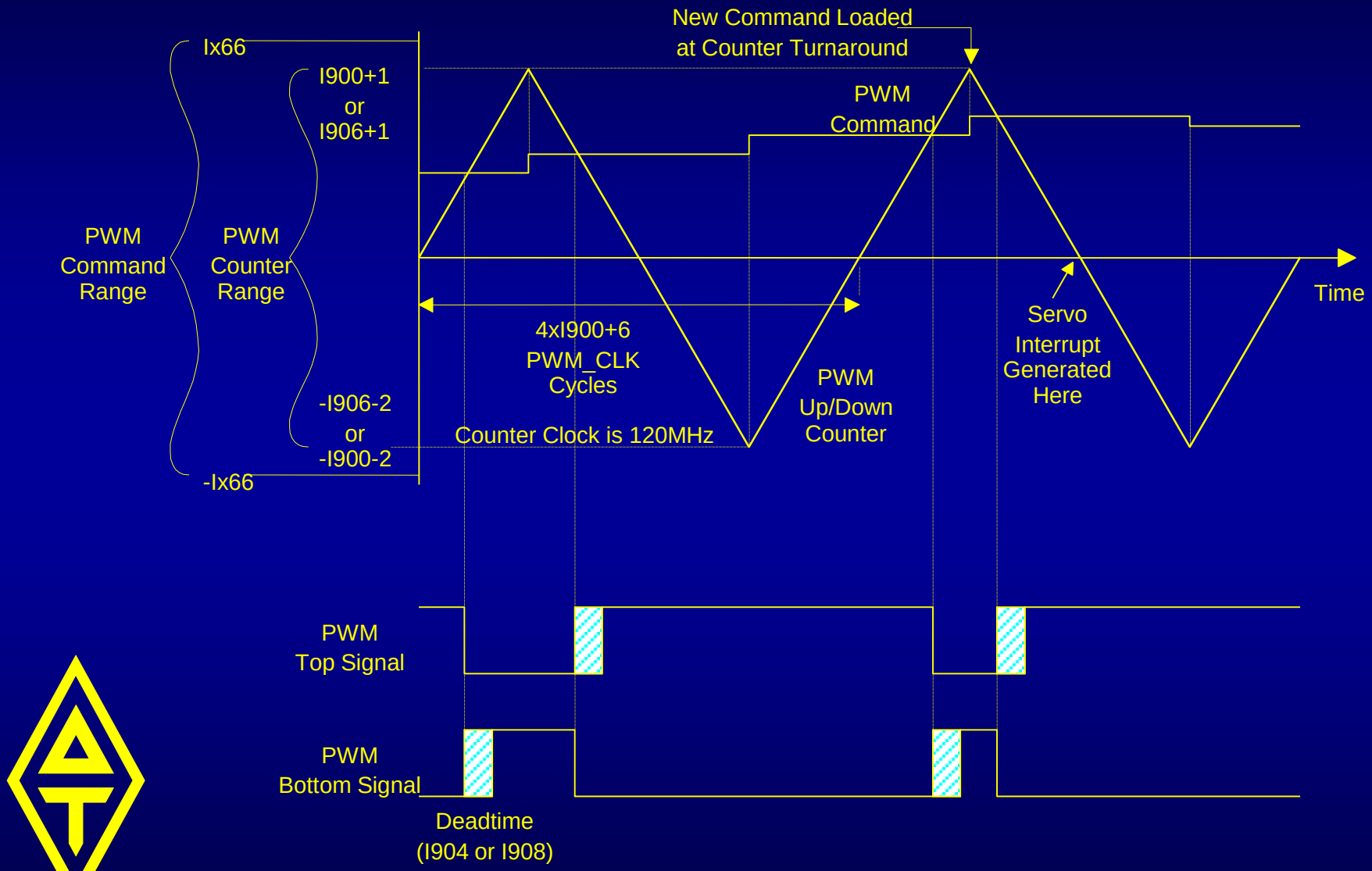


K_{pb}= Ix76•4 (0.0 <Ix76 <1.0)

K_{pf}=Ix62•4 (0.0< Ix62<1.0)

K_i=Ix61•8 (0.0< Ix61 <1.0)

PMAC2 Digital PWM Generation (单相)



Why Digital Current Loop in PMAC?

- All calculations in same processor minimizes delays
  - Higher gains possible
  - Greater stiffness, bandwidth, acceleration
- DC (field frame) loop closure
  - Minimizes high frequency problems
  - Permits higher maximum velocity
- No control functions in drive
  - No pot tweaking or personality modules
  - No “laptop tuning”
  - Drive selected for power level only



Motor x Commutation Variables

Ix70: Number of Commutation Cycles
Ix71: Counts per Ix70 Commutation Cycles
Counts/Electrical Cycle=Ix71/Ix70

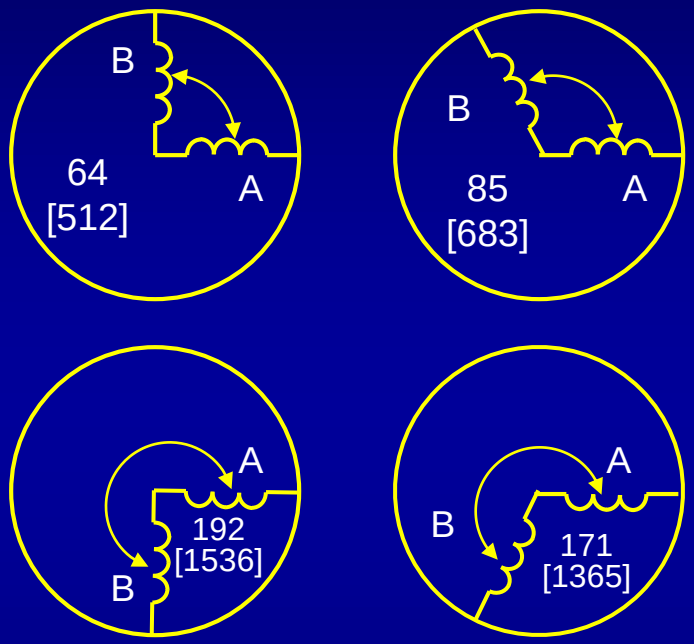
EXAMPLE: **1 pole pair, 4096 cts/mech rev**
=>Ix71/Ix70 = 4096/1

2 pole pairs, 4096 cts/mech rev
=>Ix71/Ix70 = 4096/2 = 2048/1

3 pole pairs, 4096 cts/mech rev
=>Ix71/Ix70 = 4096/3



Motor x Commutation Variables (continued)



Ix72: Commutation Phase Angle

Angle between A & B phases

Non-Turbo: 1/256 commutation cycle
[Turbo: 1/2048 commutation cycle

3 phases: 85 or 171 [683 or 1365]

2 or 4 phases: 64 or 192 [512 or 1536]

Phase Referencing I-Variables

- For synchronous motors with incremental sensors only

  Ix73: Phasing Search Command Output
  Magnitude of searching effort

  Ix74: Phasing Search Time
  Duration of search

- For synchronous motors with absolute sensors

  Ix81: Power-On Commutation Feedback Address & Mode
  [Ixx81, Ixx91] Location and format of absolute sensor

  Ix75: Phasing Offset (units: counts)
  Difference between sensor zero and commutation zero

Induction Motor Setup Variables

Ix77: Magnetization Current

Sets rotor field strength (&K_T, K_E)

Required for induction motor field

Can be used on top of permanent magnet field

Ix78: Slip Gain

Relationship between commanded torque and slip frequency

Must be zero for synchronous motors



Ix79: 2nd Phase Bias (DAC bits)
[Ix29: 1st Phase Bias (DAC bits)]

Ix80: Power-on Mode (whether commutated or not)
0,2= not enabled, requires enabling command
1,3= enabled immediately
0,1= "Two-Guess" phasing search
2,3= "Stepper-Motor" phasing search

Ix82: Current-Loop Feedback Address [PMAC2 only]
If zero, current loop not closed in PMAC

Ix83: Ongoing Commutation Feedback Address



Phase Referencing of Brushless Motors

I. Power-On Phasing Search

Ix73 controls magnitude; Ix74 controls time

A. Two Guess Method (Ix80=0,1)
- Arbitrary guess of phase reference
- Open loop command, measure acceleration
- Change guess 90^0, repeat
- Calculate phase reference from accelerations

B. Stepper Motor Method (Ix80=2,3)
- Drive like stepping motor with phase bias terms
- Bias terms control phase current magnitude
- Drive to zero position in phase cycle in two steps
- Force phase angle register to zero



Phase Referencing of Brushless Motors (continued)

II. Absolute Power-On Referencing

A. Phase Search on Assembly
- Use second search method above (end at zero position of phase cycle)
- Read absolute sensor at this position
- Set Ix75=-Pos/Ix70 (offset term)

B. No-Movement Referencing at Power-On
- PMAC automatically reads absolute sensor to do phase referencing at power-on
- Ix81 specifies address of sensor
- Ix75 added to sensor position to get phase position



PMAC Phase Referencing Methods

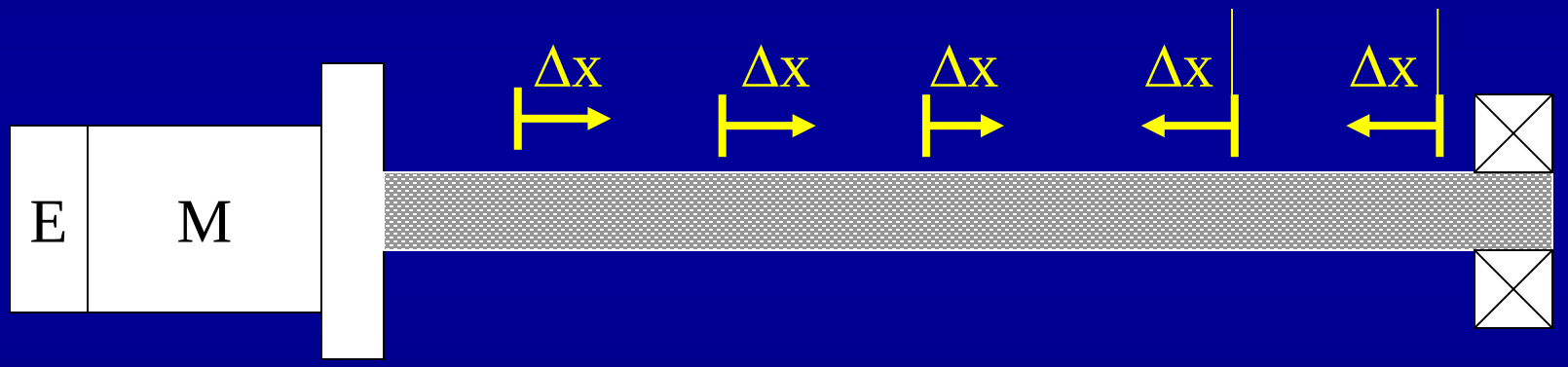
Method	Repeatability	Correction Required	Notes
Hall Sensor Read	+/- 30° e	Yes	Correct on 1st Hall edge or index
Resolver Read	+/- 3 cts	No	Reference established by stepper or symmetry search
Abs Encoder Read	+/- 1 ct	No	Reference established by stepper or symmetry search
Two Guess Search	+/- 5-10° e	Maybe	Requires low external loads Slight movement
Stepper Search	+/- 1-2° e	Maybe	Up to 1/2-cycle movement
Symmetry Search	+/- 1 ct	No	"Fine Phasing" - Not feasible in actual applications; only to establish reference

PMAC 补偿表

标准螺距补偿

e.g. Δx=f(x)

- **Get linear encoder accuracy (almost!) with rotary encoder**
- **Characterize system with linear sensor**
- **Enter errors into PMAC**



DEFINE COMP 200, #1, #1, 100000

Table Length
源电机

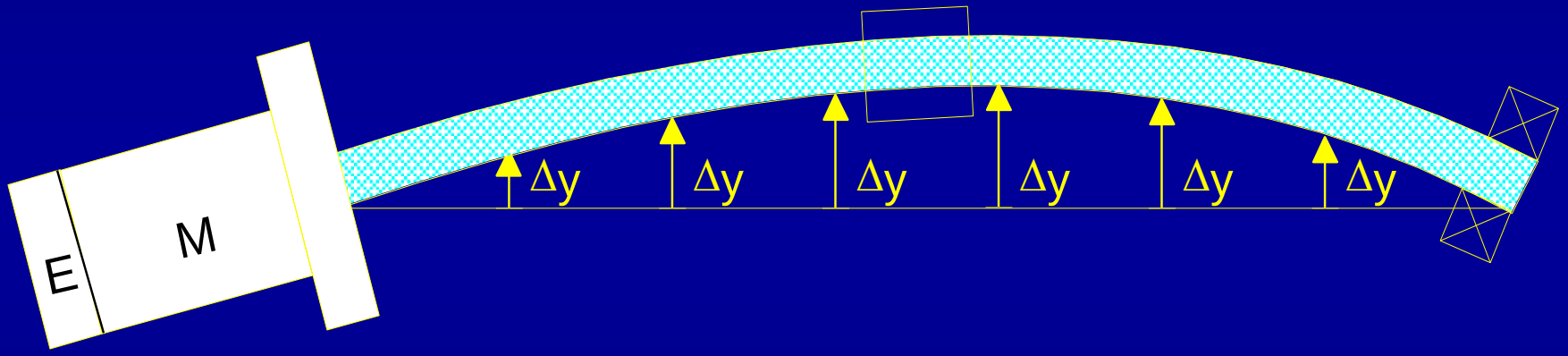
Table Span in Counts
目标电机

PMAC Compensation Tables

Cross-axis compensation

e.g. Δy=f(x)

- **Useful for bowed leadscrews**
- **Can be used to build electronic cam tables**



DEFINE COMP 100, #1, #2, 100000

Table Length

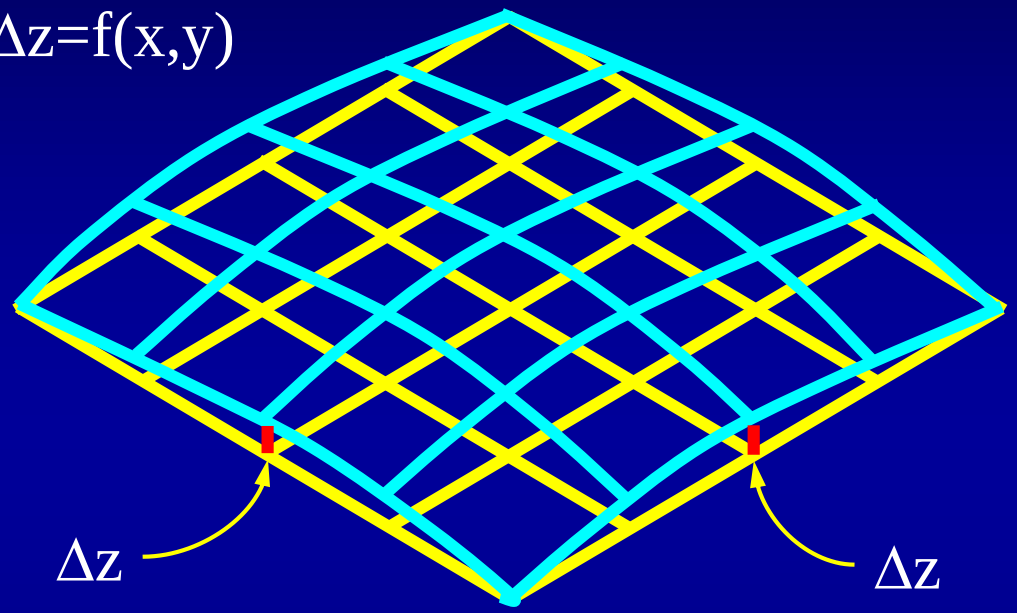
Source Motor

Table Span in Counts

Target Motor

PMAC 补偿表
2D (平面) 转换表:

e.g. Δz=f(x,y)



DEFINE COMP 20. 15, #1, #2, #3, 20000, 15000

Table columns
Table rows
1st source motor
2nd source motor

2nd motor span in counts
1st motor span in counts
Target motor

PMAC Compensation Tables
Used for laser marking systems

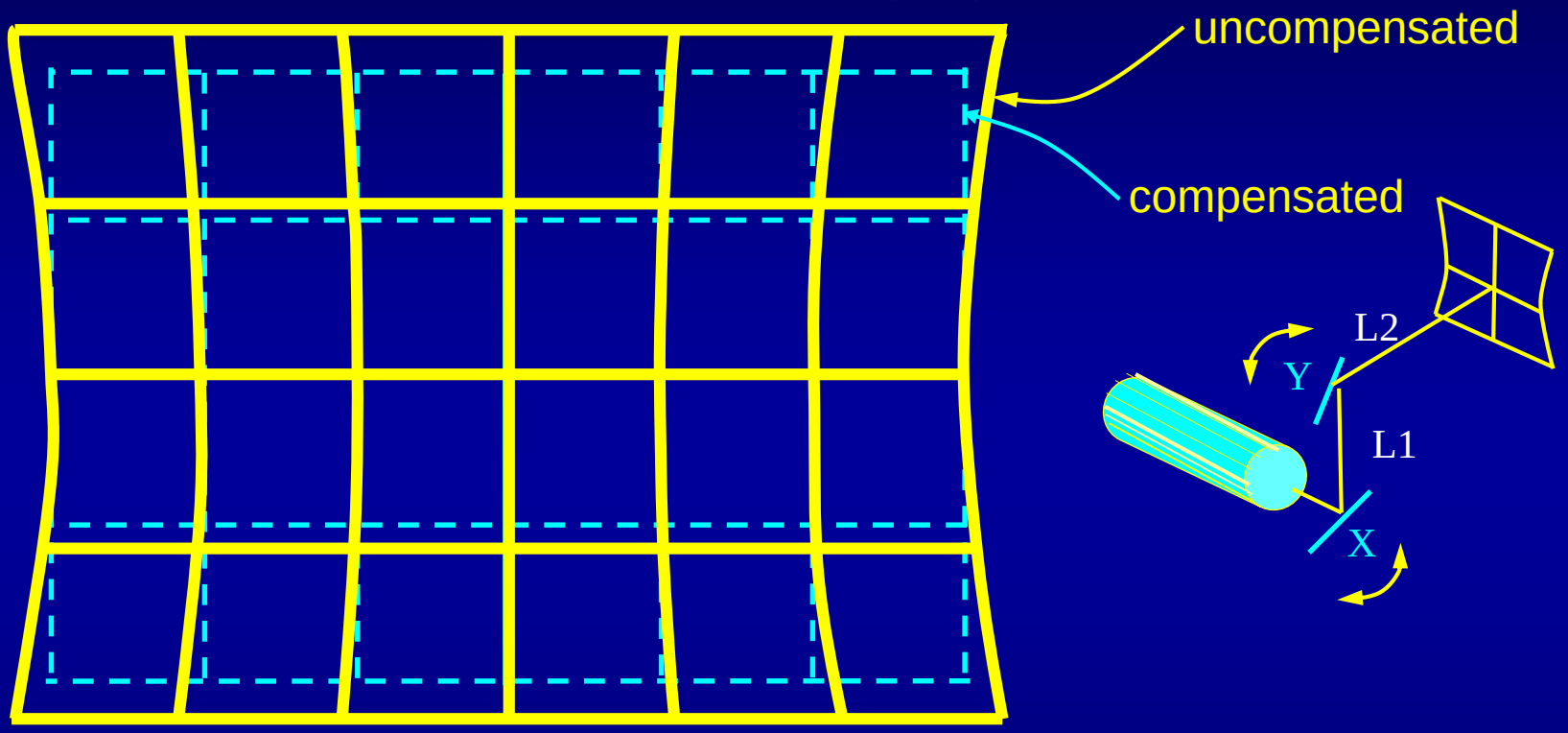


Table 1: Δy=f(y)=arctan(y/L2)-(y/L2)

Table 2: $\Delta x=f(x,y)=\arctan[x/(L1+\sqrt{L2^2-y^2})]-[x/(L1+L2)]$

PMAC Motor Backlash Parameters

Ix86: Backlash Size (1/16 count)
Added {hidden} position change on reversal

Ix85: Backlash Take-Up Rate (1/16 count/background cycle)
Rate of change of backlash on reversal
Set as high as possible without rough transition

I99: Backlash Hysteresis (1/16 count) (global)
[Ixx87] Required size of reversal before backlash change
Set non-zero to prevent dithering



PMAC Motor Backlash Tables

#x DEFINE BLCOMP {#entries}, {count span}

- Position-dependent backlash
- Added to Ix86 {constant} backlash
- Table backlash is zero at motor zero
- Used with COMP table for bidirectional compensation



Coordinate System Concept

- A coordinate system is a set of motors grouped together to achieve coordinated motion
- Motors requiring independent motion should be in separate coordinate systems
- Coordinate systems execute motion programs
- Motors assigned to coordinate system by “axis definition statement”



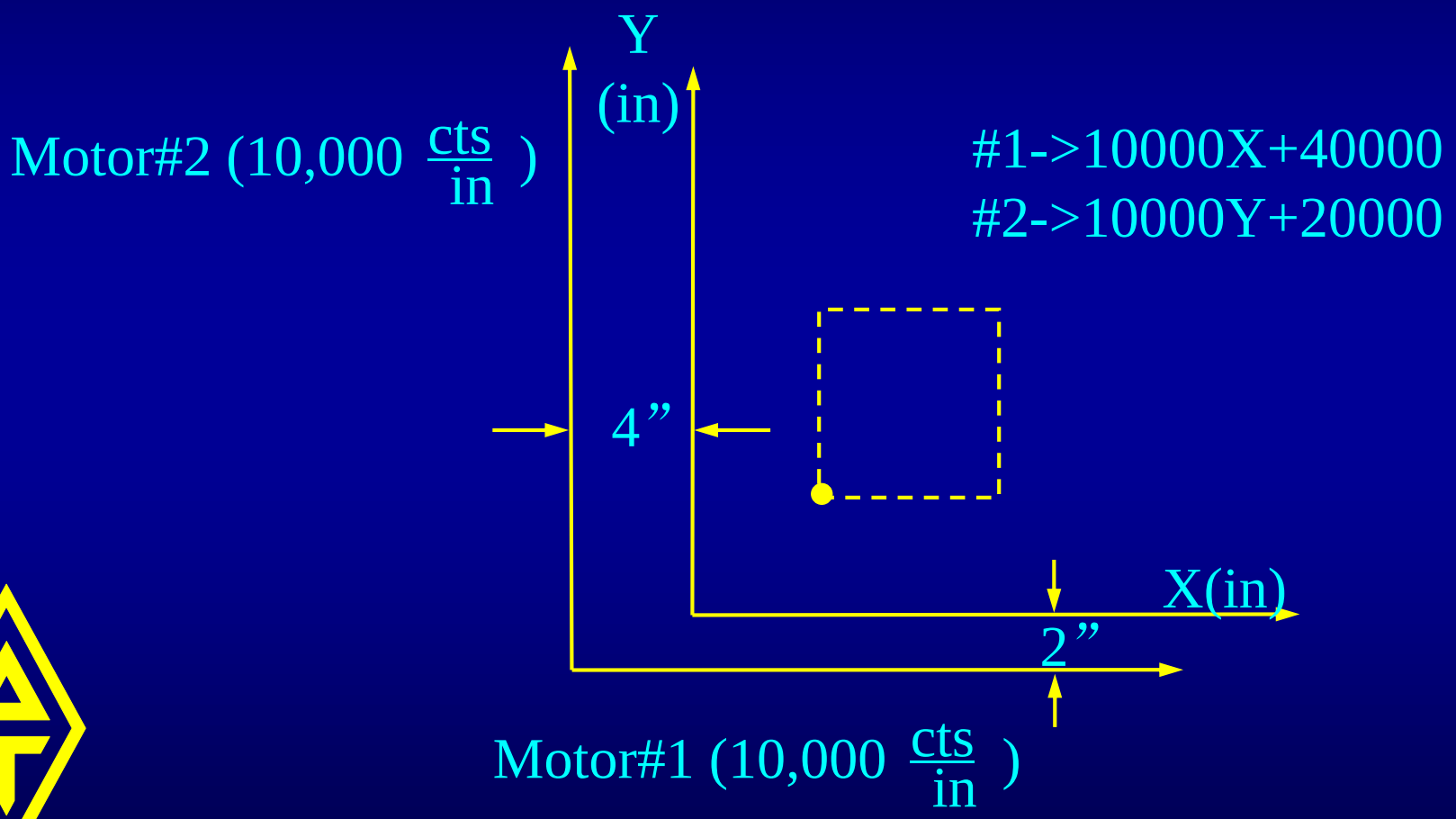
PMAC Axis Definition Statement

&1	{addresses C.S.}
#1->X	{mapping of motor to axis in C.S.}
#1->10000X	{scale factor defines X user units}
#1->10000X+5000	{fixed offset between motor and axis zero points}



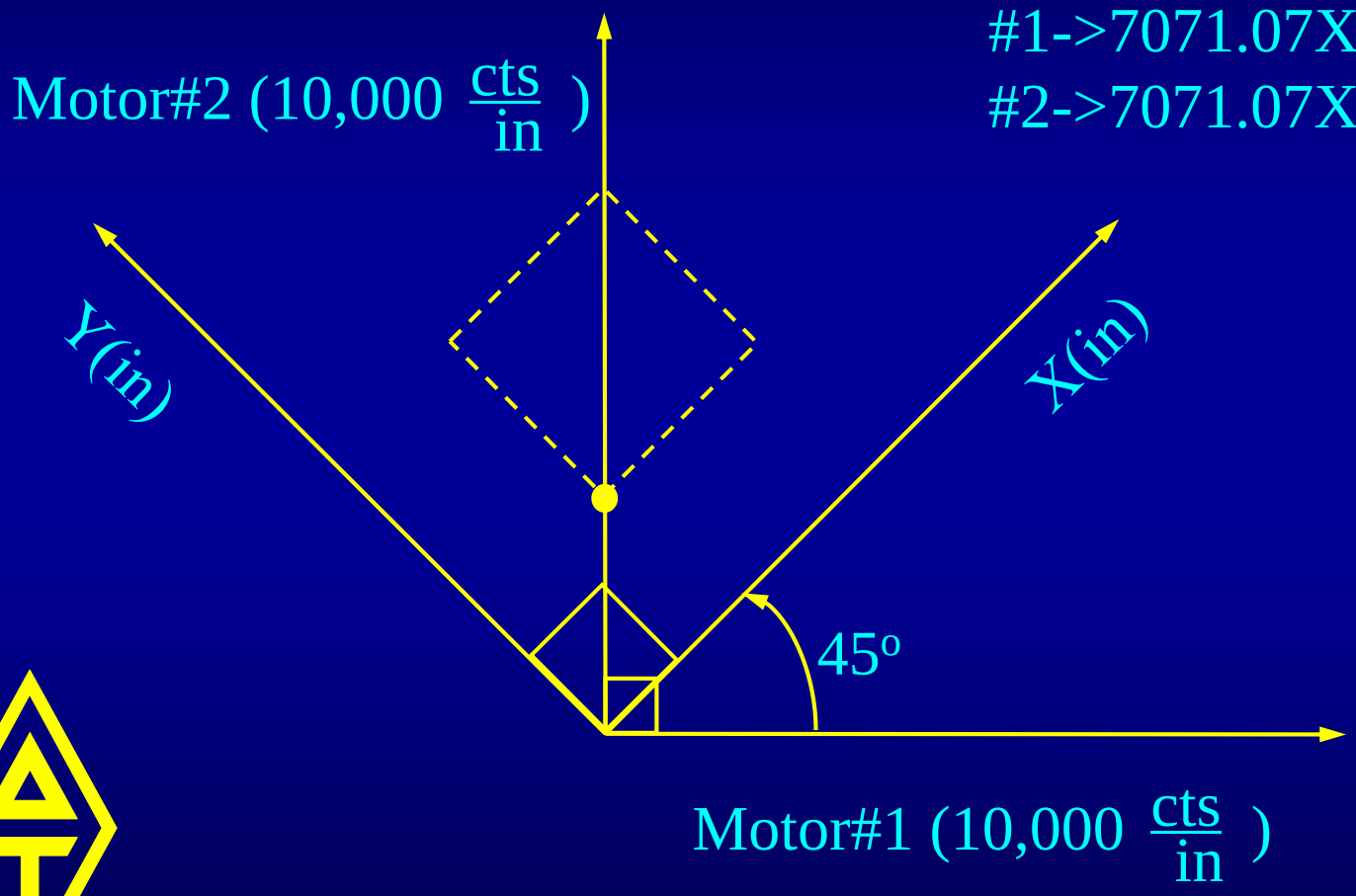
PMAC Coordinate Definition

Scaling and Translation



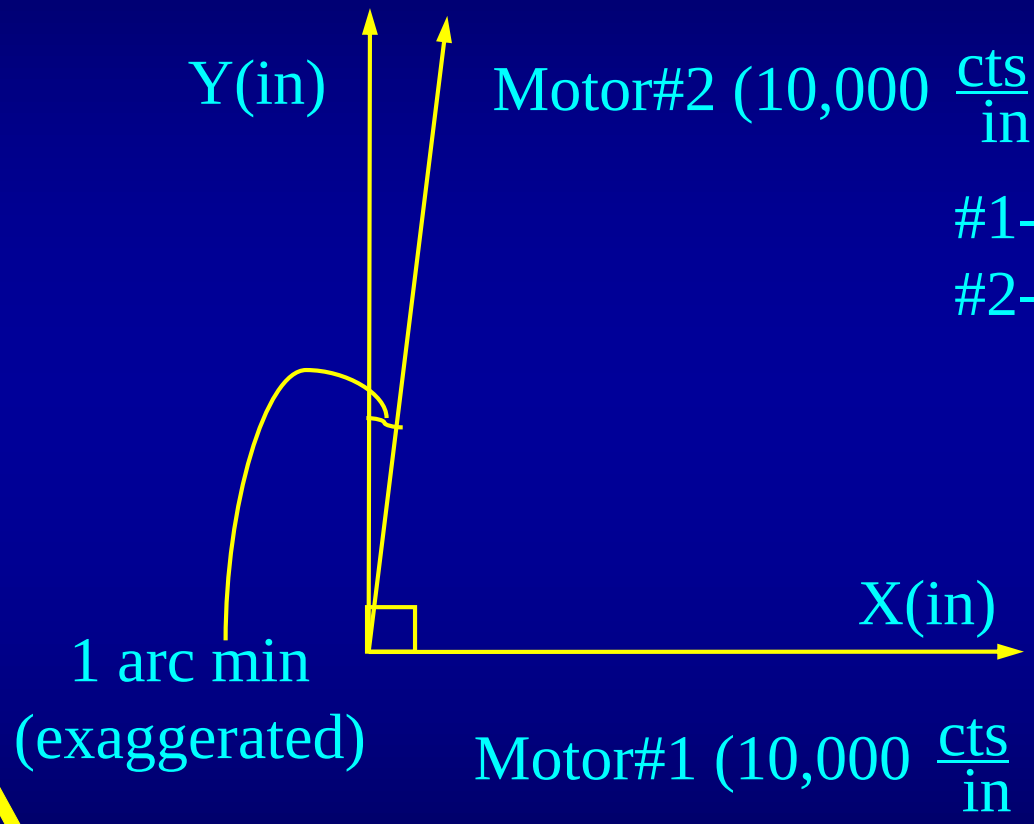
PMAC Coordinate Definition (continued)

Scaling and Rotation



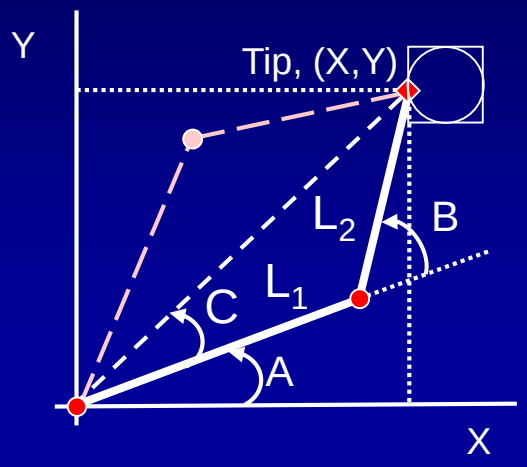
PMAC Coordinate Definition (continued)

Orthogonality Correction



Kinematic Calculations with Turbo PMAC

Allows you to Program Tip Motion Directly



Forward Kinematics

$$X = L_1\cos(A) + L_2\cos(A+B)$$

$$Y = L_1\sin(A) + L_2\sin(A+B)$$

Calculates tip position Based on A, B, L_1 & L_2

Inverse Kinematics

$$B = +\cos^{-1}\left(\frac{X^2+Y^2-L_1^2-L_2^2}{2L_1L_2}\right)$$

$$A+C = \text{atan } 2(Y,X)$$

$$C = +\cos^{-1}\left(\frac{X^2+Y^2+L_1^2-L_2^2}{2L_1\sqrt{X_2+Y_2}}\right)$$

$$A = (A+C) - C$$

Calculates A & B Angles Based on X & Y Tip Position

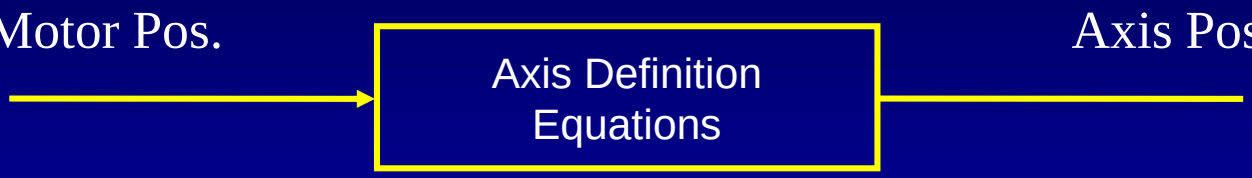
Kinematic Subroutines in Turbo PMAC

&1
OPEN FORWARD
CLEAR
Xaxispos = f_1 (Motor1pos, Motor2pos)
Yaxispos = f_2 (MotorIpos, Motor2pos)
CLOSE

OPEN INVERSE
CLEAR
Motor1pos = g_1 (Xaxispos, Yaxispos)
Motor2pos = g_2 (Xaxispos, Yaxispos)
CLOSE

MotorNPos is stored in variable PN
A,B,C,U,V,W,X,Y, Zaxispos stored in variable Q1-Q9 for C.S.

Motor-to-Axis Conversion Without Forward-Kinematic Program



Motor 1: 5000	#1->1000X	X=5
Motor 2: 10000	#2->1000Y	Y=10

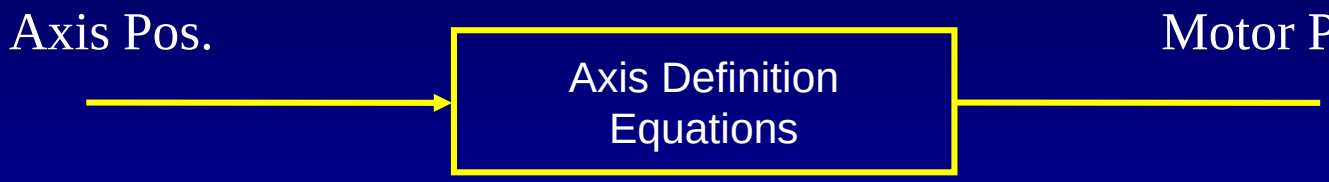
(executed on PMATCH)

Motor-to-Axis Conversion With Forward-Kinematic Program



Motor 1: 5000	(P1=5000)	(Q7=f1[P1,P2])	X=(Q7)
Motor 2: 10000	(P2=10000)	(Q8=f2[P1,P2])	Y=(Q8)

Axis-to-Motor Conversion Without Inverse Kinematics



X10 Y20

#1->1000X
#2->1000Y

Motor 1: 10000
Motor 2: 20000

(executed every move / segment)

Axis-to-Motor Conversion With Inverse Kinematics



X10 Y20	(Q7=10)	#1->I	(P1=f1[Q7,Q8])	Motor 1: (P1)
	(Q8=20)	#2->I	(P2=f2[Q7,Q8])	Motor 2: (P2)

PMAC Axis Properties

Permitted Axis Names: **X,Y,Z,U,V,W,A,B,C**

All axes support mapping, scaling, and offset

X,Y,Z: Traditionally Main Linear Axes

- Matrix Axis Definition
- Real Time Matrix Axis Transformation
- Circular Interpolation
- Cutter Radius Compensation



PMAC Axis Properties (continued)

U,V,W: Traditionally Secondary Linear Axes

Matrix Axis Definition

A,B,C: Traditionally Rotary Axes

(A rotates about X, B about Y, C about Z)

Position Rollover (Ix27)



Multiple Axis Definitions

```
&1
#1->x
&2
#2->x
```

This is permitted.
The two motors will act as X-axes in independent coordinate systems.

```
&1
#1->x
#2->x
```

This is permitted.
The motors will act on identical X-axis trajectories, as in a gantry system.



Multiple Axis Definitions (continued)

```
&1
#1->X
#1->Y
```

This is NOT permitted.
The second axis definition command will cancel out the first.

```
&1
#1->X
&2
#1->X
```

This is NOT permitted.
The motors could receive conflicting commands. PMAC will reject the second definition.



Programmable Axis Offsets

- **Axis origin for programming purposes can be changed**
- **Change made by redefining present commanded position**
- **On-Line command {axis}={constant}**
  e.g. X=10 Y=20
- **Buffered motion program command PSET {axis}{data}…**
  e.g. PSET X10 Y(P1)
- **These commands change "position bias" register–difference between motor origin and axis origin**
- **Also possible to write directly to position bias register (must then use PMATCH command to re-align motor and axis)**



Axis Matrix Transformations

Working from base coordinate definition:

$$\begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix} = \begin{bmatrix} Q(n) & Q(n+1) & Q(n+2) \\ Q(n+3) & Q(n+4) & Q(n+5) \\ Q(n+6) & Q(n+7) & Q(n+8) \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + \begin{bmatrix} Q(m) \\ Q(m+1) \\ Q(m+2) \end{bmatrix}$$

Rotation Matrix

Displacement Vector

AROTn : Absolute Rotation

ADISm : Absolute Displacement

矩阵轴转换

Working from present transformed coordinates:

$$\begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix} = \begin{bmatrix} Q(n) & Q(n+1) & Q(n+2) \\ Q(n+3) & Q(n+4) & Q(n+5) \\ Q(n+6) & Q(n+7) & Q(n+8) \end{bmatrix} \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix} + \begin{bmatrix} Q(m) \\ Q(m+1) \\ Q(m+2) \end{bmatrix}$$

Rotation Matrix

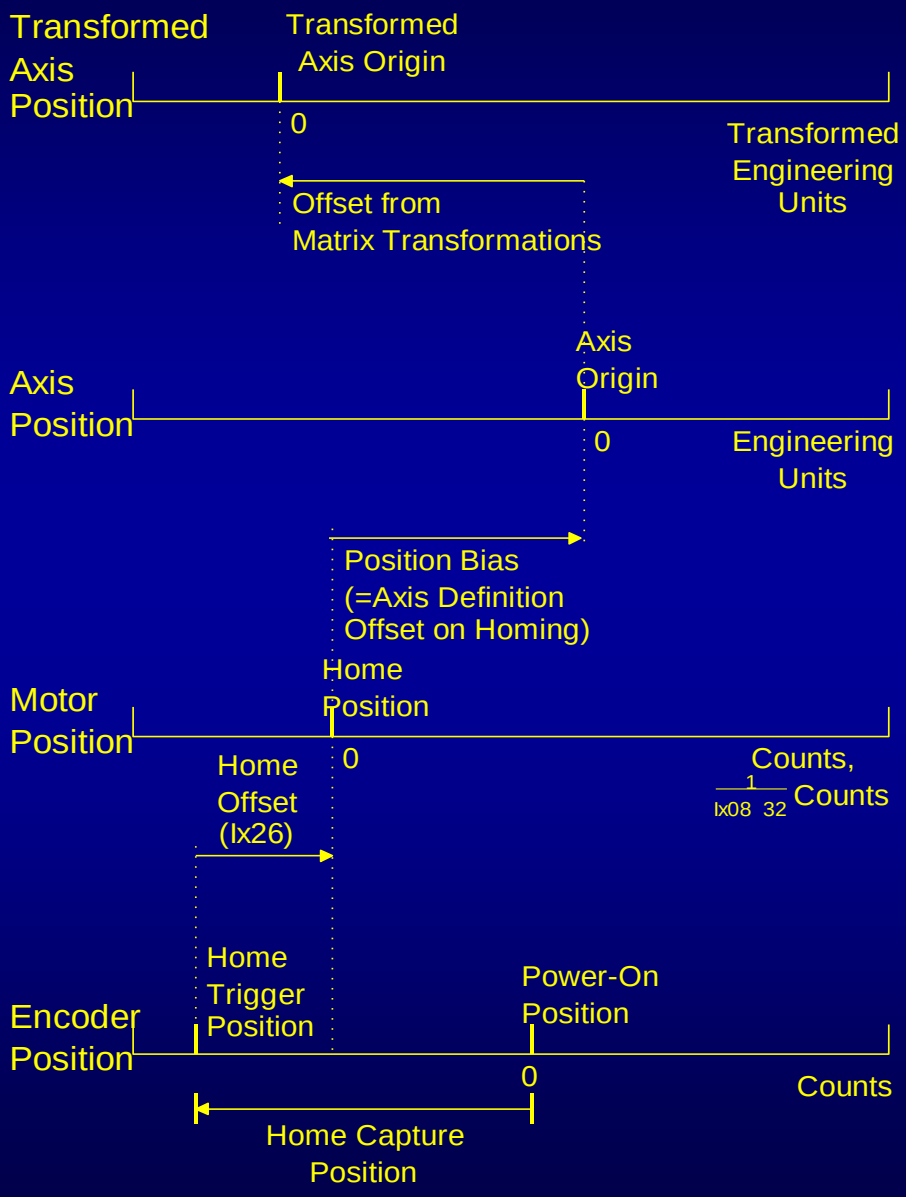
Displacement Vector

IROTn : Incremental Rotation

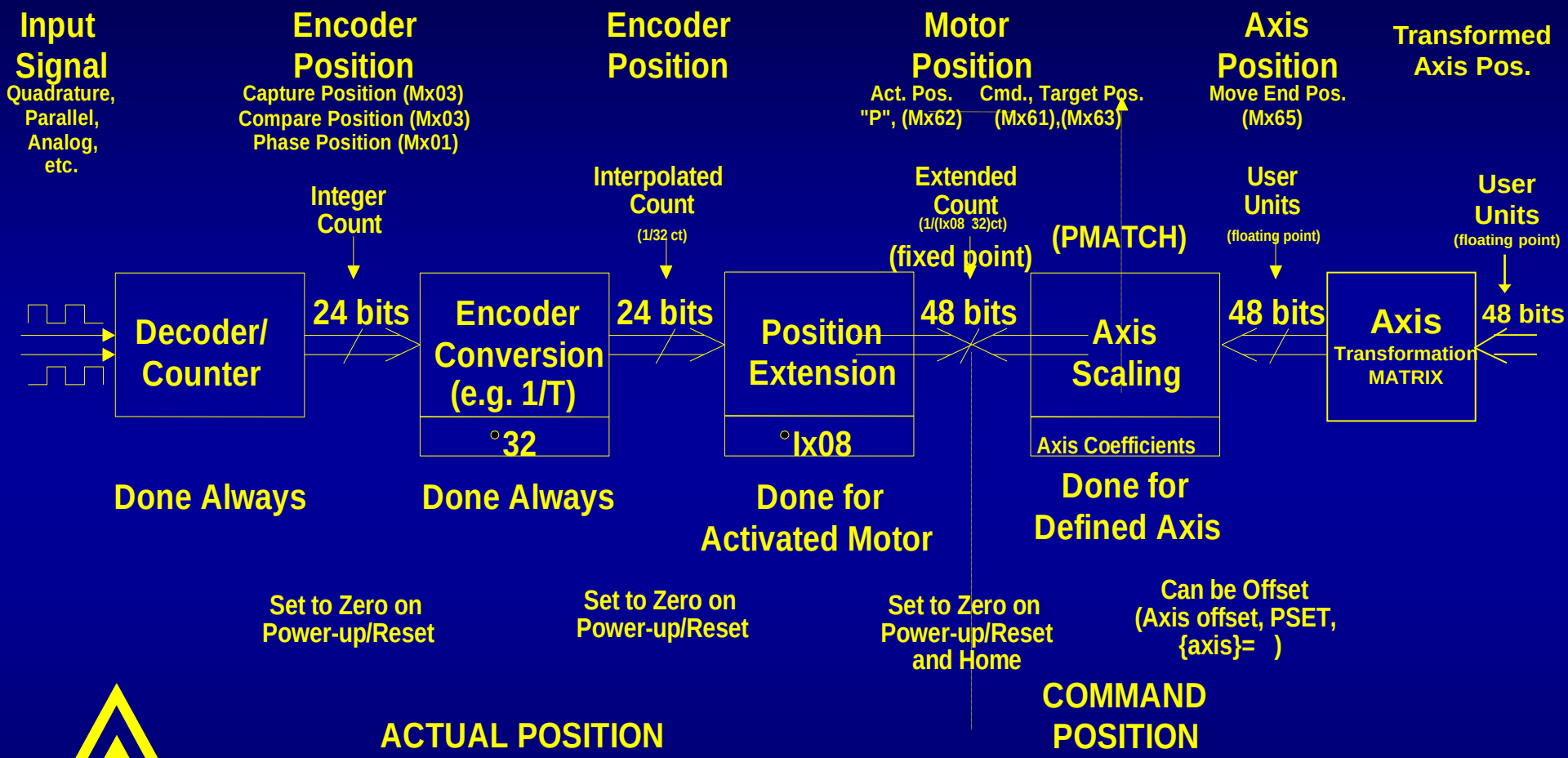
IDISm : Incremental Displacement



PMAC Position Scales



PMAC 位置处理


Input Signal
Quadrature, Parallel, Analog, etc.
Encoder Position
Capture Position (Mx03)
Compare Position (Mx03)
Phase Position (Mx01)
Encoder Position
Motor Position
Act. Pos. "P", (Mx62)
Cmd., Target Pos. (Mx61),(Mx63)
Axis Position
Move End Pos. (Mx65)
Transformed Axis Pos.
Integer Count
Interpolated Count
(1/32 ct)
Extended Count
(1/(Ix08 32)ct)
(fixed point)
(PMATCH)
User Units
(floating point)
User Units
(floating point)
Decoder/ Counter
24 bits
Encoder Conversion (e.g. 1/T)
°32
24 bits
Position Extension
°Ix08
48 bits
Axis Scaling
Axis Coefficients
48 bits
Axis
Transformation MATRIX
48 bits
Done Always
Done Always
Done for Activated Motor
Done for Defined Axis
Set to Zero on Power-up/Reset
Set to Zero on Power-up/Reset
Set to Zero on Power-up/Reset and Home
Can be Offset (Axis offset, PSET, {axis}=)
ACTUAL POSITION
COMMAND POSITION

PMAC Motion Programs

- Automatic sequenced execution of motions
- Program math and logic executed with move sequence (at move boundaries)
- Motion statements like machine tool “G codes”
- Math and logic statements like BASIC
- Program run by a coordinate system
- Axes commanded in engineering units



PMAC PLC Programs

- Automatic repetitive execution of non-motion commands
- For actions not synchronous with programmed motion sequence
- Math and logic statements like BASIC



PMAC Motion Program Statements

Move Commands

```
X1000 Y2000 Z3000
U(P1*3.14159) V(20*SIN(Q6))
DWELL, DELAY
```

Modal Commands

```
ABS, INC, FRAX, NORMAL
LINEAR, RAPID, CIRCLEn, SPLINEn, PVT
TA, TS, TM, F
```



Variable Value Assignment

```
{variable} = {expression}
```

运动程序逻辑控制

- 分枝: IF ({condition})
  [ELSE]
  ENDIF
- 循环: WHILE ({condition})
  ENDWHILE
- 标志 N {constant}
  O {constant}
- 子程序: GOSUB {data}
  CALL {data}
  G, M, T, D
  RETURN
- 跳转: GOTO {data}



Motion Program Move Sequencing

程序运算开始于下一中断周期结束后:

1) 接受运行指令
2) 当运行（插补）到新程序块（Block）时，块请求标志被设定。



Motion Program Move Sequencing (continued)

程序运算持续到:

1) 下一个运动, DELAY or DWELL is found and computed **(块请求标志被清除;Calculation resumes after next Block Request flag)**

2) Two jumps back in program are found

a) ENDWHILES (including implied)

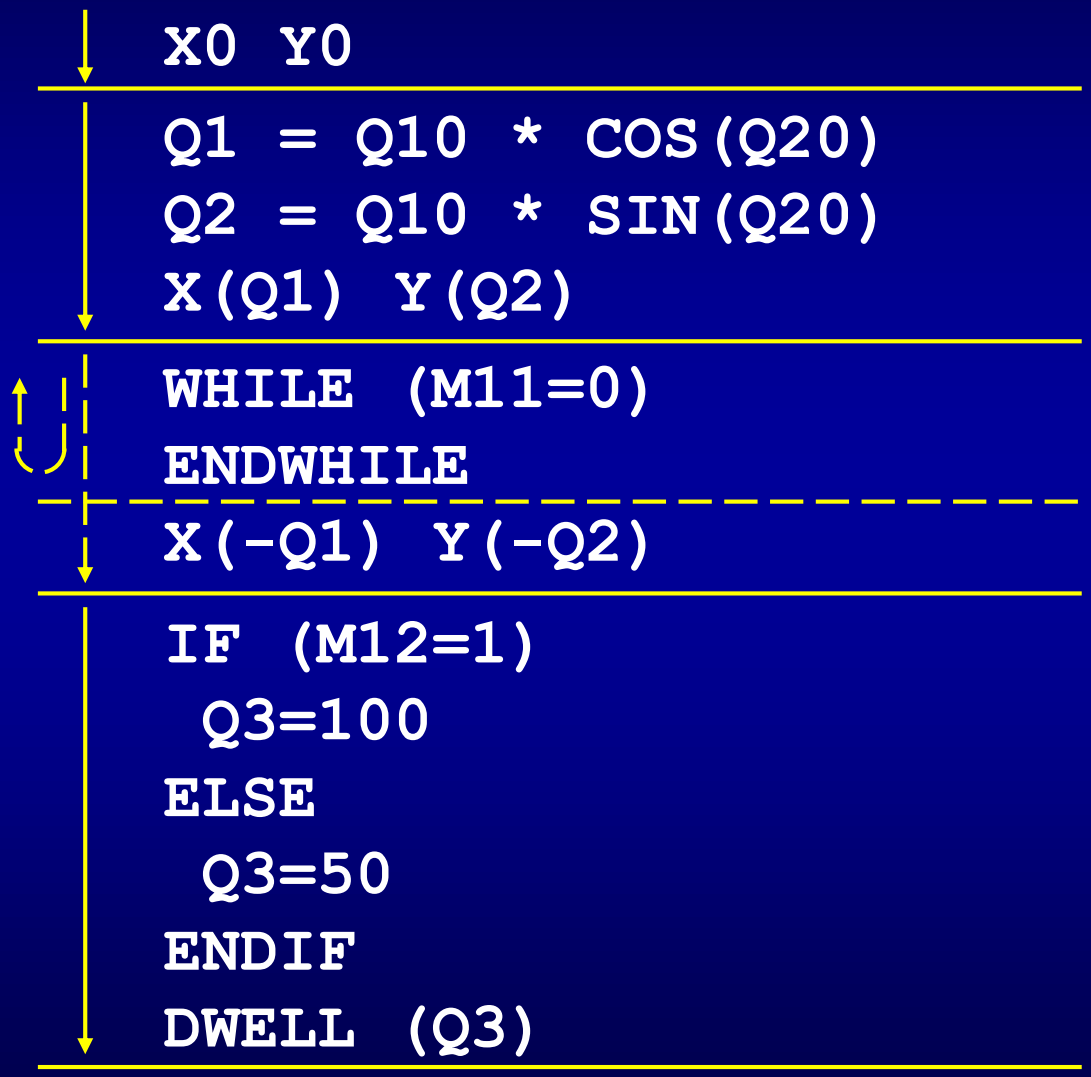
and/or

b) GOTO's to higher lines

(Block Request flag is left set;
Blending is disabled;
Calculations resume after move stops;
if stopped already, resume at next RTI)



Motion Program Sequencing Example



Observing the PMAC "Double Jump Back" Rule

```
P1=1
WHILE (P1<11)
   P2=0
    WHILE (P2<360)
     P3=10+P1*COS(P2)
     X(P3)
     P2=P2+10
    ENDWHILE
   P1=P1+1
ENDWHILE
```

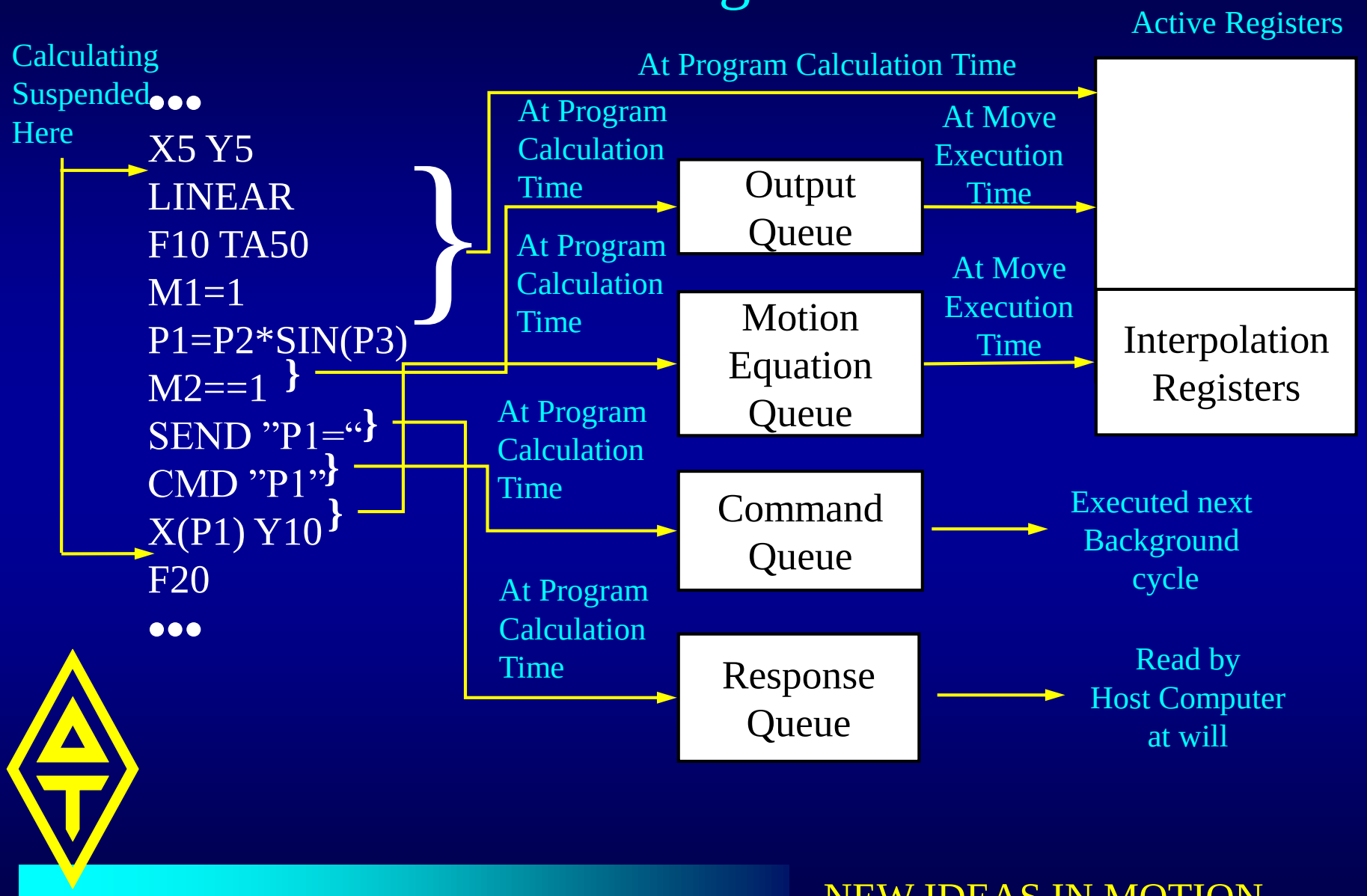


(Blending stops each time inner loop is exited: two Endwhile encountered before next move)

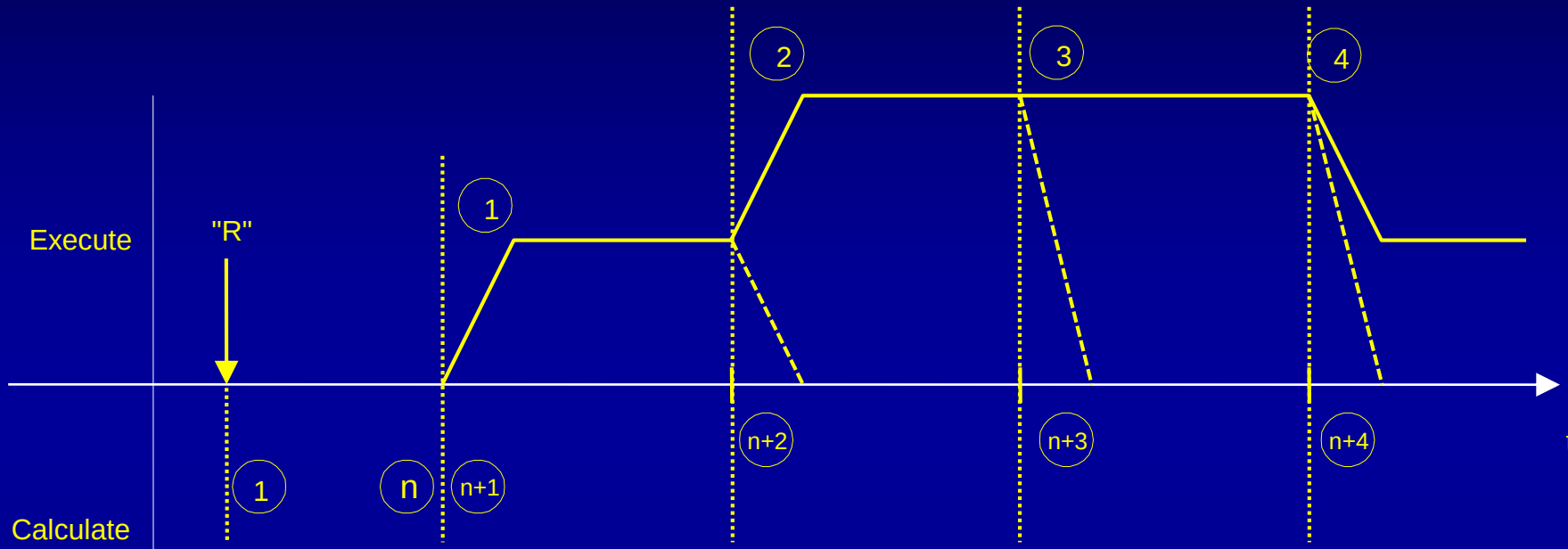
```
P1=1
WHILE (P1<11)
   P2=0
    WHILE (P2<350)
     P3=10+P1*COS(P2)
     X(P3)
     P2=P2+10
    ENDWHILE
   P3=10+P1*COS(P2)
   X(P3)
   P1=P1+1
ENDWHILE
```

(Blending is continuous thru entire example)

PMAC Motion Program Execution



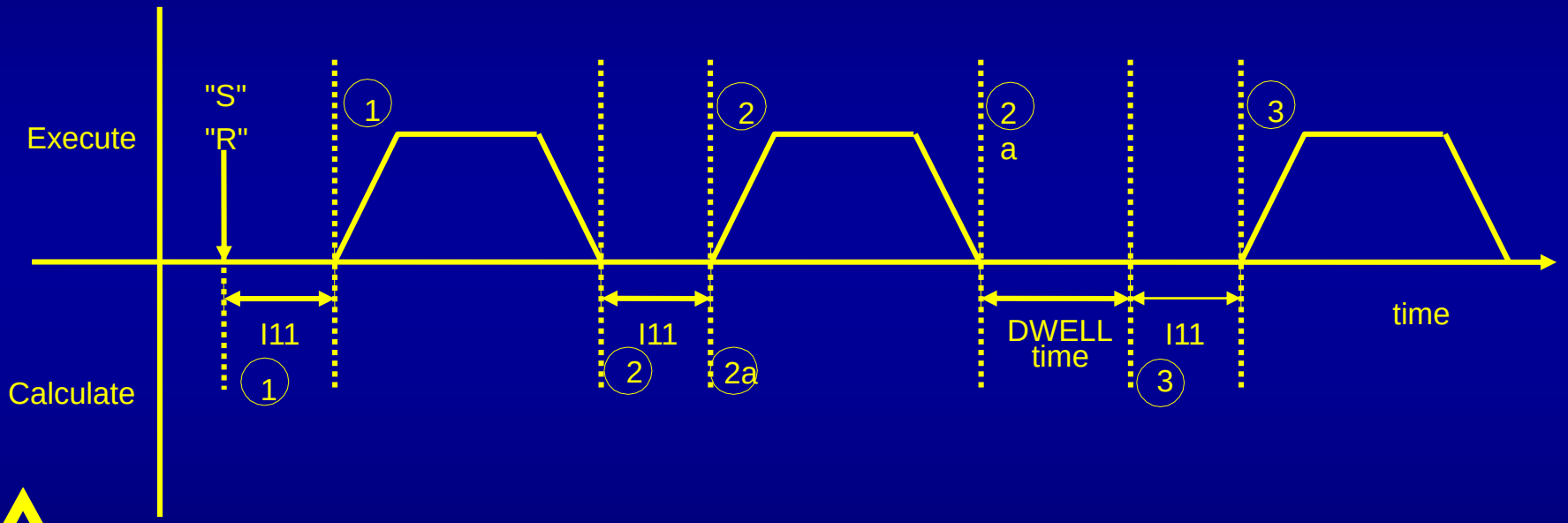
PMAC Motion Program Precalculation
"n" moves ahead for Blended Moves



How big is "n"?
- 0 for point-to-point moves, no radius compensation
- Add 1 for blending
- Add 1 for basic acceleration control
- Add 1 for Cutter radius compensation
- As big as necessary to cover stopping distance for robust acceleration control

PMAC Motion Program Precalculation

No moves ahead Ix92=1, RAPID, HOME, DWELL, “S”



Example 1: A Simple Move

This example shows how to program a simple move on the program specifies how to do the move, then commands the move.

```
********** Set-up and Definitions **********

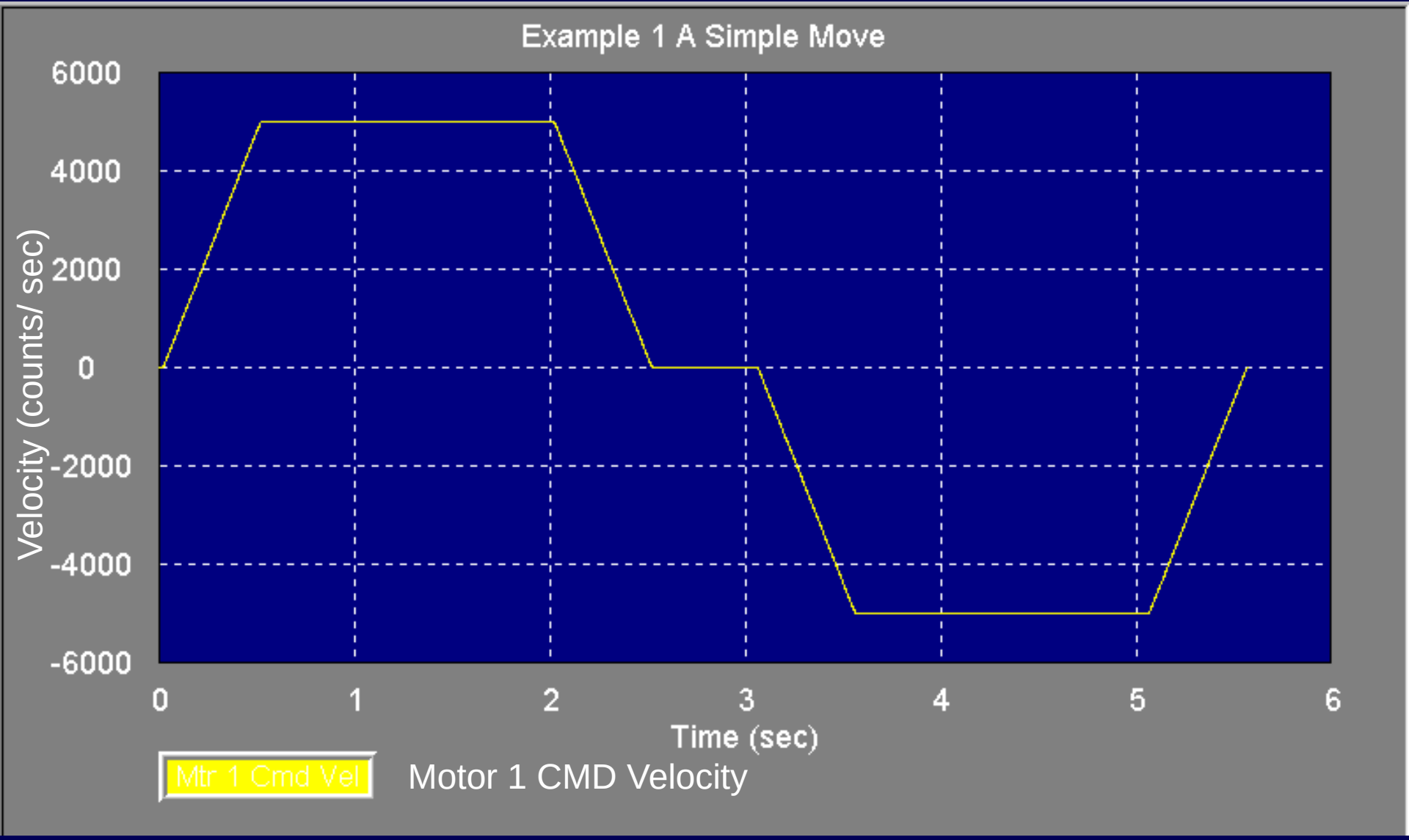
DEL GAT  ; Erase any defined gather buffer
&1       ; Coordinate System 1
CLOSE    ; Make sure all buffers are closed
#1->X    ; Assign motor 1 to the X-axis - 1 program unit
         ; of X is 1 encoder count of motor #1

********** Motion Program Text **********

OPEN PROG 1      ; Open buffer for program entry, Program #1
CLEAR    ; Erase existing contents of buffer
LINEAR   ; Blended linear interpolation move mode
ABS      ; Absolute mode - moves specified by position
TA500    ; Set 1/2 sec (500 msec) acceleration time
TS0      ; Set no S-curve acceleration time
F5000    ; Set feedrate (speed) of 5000 units(cts)/sec
X10000   ; Move X-axis to position 10000
DWELL500 ; Stay in position for 1/2 sec (500 msec)
X0       ; Move X-axis to position 0
CLOSE    ; Close buffer - end of program
```

To run this program:

```
&1 B1 R  ; Coord. System 1, point to Beginning of Program 1, Run
```



Motor 1 CMD Velocity

Example 2: A More Complex Move

This example introduces incremental and time-specification of moves, looping logic, using variables, scaling of axes, and simple arithmetic. Note that logical and mathematical operations do not delay moves.

```
;*************** Set-up and Definitions ****************

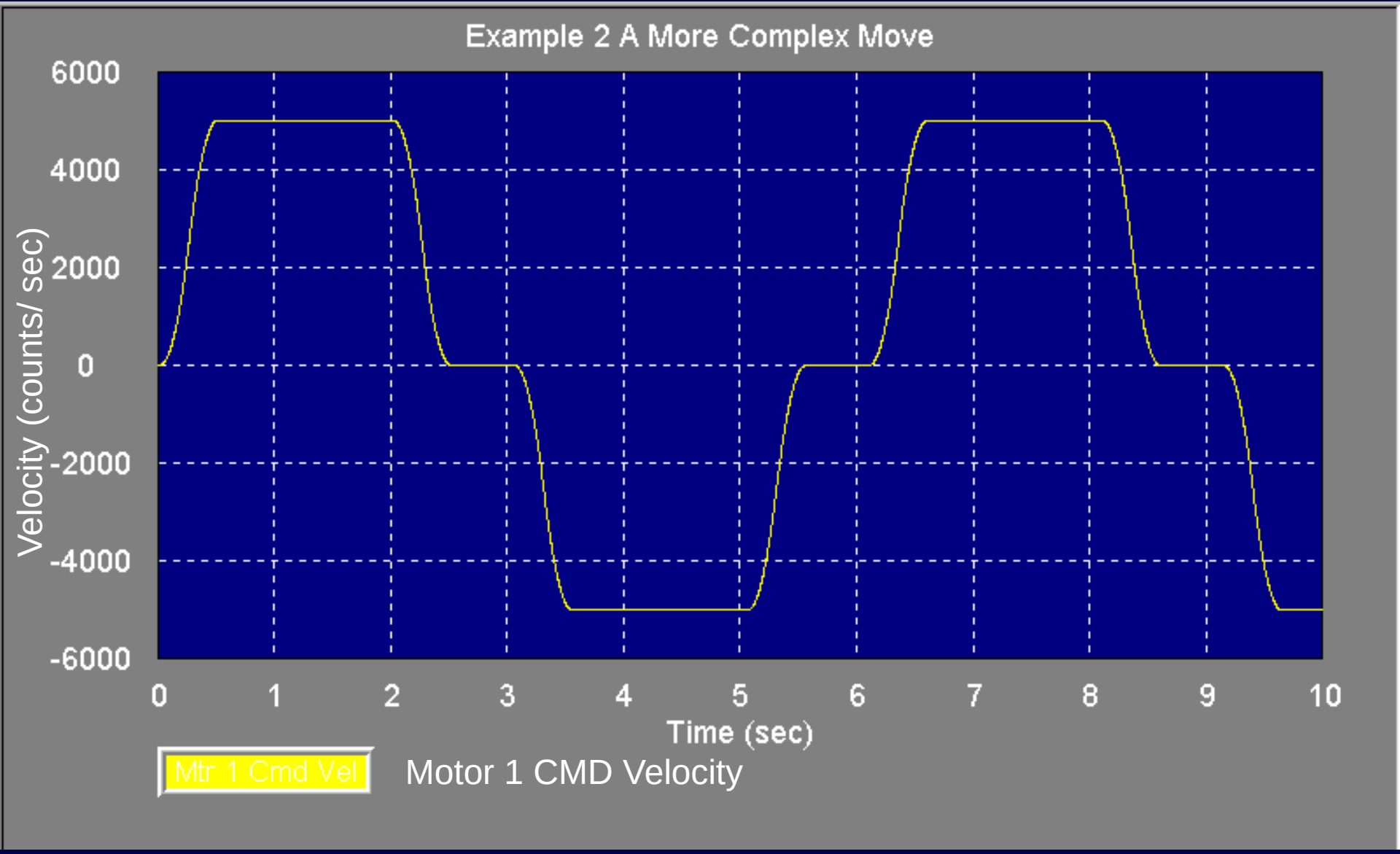
&2                ; Coordinate system 2
                  ; NOTE: No motor can be simultaneously defined
                  ; in more than one coordinate system
CLOSE             ; Make sure all buffers are closed
#5->1000X         ; 1 unit (cm) of X is 1000 counts of motor 5
```

Example 2: A More Complex Move (continued)

```
;****************** Motion Program Text ********************
OPEN PROG 2           ; Open buffer for entry, Program #2
CLEAR                 ; Erase existing contents of buffer
LINEAR                ; Blended linear interpolation move mode
INC                   ; Incremental mode - moves specified by distance
TA500                 ; 1/2 sec (500 msec) acceleration time
TS250                 ; 1/4 sec in each half of S-curve
P1=0                  ; Initialize a loop counter variable
WHILE (P1<10)         ; Loop until condition is false (10 times)
   X10                ; Move X-axis 10 cm (=10,000 cts) positive
   DWELL500           ; Hold position for 1/2 sec
   X-10               ; Move X-axis back 10 cm negative
   DWELL500           ; Hold position for 1/2 sec
   P1=P1+1            ; Increment loop counter
ENDWHILE              ; End of loop
CLOSE                 ; Close buffer - end of program
```

To run this program:

```
&2 B2 R               ; Coordinate System 2, point to Beginning of
Program 2,
```


Example 2 A More Complex Move
6000
4000
2000
0
-2000
-4000
-6000
Velocity (counts/ sec)
0
1
2
3
4
5
6
7
8
9
10
Time (sec)
Mtr 1 Cmd Vel
Motor 1 CMD Velocity

Rotary Motion Program Buffers

For real-time downloading of programs

DEFINE ROT {size} reserves the specified number of words of memory for the addressed C.S.'s buffer

B0 tells the coordinate system to be ready to execute the rotary buffer. (All buffers must be CLOSED for this command.)

OPEN ROT prepares all rotary buffers for entry

CLEAR erases any existing contents



Rotary Motion Program Buffers (continued)

{program lines} are entered into buffer for addressed C.S.

R starts execution for addressed C.S.

PR returns the number of lines yet to be executed in addressed C.S.

<BREQ> interrupt generated until I17 lines ahead; generated again when execution catches up within I16 lines; for addressed C.S. when I2=1

CLOSE prevents entry into buffer; does not stop execution



Running “Naked” Rotary Buffer

- Rotary buffer that has executed all moves sent to it is in indefinite “wait” state
- New move sent to it is immediately executed
- Makes “buffered” program commands execute like on-line commands
- End of move evaluated by status bits



PMAC 运动模式

- LINEAR 直线运动模式
  - 梯形/三角形速度曲线
  - Straight-line path in Cartesian system
  - Blends with LINEAR + CIRCLE moves

- CIRCLE mode
  - Sinusoidal velocity profiles
  - Circular arc path in Cartesian system
  - Blends with LINEAR + CIRCLE moves



PMAC Move Modes (continued)

- RAPID mode
  - Trapezoidal/triangular velocity profiles
  - (Near) straight-line path in Cartesian system
  - Minimum time point-to-point; no blending
- SPLINE modes
  - Parabolic velocity profiles
  - Cubic B-spline path
  - Blends with SPLINE moves, continuous acceleration
- PVT mode
  - Parabolic velocity profiles
  - Hermite spline path
  - Blends with PVT moves, continuous velocity



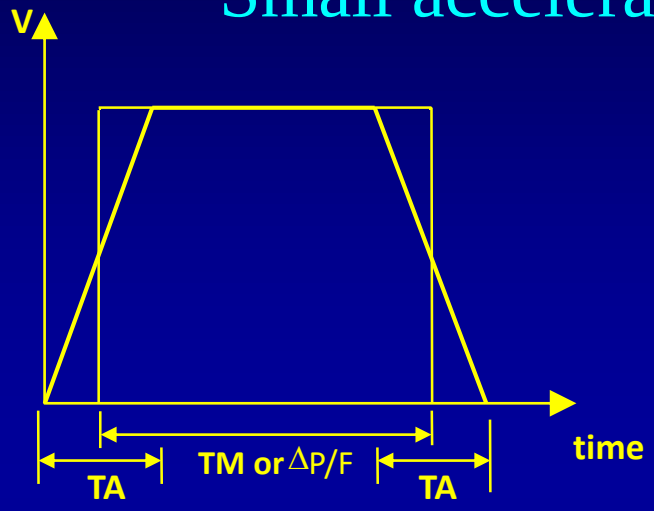
LINEAR Mode Moves

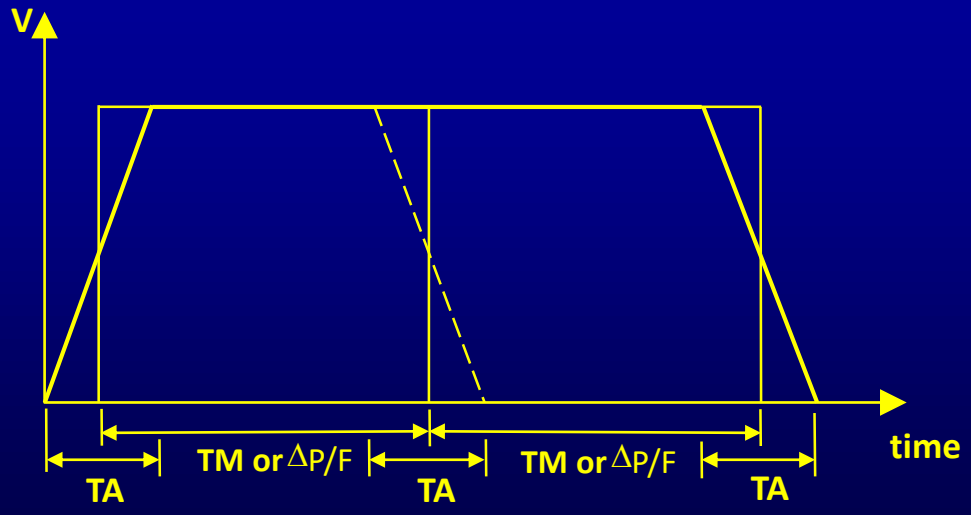
- End-point (ABS) or distance (INC) specification
- Vector Speed (F) or move time (TM) specification
- Acceleration time(TA, TS) or rate (Ix17) specification
- With F specification, constant vector speed, regardless of move direction
- With TM=TA, TS=0, quadratic B-spline path



Linear Mode Trajectories

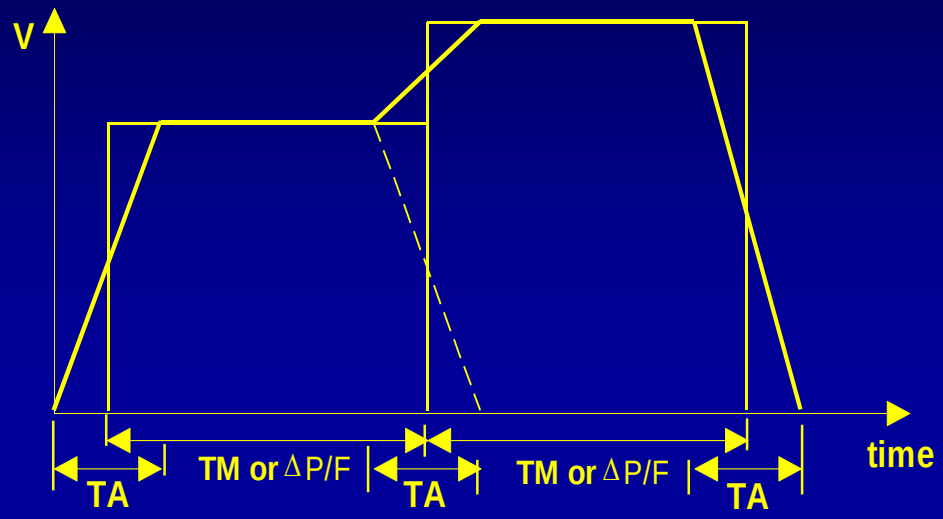
Small acceleration time

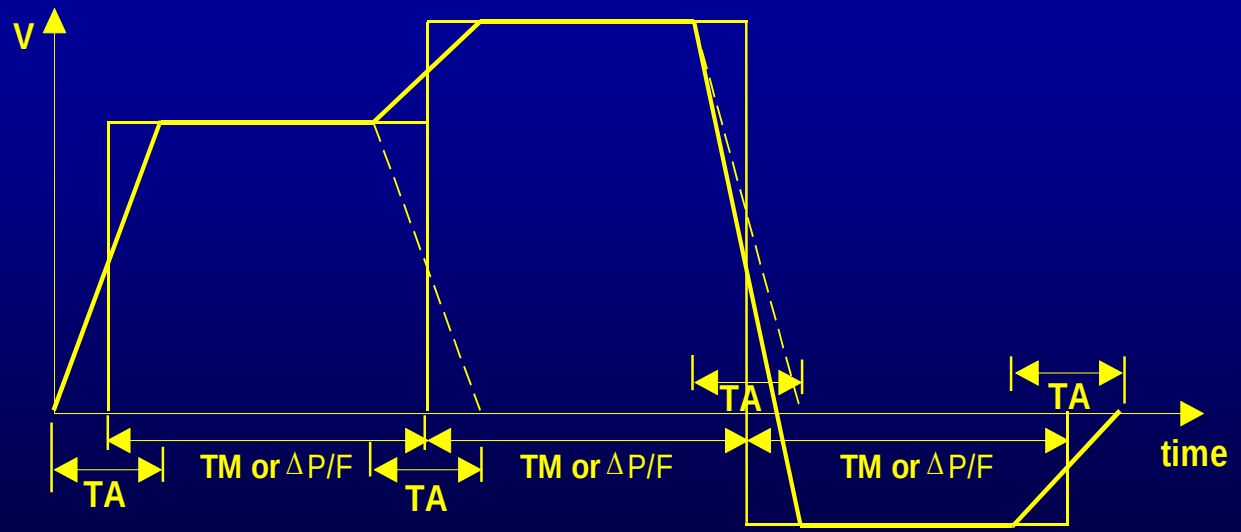




Linear Mode Trajectories (continued)

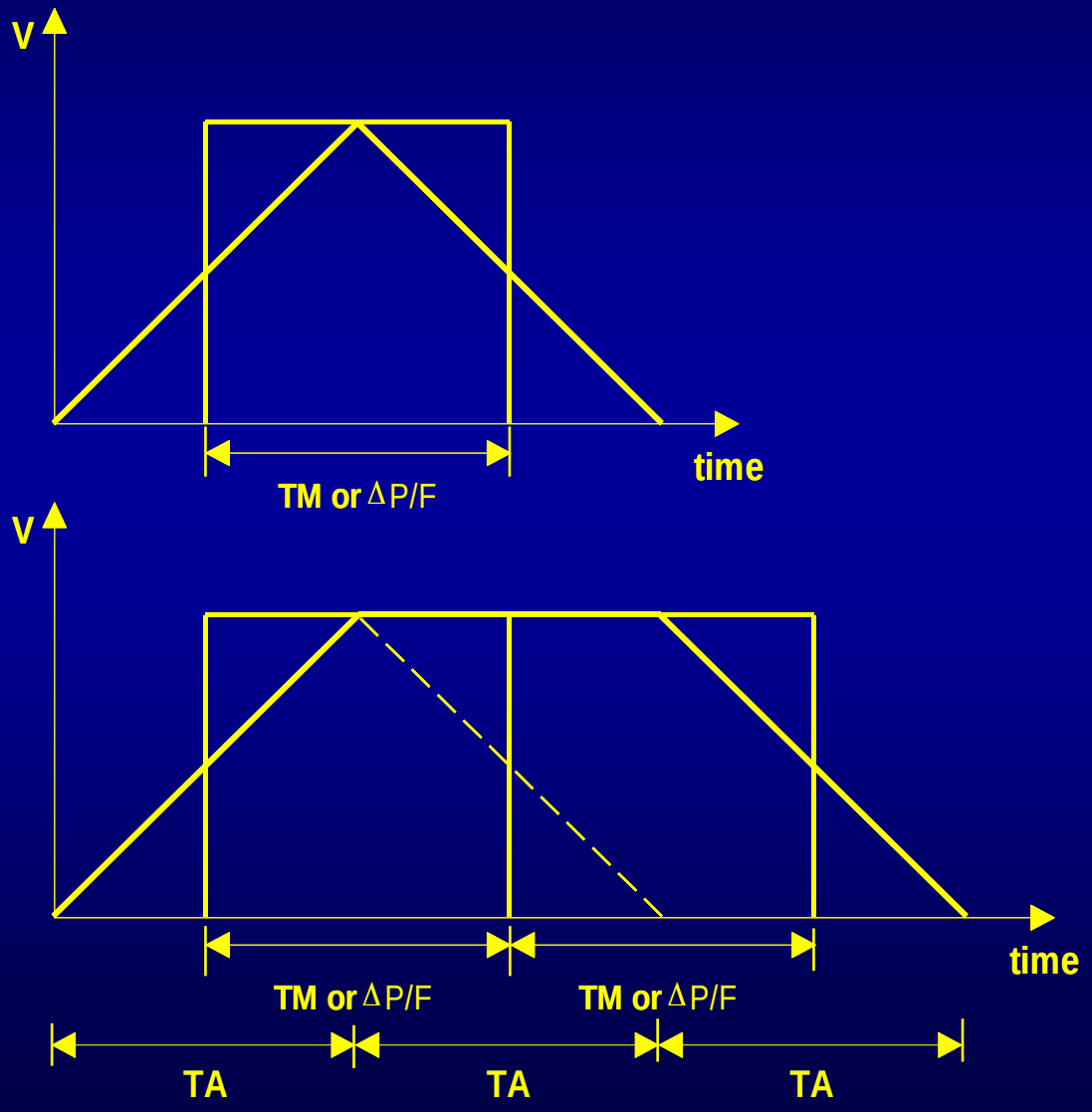
Small acceleration time



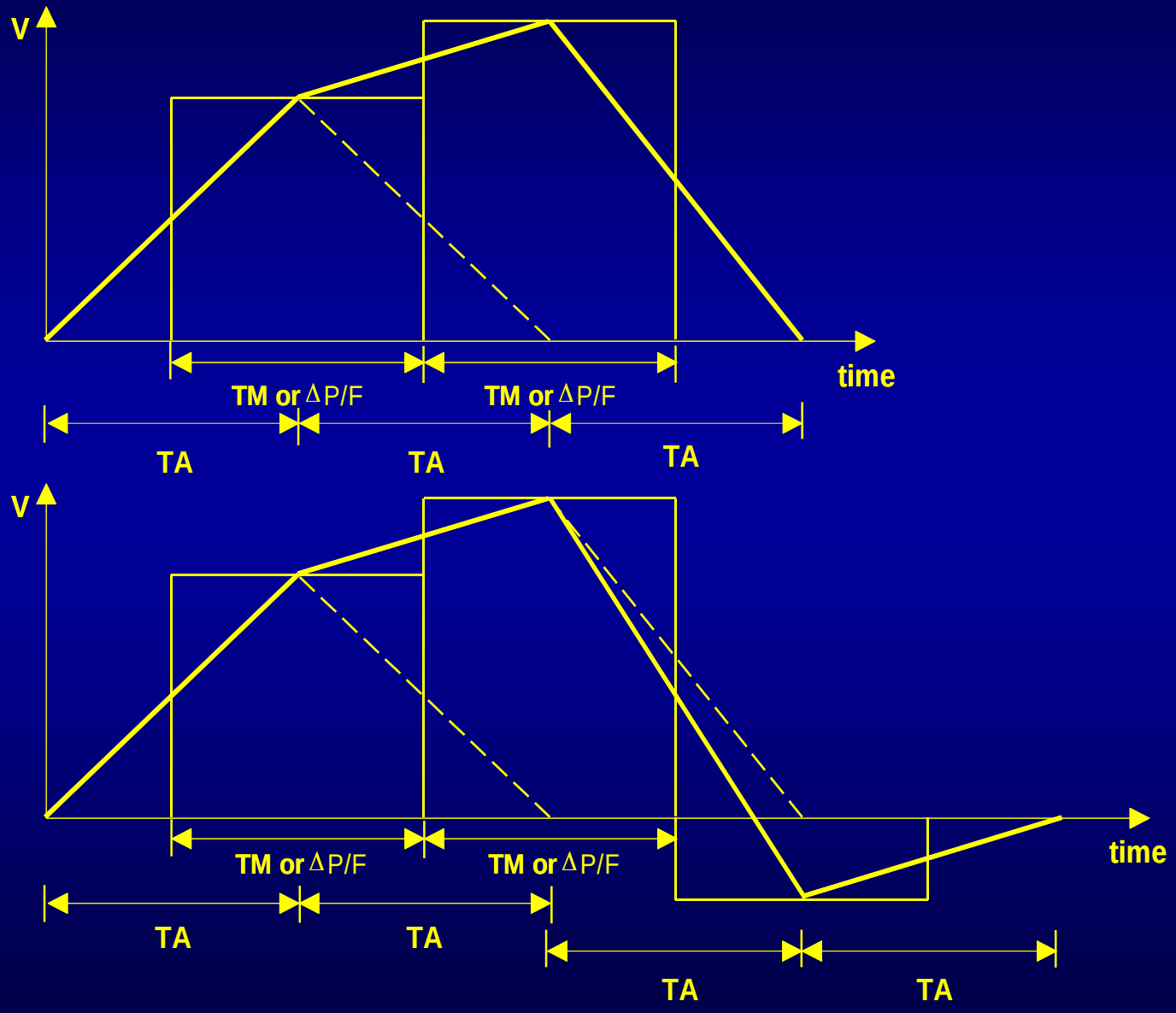


Linear Mode Trajectories

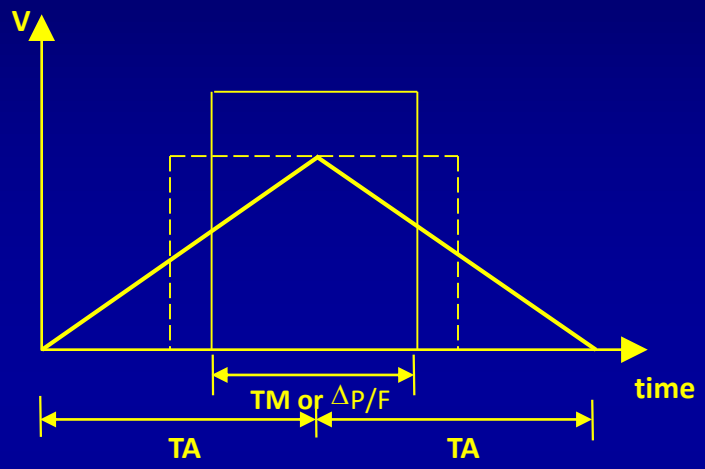
Acceleration Time Matches Move Time

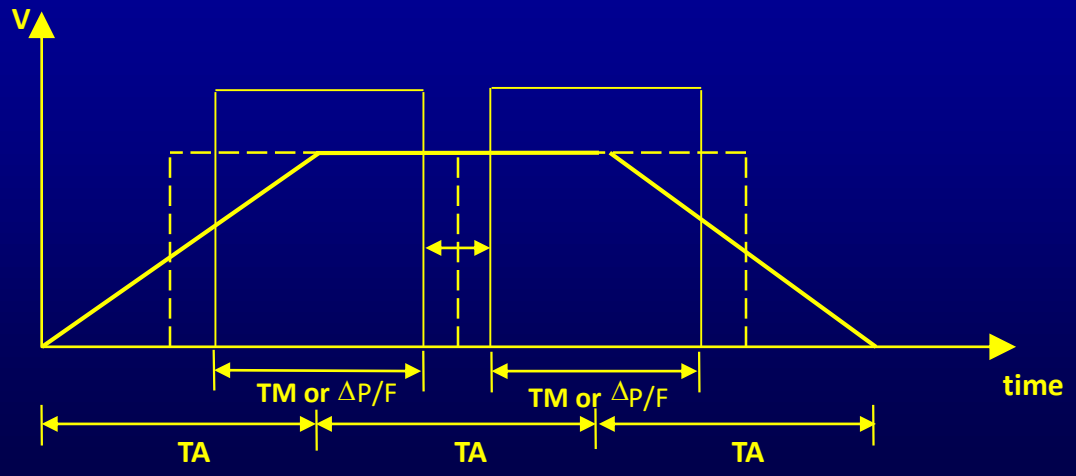


Linear Mode Trajectories (continued)



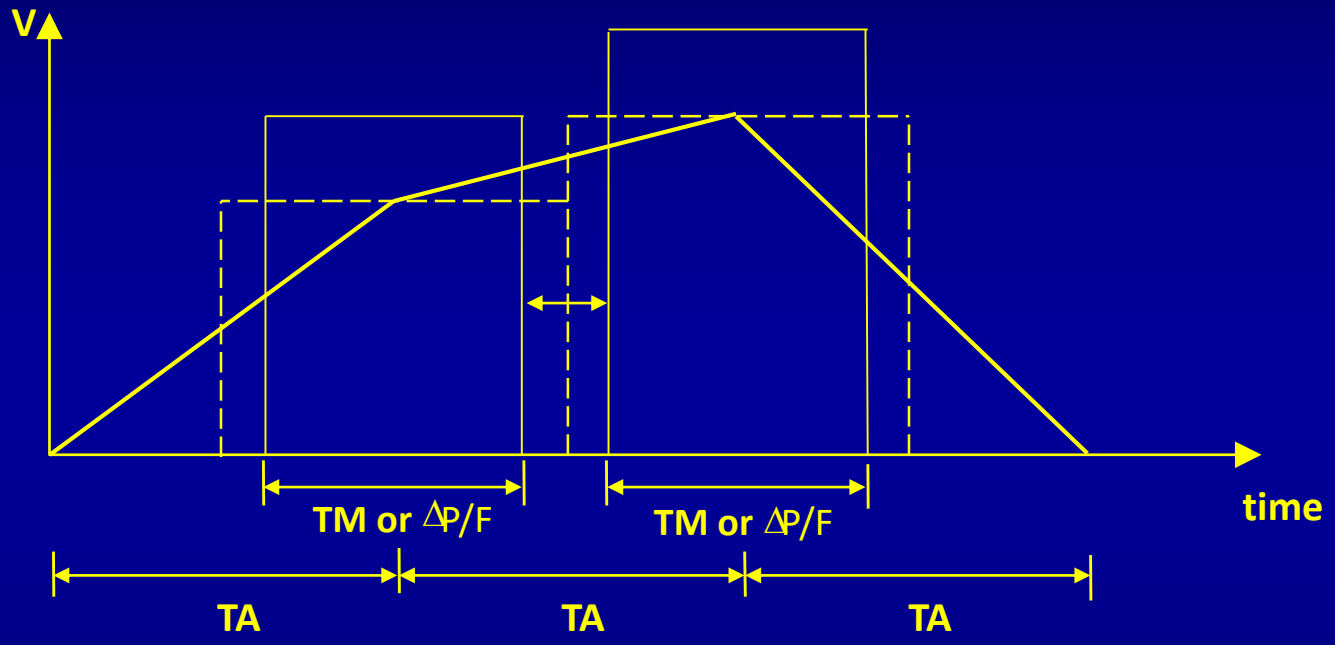
Linear Mode Trajectories
Large (velocity limiting) acceleration time





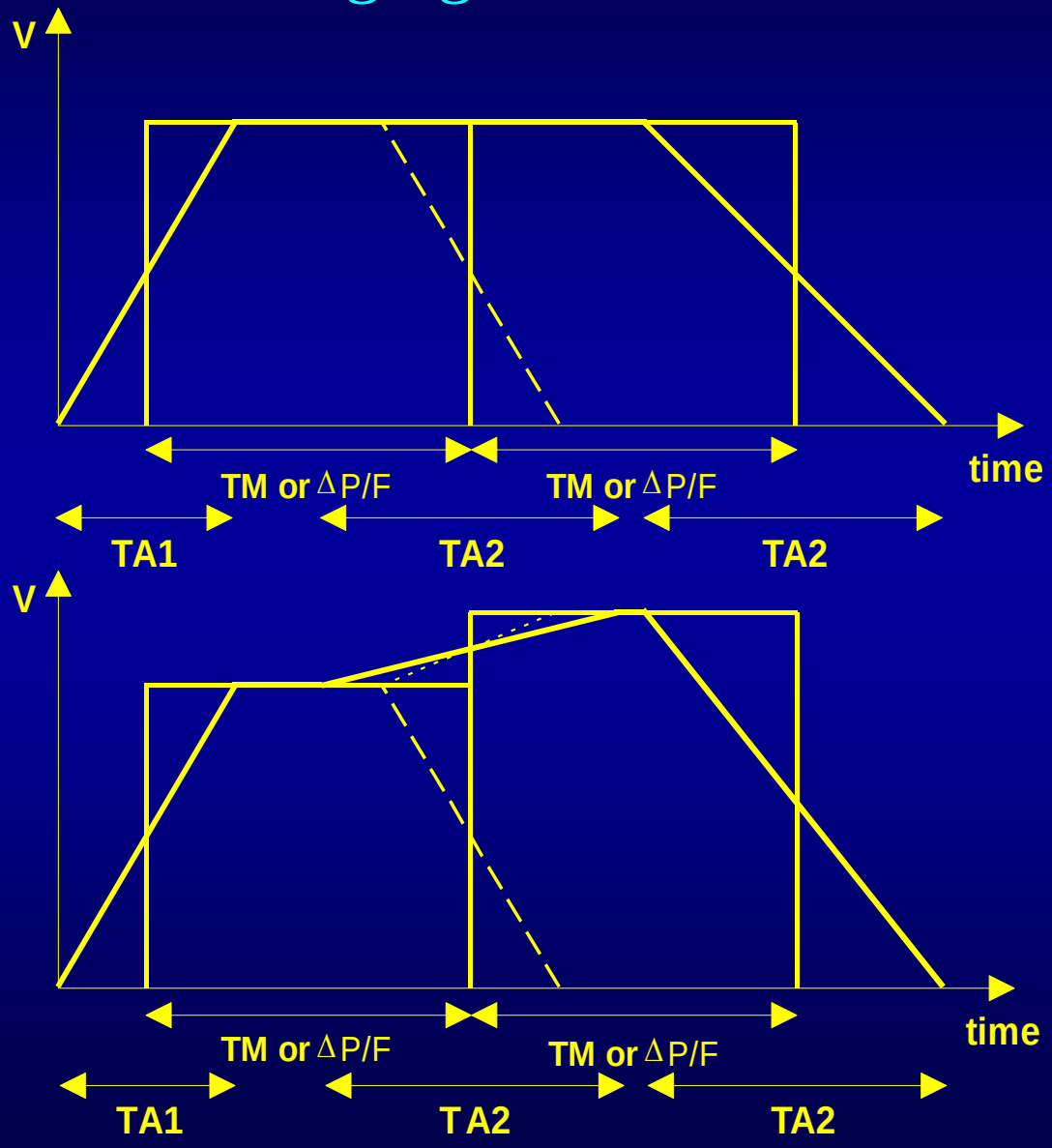
Linear Mode Trajectories (continued)

Large (velocity limiting) acceleration time



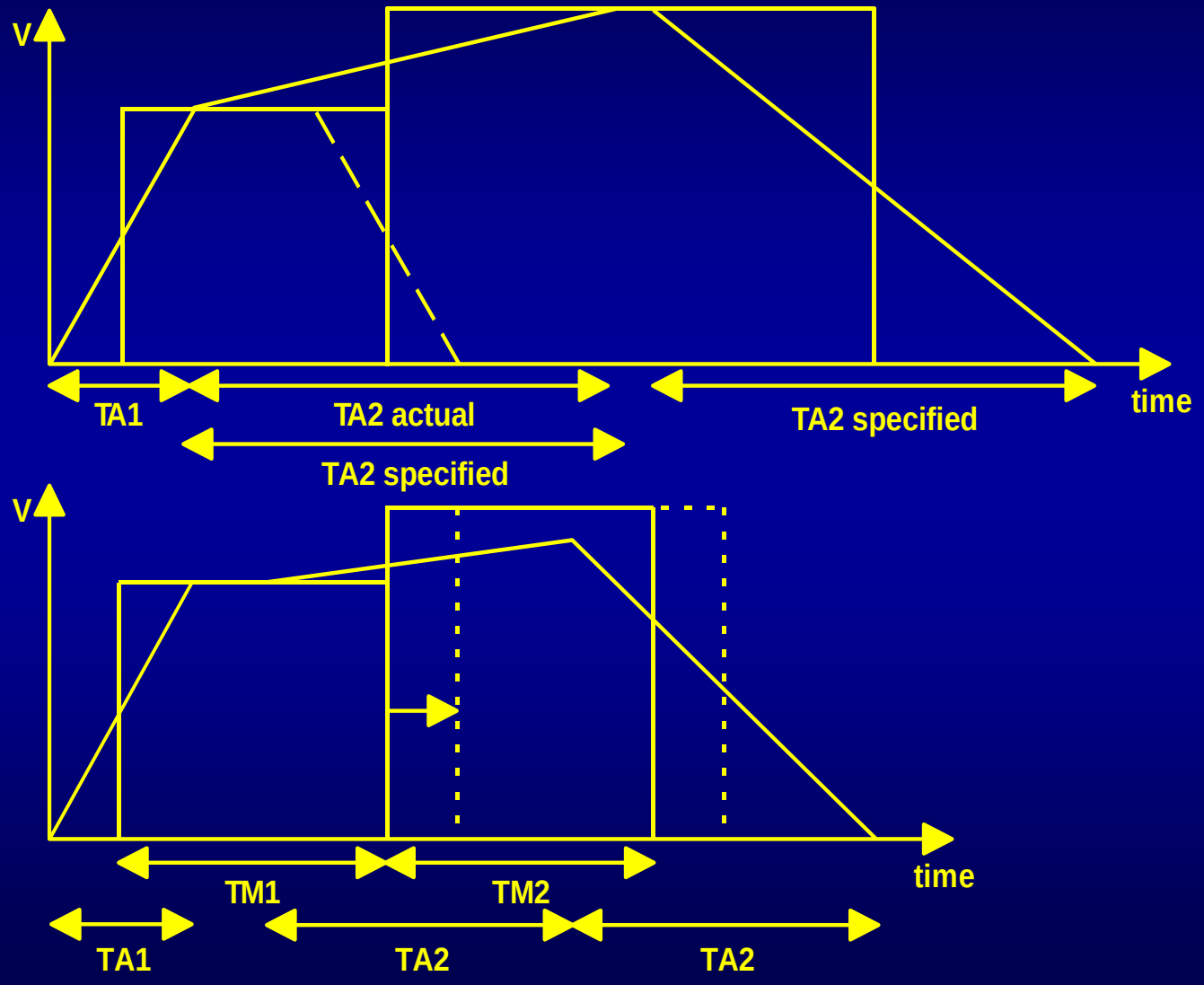
Linear Mode Trajectories

Changing acceleration times

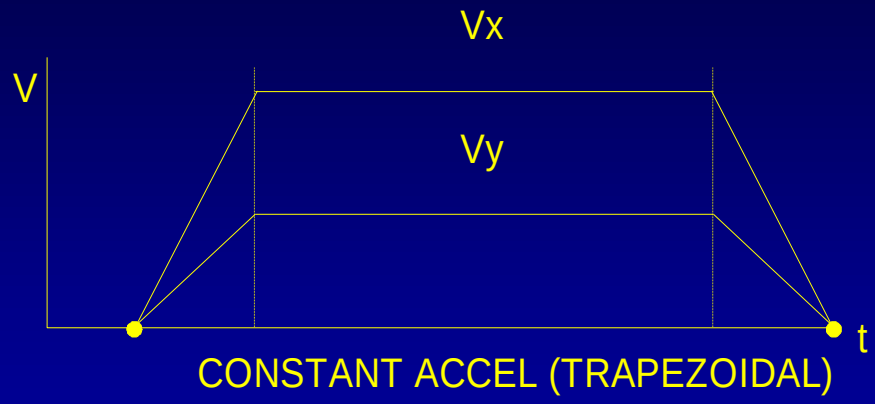


Linear Mode Trajectories (continued)

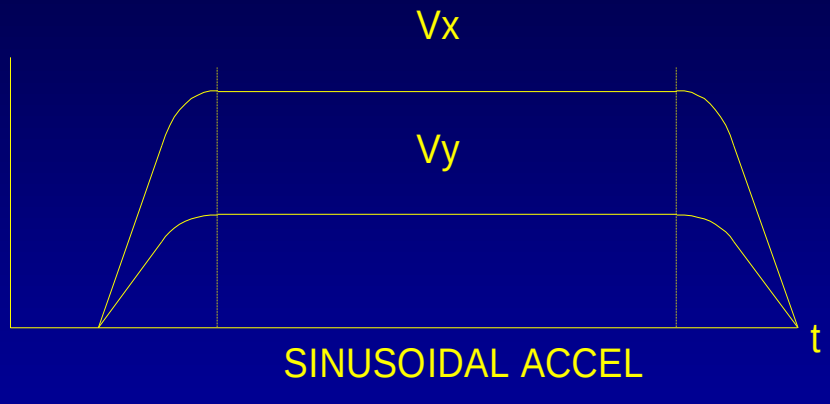
Changing acceleration times



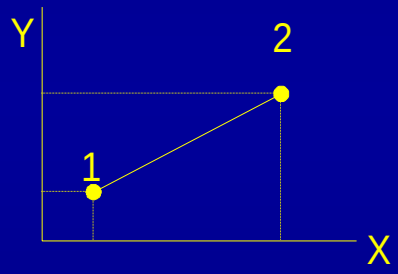
Profiled Linear Interpolation

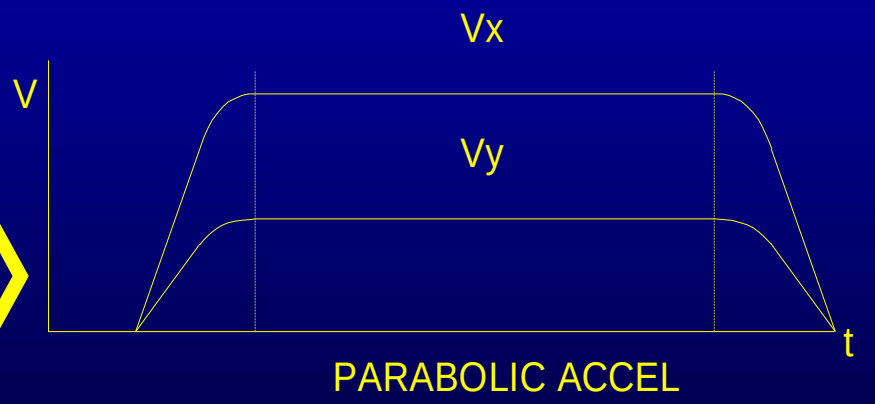


CONSTANT ACCEL (TRAPEZOIDAL)

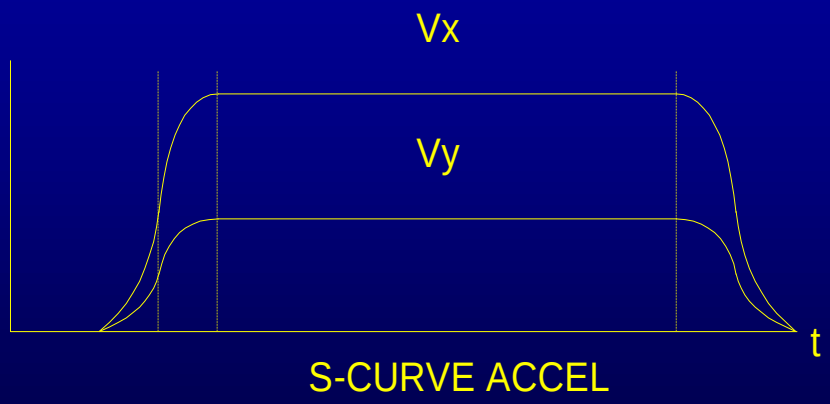


SINUSOIDAL ACCEL



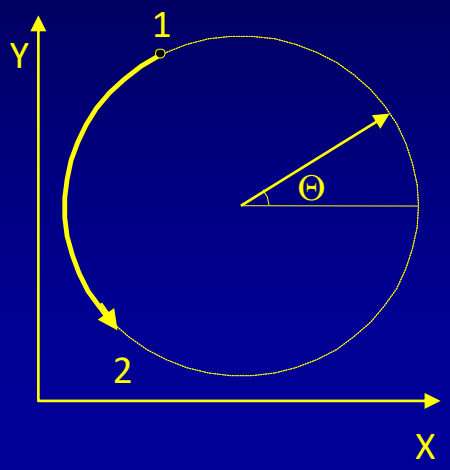


PARABOLIC ACCEL

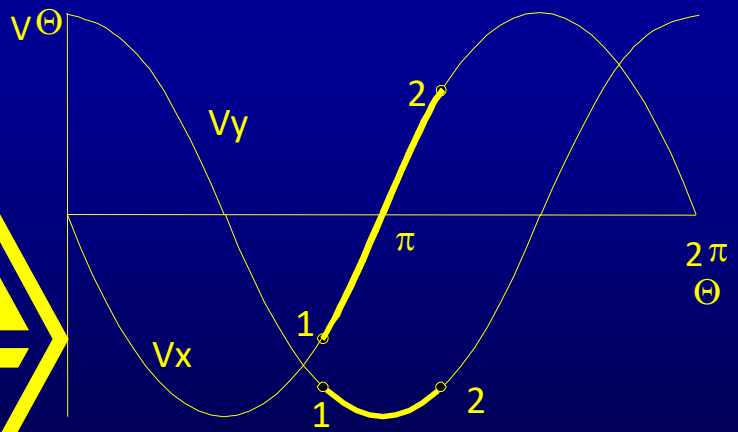


S-CURVE ACCEL

PROFILED CIRCULAR INTERPOLATION

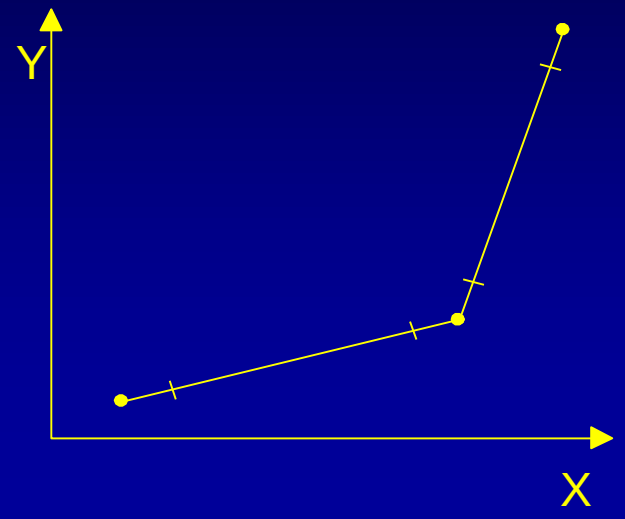


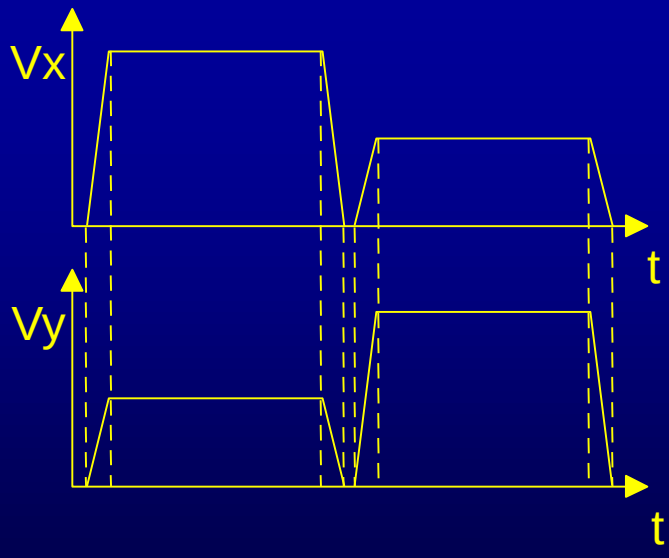




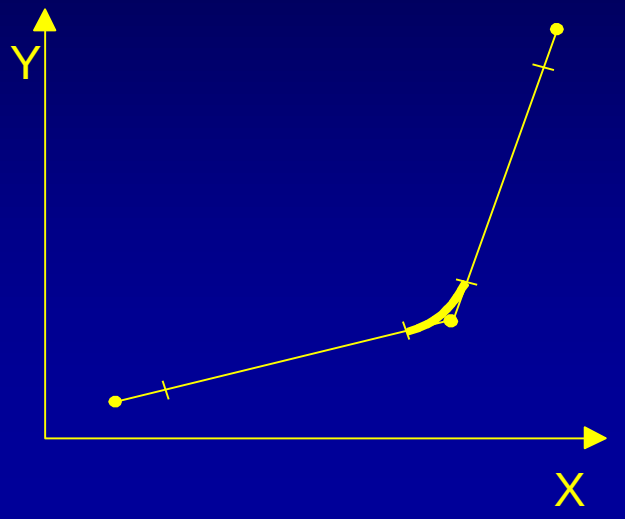
$$V_x(t) = V_x(\Theta) \cdot V_{prof}(t)$$
$$= R(-\sin\Theta) \cdot V_{prof}(t)$$
$$V_y(t) = V_y(\Theta) \cdot V_{prof}(t)$$
$$= R\cos\Theta \cdot V_{prof}(t)$$

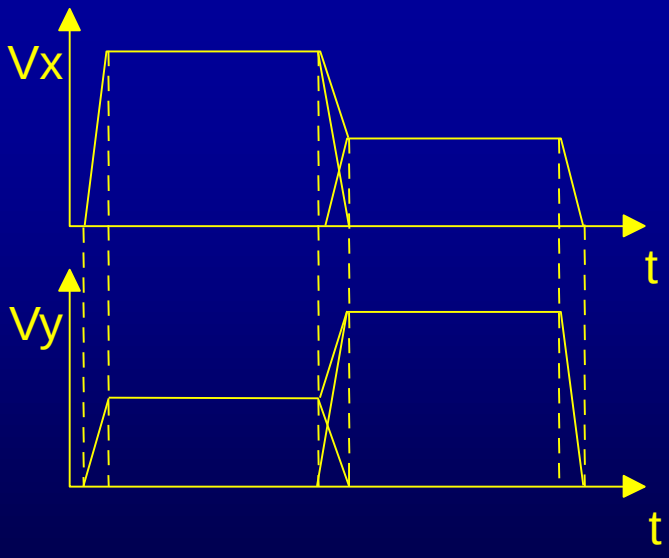
MultiAxis Linear Moves
Non-Blended vs. Blended





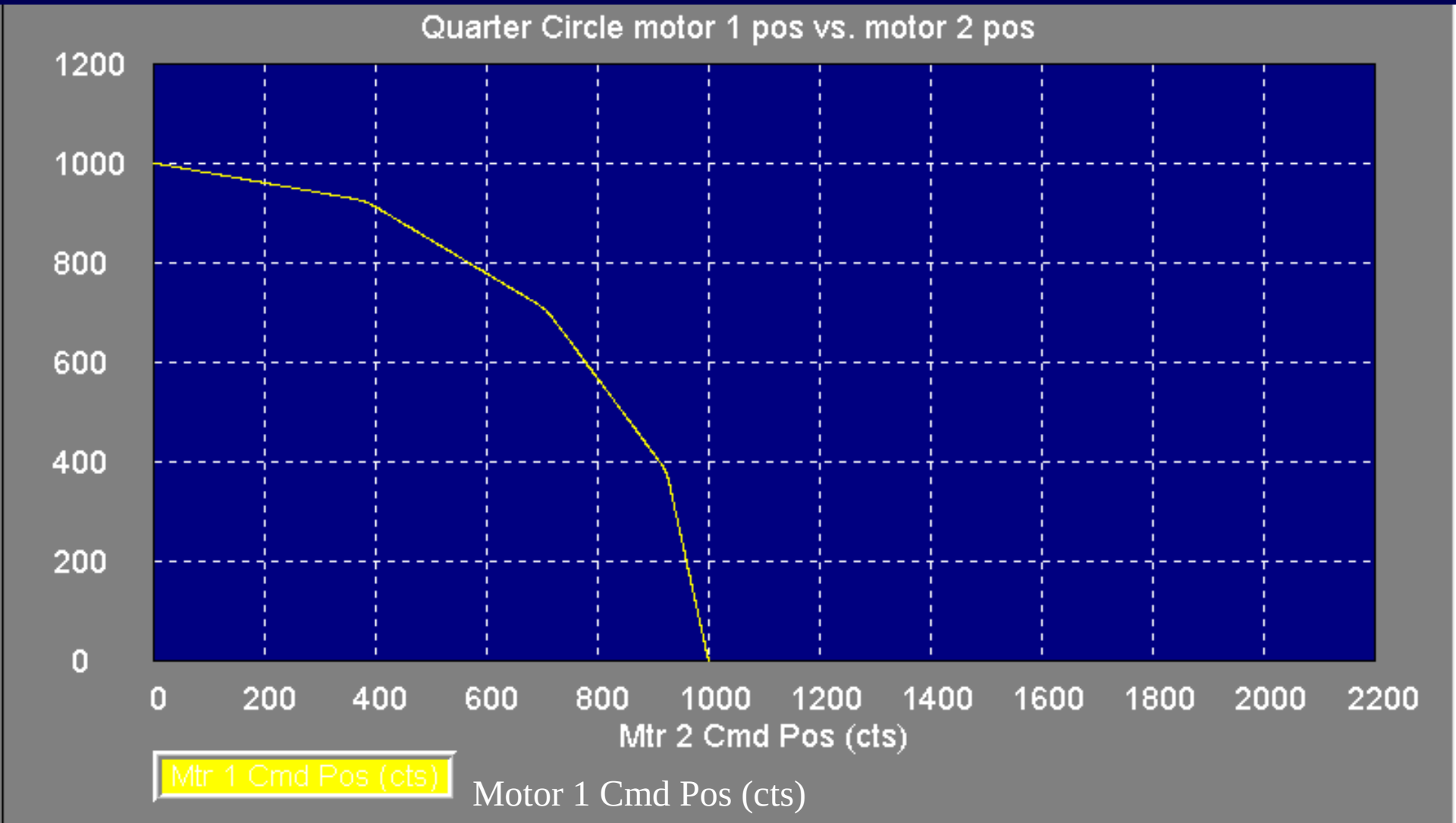
Non-Blended





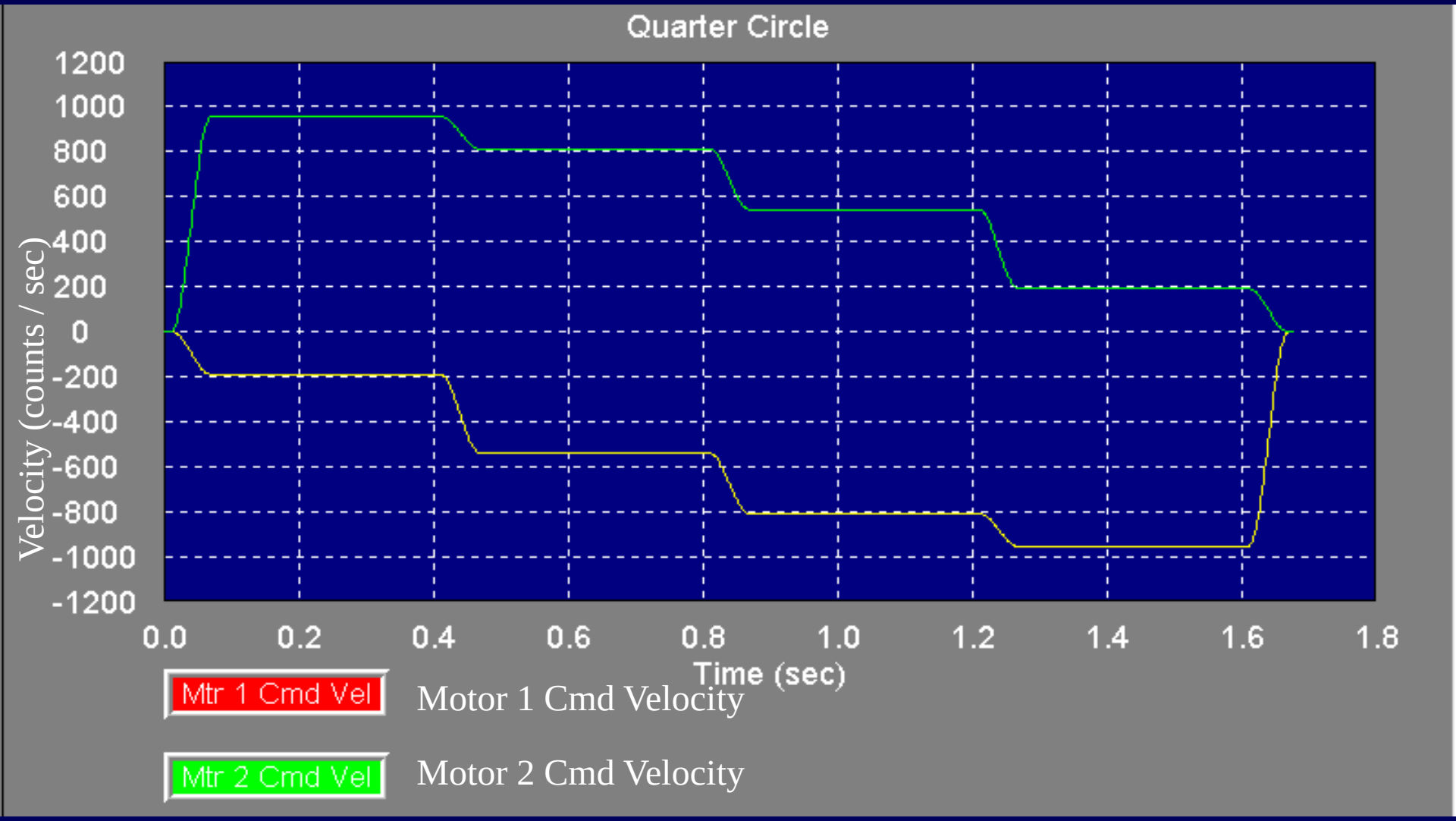
Blended

Quarter Circle Approximation - Small Accel Time



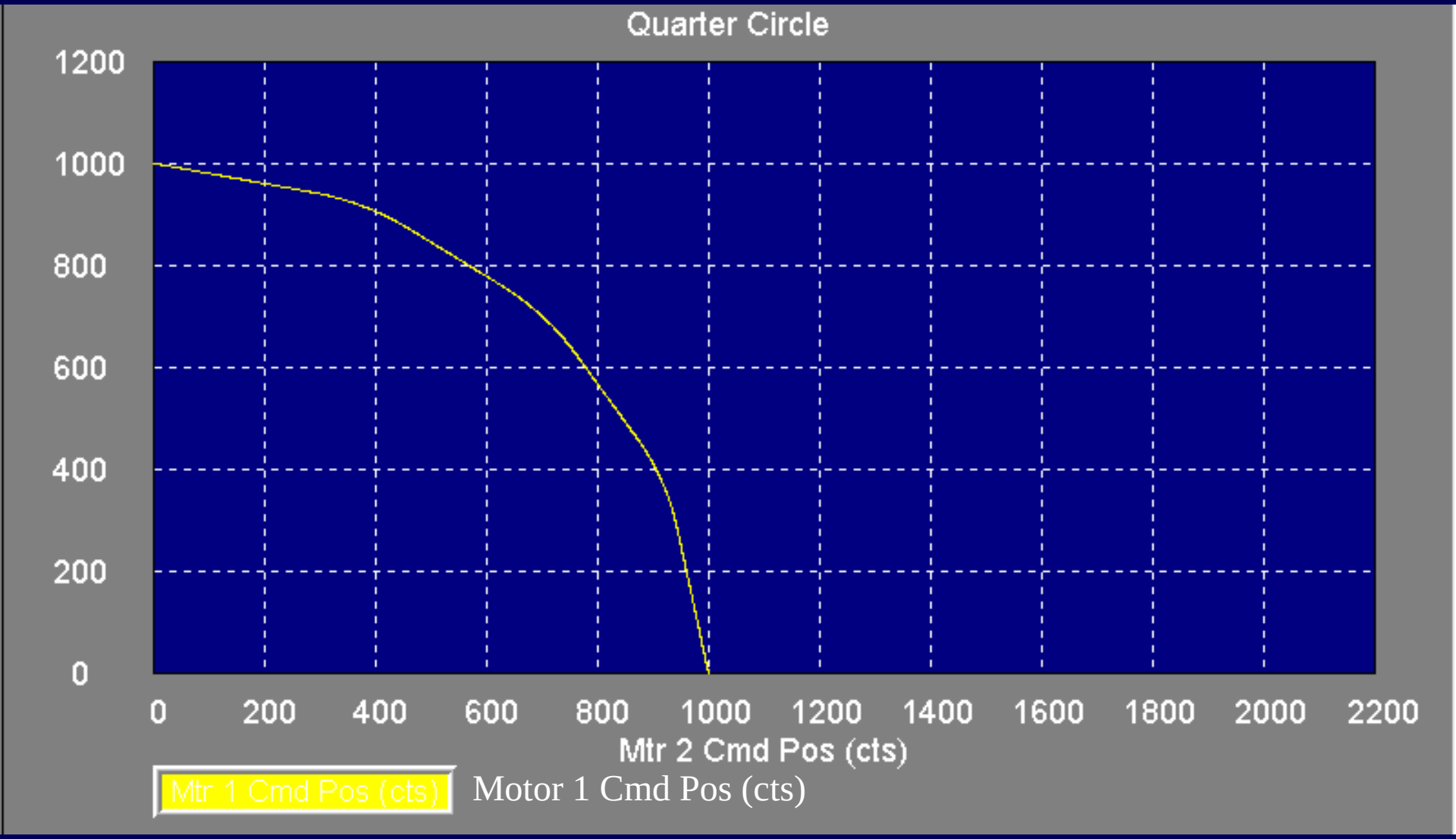
y vs. x Position Plot

Quarter Circle Approximation - Small Accel Time



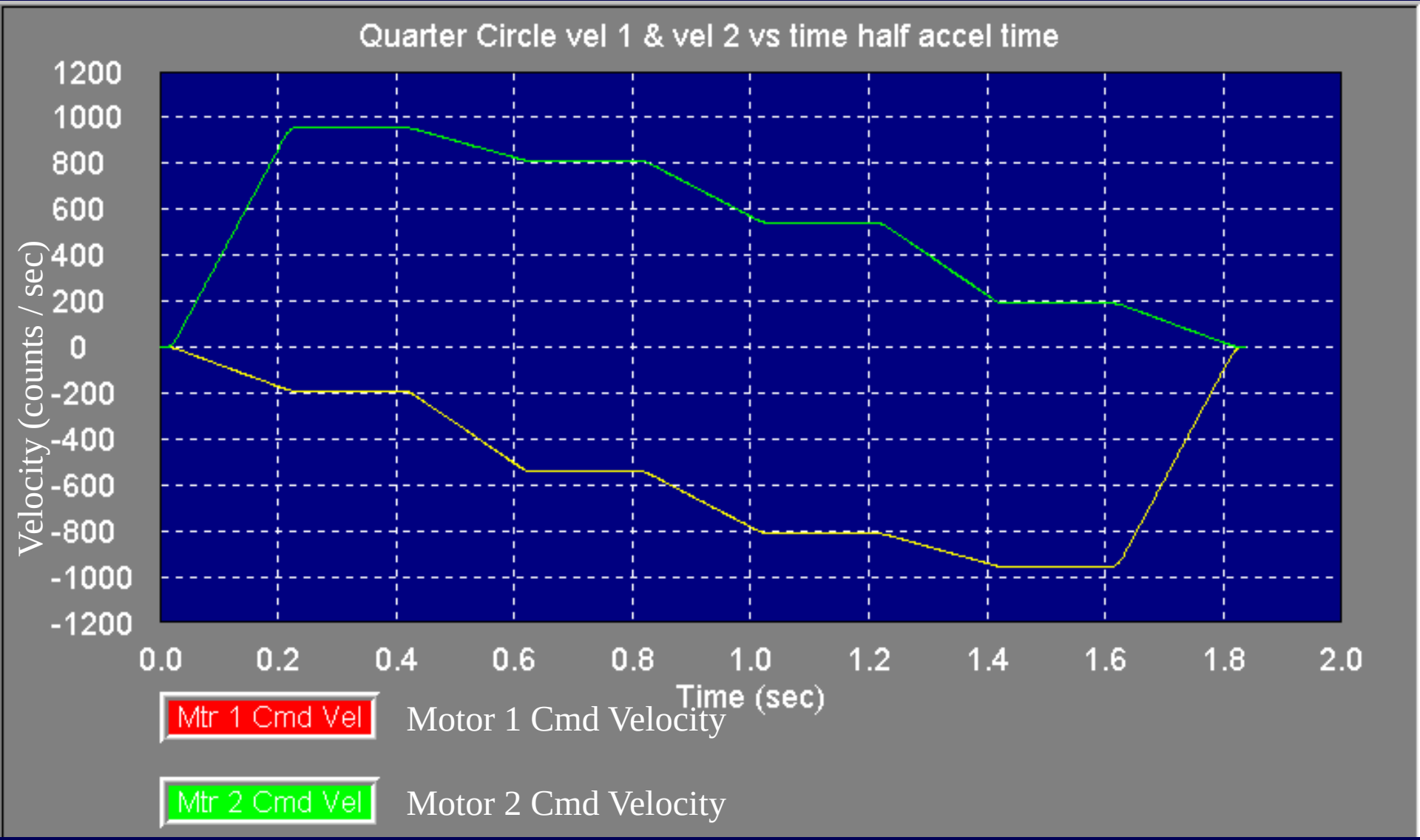
Velocity vs. Time Plots

Quarter Circle Approximation - Half Accel Time



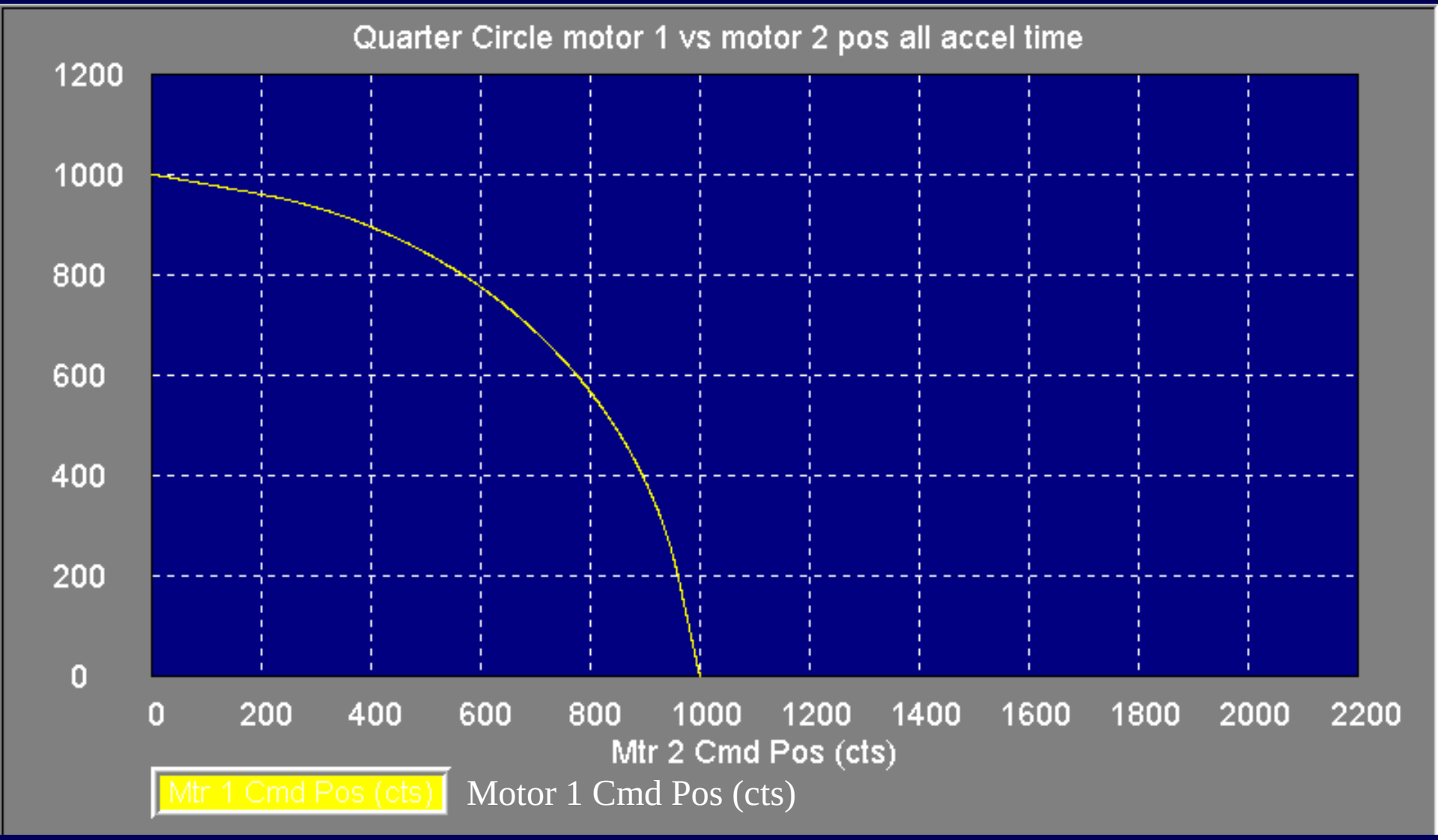
y vs. x Position Plot

Quarter Circle Approximation - Half Accel Time



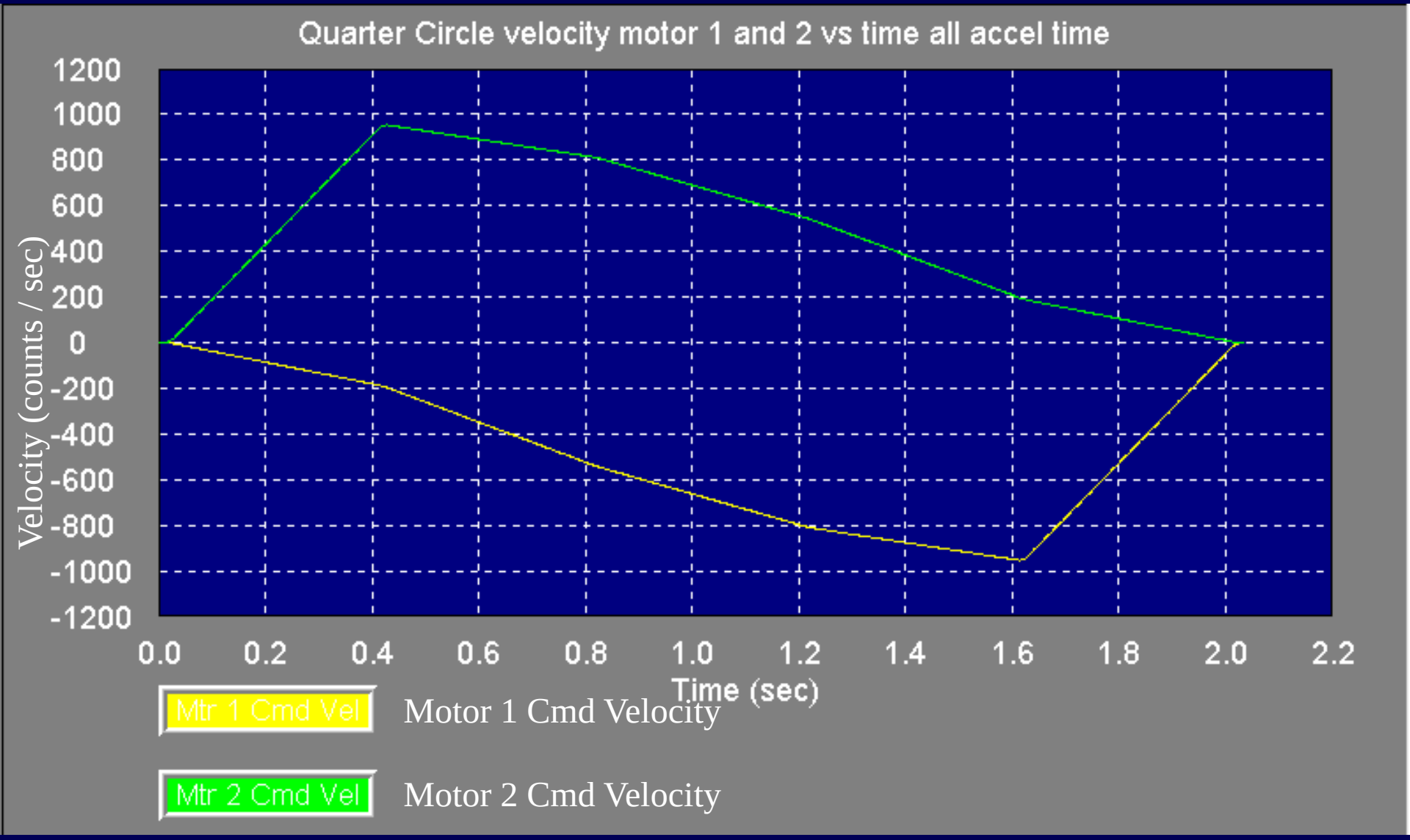
Velocity vs. Time Plots

Quarter Circle Approximation - Full Accel Time



y vs. x Position Plot

Quarter Circle Approximation - Full Accel Time



Velocity vs. Time Plots

Vector Feedrate Axes

Note: X,Y & Z are default feedrate axes

INC
FRAX(X,Y)
X3 Y4 F10

Dist= $\sqrt{3^2+4^2}=5$
Move Time = 5/10 = 0.5
Vx= 3/0.5 = 6
Vy= 4/0.5 = 8

INC
FRAX(X,Y)
X3 Y4 Z12 F10

Dist= $\sqrt{3^2+4^2}= 5$
Move Time = 5/10 = 0.5
Vx= 3/0.5 = 6
Vy= 4/0.5 = 8
Vz= 12/0.5 = 24

Vector Feedrate Axes (continued)

INC
FRAX(X,Y,Z)
X3 Y4 Z12 F10

Dist= $\sqrt{3^2+4^2+12^2}$ = 13
Move Time = 13/10 = 1.3
Vx= 3/ 1.3 = 2.31
Vy= 4/ 1.3 = 3.08
Vz= 12/1.3 = 9.23

INC
FRAX(X,Y,Z)
C10 F10

Dist = 0
Move Time = 0/10 = 0(<TA)

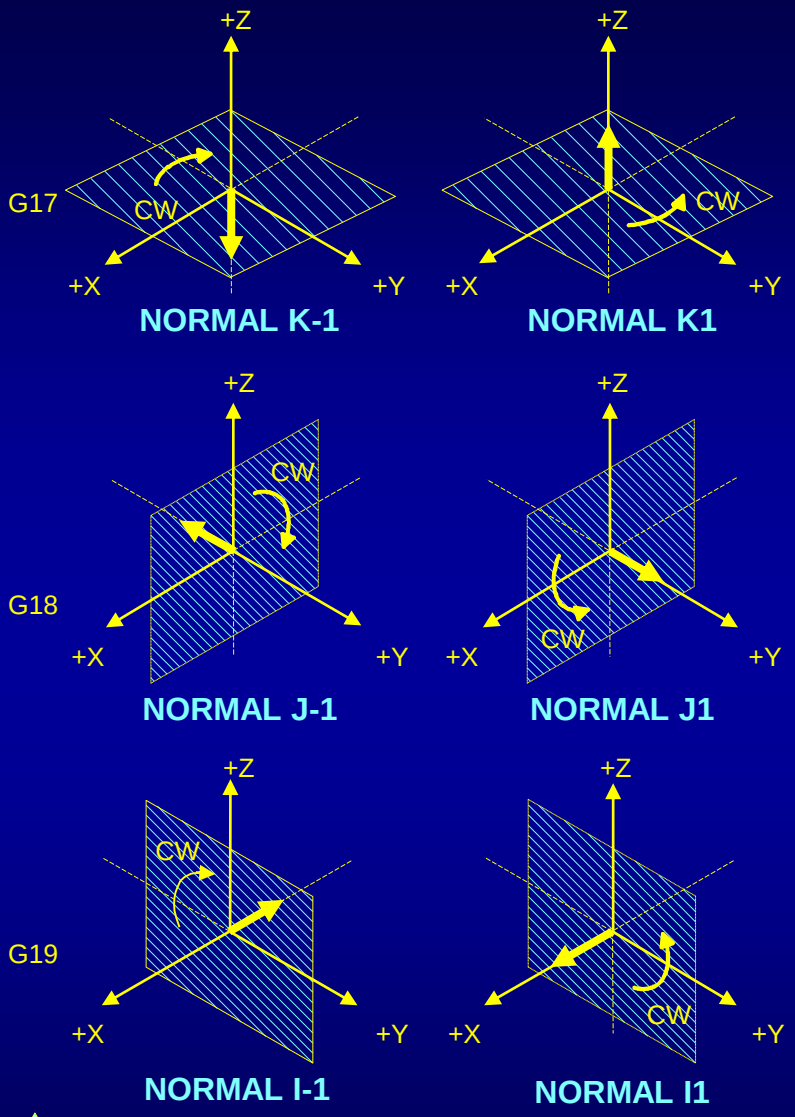
Non-Turbo: C is feedrate axis for this move
Turbo: Isx86 (alternate feedrate) controls speed

CIRCLE Mode Moves

- Circle plane specification (NORMAL)
- End-point (ABS) or distance (INC) specification
- Center vector (I,J,K) or radius (R) specification
- Vector speed (F) or move time (TM) specification
- Acceleration time specification (TA, TS)
- Axes not in circle plane linearly interpolated



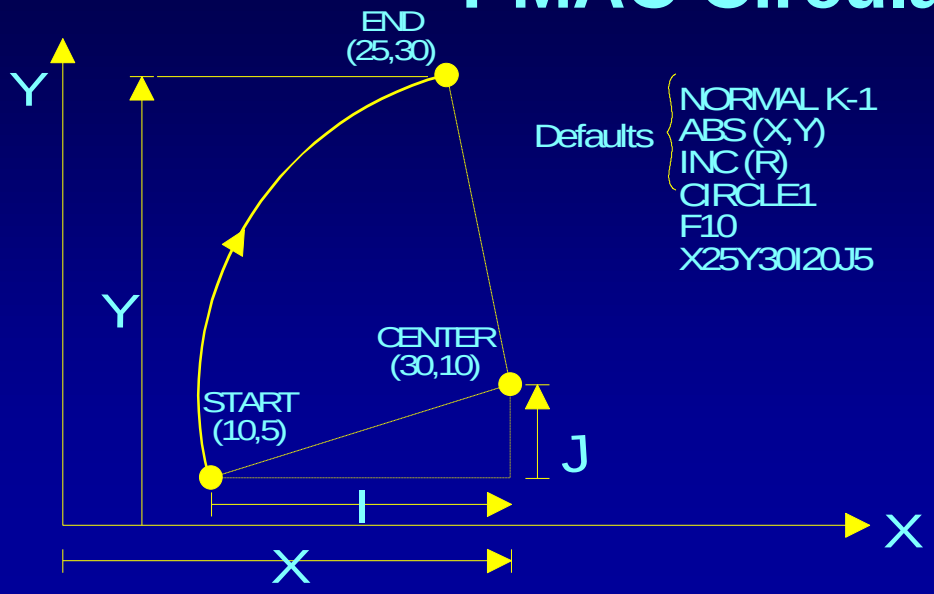
CIRCULAR INTERPOLATION: Plane Specification



NORMAL VECTORS FOR CIRCULAR MOVES:
THE PLANES AND CLOCK WISE ARCS THEY DEFINE

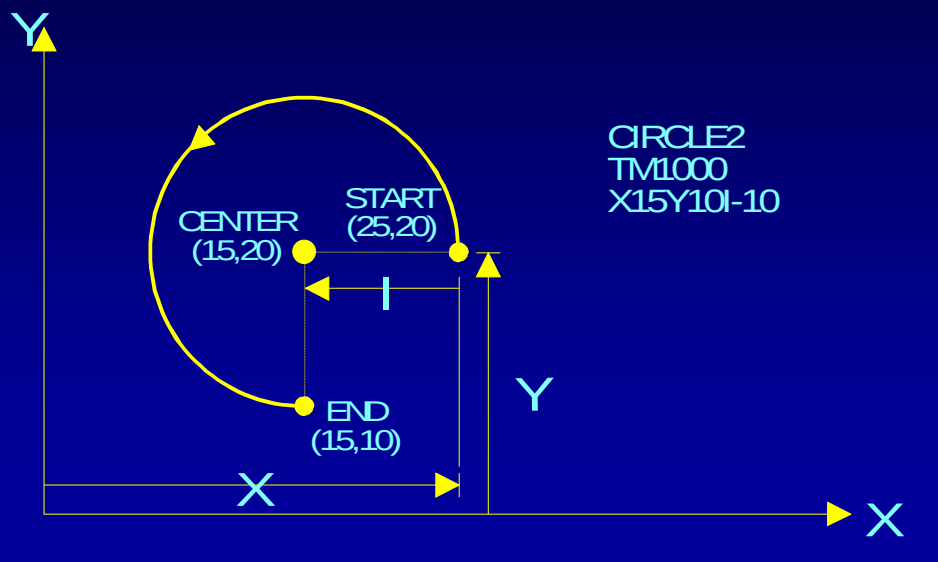
(NRML-VEC)

PMAC Circular Interpolation

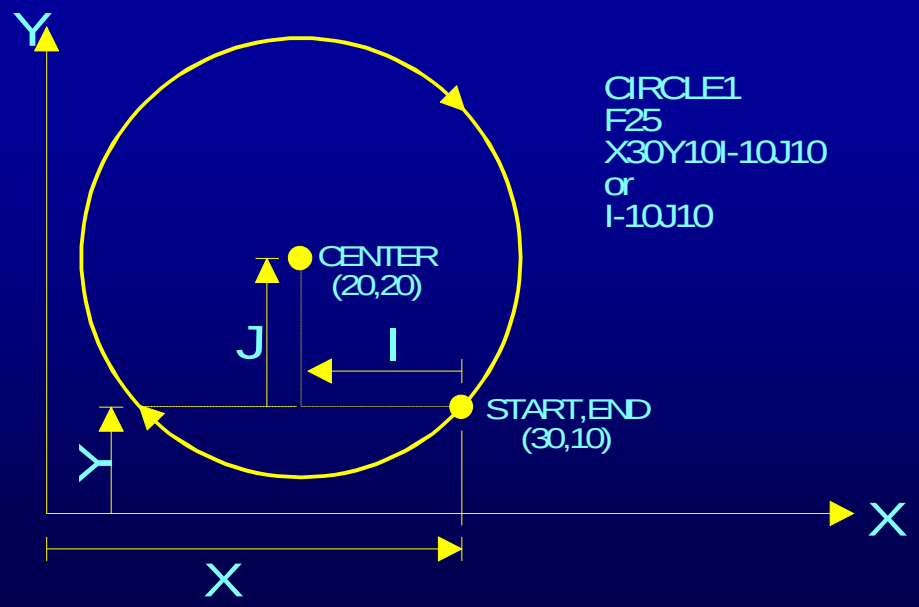


Defaults
- NORMAL K-1
- ABS (X,Y)
- INC (R)
- CIRCLE1
- F10

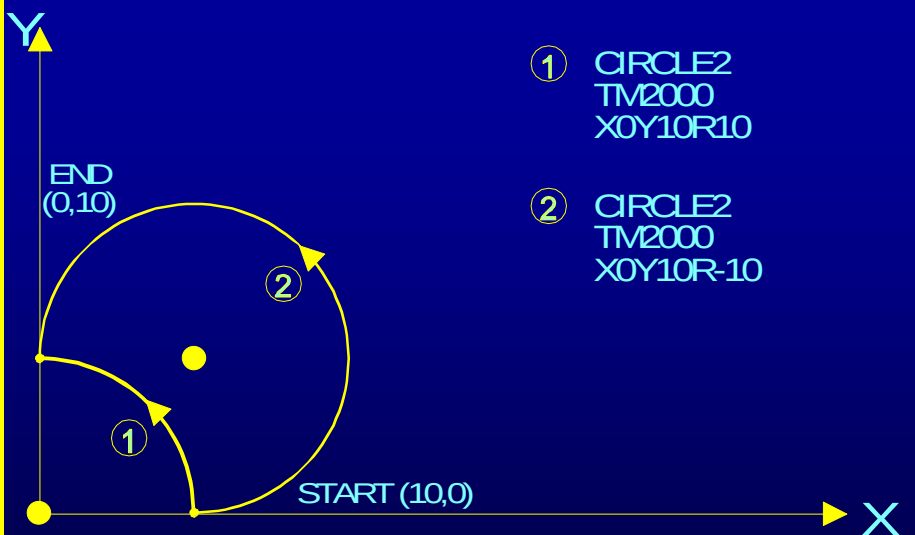
X25Y30I20J5



CIRCLE2
TM1000
X15Y10I-10



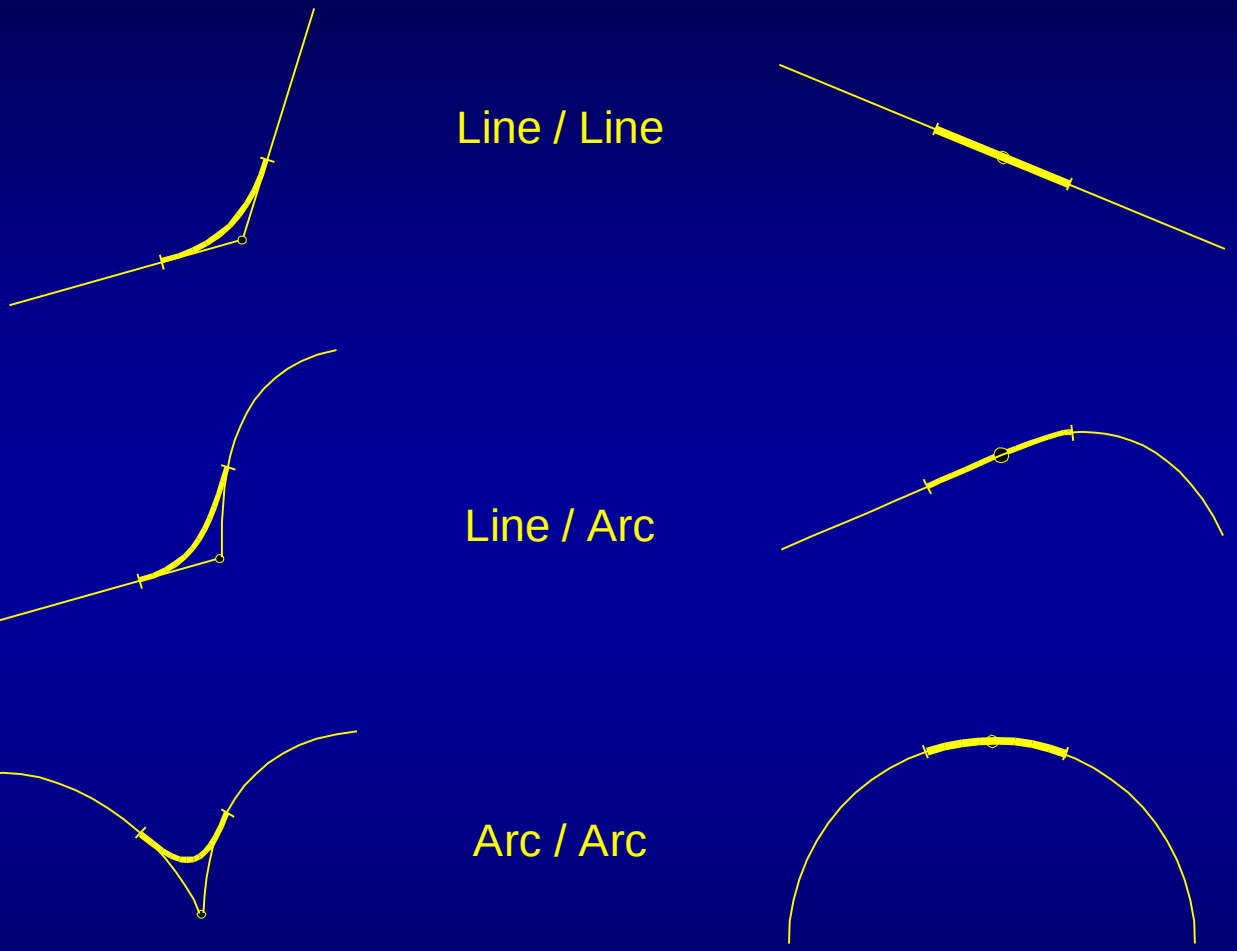
CIRCLE1
F25
X30Y10I-10J10
or
I-10J10



① CIRCLE2
TM2000
X0Y10R10

② CIRCLE2
TM2000
X0Y10R-10

Blending Between Lines and Arcs



Why Use Segmentation Mode (I13>0)?
[Isx13>0 on Turbo]

- **Permits circular interpolation (I13 is “fit” time)**
- **Permits cutter radius compensation (I13 is “fit” time)**
- **Permits jog during feedhold (“\” command to feedhold)**
- **Permits quit at end of executing move (“/” command)**
- **Permits reporting of currently executing move command**
- **In rotary buffer, permits change from calculated deceleration of final move to blending into newly sent move.**
- **Permits inverse kinematics (I13 is “fit” time)**
- **Permits segment buffering for special lookahead and retrace**



Example 4: Linear and Circular Interpolation

This example introduces linear and circular interpolation in a Cartesian X-Y application

```
;************ Set-up and Definitions ************

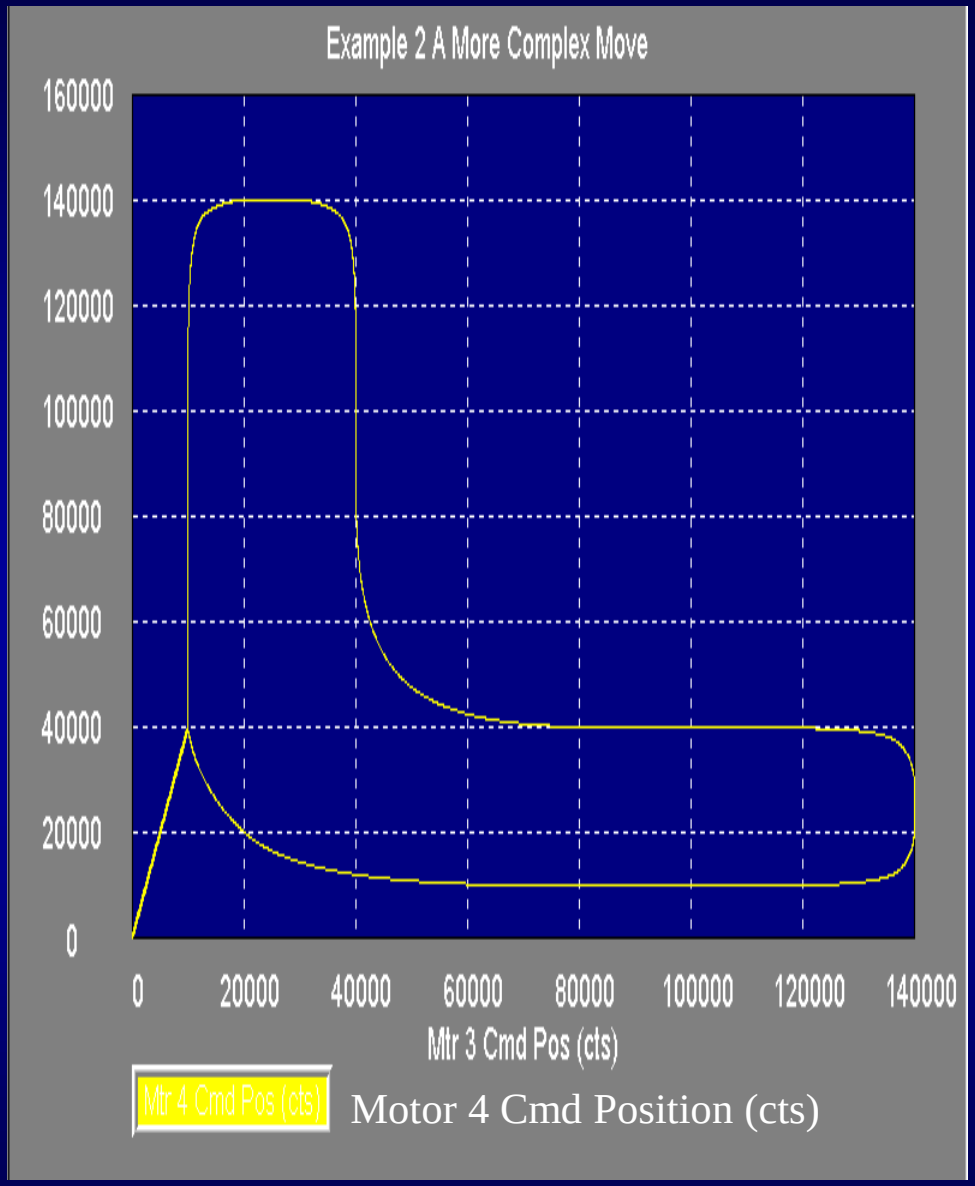
CLOSE                   ; Make sure all buffers are
                        ; closed
&1                      ; Coordinate System 1
#3->10000X              ; Use motor 3 for X-axis
#4->10000Y              ; Use motor 4 for Y-axis
```



Example 4: Linear and Circular Interpolation (continued)

```
OPEN PROG 4 CLEAR      ; Prepare buffer for entry
RAPID X1 Y4            ; Fast move to starting point
F500                   ; Speed for linear and circular moves
LINEAR Y13             ; Straight move
CIRCLE1 X2 Y14 I1 J0   ; CW arc
LINEAR X3              ; Straight move
CIRCLE1 X4 Y13 I0 J-1  ; CW arc
LINEAR Y7              ; Straight move
CIRCLE1 X7 Y4 I3 J0    ; CCW arc
LINEAR X13             ; Straight move
CIRCLE1 X14 Y3 I0 J-1  ; CW arc
LINEAR Y2              ; Straight move
CIRCLE1 X13 Y1 I-1 J0  ; CW arc
LINEAR X4              ; Straight move
CIRCLE1 X1 Y4 I0 J3    ; CW arc
DWELL100               ; Hold 100 ms
RAPID X0 Y0            ; Return home
CLOSE
;To run this program:

&1 B4 R ; Coord. System 1, point to Beg. of Program 4, Run
```


Example 2 A More Complex Move
160000
140000
120000
100000
80000
60000
40000
20000
0
0
20000
40000
60000
80000
100000
120000
140000
Mtr 3 Cmd Pos (cts)
Mtr 4 Cmd Pos (cts)
Motor 4 Cmd Position (cts)

What Is Lookahead?

- Standard feature on all Turbo PMACs
- Algorithm that “scouts ahead” in motion programs
- Checks for violations of motor limits:
  - Position (software overtravel) limits
  - Velocity limits
  - Acceleration limits
- When violation is detected, motion is slowed at problem point
- Algorithm re-calculates motion leading up to problem point to keep all motion within limits



What Can Lookahead Do?

- Lookahead permits acceleration and deceleration over multiple move blocks
- Lookahead permits deceleration into tight corner and acceleration out of it
- Tight corners can be made within acceleration constraints without slowing moves away from corners
- Lookahead permits stopping at position limits, not past them
- Lookahead does not change the programmed path, just how fast the path is traversed

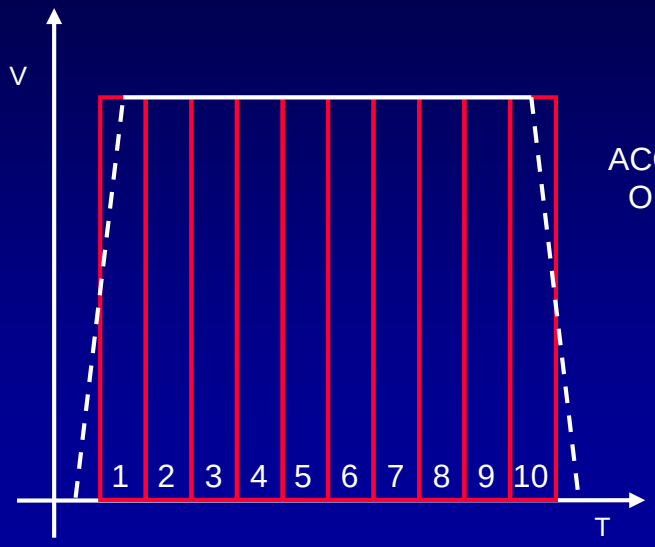


What is "Segmented Lookahead"?

- Lookahead utilizes PMAC's segmentation (coarse interpolation) function
- Lookahead buffer stores relatively uniform motion segments, not arbitrary program move blocks
- Uniformity of segments permits:
  - More efficient calculations
  - Better optimized solutions with long and short blocks
- Tight spacing of segments permits:
  - Accel and decel within program blocks, not just at boundaries
  - Linear approximations for arc moves
- Segments are reversible; motion program is not!



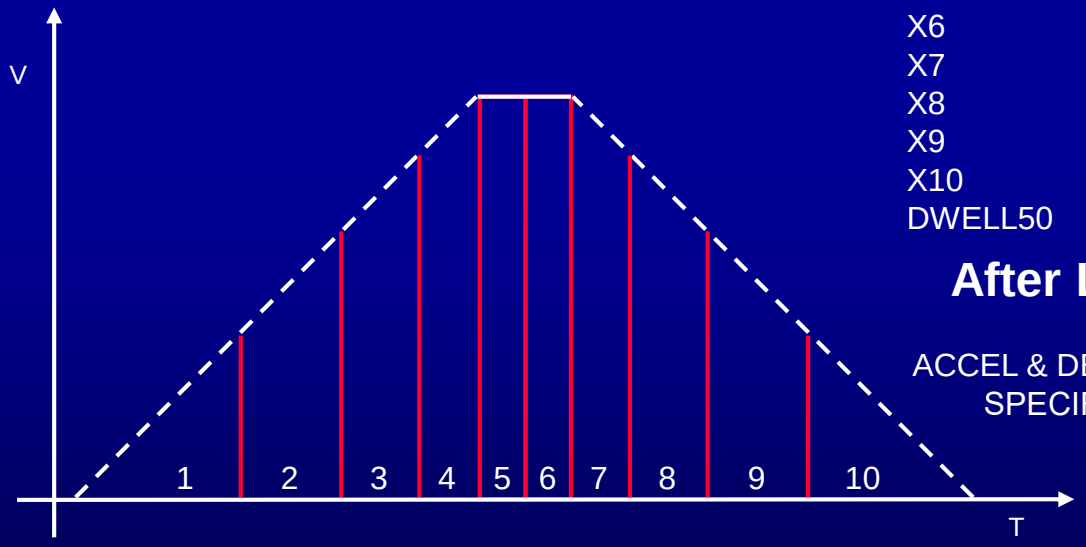
Lookahead for Multi-Block Accel/Decel



ACCEL & DECEL TIME MUST BE EQUAL TO OR SHORTER THAN BLOCK MOVE TIME

Before Lookahead

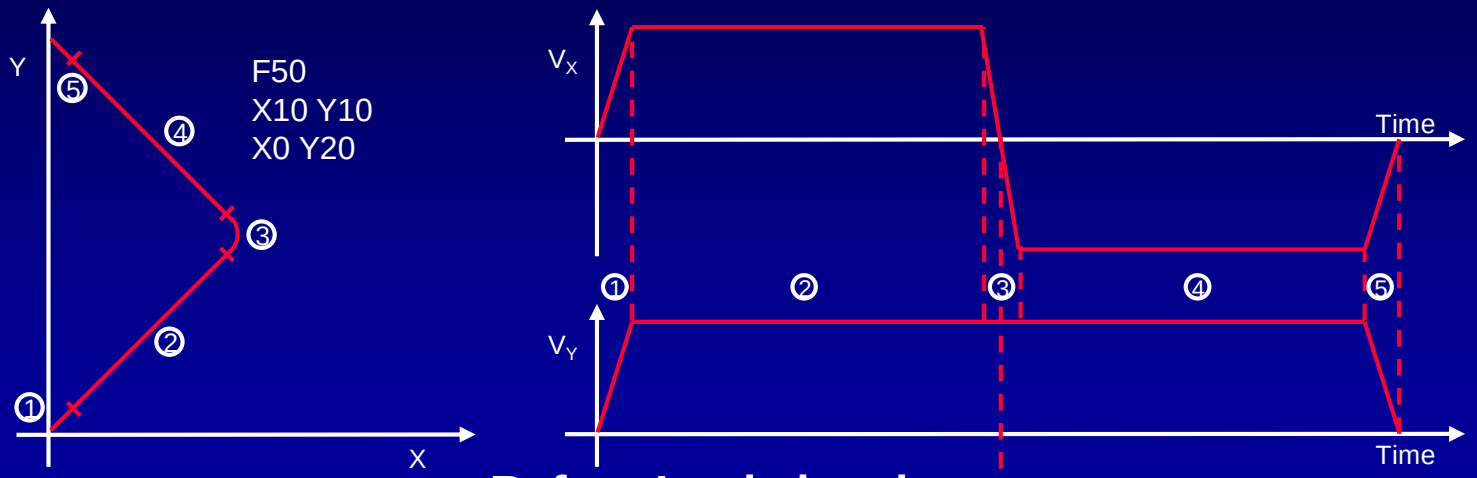
F100
X1
X2
X3
X4
X5
X6
X7
X8
X9
X10
DWELL50

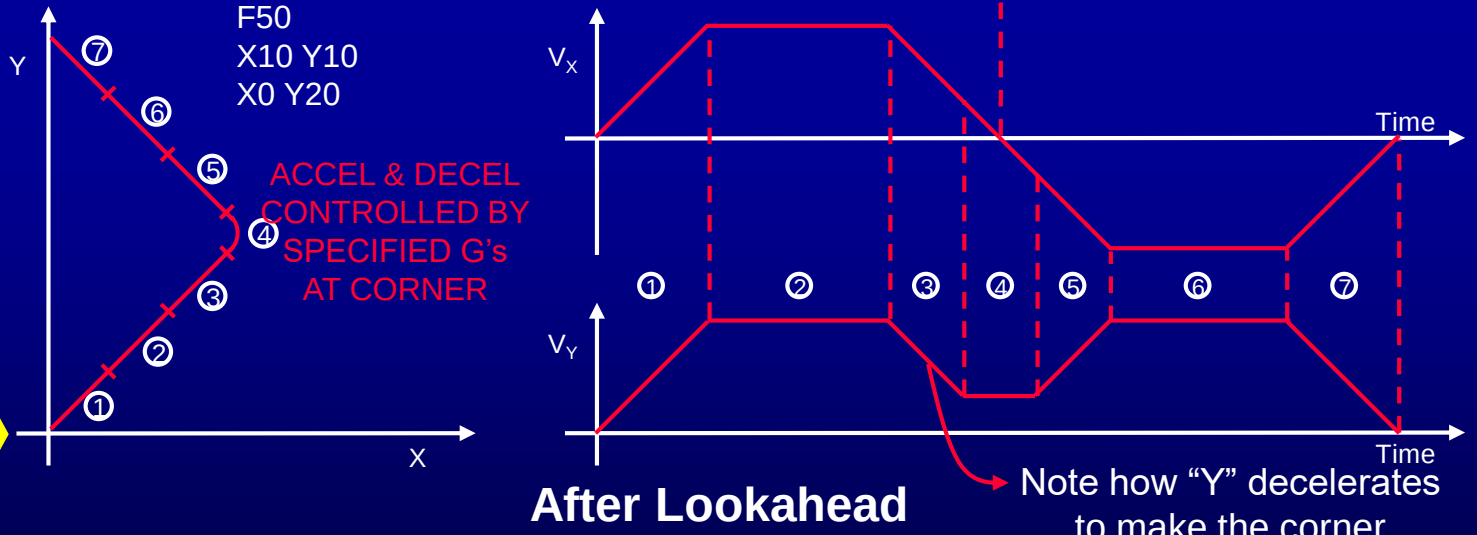


After Lookahead

ACCEL & DECEL ARE CONTROLLED BY SPECIFIED ACCEL LIMIT (G's)

Lookahead & Small, Tight Corners





Lookahead Key Variables

- Ixx13 Motor xx Positive Position Limit
- Ixx14 Motor xx Negative Position Limit
- Ixx16 Motor xx Maximum Velocity
- Ixx17 Motor xx Maximum Acceleration
- Isx13 C.S. ‘x’ Segmentation Time
- DEFINE LOOKAHEAD {# of segs.}, {# of outputs}
- Isx20 C.S. ‘x’ Lookahead Distance (segments)



How Much Lookahead is Needed?

- Need to look ahead the stopping distance/time
- Worst-case stopping time is max velocity (Ixx16) divided by max acceleration (Ixx17)
- Select the largest time for all motors
- Number of segments needed ahead is:
  Stopping Time / (2 * Segment Time)
- Tell PMAC to look ahead 4/3 this number of segments so always have the proper number correct
- Size the lookahead buffer to hold this number of segments plus any segments to reverse through



Turbo Lookahead Speed Capability

- 2 axes: 2000 blocks/second (1 msec seg, 200%)
- 3 axes: 1000 blocks/second (1 msec seg)
- 4 axes: 500+ blocks/second (2 msec seg)
- 5 axes: 500+ blocks/second (2 msec seg)
- 6 axes: 500 blocks/second (2 msec seg)
- 8 axes: 333 blocks/second (3 msec seg)
- 12 axes: 250 blocks/second (4 msec seg)
- 16 axes: 200 blocks/second (5 msec seg)

Segment time = minimum block time;

Default servo update 2.25 kHz, no commutation

Turbo Lookahead Length Capability

- Length of lookahead limited only by memory capacity!
- Longer lookahead does not require more processor speed!
- Standard data memory, 90% for lookahead:
  - 2 axes: 3000 segments
  - 4 axes: 2000 segments
  - 6 axes: 1500 segments
- Expanded data memory, 90% for lookahead:
  - 2 axes: 24,000 segments
  - 4 axes: 16,000 segments
  - 6 axes: 12,000 segments



Reversal (Retrace) in Lookahead

- Oversized lookahead buffer provides “history” of trajectory
- Position information in buffer is reversible (unlike actual motion program)
- The ‘<’ command causes reversal along path
- Reverse operation obeys same limits as forward operation
- Reversal length limited only by buffer memory
- Can only reverse in continuous blended sequence
- Quick stop in either direction with ‘\’ command
- Forward operation again by ‘>’, ‘R’, or ‘S’
- Seamless continuity into new parts of program



Lookahead on Non-Turbo PMAC

- Option 6L is lookahead firmware
- Slight incompatibility with standard firmware
- Lookahead limited to 1 coordinate system
- Lower speed than on Turbo
- No position limits at lookahead time
- Available on any PMAC or PMAC2 version
- Most important is availability on low-cost Mini and Lite controllers



Non-Turbo Lookahead Speed Capability

- 40 MHz, 3 axes: 100 blocks/second
- 60 MHz, 3 axes: 200 blocks/second
- 80 MHz, 3 axes: 333 blocks/second
- Scales proportionately with number of axes
- Testing done with standard 2.25 kHz servo update, 9 kHz phase update



Altdest.pmc

```
; This program allows altered destination in moves.
; The card is always moving toward an X position of
; P1 and a Y position of P2, but it does it in small
; increments so when P1 or P2 is changed, it will
; change direction within a few milliseconds.  P1
; and P2 can be changed from the host on the fly.
```



Altdest.pmc

```
CLOSE
OPEN PROG 2
CLEAR
P9=100              ; Vector distance per increment
P11=0               ; Starting X position
P12=0               ; Starting Y position
TA10                ; C.S. 1 accel time in msec
TS0                 ; C.S. 1 S-curve time in msec
TM10                ; Increment time in msec
WHILE (M1=1)        ; Whatever condition you like
  P21=P1-P11        ; X distance to go
  P22=P2-P12        ; Y distance to go
  P0=SQRT(P21*P21+P22*P22)     ; Vector distance to go
  IF(P0>P9)                    ; If longer than increment
    P0=P9/P0                   ; Fraction of distance
    P11=P11+P21*P0             ; X increment
    P12=P12+P22*P0             ; Y increment
  ELSE                         ; Distance < increment
    P11=P1                     ; Just go to destination
    P12=P2
  ENDIF
  X(P11) Y(P12)                ; Make move
ENDWHILE
CLOSE
```

ChgSpeed

This program is an example of moving at high speed until some sort of external command, then reacting very quickly to the command, but changing speed very slowly. It works by splitting the move into very small segments, and using WHILE loops to continue in one mode until the signal is received.



```
CLOSE
OPEN PROG 10
CLEAR
P4=150000
P5=3000
P6=600
INC(X)                  ; X-axis in incremental mode
TM(P5/P6)               ; move segment time (5 msec here!)
TA(P5/P6)               ; accel time (<= TM)
P3=P4/(P6*P6)           ; multiplier to get distance
P1=0                    ; start at zero speed
P2=0                    ; start at zero distance
N10 WHILE (P0 != 0)     ; P0=0 is signal to decelerate
  IF (P1<P6)            ; not up to full speed?
    P1=P1+1             ; increment to increase speed
    P2=P1*P3            ; distance to travel in segment
  ENDIF
  X(P2)                 ; incremental move segment
ENDWHILE
WHILE (P0=0)            ; P0=1 is signal to accelerate
  IF (P1>0)             ; still moving?
    P1=P1-1             ; decrement to decrease speed
    P2=P1*P3            ; distance to travel in segment
  ENDIF
  X(P2)                 ; incremental move segment
ENDWHILE
GOTO 10                 ; loops back up when P0=1 again
CLOSE
```

Rapid Mode Moves

- Endpoint (ABS) or distance (INC) specification
- Max. speed (Ix16) or jog speed (Ix22) specification
- Acceleration time (Ix20, Ix21) or rate (Ix19) specification
- Only axis with highest distance/speed ratio goes at specified speed; others match this time
- Straight-line path if acceleration times match
- Not constant vector speed at different directions
- Move until trigger: e.g. X100^2.5
- Altered destination: e.g. ! X19.93 Y31.25

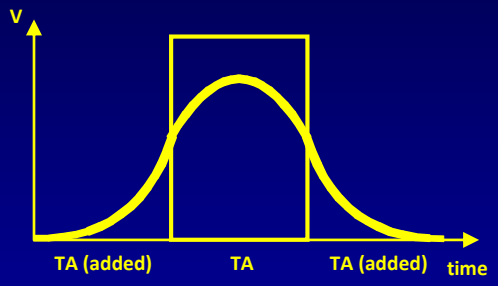


SPLINE Mode Moves

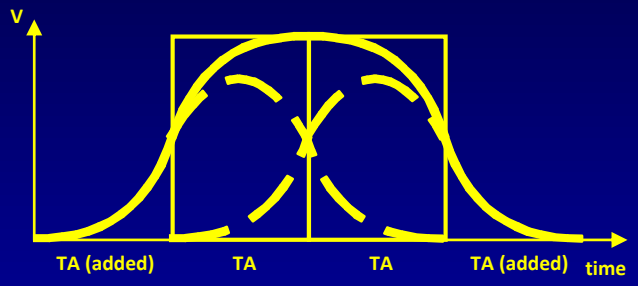
- End-point (ABS) or distance (INC) specification
- Non-Turbo: Move time (TA) specification, nearest millisecond
- Turbo: Move time (TM) specification, ¼ µsec resolution
- SPLINE1 mode requires constant move time
- SPLINE2 mode permits variable move time, should be only slowly changing
- Position, velocity, and acceleration continuous at move boundaries
- Blending rounds to inside of programmed points
- Generates cubic B-spline path
- Most efficient computation of any PMAC mode



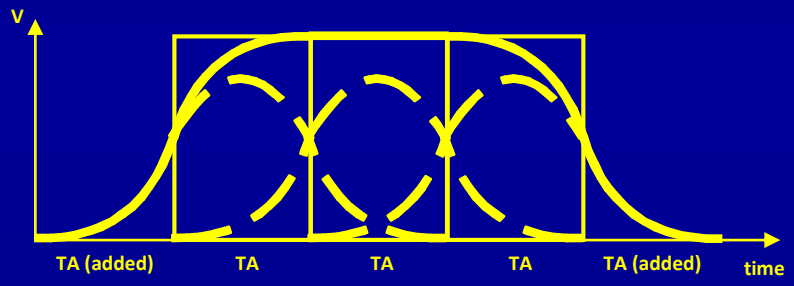
Cubic Spline Trajectories



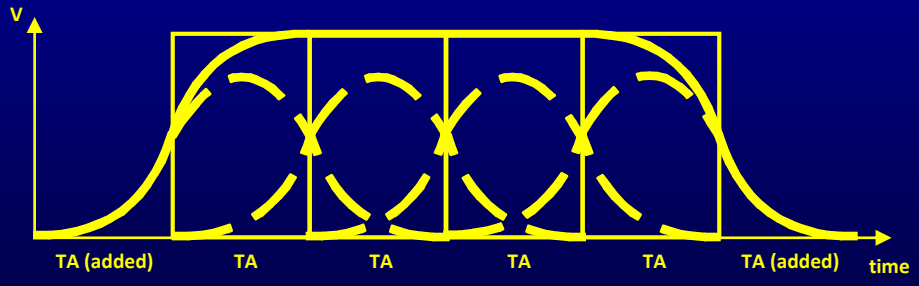
One Programmed Segment



Two Programmed Segments

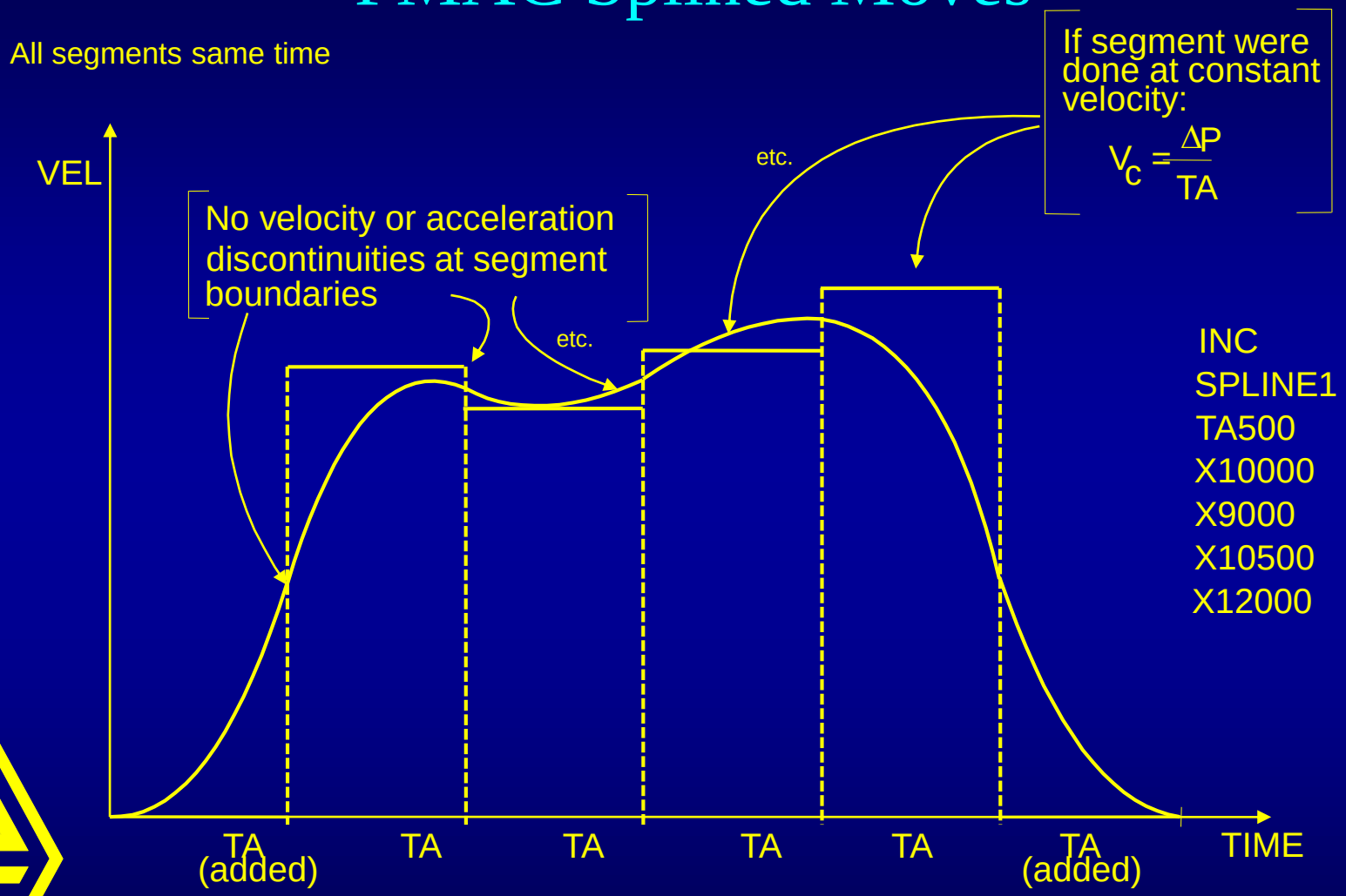


Three Programmed Segments



Four Programmed Segments

PMAC Splined Moves



PVT 模式运动

- End-point (ABS) or distance (INC) specification
- Signed end velocity specification each axis
- [Non-Turbo]: Movetime (TA) specification, millisecond resolution
- [Turbo]: Movetime (TM) specification, 1/4-microsecond resolution
- Position and velocity continuous at move boundaries

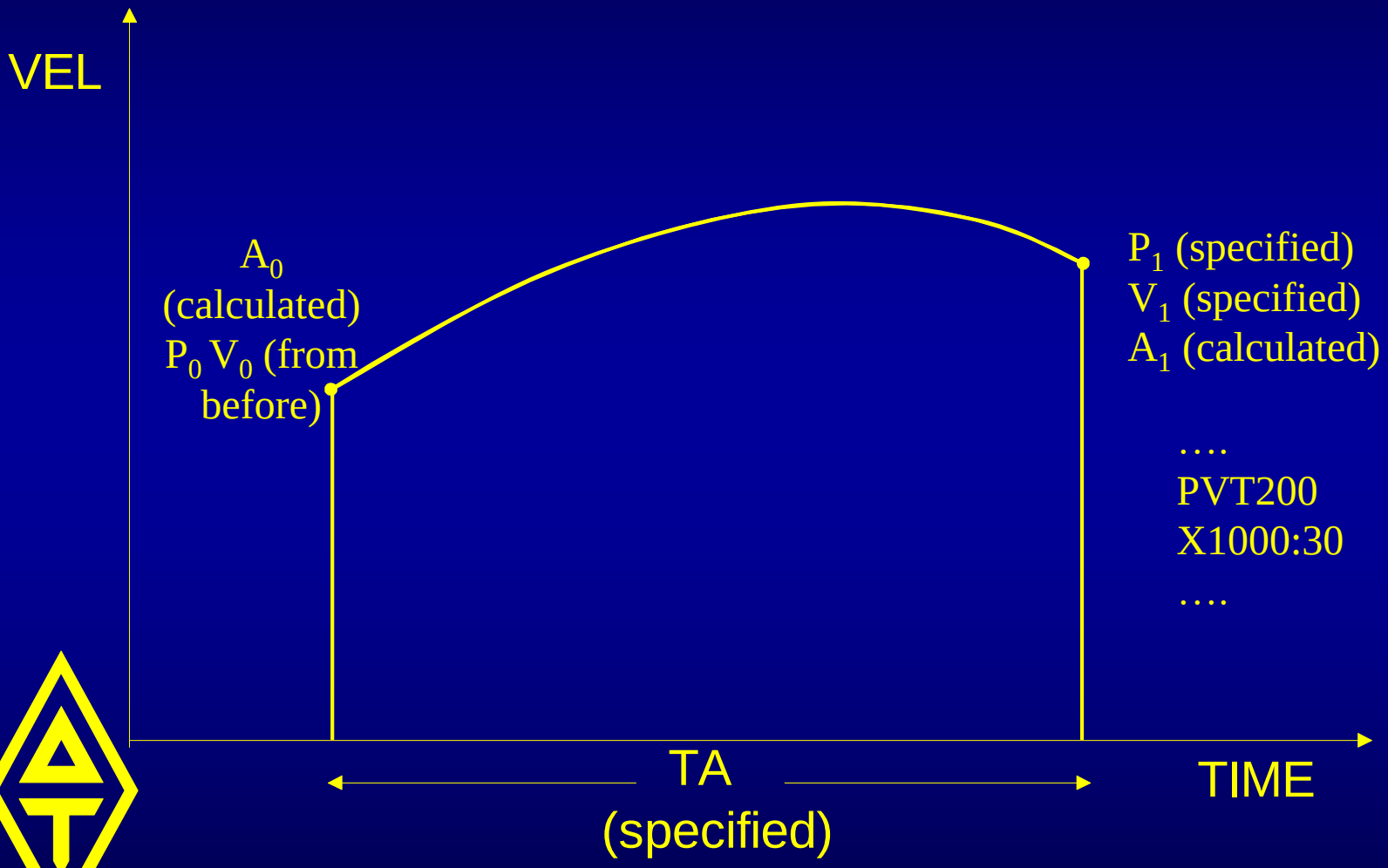


PVT Mode Moves (continued)

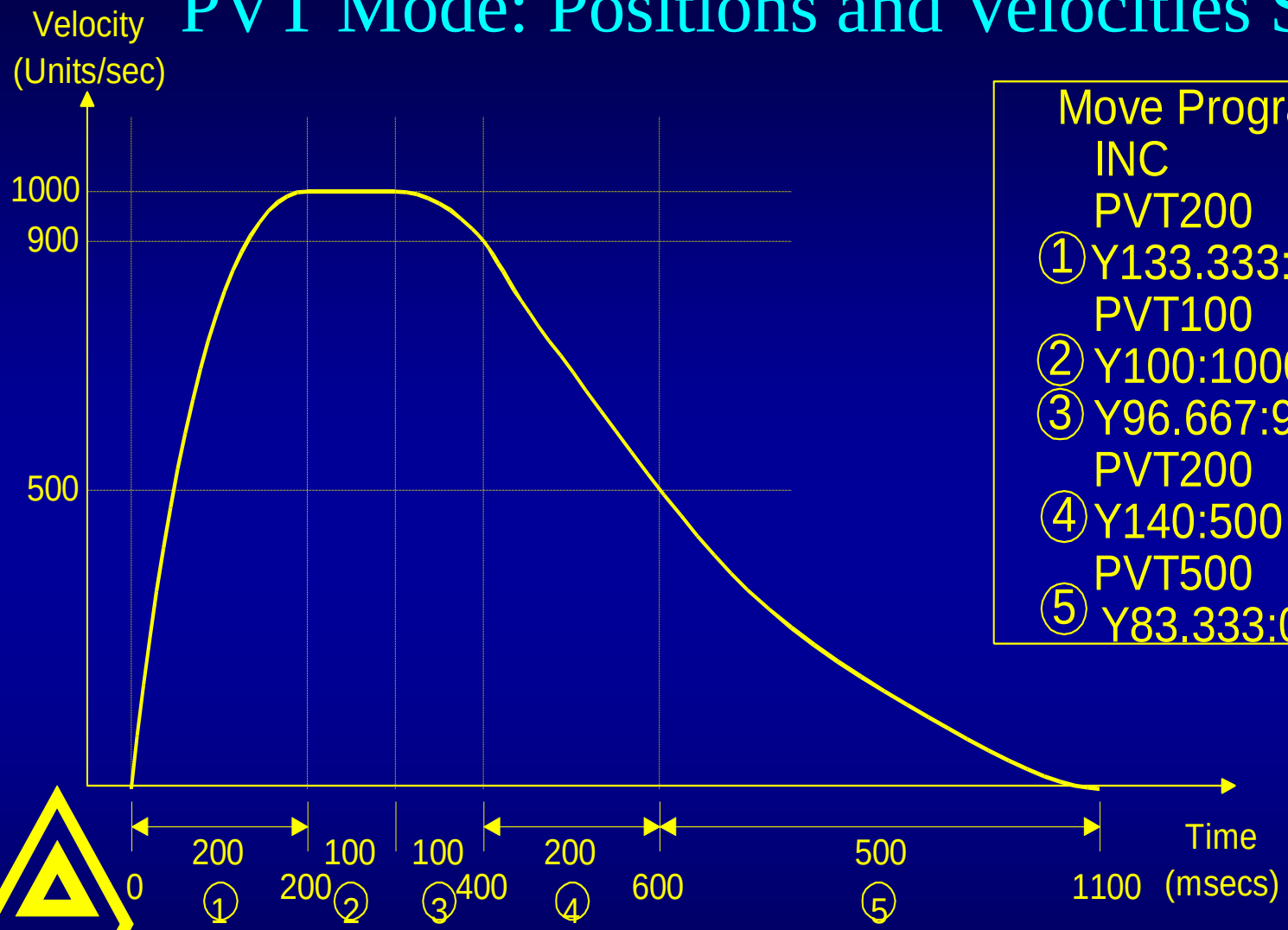
- Blending passes exactly through programmed points
- Good to create arbitrary profiles
- Generates Hermite-spline path
- Very efficient computation



Position, Velocity, and Time (PVT mode) [Parabolic Velocity]

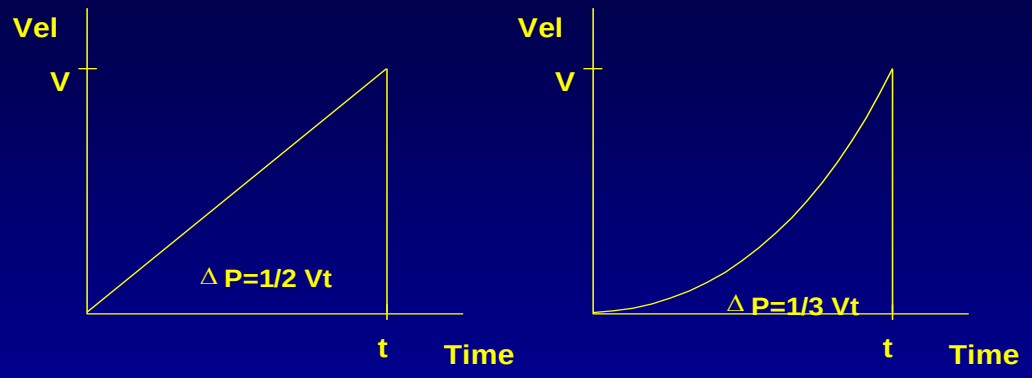


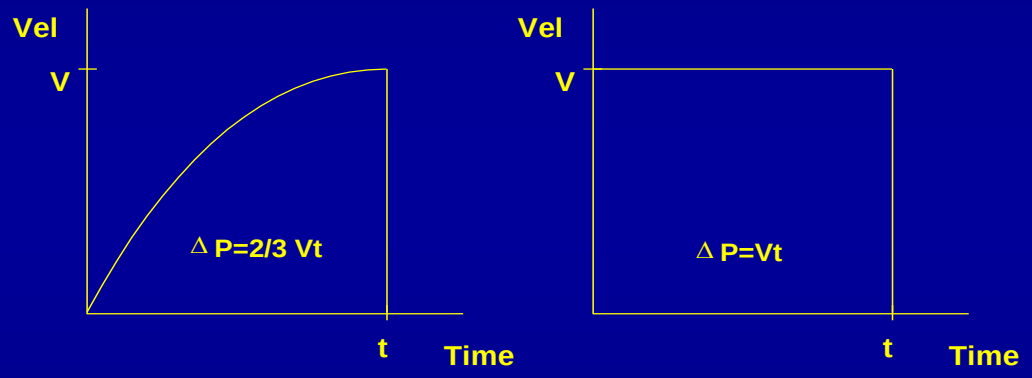
PVT Mode: Positions and Velocities Specified

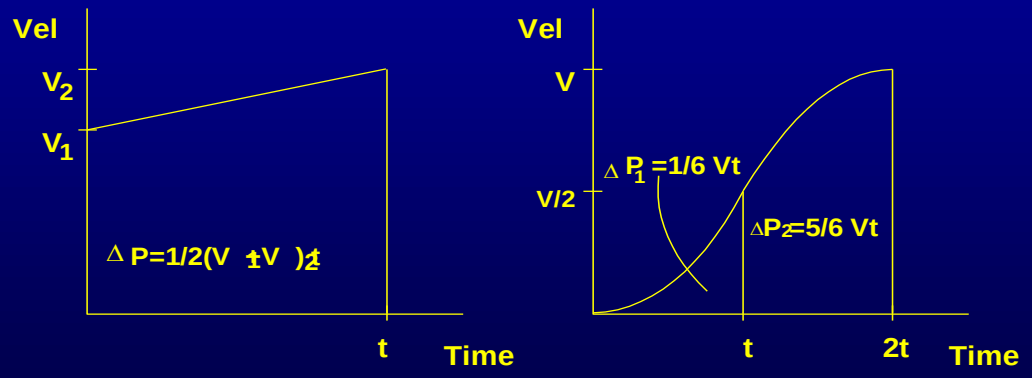



```
Move Program
   INC
   PVT200
①  Y133.333:1000
   PVT100
②  Y100:1000
③  Y96.667:900
   PVT200
④  Y140:500
   PVT500
⑤  Y83.333:0
```

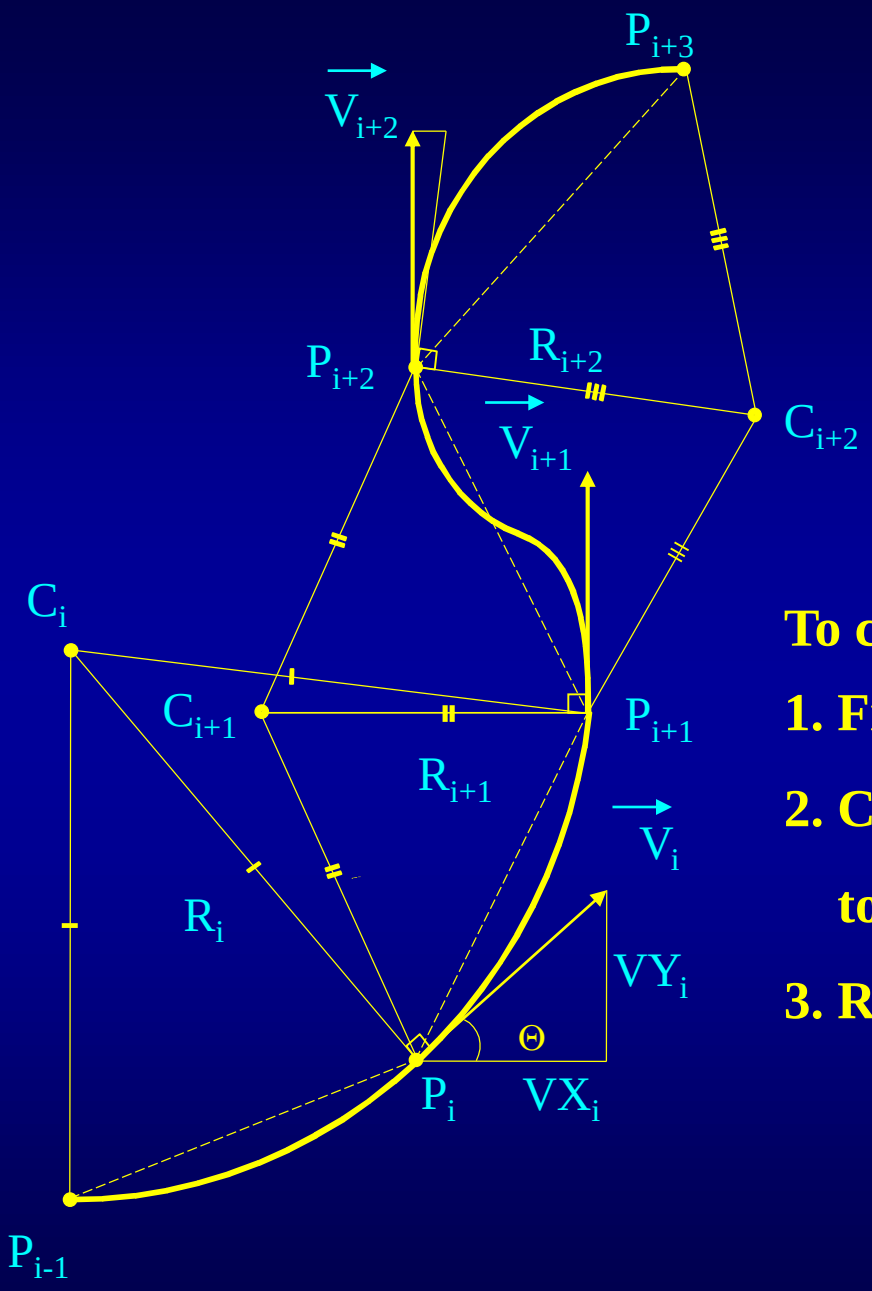
Common PVT Segment Shapes







PVT Mode Contouring (Hermite Spline)

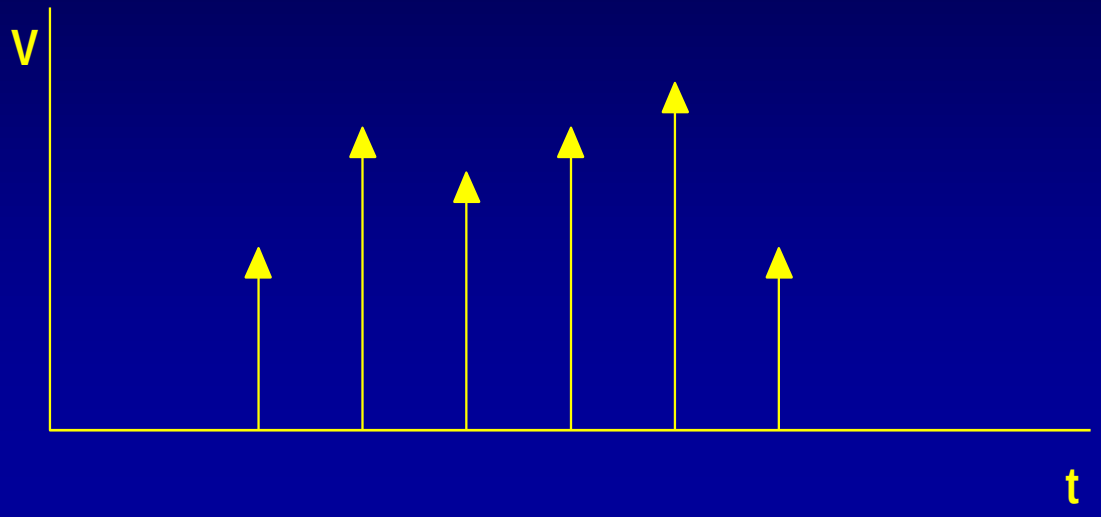


To compute axis velocities at point P:

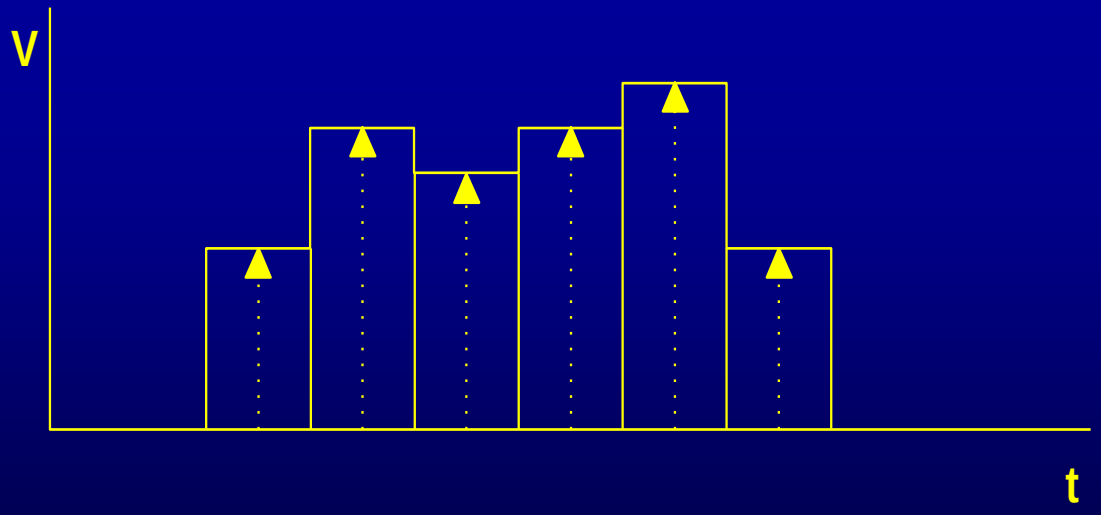
1. **Find common center of P_{i-1}, P_i, and P_{i+1}**
2. **Compute velocity vector as normal to radius vector**
3. **Resolve velocity vector into components**

Contouring Interpolation Methods

Steps
(0th Order Update)

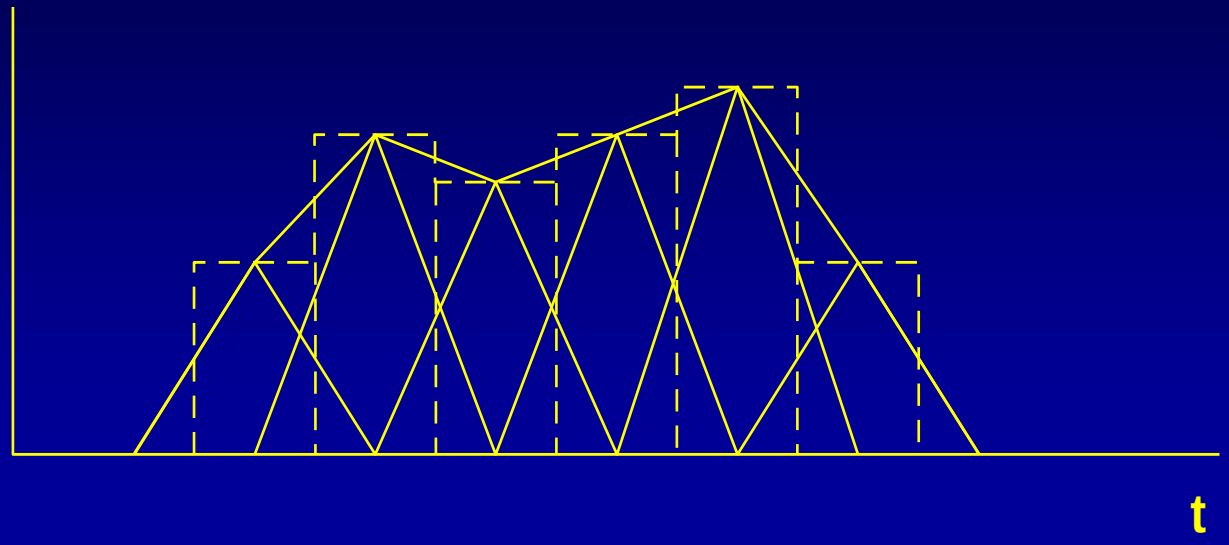


Constant Velocity
(1st Order Update)

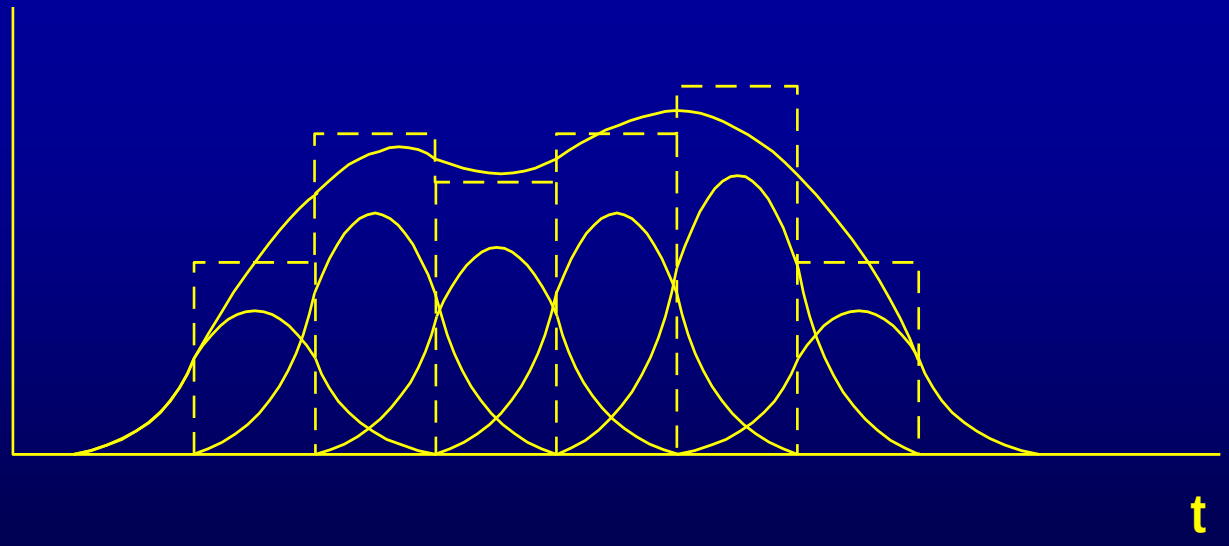


Contouring Interpolation Methods (continued)

Constant Accel
(2nd Order Update)



3-Point Spline
(3rd Order Update)



Subroutines and Subprograms

GOSUB 300 jumps to:

N 300 of same motion program,

CALL 500 jumps to:

PROG 500, at the top (N0)

CALL 500.1 jumps to:

PROG 500, label N10000

CALL 500.12 jumps to:

PROG 500, label N12000

CALL 500.12345 jumps to:

PROG 500, label N12345

On-line command B700 points to:

PROG 700, (N0) ready to run

On-line command B700.34 points to:

PROG 700, N34000, ready to run



Passing Arguments to Subroutines

```
{PROG 1}
CALL 500 D10E20

{PROG 500}
READ (D,E)
     sets Q104 to 10
     sets Q105 to 20
```

A argument reads into Q101
B argument reads into Q102
• • •
Y argument reads into Q125
Z argument reads into Q126



PMAC Subroutine Parameter Pass Check

- **Bit (n-1) of Q100 set to 1 if "nth" letter passed argument in last READ statement**

IF(Q100 & 16>0)

is true when "E" (5th letter) has been passed ($1=2^{5-1}$)

- **Q100 set to 0 at beginning of READ**
- **Successful read of letter value sets corresponding bit of Q100 to 1**

Z	Y		F	E	D	C	B	A	*Letter*
0	0	• • •	0	1	1	0	0	0	*Q100*

25	24	• • •	5	4	3	2	1	0	*Bit #*
2^{25}	2^{24}	• • •	32	16	8	4	2	1	*Bit Value (Dec)*
2^{25}	2^{24}	• • •	20	10	8	4	2	1	*Bit Value (Hex)*

```
PROG1
CALL 500 D10 E20
```

```
PROG500
READ (D,E)
```

G,M,T and D Codes

- For RS-274 compatible motion programs

G 73 is equivalent to
CALL 1000.73000

M 3 is equivalent to
CALL 1001.03000

T 01 is equivalent to
CALL 1002.01000

D 12 is equivalent to
CALL 1003.12000



G, M, T, and D Codes (continued)

- System designer writes custom routines in PROG 1000 to PROG 1003
- Part programmers and machine operators can treat system as regular “G-Code” machine
- PROG 1000 and up can password protected
  (also PLC’s 0 - 15)



Example 5: A Simple G-Code Program

```
;*************** Part Program Text ******************
;Note that the  part program does not need to know
;how the G-codes and M-codes are executed
OPEN PROG 5 CLEAR ;Prepare motion program 5 for entry
G17 G90           ; XY plane, absolute move spec
G97 S1800         ; Set spindle speed of 1800 rpm
F500              ; Cutting speed 500 mm/min
G00 X10.00 Y5.00  ; Rapid move to (10, 5)
M03               ; Start spindle
G04 P2.0          ; Wait 2 seconds
G01 Z0            ; Lower cutter
X30.25 Y5.00      ; Linear XY move
```



Example 5: A Simple G-Code Program (continued)

```
G03 X35.25 Y10.00 J5          ; CCW arc move
G01 X35.25 Y50.10             ; Linear move
G03 X30.25 Y55.10 I-5         ; CCW arc move
G01 X10.00 Y55.10             ; Linear move
G03 X5.00 Y50.10 J-5          ; CCW arc move
G01 X5.00 Y10.00              ; Linear move
G03 X10.00 Y5.00 I5           ; CCW arc move
G01 Z5 M05                    ; Cutter up, stop
G00 X0 Y0                     ; Back to home
CLOSE
```



Example 5: A Simple G-Code Program (continued)

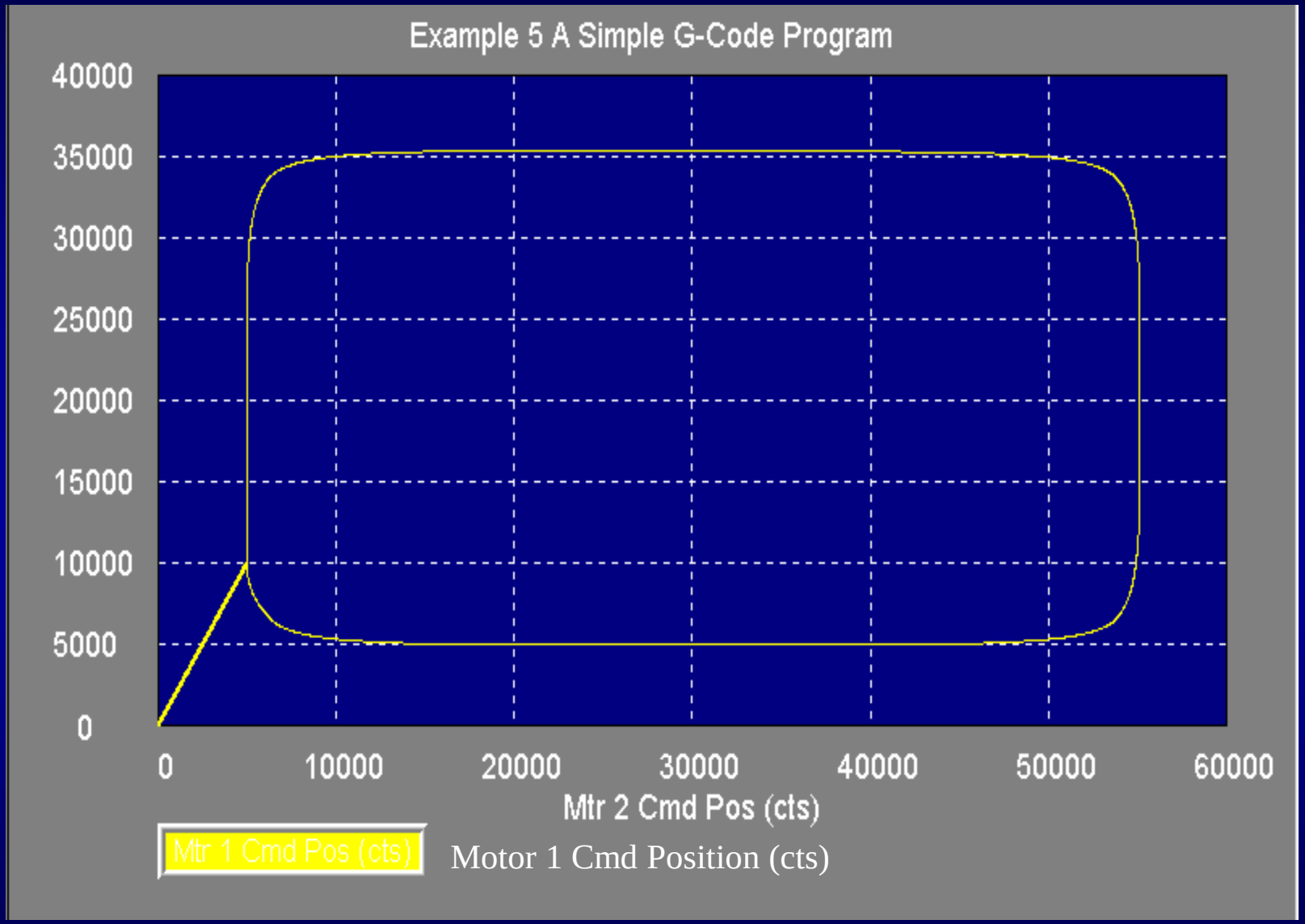
```
OPEN PROG 1000 CLEAR  ; prepare buffer 1000 for entry
RAPID RETURN          ; G00Rapid mode (N0 is implied)
N01000 LINEAR  RETURN ; G01 Linear interpolation mode
N02000 CIRCLE1 RETURN ; G02 CW circle mode
N03000 CIRCLE2 RETURN ; G03 CCW circle mode
N04000 READ(P)        ; G04 Dwell for P seconds
IF (Q100 & 32768 > 0) ; P parameter specified?
DWELL (Q116*1000)     ; PMAC specifies dwell in msec
ENDIF
RETURN
N17000 NORMAL K-1 RET ; G17 Specify XY plane
N18000 NORMAL J-1 RET ; G18 Specify ZX plane
N19000 NORMAL I-1 RET ; G19 Specify YZ plane
N90000 ABS RET        ; G90 Absolute mode
N91000 INC RET        ; G91 Incremental mode
N97000 READ(S)        ; G97 Spindle speed set
IF (Q100 & 262144 > 0); S parameter specified?
I422=Q119/30          ; #4 jog speed in cts/msec
ENDIF
RETURN
CLOSE
```

Example 5: A Simple G-Code Program (continued)

```
OPEN PROG 1001 CLEAR   ; Prepare buffer 1001 for entry
N03000 CMD "#4J+" RET  ; Start spindle clockwise
                       ; (closed loop)
N04000 CMD "#4J-" RET  ; Start spindle
                       ; counterclockwise (ditto)
N05000 CMD "#4J/" RET  ; Stop spindle
CLOSE
;To run this program:

&1 B5 R  ;Coordinate System 1, point to Beginning of
Program 5, Run
```





Motor 1 Cmd Position (cts)

Coordinate System Variables

Ix87 Default Acceleration Time (Program)

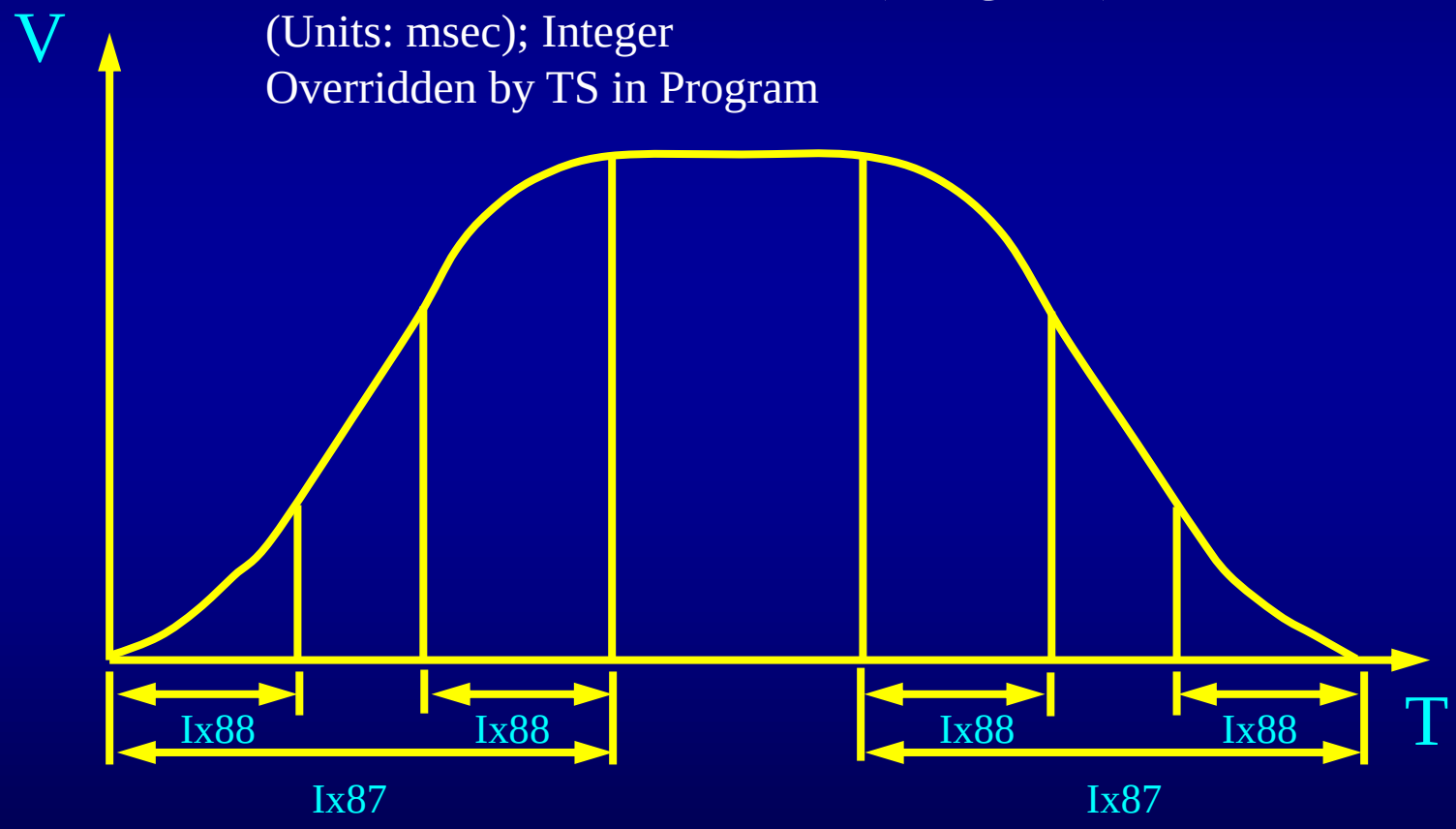
(Units: msec); Integer

Overridden by TA in Program

Ix88 Default S-Curve Time (Program)

(Units: msec); Integer

Overridden by TS in Program



Ix90 FEEDRATE TIME UNITS (Unit: msec)

$$\text{F UNIT} = \frac{\text{LENGTH UNIT}}{\text{TIME UNIT}} \Rightarrow \frac{\text{AXIS DEF. SCALING}}{\text{Ix90 (msec)}}$$

EXAMPLE: #1 -> 2000X
Ix90 = 1000

$$\text{F UNIT} = \frac{2000\text{cts}}{1000\text{msec}} \Rightarrow \frac{\text{mm}}{\text{sec}}$$

Ix92 BLEND DISABLE

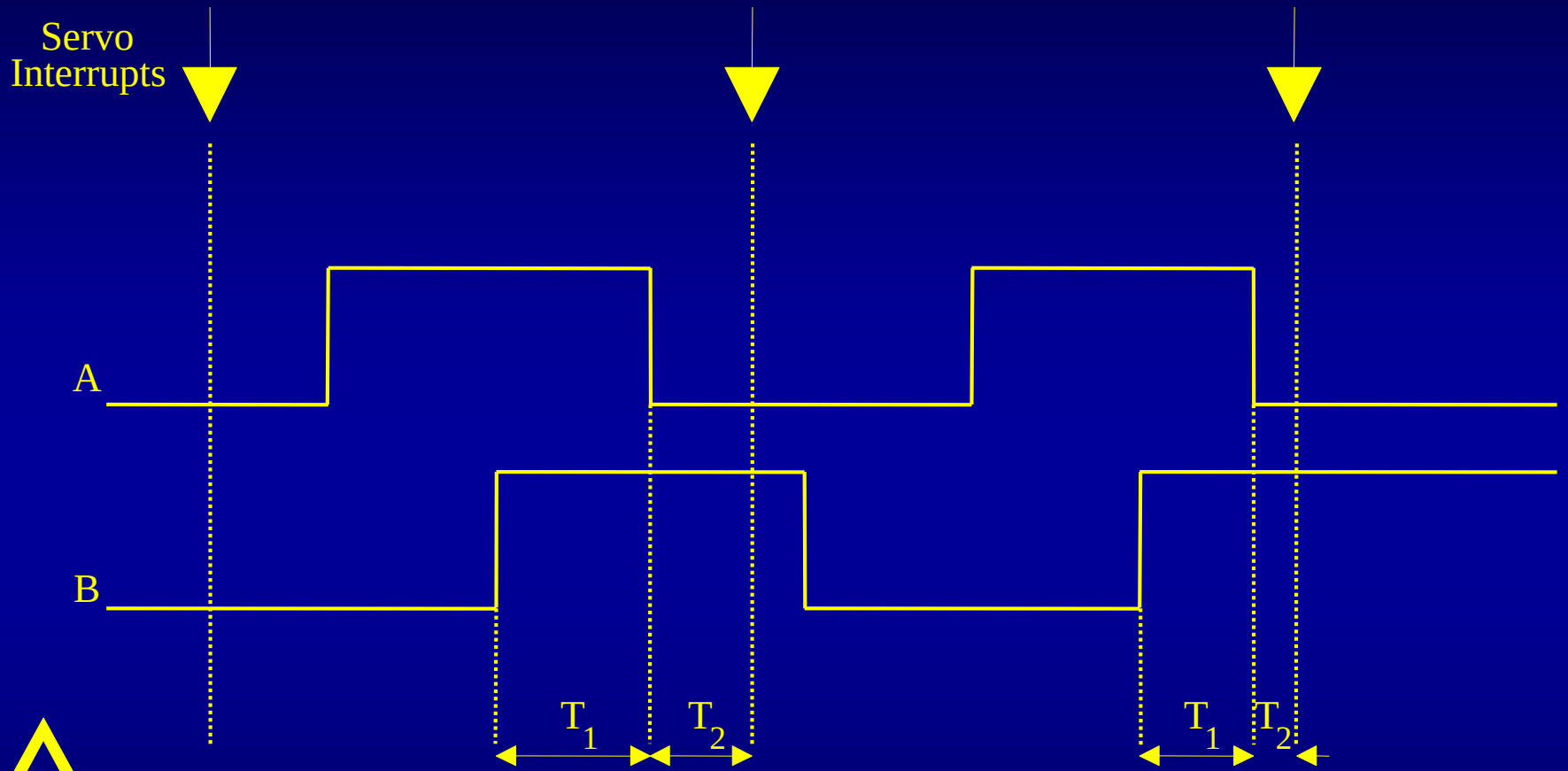
Ix92 = 0 => Blend enable



Ix92 = 1 => Blend disable

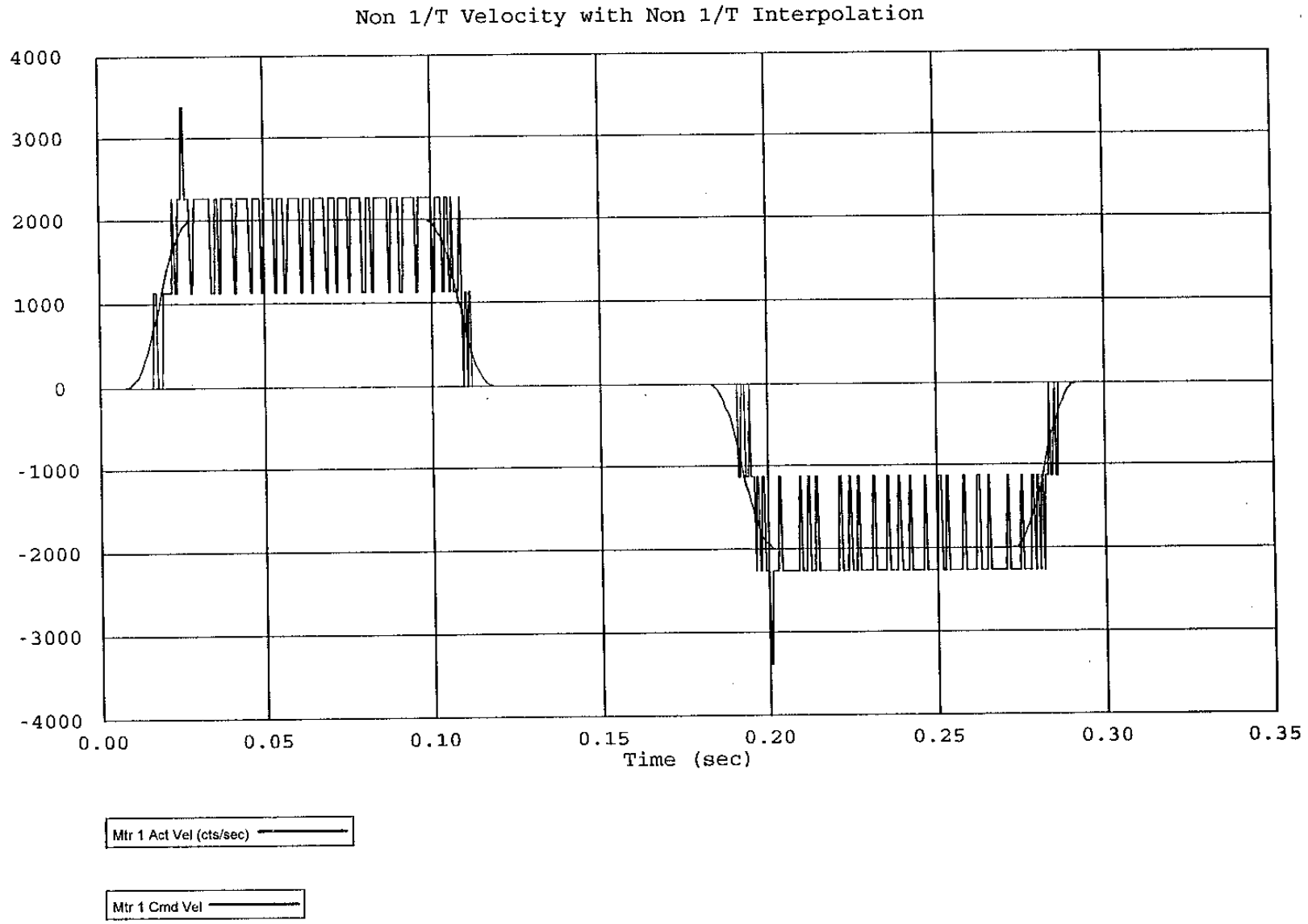


PMAC 1/T EXTENSION

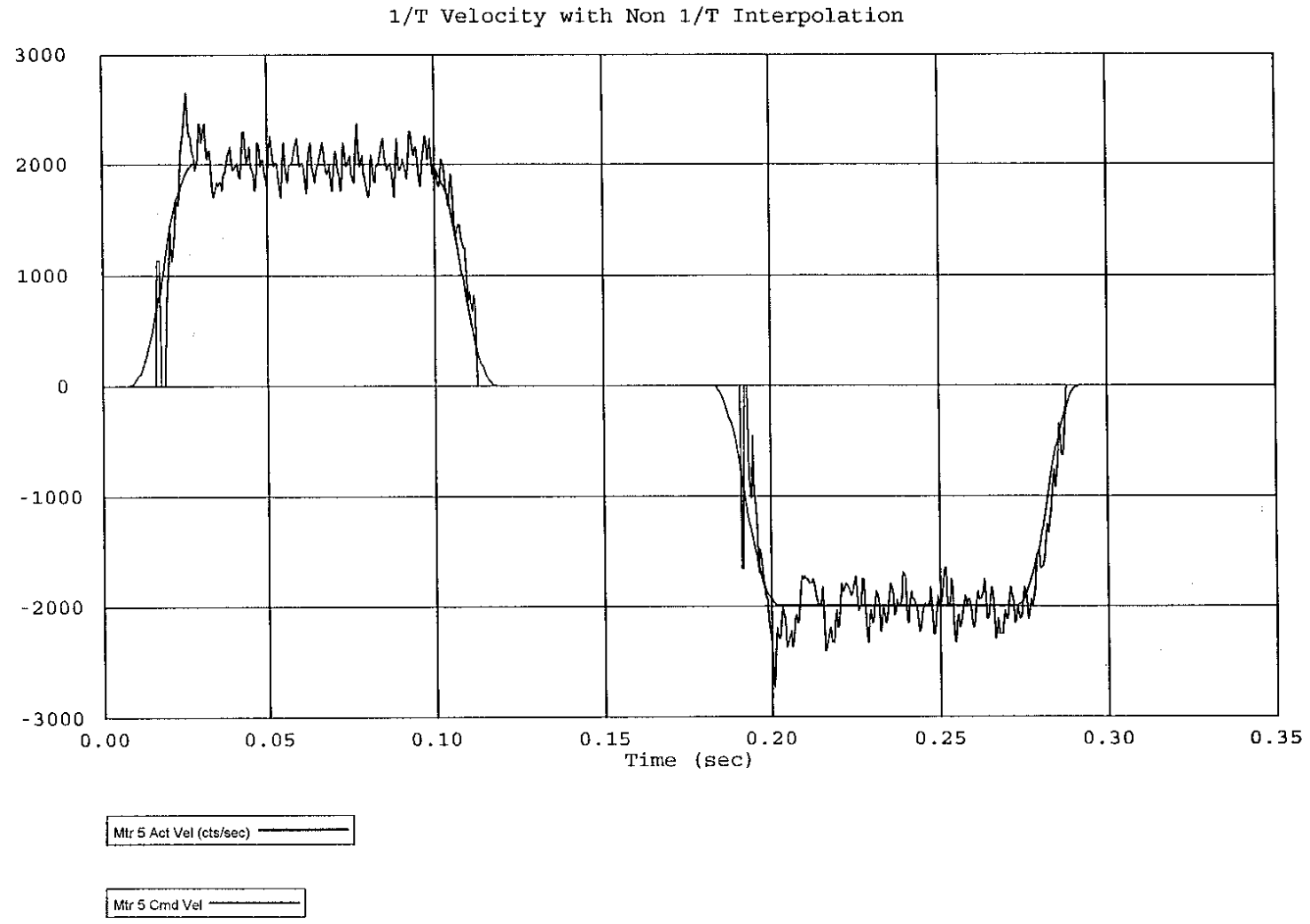


$$V_n = \frac{K}{T_1} \qquad P_n = \text{Counter} \pm \frac{T_2}{T_1}$$

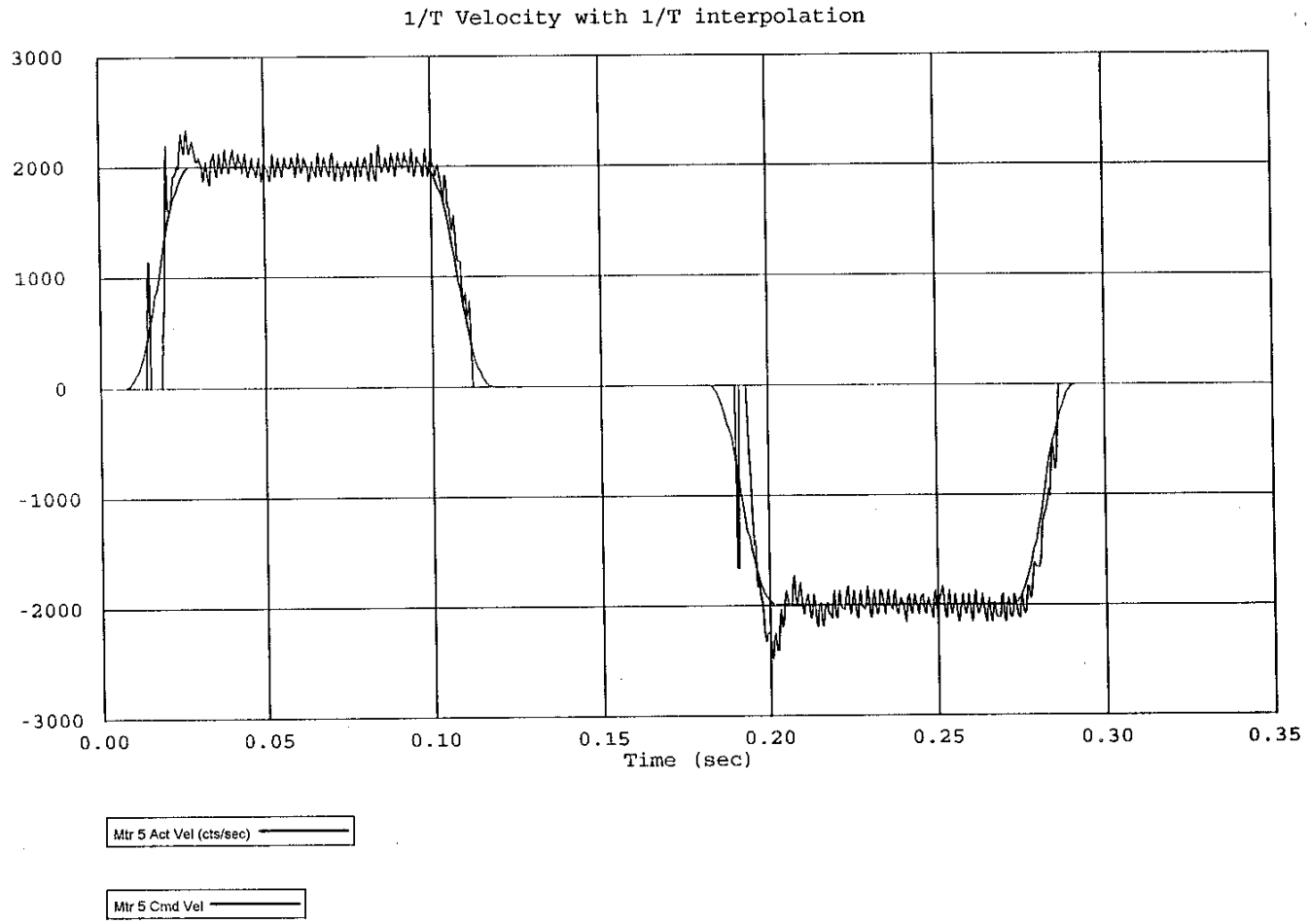
PMAC Executive for Windows
Release 1.50, 07/03/1996 - 11:03
Copyright © 1994,95,96. Delta Tau Data Systems, Inc.
Development:1



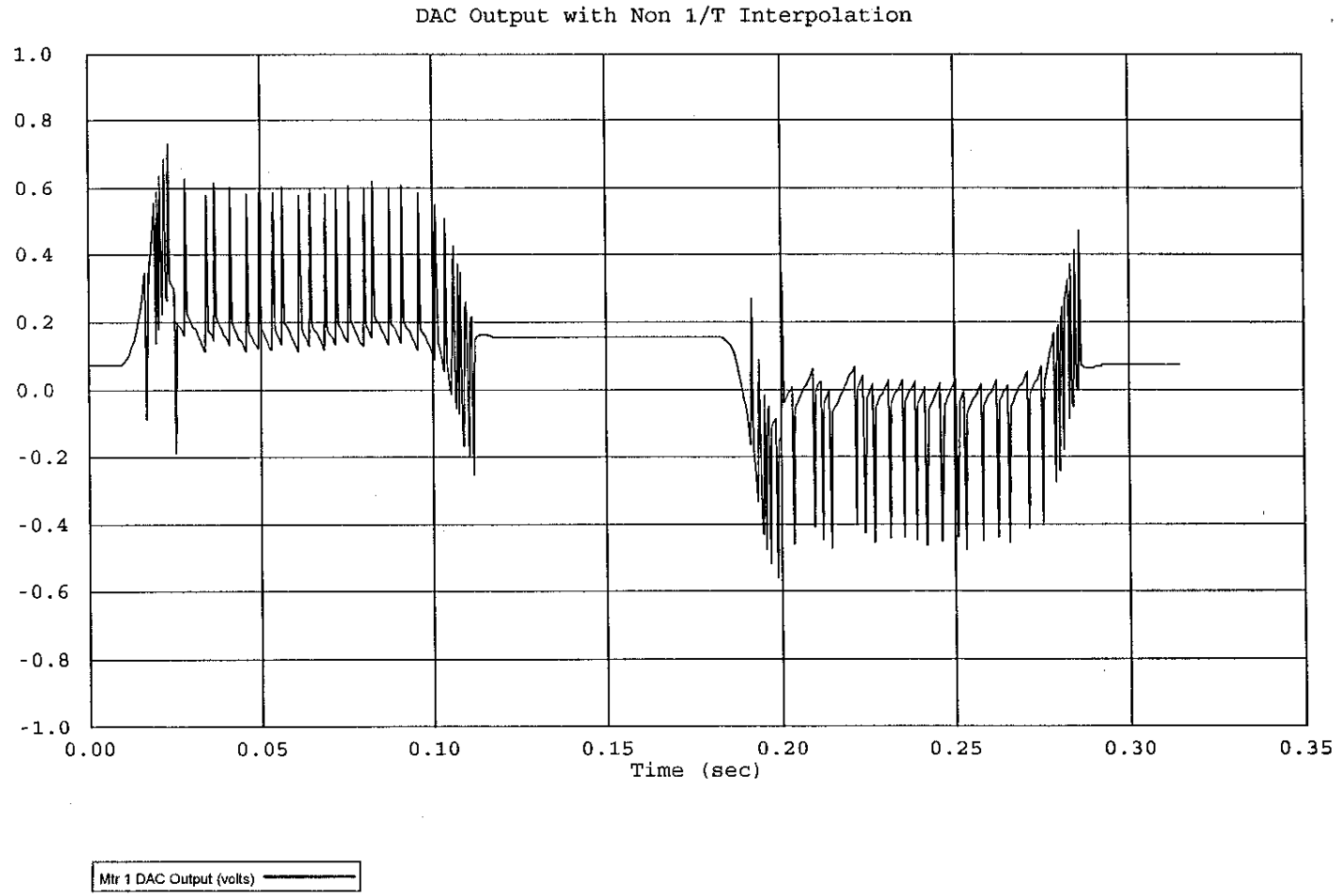
MAC Executive for Windows
elease 1.50, 07/03/1996 - 11:19
opyright © 1994,95,96. Delta Tau Data Systems, Inc.
evelopment:1



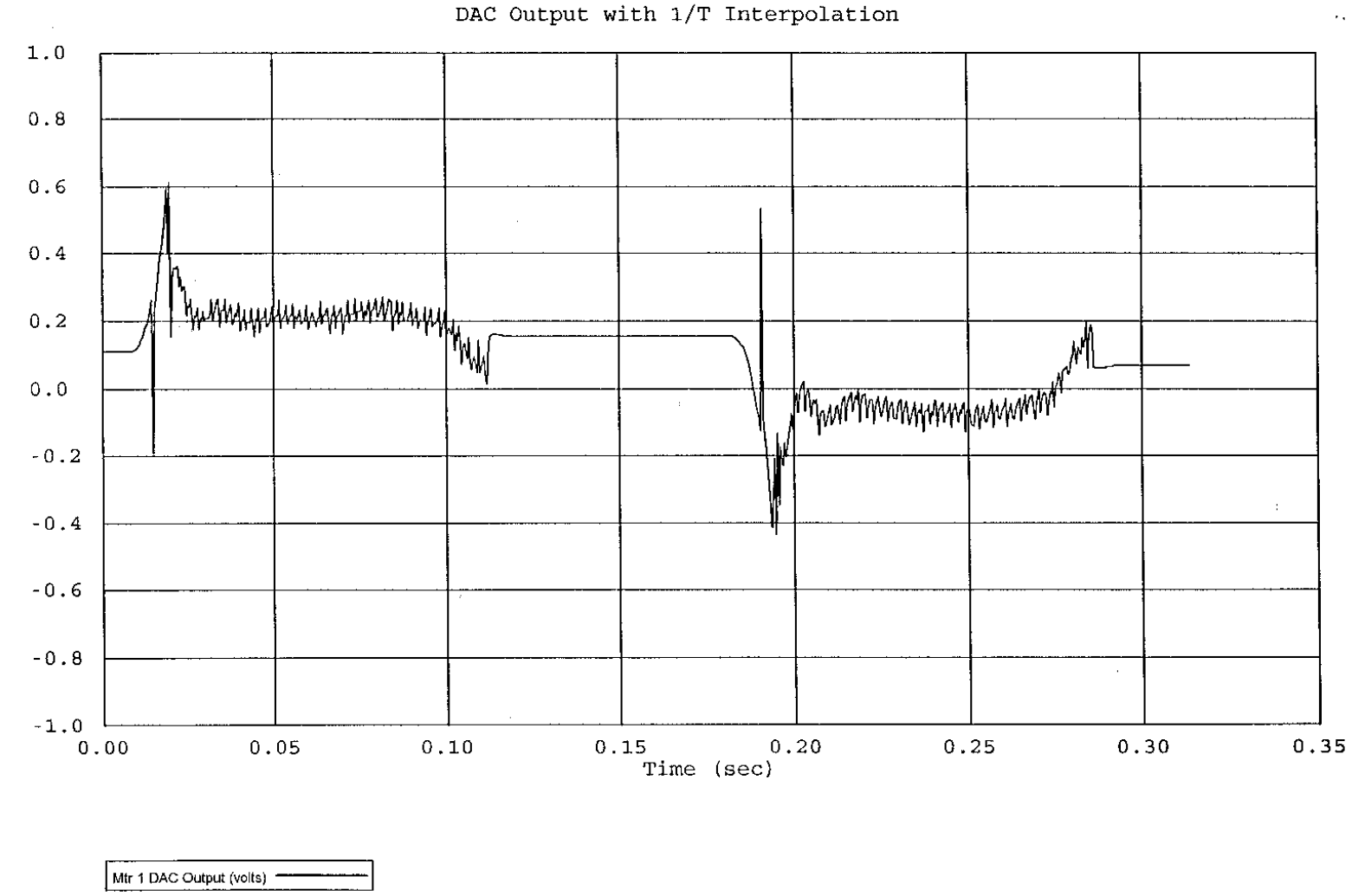
PMAC Executive for Windows
Release 1.50, 07/03/1996 - 11:03
Copyright © 1994,95,96. Delta Tau Data Systems, Inc.
Development:1



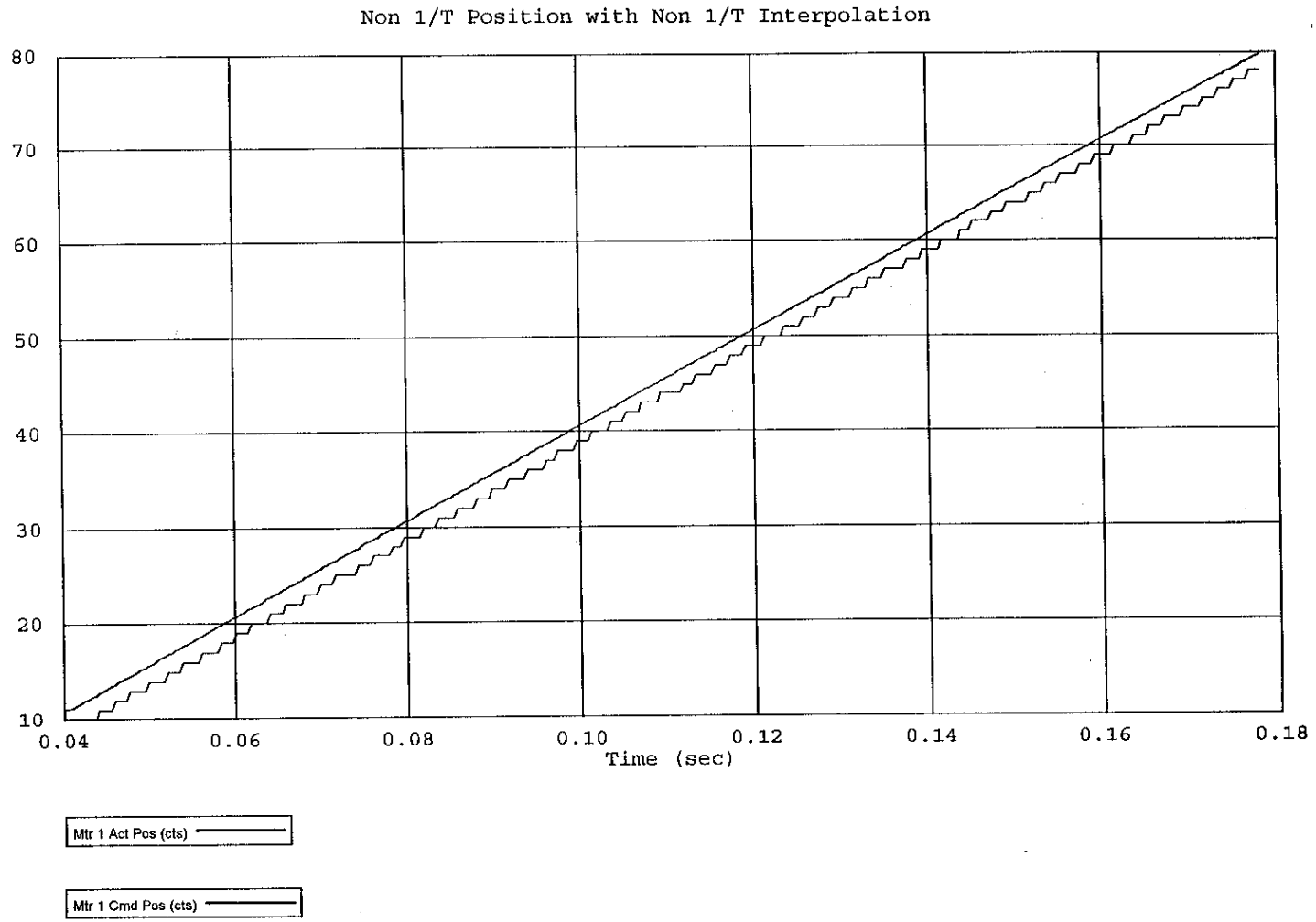
PMAC Executive for Windows
Release 1.50, 07/03/1996 - 11:03
Copyright © 1994,95,96. Delta Tau Data Systems, Inc.
Development:1



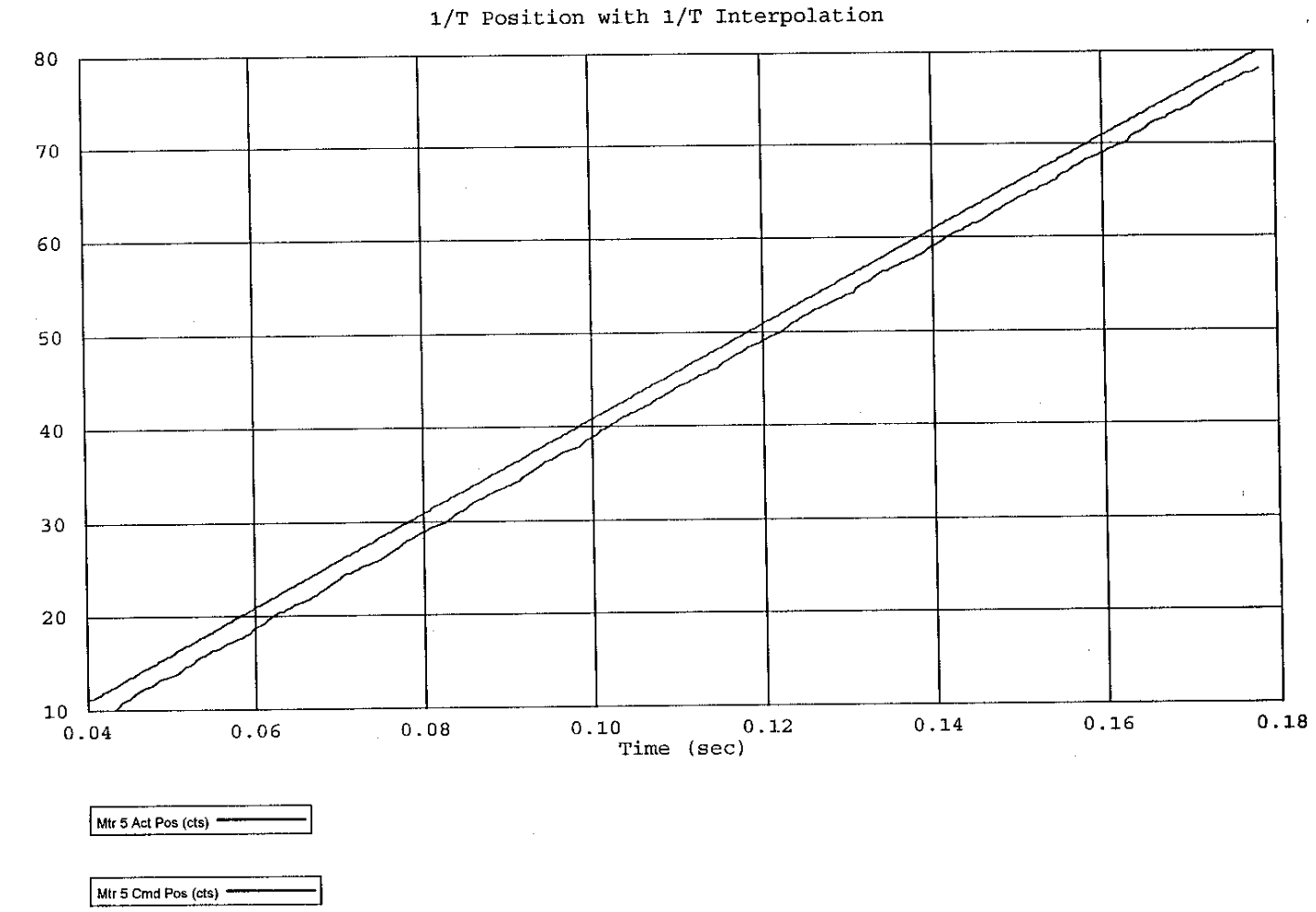
PMAC Executive for Windows
Release 1.50, 07/03/1996 - 11:17
Copyright © 1994,95,96. Delta Tau Data Systems, Inc.
Development:1



PMAC Executive for Windows
Release 1.50, 07/03/1996 - 11:02
Copyright © 1994,95,96. Delta Tau Data Systems, Inc.
Development:1

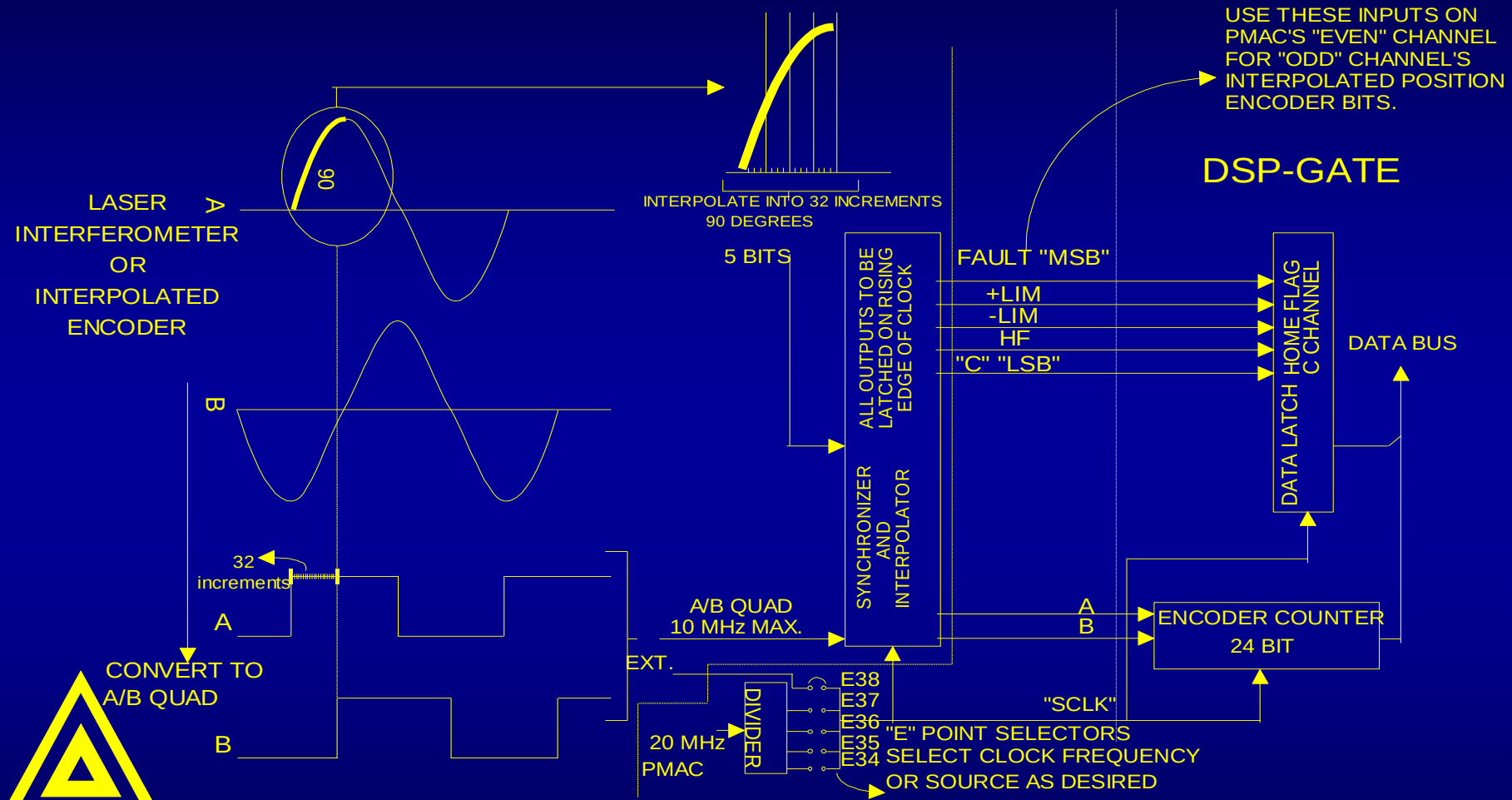


PMAC Executive for Windows
Release 1.50, 07/03/1996 - 11:02
Copyright © 1994,95,96. Delta Tau Data Systems, Inc.
Development:1



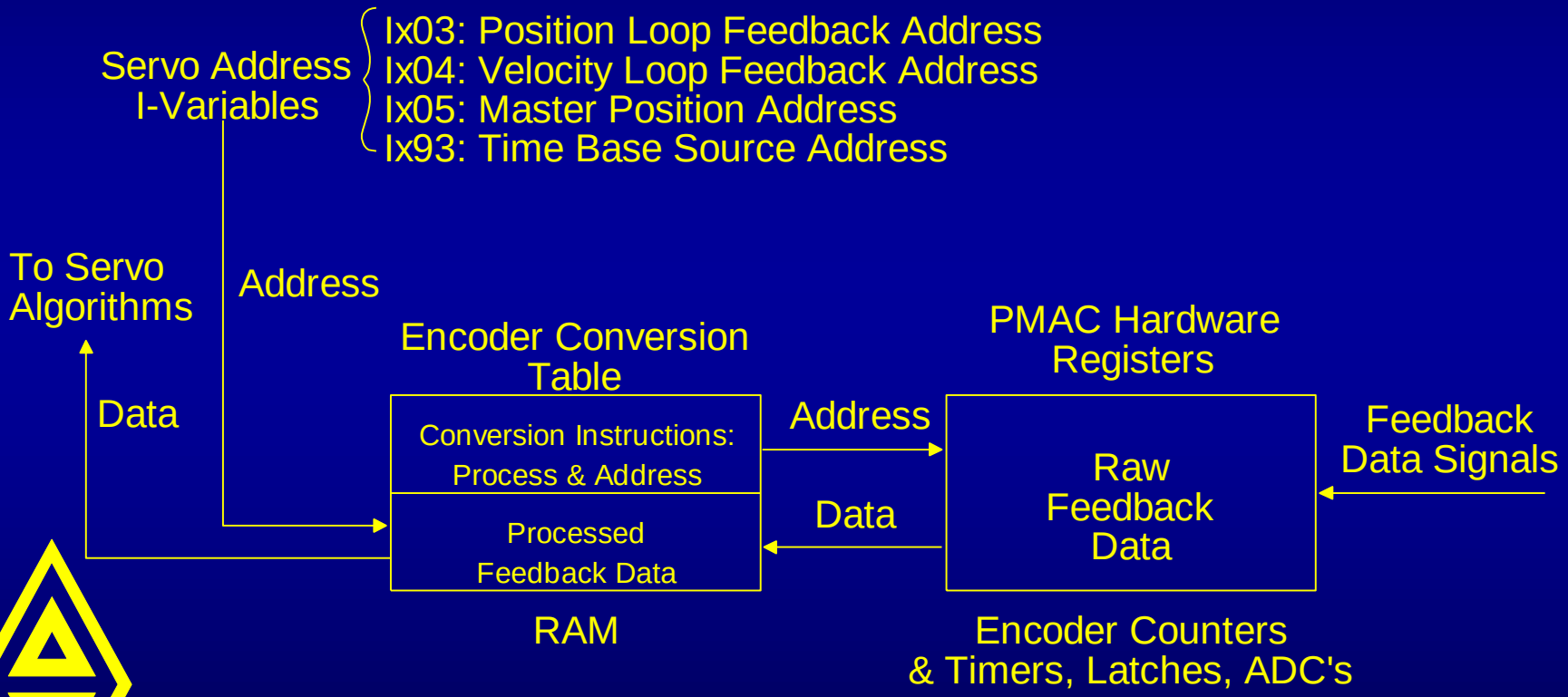
INTERPOLATED ENCODER FEEDBACK

NOTE: A 4 AXIS PMAC CAN HANDLE ONLY 2 AXIS OF CONTROL
WHEN USING BIT INTERPOLATION FOR ENCODER.



PMAC Encoder Conversion Table Principle

Pre-processing of raw feedback data before servo use



PMAC Encoder Conversion Table

X - Memory (Results) **Y - Memory (Set-up)**

1. Single Line Entry

Bits 5 - 23	Bits 0 - 4	Bits 16 - 23	Bits 0 - 15
Result: Integer	Fraction	Method	Source Address

2. Multi-Line Entry

Bits 5 - 23	Bits 0 - 4	Bits 16 - 23	Bits 0 - 15
(Intermediate Result) ○ ○ ○		Method	Source Address (Conversion Factors) • • •
Result: Integer	Fraction		

Conversion Methods

Method		Example Method and Address
$00	Incremental with 1/T extension	$00C004
$10	ACC-28 A/D Register	$10C006
$20	Parallel Y-source no filtering 2nd entry: Active-bit mask	$20FFD0 $003FFF
$30	Parallel Y-source with filtering 2nd entry: Active-bit mask 3rd entry: Max ΔP/cycle	$30FFD1 $000FFF $000200



Conversion Methods (continued)

Method		Example Method and Address
$40	Time-Base Conversion 2nd entry: Scale Factor	$400723 $000800
$50	Integrated A/D Conversion 2nd entry: Reference Bits	$50C007 $000200
$60	Parallel X-source, no filtering 2nd entry: Active Mask	$60C001 $00FFFF
$70	Parallel X-source with filtering 2nd entry: Active-bit mask 3rd entry: Max ΔP/cycle	$700721 $07FFFF $000040



Conversion Methods (continued)

Method		Example Method and Address
$80	Incremental with parallel extension	$80C010
$90/A0/B0	Trigger-started time base ($90:frozen; $B0:armed; $A0: post trigger) 2nd entry: Scale factor	$A0C00C $0008000
$C0	Incremental with no extension	$C0C020
$D0	Exponential Filter 2nd entry: Max ΔP/cycle 3rd entry: Filter Gain	$700721 $07FFFF $000040



What is PMAC Time-Base Control?

- Interpolation function computes commanded positions each servo cycle as:

  $CP_n = CP_{n-1} + CV_n\Delta t$

- Δt represents time elapsed in servo cycle

  24-bit register: 2^{23} = 1 msec

- Physical time between servo cycles set by hardware
- Varying Δt register value from true time elapsed provides “time base control” (a.k.a. “feedrate override”)



PMAC Time-Base Control Variables

- I10 Servo Interrupt Time
  - Tells interpolator true time of servo cycle
  - Scaled so that 2^{23} (8M) = 1msec
  - Default I10 = 3,713,707 (442 μsec)
- Ix93 C.S.x Time-Base Address
  - Tells what register is read for time base value
  - Contents of register have units of I10



What are Common Time-Base Sources?

- % Commands (default)
- Operator Pot thru A/D or V/F
- Operator Switch thru PLC program
- Master Encoder Count Frequency
- External Clock Frequency
- Process variable servo loop output



Time Base Host Commands

- % {constant} command puts (I10*{constant}/100) into special register for addressed C.S.
- %100 is “real time”
- % query command reports back preset time base value (regardless of source)
- Ix93 = default, this value is used for time base Δt
- Time base value slews to new command at Ix94 units per servo cycle

What is External Time Base?

- PMAC's motion language describes commanded position as function of time:

  CP=f(t)

- PMAC's basic units of time are milliseconds
- In Slaving applications, we want to describe commanded position as function of master position:

  CP=f(MP)

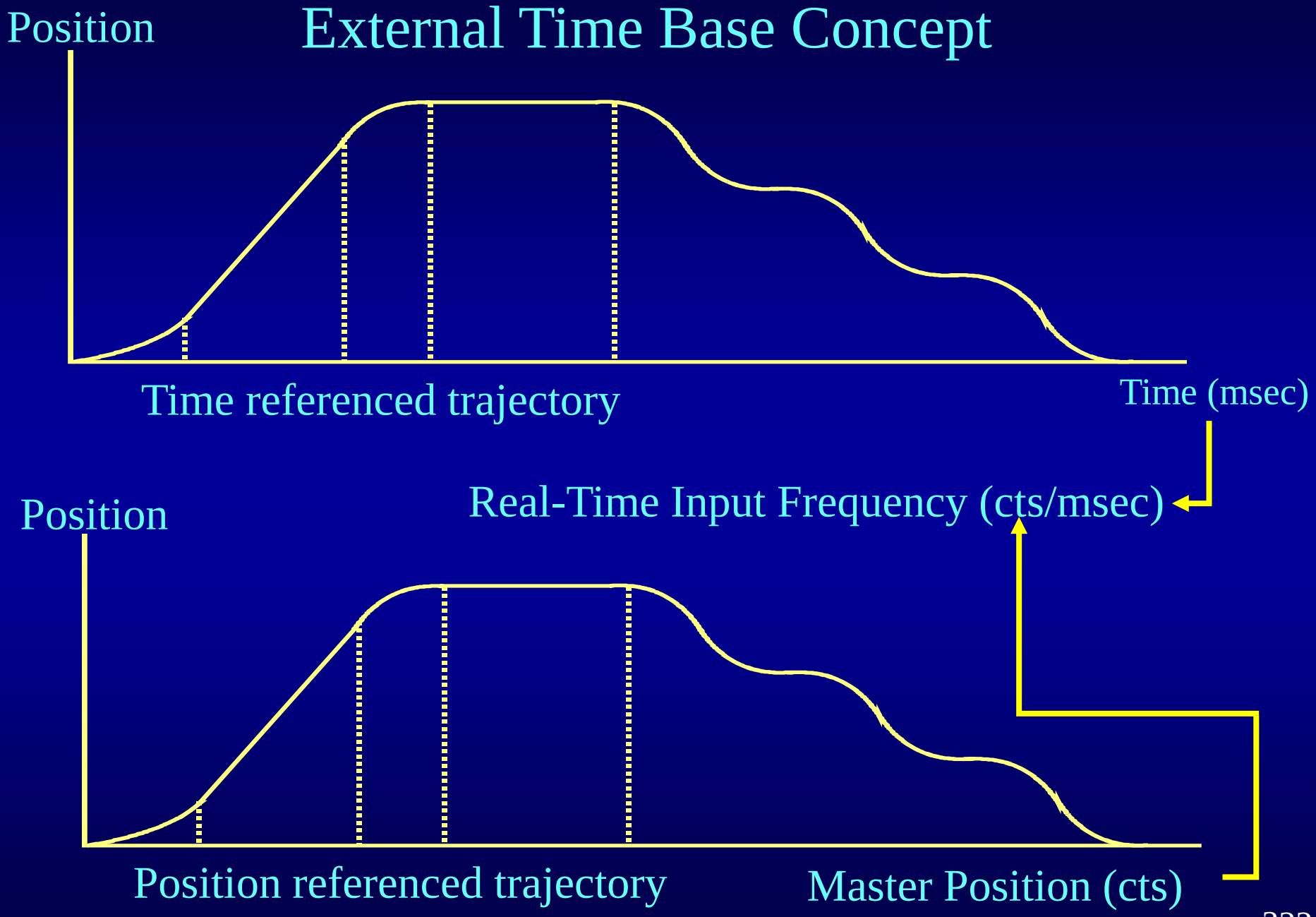


What is External Time Base? (continued)

- Since language is still in terms of time, we must define relationship between “time” (units: msecs) and master position (units: cts)
- This is Real Time Input Frequency (RTIF) in cts/msec
- If we use a RTIF of 32 cts/msec, when we refer to a msec in the program, we are really referring to 32 counts of the master



External Time Base Concept



How Does External Time Base Work?

- Conversion Table computes

$$\Delta MP_n = 2*SF*(MP_n - MP_{n-1})$$

- Ix93 contains address of ΔMP_n register in conversion table
- Coordinate system uses ΔMP_n as "Δt" in

$$CP_n = CP_{n-1} + CV_n \Delta t$$

- Trajectories are now function of master position



Time Base Conversion Table Setup

HEX	DEC	X-Register	Y-Register
$720	1824		
•••	•••		
$723	1827	Interpolated Cts(n)	$00 C00C {1/T} {Enc 4 counter}
•••	•••		
$728	1832	Interpolated Cts(n-1)	$40 0723 {Time Base} {Int. Enc. 4}
$729	1833	Result (DMP)	$001000 {Scale factor}

I193=1833

Result =2*S.F.*[Interp Cts(n)-Interp Cts(n-1)]

$=2*(2^{17}/RTIF)*(2^{5}*IF*I10/2^{23})$

$=I10*(IF/RTIF)$

PMAC External Time Base Example

Cutting on the fly: rotary cutter slaved to web

Web encoder wheel: 5 inch circumference

500 lines/rev = 2000 cts/rev (often power of 2)

Maximum web speed= 200 in/sec

$$200\frac{\text{in}}{\text{sec}} \bullet \frac{\text{rev}}{5\text{in}} \bullet 2000\frac{\text{cts}}{\text{rev}} \bullet \frac{\text{sec}}{1000\text{msce}} = 80\frac{\text{cts}}{\text{msec}}$$

Define “real time input frequency” (RTIF)

RTIF= 128 cts/msec (usually power of 2)

Calculate “time-base scale factor (TBSF)

$$\text{TBSF} = \frac{2^{17}}{\text{RTIF}\left(\frac{\text{cts}}{\text{msec}}\right)} = \frac{131{,}072}{128} = 1024\,(\text{must be integer})$$

PMAC External Time Base Example (continued)

At RTIF, web speed is:

$$200\frac{\text{in}}{\text{sec}} \bullet \frac{128}{80} = 320\frac{\text{in}}{\text{sec}} \; or \; 3.125\frac{\text{msec}}{\text{in}}$$

Assume this web speed to write cutter program:

To match web speed with 10-inch circumference cutter:

$$320\frac{\text{in}}{\text{sec}} \bullet \frac{\text{rev}}{10\text{in}} = 32\frac{\text{rev}}{\text{sec}}$$

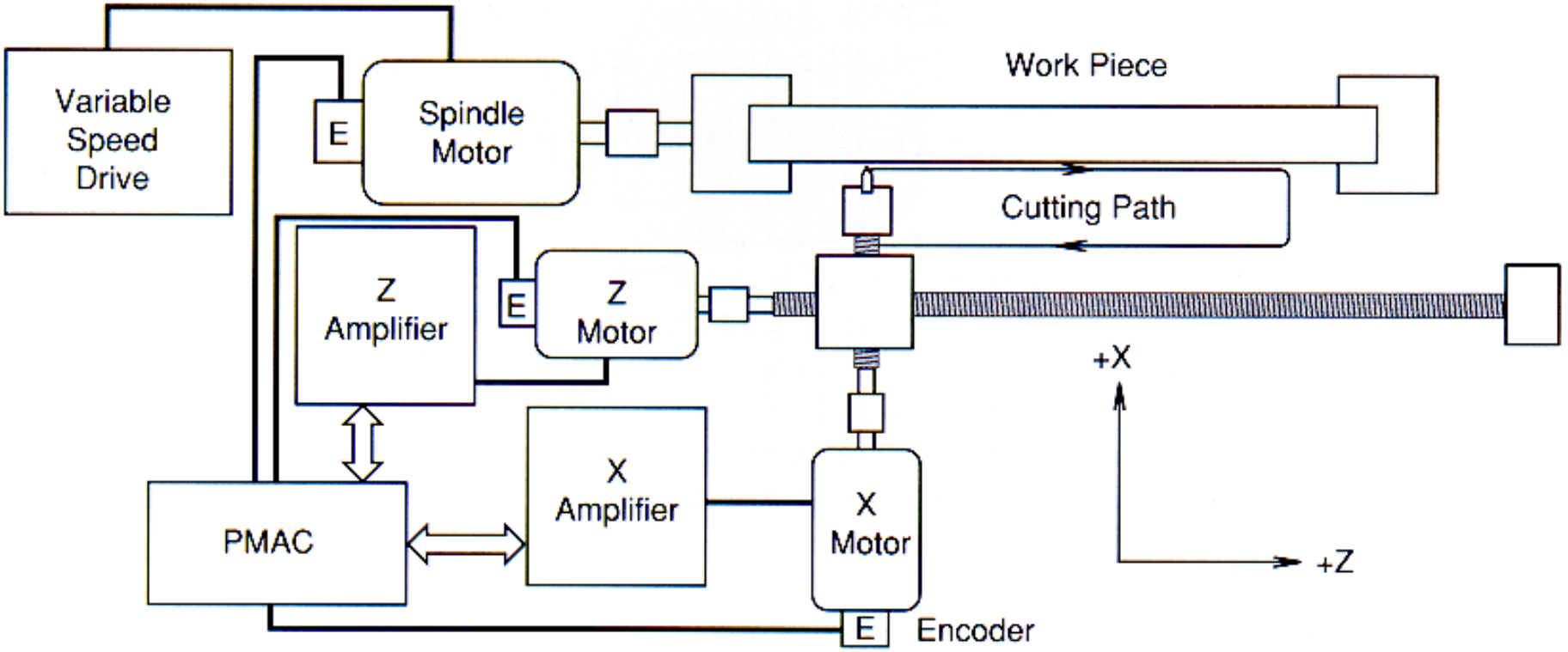
To cut an 11-inch piece, cutter cycle must be:

$$3.125\ \frac{\text{msec}}{\text{inch}} \bullet \frac{11\,\text{in}}{\text{cycle}} = 34.375\frac{\text{msec}}{\text{cycle}}$$

Example 10: Precision Slaving

This example shows how the PMAC can use an external time-base source for sophisticated precision slaving applications. Here it is used for multiple-pass screw threading on a lathe with an open-loop spindle. Coil winding applications are virtually identical as are cut-on-the-fly applications for conveyorized or extruded material. The PMAC slaves its cutter axes (X&Z) to an encoder on the spindle so that cutter speed tracks spindle speed, resulting in constant pitch. This is done simply in time-base mode. Define a "real-time" speed for the master (spindle) and set the time-base constant based on this. Write the program for the slave axes as if the master were always operating at the real-time speed. The time base control will compensate automatically for changes in the spindle speed.




Variable Speed Drive
E
Spindle Motor
Work Piece
Cutting Path
Z Amplifier
E
Z Motor
+X
X Amplifier
X Motor
PMAC
+Z
E
Encoder

Example 10: Precision Slaving (continued)

```
;********************** Set-up and Definitions **********************
&1                      ; Define axes in Coordinate System 1
#1->10000X              ; Motor 1 is X at 10000 cts/in (radial)
#2->10000Z              ; Motor 2 is Z at 10000 cts/in (traverse)
Spindle encoder has 1024 lines/rev or 4096 cts/rev.  At real-time speed of 3000
rpm (50 rps), encoder frequency is 204.8 cts/msec.  By formula, the time base
constant is 131,072/204.8 = 640.
WY:1833,640             ; Set time base constant (at address 1833)
I193=1833               ; Tell Coordinate System 1 to use this
                        ; address as the time-base source.
;********************** Motion Program Text **************************
This program is written to cut a 5-pitch (5 threads per inch) screw.  It
assumes the spindle speed is 3000 rpm, or 50 rps.  Cutting speed is therefore
50(rev/sec)/5(rev/in)=10 in/sec.  Programmed time for each loop must be an
exact multiple of 20 msec for cutter to match thread each pass (50rev/sec <=>
20msec/rev).
```



Example 10: Precision Slaving (continued)

```
OPEN PROG 77 CLEAR      ; Prepare buffer 77 for entry
P100=3.00               ; X (radial) dimension of outside of stock
P101=P100               ; Starting location of cut
RAPID X(P100-0.1) Z2    ; Move quickly to starting position
LINEAR
WHILE (P101<3.10)       ; Loop until 0.1 inches deep
        P101=P101+0.01  ; Increase depth 0.01 inch
        TM100           ; 100 msec "plunge" time
        X(P101)         ; Radial axis into stock ("plunge")
        TM(24*1000/10)  ; 24 inches at 10 in/sec (in msec)
        Z26             ; Make threading pass for 24 inches (26-2)
        TM60            ; 60 msec retract time
        X(P100-0.1)     ; Retract to just outside of stock
        TM(24*1000/30)  ; 24 inches at 30 in/sec (in msec)
        Z2              ; Reverse move for next pass
                        ; Total time for moves in loop is
                        ; 100+2400+60+800=3360 msec, which is an
                        ; exact multiple of 20 msec, or 1 spindle rev.
ENDWHILE                ; End of loop
RAPID X0 Z0             ; Return to home position
CLOSE
```

Constraints on Choosing Time Base Real-Time Input Frequency

The choice of RTIF in master counts per millisecond is quite arbitrary. There are only a few constraints:

1) The Time Base Scale Factor (2^{17}/RTIF) must be an integer to prevent drift

a) RTIF=100cts/msec=>TBSF=1310.72 ✘ (BAD)

b) RTIF=128cts/msec=>TBSF=1024 ✓ (Good)

2) $\frac{Maximum\ \ Input\ \ Frequency}{Real\ \ Time\ \ Input\ \ Frequency} < Servo\ \ Update\ \ Frequency\ (kHz)$

to prevent saturation

$$\therefore \text{RTIF} > \frac{MIF}{SUF\ (kHz)}$$

a) SUF=2.25 kHz, MIF = 100 $\frac{cts}{m\,\sec}$**, RTIF =32** $\frac{cts}{m\,\sec}$ **=>RTIF<** $\frac{MIF}{SUF}$ **✘ (BAD)**

b) SUF=2.25 kHz, MIF = 100 $\frac{cts}{m\,\sec}$**, RTIF =64** $\frac{cts}{m\,\sec}$ **=>RTIF>** $\frac{MIF}{SUF}$ **✓ (Good)**

Constraints on Choosing Time Base Real-Time Input Frequency (continued)

3) SPLINE and PVT segment times (at RTIF) must be an integer number of milliseconds, so here master cycle time must be an integer number of milliseconds

a) Master Encoder = 4000 $\frac{cts}{rev}$ Max Speed = 3000 rpm = 50 rps
RTIF=MIF=200 $\frac{cts}{m\,\sec}$ =>TBSF655.36 ✘ (BAD)

b) Master Encoder = 4000 $\frac{cts}{rev}$ Max Speed = 3000 rpm = 50 rps
Real Time Speed = 64 rps RTIF = 256 $\frac{cts}{rev}$ TBSF=512
Rev Time=15.625 msec ✘ (BAD)

c) Master Encoder =4096 $\frac{cts}{rev}$ Max Speed = 3000 rpm = 50 rps
RTIF=MIF=204.8 $\frac{cts}{m\,\sec}$ => TBSF=640 ✓ Rev Time=20msec ✓ (GOOD)

Glossary:

RTIF: Real Time Input Frequency (cts/msec)
TBSF: Time Base Scale Factor
MIF: Maximum Input Frequency (cts/msec)
SUF: Servo Update Frequency

Time Base I-Variables (continued)

Ix94 Time Base Slew Rate / Slew Limit

- Determines maximum rate of time base change
- Expressed in (units of I10)/servo cycle
- Set relatively low for host-commanded time-base (default is 1644) to prevent step velocity change
- Set high for external time base to prevent loss of synchronization



Time Base I-Variables (continued)

Ix95 Feed Hold Slew Rate

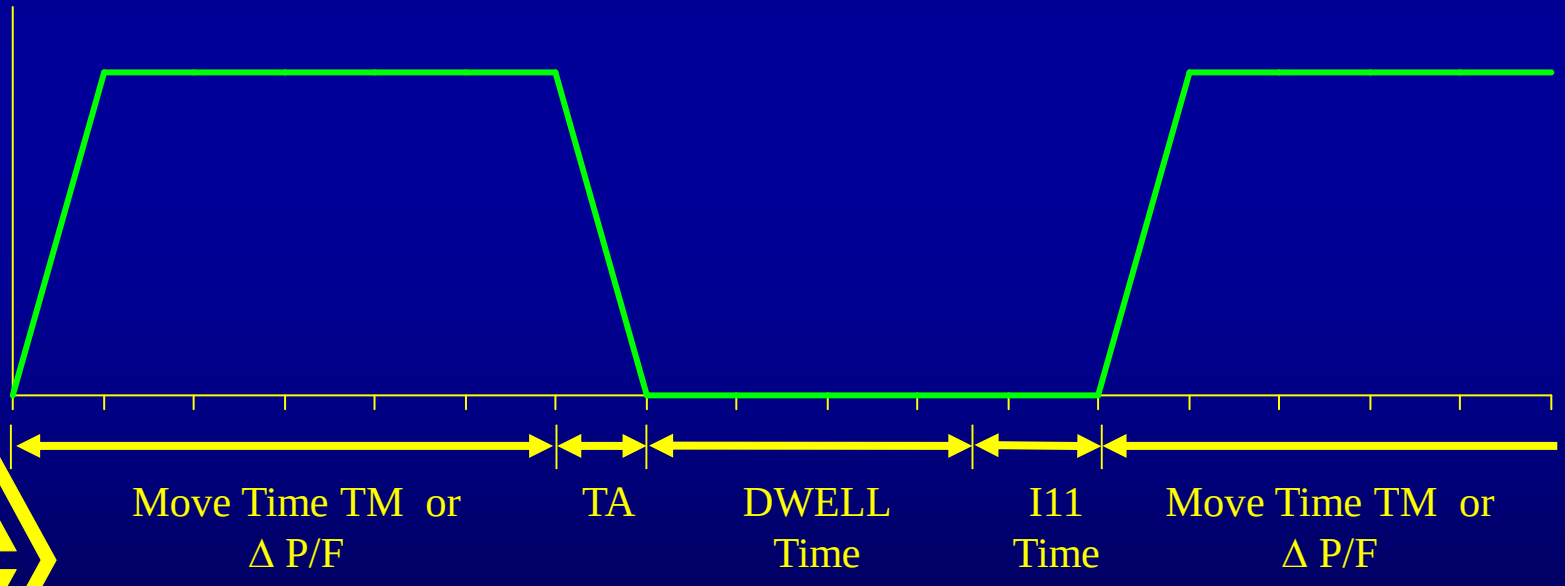
- Works like Ix94, has same units
- Affects only deceleration on ‘H’ command and re-acceleration on following ‘R’
- ‘Hold’ is effectively a ‘%0’ command
- Ix95 permits controlled hold even when Ix94 is high for external time base



Dwell vs. Delay

DWELL

- Always uses fixed time base (I10)
- Time does not include preceding deceleration time
- Next move not calculated until after end of DWELL
- Does not keep sync in external time base



Dwell vs. Delay

DELAY

- Uses variable time base (% value)
- Time includes preceding deceleration time
- Minimum time is current TA time
- Upcoming move calculated at beginning of DELAY



Triggered Time Base

- Untriggered time base is not well referenced to a starting master position
- Triggered time base uses hardware-captured master position as starting reference
- Subsequent movement is referenced to this starting master position



Setting up Triggered Time-Base

- Conversion table entry
  - Method $A0 (running triggered)
  - Address of channel in DSPGATE (e.g. $C000)
  - Scale factor as for untriggered time base
- Trigger Control
  - Encoder capture control (I902 etc. / I9n2)
  - Capture flag select (I903 etc. /I9n3)
- Ix93 contains address of entry's second line
- Define M-Variable to method byte in table



Implementing Triggered Time Base

1. Motion Program freezes time base and calculates first move

```
DWELL 20          ; Stop motion and look ahead
M99 = $90         ; Freeze time base
X10               ; Calculate first move
```

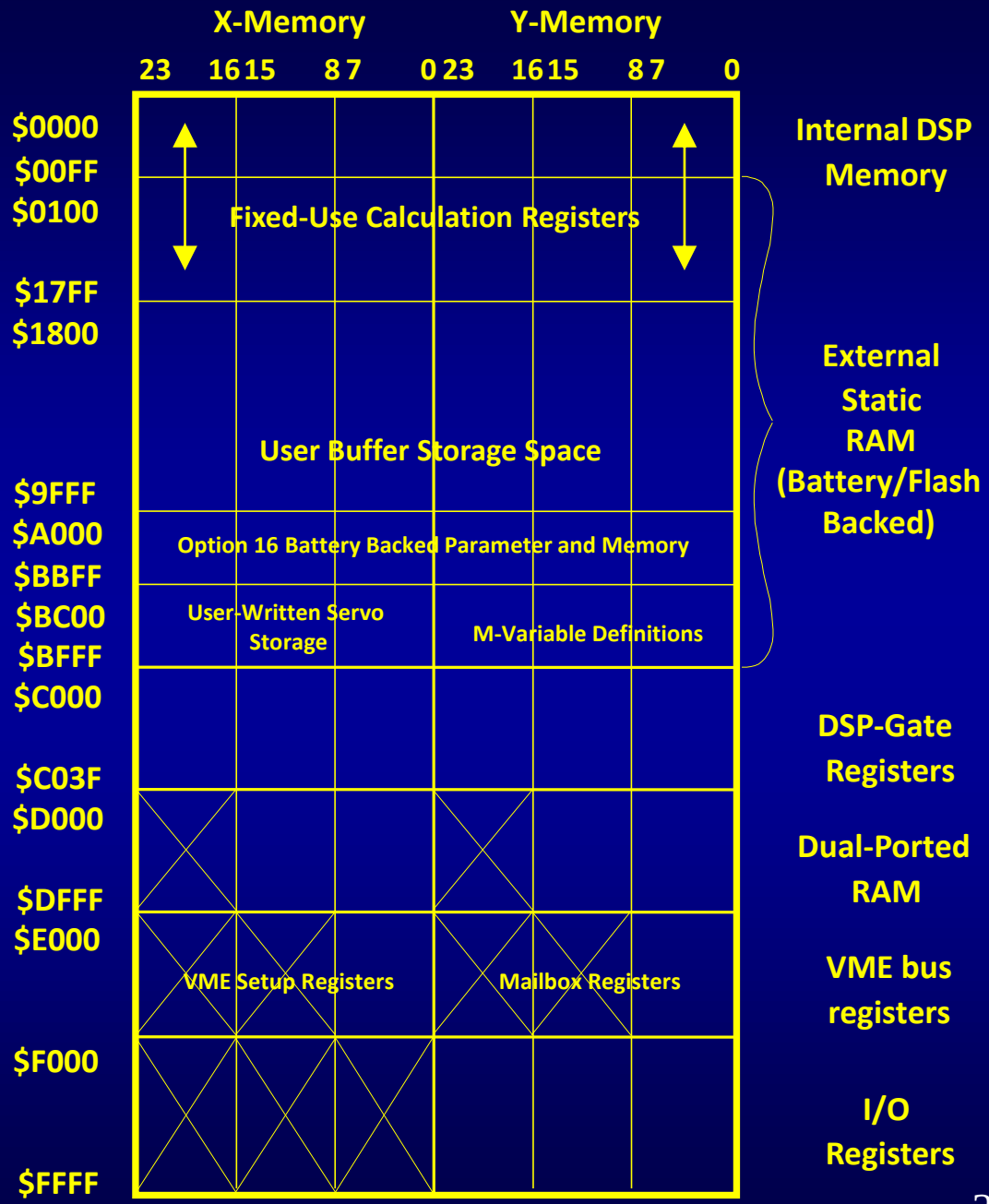
2. PLC program arms time base

```
IF  (M99 = $90)   ; If frozen
   M99 = $B0      ; then arm
ENDIF
```

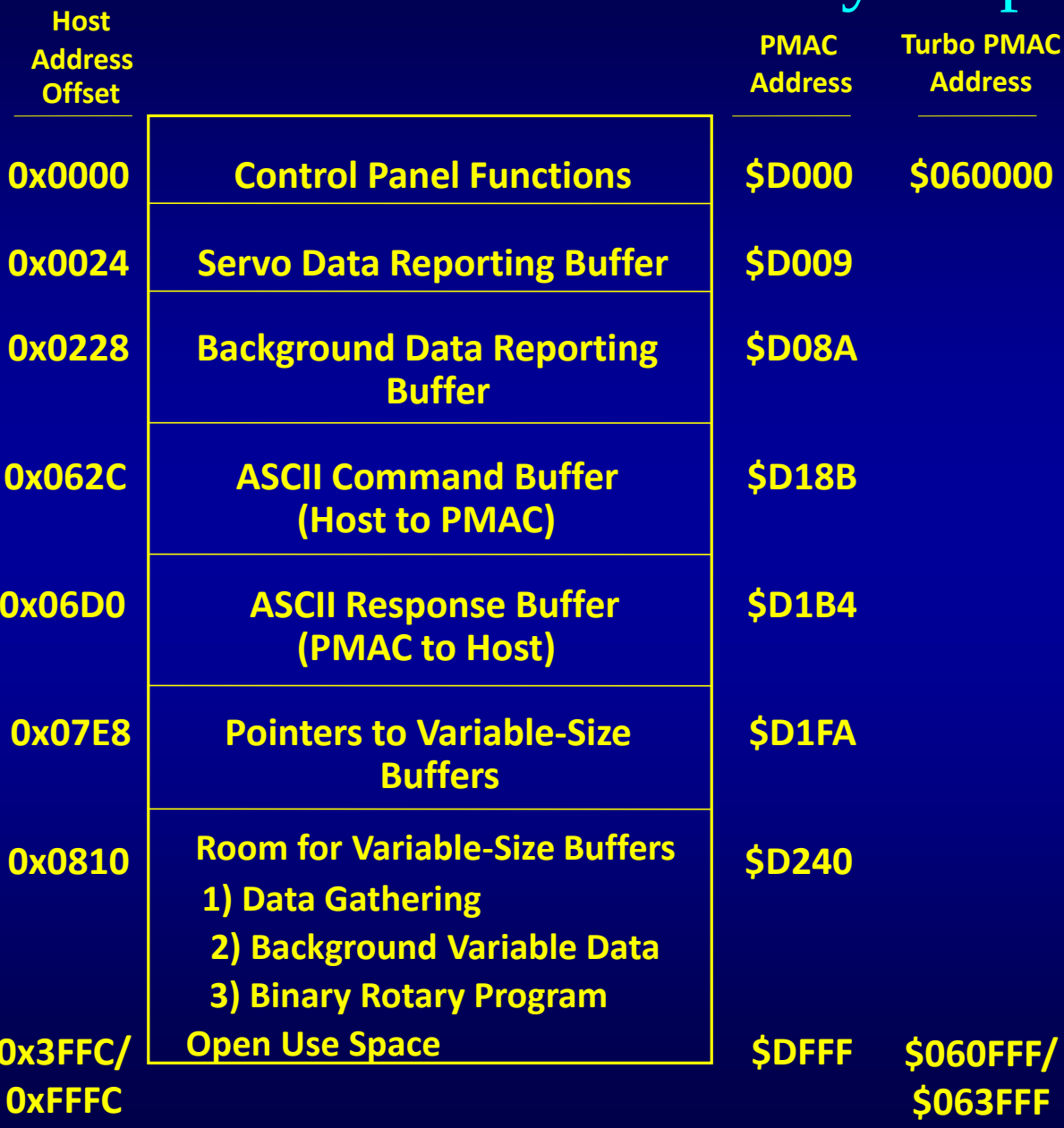
3. Conversion table releases time base on trigger
   - Captured position is starting pos (time)
   - Method byte changed to $A0 (running)



PMAC Memory Map



PMAC Dual-Ported RAM Memory Map



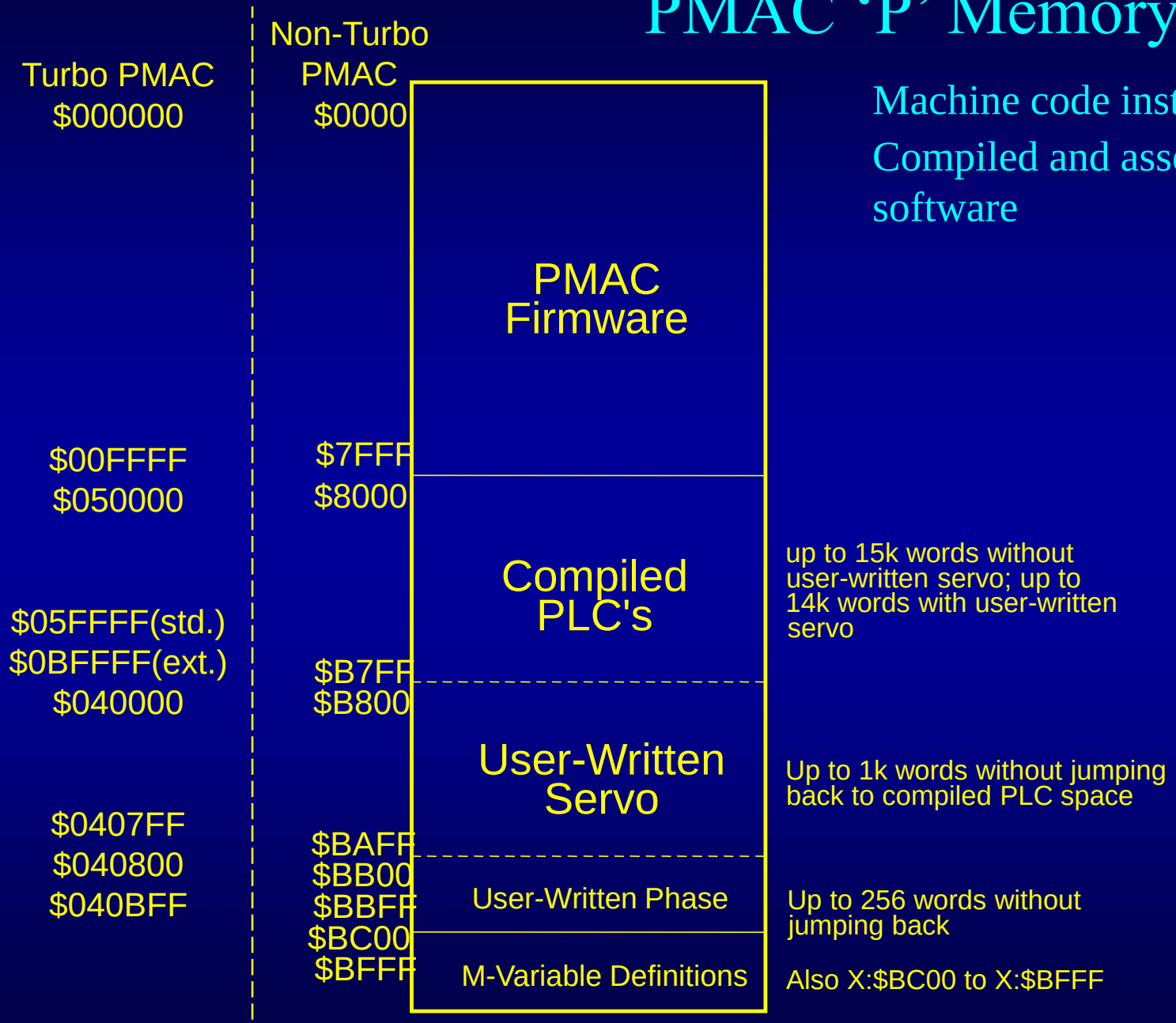
DPRAM Address Mapping

Host Mapping	Physical IC Mapping	PMAC Mapping
16k x 8	8k x 16	4k x 32
• For byte addressing •Only even addresses used •User-defined base address	•13 address lines •16 data lines	•X & Y register at each Numerical address •Fixed address range $D000-$DFFF [$60000-$60FFF]

PMAC 'P' Memory Space

Machine code instructions only

Compiled and assembled software



PMAC User Program and Buffer Space

Turbo PMAC Address
$006000 (I46 = 3)
$008000 (I46 = 1,2)
$00A000 (I46 = 0)

Data Memory (including interpreted programs)

$0107FF (Standard)
$03FFFF (Extended)

PMAC Address
$1800

X Y

PROGm
PROGn
• • •
PROGz
PLC0
PLC1
• • •
PLC31

GATHER
&1ROTARY
• • •
&8ROTARY
TBUF
#1 BLCOMP
• • •
#8 BLCOMP
#1 TCOMP
• • •
#8 TCOMP
#1 COMP
• • •
#8COMP
UBUFFER

$9FFF

256 Motion Programs can be held
All PROGs must be stopped before any can be OPENed

All PROGs must be CLOSEd before any can be Run

A PLC program can be OPENed while others are running

DELETE

Buffers must be DEFINED from end of memory toward beginning.
Buffers must be DELETED from beginning of memory to end

DEFINE

M-Variable Definition Types

X: {address}, {offset}, {width}, {format}

Y: {address}, {offset}, {width}, {format}

integer 1-24 bits wide
{offset} is starting bit number
default width 1; default format U (unsigned)

D: {address}

signed integer, 48 bits wide



M-Variable Definition Types (continued)

L: {address}

floating-point value 48 bits wide

DP: {address} (dual ported RAM)

signed integer 32 bits wide

low 16 bits of X, low 16 bits of Y (Intel format)

F: {address} (dual ported RAM)

floating-point value IEEE 32-bit format

low 16 bits of X, low 16 bits of Y (Intel format)



Suggested M-Variable Definitions

This file contains a suggested set of M-Variable definitions. These definitions can be very useful for getting access to important I/O and registers. It is not required that you use these particular definitions. Most example programs will use these definitions.



```
CLOSE                          ; To ensure commands are on-line
M0..1023->*                    ; To clear all existing definitions
M0->X:$0,0,24,U      ; Servo cycle counter

; General Purpose inputs and outputs (PMAC-PC,-Lite,-VME)
M1->Y:$FFC2,8,1      ; Machine Output 1
M2->Y:$FFC2,9,1      ; Machine Output 2
M3->Y:$FFC2,10,1     ; Machine Output 3
M4->Y:$FFC2,11,1     ; Machine Output 4
M5->Y:$FFC2,12,1     ; Machine Output 5
M6->Y:$FFC2,13,1     ; Machine Output 6
M7->Y:$FFC2,14,1     ; Machine Output 7
M8->Y:$FFC2,15,1     ; Machine Output 8
M9->Y:$FFC2,8,8,U ; Machine Outputs 1-8 treated as byte
M11->Y:$FFC2,0,1     ; Machine Input 1
M12->Y:$FFC2,1,1     ; Machine Input 2
M13->Y:$FFC2,2,1     ; Machine Input 3
M14->Y:$FFC2,3,1     ; Machine Input 4
M15->Y:$FFC2,4,1     ; Machine Input 5
M16->Y:$FFC2,5,1     ; Machine Input 6
M17->Y:$FFC2,6,1     ; Machine Input 7
M18->Y:$FFC2,7,1     ; Machine Input 8
M19->Y:$FFC2,0,8,U             ; Machine Inputs 1-8 treated as byte
```



```
; Registers associated with Encoder/DAC1 (Usually Motor #1)
M101->X:$C001,0,24,S      ; ENC1 24-bit counter position
M102->Y:$C003,8,16,S      ; DAC1 16-bit analog output
M103->X:$C003,0,24,S      ; ENC1 capture/compare position register
M104->X:$0720,0,24,S      ; ENC1 interpolated position (1/32 ct)
M105->Y:$C006,8,16,S      ; ADC1 16-bit analog input
M106->Y:$C000,0,24,U      ; ENC1 time between counts (SCLK cycles)
M110->X:$C000,10,1        ; ENC1 count-write enable control
M111->X:$C000,11,1        ; EQU1 compare flag latch control
M112->X:$C000,12,1        ; EQU1 compare output enable
M113->X:$C000,13,1        ; EQU1 compare invert enable
M114->X:$C000,14,1        ; AENA1/DIR1 Output
M116->X:$C000,16,1        ; EQU1 compare flag
M117->X:$C000,17,1        ; ENC1 position-captured flag
M118->X:$C000,18,1        ; ENC1 Count-error flag
M119->X:$C000,19,1        ; ENC1 3rd channel input status
M120->X:$C000,20,1        ; HMFL1 input status
M121->X:$C000,21,1        ; -LIM1 input status
M122->X:$C000,22,1        ; +LIM1 input status
M123->X:$C000,23,1        ; FAULT1 input status
```

```
; Motor #1 Status Bits
M130->Y:$0814,11,1      ; #1 Stopped-on-position-limit bit
M131->X:$003D,21,1      ; #1 Positive-end-limit-set bit
M132->X:$003D,22,1      ; #1 Negative-end-limit-set bit
M133->X:$003D,13,1      ; #1 Desired-velocity-zero bit
M135->X:$003D,15,1      ; #1 Dwell-in-progress bit
M137->X:$003D,17,1      ; #1 Running-program bit
M138->X:$003D,18,1      ; #1 Open-loop-mode bit
M139->Y:$0814,14,1      ; #1 Amplifier-enabled status bit
M140->Y:$0814,0,1       ; #1 In-position bit
M141->Y:$0814,1,1       ; #1 Warning-following error bit
M142->Y:$0814,2,1       ; #1 Fatal-following-error bit
M143->Y:$0814,3,1       ; #1 Amplifier-fault-error bit
M145->Y:$0814,10,1      ; #1 Home-complete bit
```



```
; Motor #1 Move Registers
M161->D:$0028 ; #1 Commanded position (1/[Ix08*32] cts)
M162->D:$002B ; #1 Actual position (1/[Ix08*32] cts)
M163->D:$080B ; #1 Target (end) position (1/[Ix08*32] cts)
M164->D:$0813 ; #1 Position bias (1/[Ix08*32] cts)
M165->L:$081F ; &1 X-axis target position (engineering units)
M166->X:$0033,0,24,S    ; #1 Actual velocity (1/[Ix09*32] cts/cyc)
M167->D:$002D ; #1 Present master pos (1/[Ix07*32] cts)
M168->X:$0045,8,16,S    ; #1 Filter Output (DAC bits)
M169->D:$0046 ; #1 Compensation correction
M170->D:$0041 ; #1 Present phase position
```



```
; Coordinate System &1 Status Bits
M180->X:$0818,0,1        ; &1 Program-running bit
M181->Y:$0817,21,1       ; &1 Circle-radius-error bit
M182->Y:$0817,22,1       ; &1 Run-time-error bit
M187->Y:$0817,17,1       ; &1 In-position bit (AND of motors)
M188->Y:$0817,18,1       ; &1 Warning-following-error bit (OR)
M189->Y:$0817,19,1       ; &1 Fatal-following-error bit (OR)
M190->Y:$0817,20,1       ; &1 Amp-fault-error bit (OR of motors)
; Motor #1 Axis Definition Registers
M191->L:$0822 ; #1 X/U/A/B/C-Axis scale factor (cts/unit)
M192->L:$0823 ; #1 Y/V-Axis scale factor (cts/unit)
M193->L:$0824 ; #1 Z/W-Axis scale factor (cts/unit)
M194->L:$0825 ; #1 Axis offset (cts)
; Coordinate System &1 Variables
M197->X:$0806,0,24,S    ; &1 Host commanded time base (I10 units)
M198->X:$0808,0,24,S    ; &1 Present time base (I10 units)
```



Example 7: Rotary Tangent to X-Y System

This example shows how easy it is to keep a rotary axis tangent (or normal) to the movement of an X-Y system. The rotary axis is controlled by a separate program running simultaneously that monitors X-Y variables. The XY part program DOES NOT need to know anything about the rotary axis - it is all automatic.

```
;********************  Set-up and Definitions **********************
CLOSE
&1 #1->10000X     ; Motor 1 is X axis in Coordinate system 1
#2->10000Y        ; Motor 2 is Y axis in Coordinate system 1
&2 #3->40C        ; Motor 3 is C axis in Coordinate system 2
I15=0             ; Do trig calculations in degrees
I327=14400        ; C axis rolls over every 14400 cts
M161->D:$0028     ; Motor 1 present commanded position
M163->D:$080B     ; Motor 1 target position
M261->D:$0064     ; Motor 2 present commanded position
M263->D:$08CB     ; Motor 2 target position
P10=3072          ; 1 count in Mx61, Mx63 units (div by 0 check)
```

Example 7: Rotary Tangent to X-Y System (continued)

```
;*************** Motion Program Text for Rotary Axis *****************
OPEN PROG 10 CLEAR
SPLINE1 TA20     ; Cubic spline segments of 20 msec
WHILE (P1>0)     ; As long as in this mode
        Q0=M163-M161    ; X target position - X current position
        Q1=M263-M261    ; Y target position - Y current position
        IF (ABS(Q0)>P10 OR ABS(Q1)>P10) ; Target position < > current position?
        Q2=ATAN2(Q1)    ; Calculate angle - ATAN2 (Q1, Q2)
        ENDIF
        C(Q2)     ; Move rotary axis to new angle
ENDWHILE
CLOSE
```



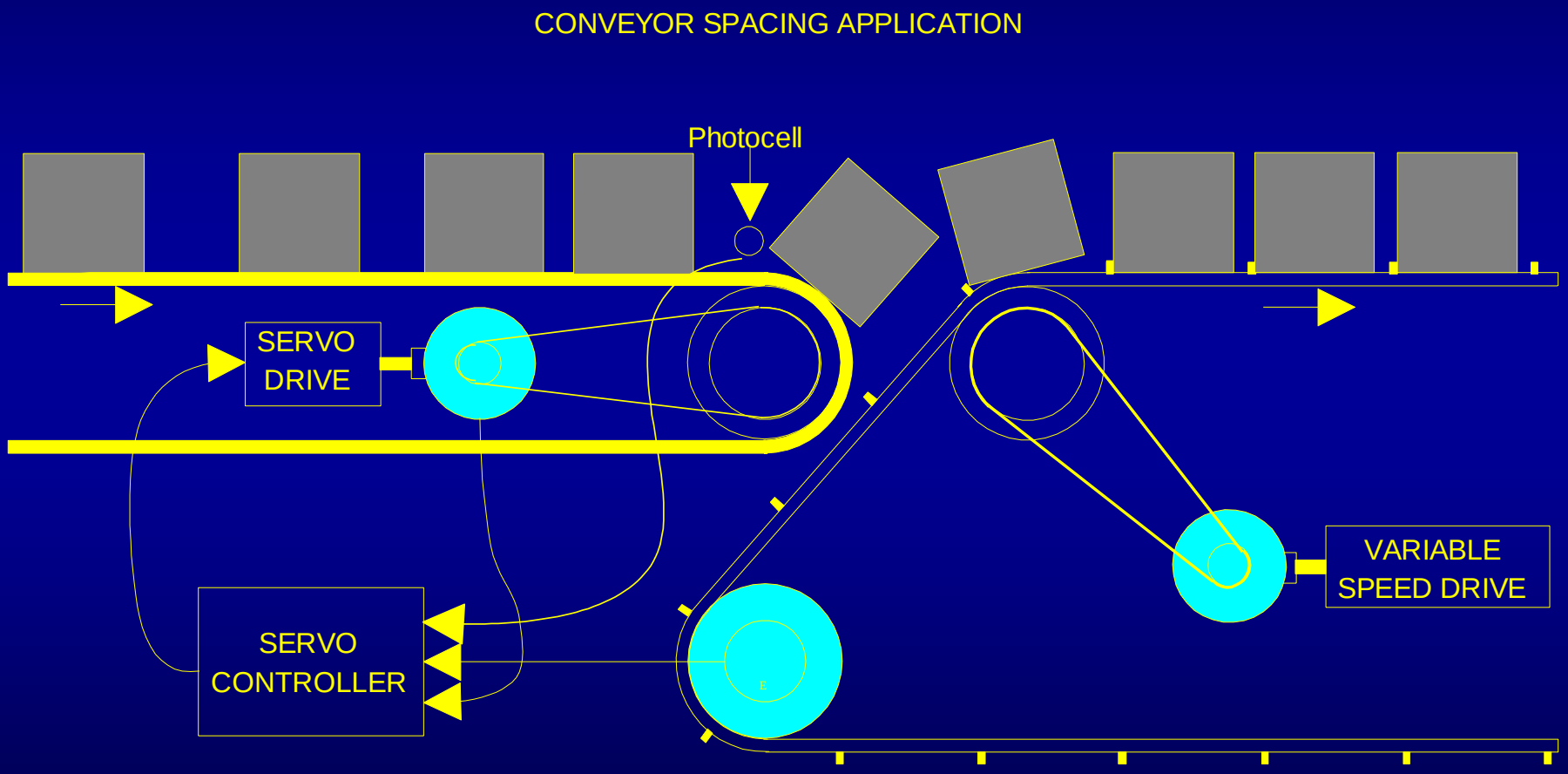
Example 11: Matching Positions on the Fly

This example shows how the position-capture function of PMAC can be used to match positions on the fly with a moving system not under PMAC control. This particular example has a short belt that takes items arriving at uneven intervals and evens out the spacing to place them between the lugs of a conveyor belt driven by a variable-speed drive. The same technique is applicable to any application requiring the matching of position of two moving devices when the spacing is not exactly known ahead of time.

The application makes use of PMAC's hardware position-capture registers, which register the position on an external trigger to the count, regardless of speed. This permits very high speed registration and correction. The belt in this application would probably also be slaved to the lugged conveyor in time-base mode (see Example 10) for speed matching -- this would keep any position corrections relatively small.



Example 11 Matching Position on the Fly



Example 11: Matching Positions on the Fly (continued)

```
;************ Set-up and Definitions *************
CLOSE
&1 #1->X                ; Motor 1 is X-axis scaled in
                        ; counts
M11->Y:$FFC2,0,1        ; M11 is Machine Input 1
M203->X:$C007,0,24,S    ; Encoder 2 captured-position
                        ; register
M217->X:$C004,17,1      ; Encoder 2 position-capture flag
I907=2                  ; Capture ENC2 on rising edge of
                        ; flag
I908=0                  ; Use HMFL2 to capture ENC2
P1=5000                 ; 5000 counts of ENC2 per lug
P2=1325                 ; "No-correction" position within
                        ;lug cycle, when captured
```



Example 11: Matching Positions on the Fly (continued)

```
;************* Motion Program Text **********************
OPEN PROG 346 CLEAR
INC                 ; Incremental move mode
SPLINE1 TA10        ; Cubic spline segments of 10 msec
WHILE (M11=1)       ; Loop once per item as long as input
                    ; set
  WHILE (M217=0)    ; Loop until photocell blocked
    X(P10)          ; P10 is nominal distance in 10 msec
  ENDWHILE          ; Loop back to check for trigger again
                    ; Found trigger, so calculate and adjust
  P3=M203 % P1      ; Read captured position of lug conveyor
                    ; and do modulo by counts per lug to
                    ; find out where in lug cycle we are.
  P4=P3-P2          ; Compare to no-correction position --
                    ; difference is error to make up
  X(P10+P4/4)       ; Make up first 1/4 of error in 10 msec
  X(P10+P4/2)       ; Make up 1/2 of error in 10 msec
  X(P10+P4/4)       ; Make up last 1/4 of error in 10 msec
ENDWHILE            ; Loop back for next item
```



PMAC PLC Programs

- Perform many tasks like hardware PLC (and more)
- Cycle through calculations repeatedly and rapidly, regardless of status of motion programs
- Uses include:
  - monitoring inputs
  - setting outputs
  - changing gains
  - monitoring card status
  - commanding actions
  - sending messages



PMAC PLC Programs (continued)

- Foreground PLCs (PLC0, PLCC0)
  - Operate on real time interrupt
  - Repetition rate controlled by I8
  - For a few time-critical tasks - KEEP SHORT
- Background PLCs (PLC 1 – 31, PLCC 1-31)
  - Operate between interrupt tasks
  - Repetition rate is a function of
    - Servo frequency
    - Number and type of motors
    - Calculation requirements of motion programs
    - Length and complexity of PLC programs



PMAC PLC Program Statements

1. Conditional Statements (Nestable)

   IF ({condition})
   WHILE ({condition})
   AND ({condition})
   OR ({condition})

   where {condition}=
   {expression}{comparator}{expression}
   [AND/OR {expression}{comparator}{expression}...



PMAC PLC Program Statements (continued)

2. Logical Control Statements
   - ELSE
   - ENDIF
   - ENDWHILE

3. Action Statements
   - {variable}={expression}
   - COMMAND "{on-line command}"
   - SEND "{message}"
   - DISPLAY "{message}"



PLC Program Control

- I5 = 0 No PLCs can be enabled
  = 1 PLC 0 can be enabled
  PLC 1 - 31 cannot be enabled
  = 2 PLC 0 cannot be enabled
  PLC 1 - 31 can be enabled
  = 3 All PLCs can be enabled
- All existing PLCs permitted by I5 are enabled on power-up or reset



PLC Program Control (continued)

- On-line commands, motion program and PLC program statements:

  ENABLE PLC n, ENABLE PLCC n

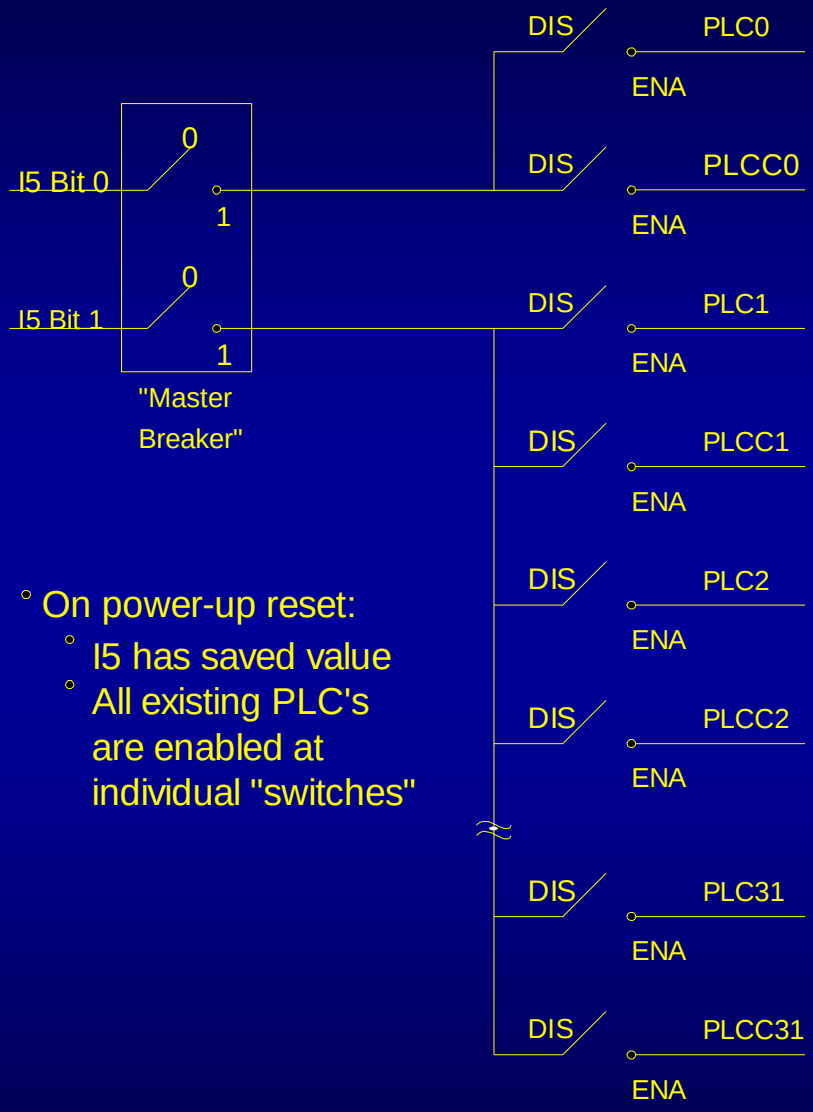
  DISABLE PLC n, DISABLE PLCC n

  control programs individually or in groups

  (at beginning/end of scan only)
- <CONTROL-D> disables all PLC programs
- OPEN PLC n disables that PLC
- CLOSE does not enable that PLC
- PAUSE PLC n can stop execution in middle of scan, to continue on RESUME PLC n



PMAC PLC Control



- On power-up reset:
  - I5 has saved value
  - All existing PLC's are enabled at individual "switches"

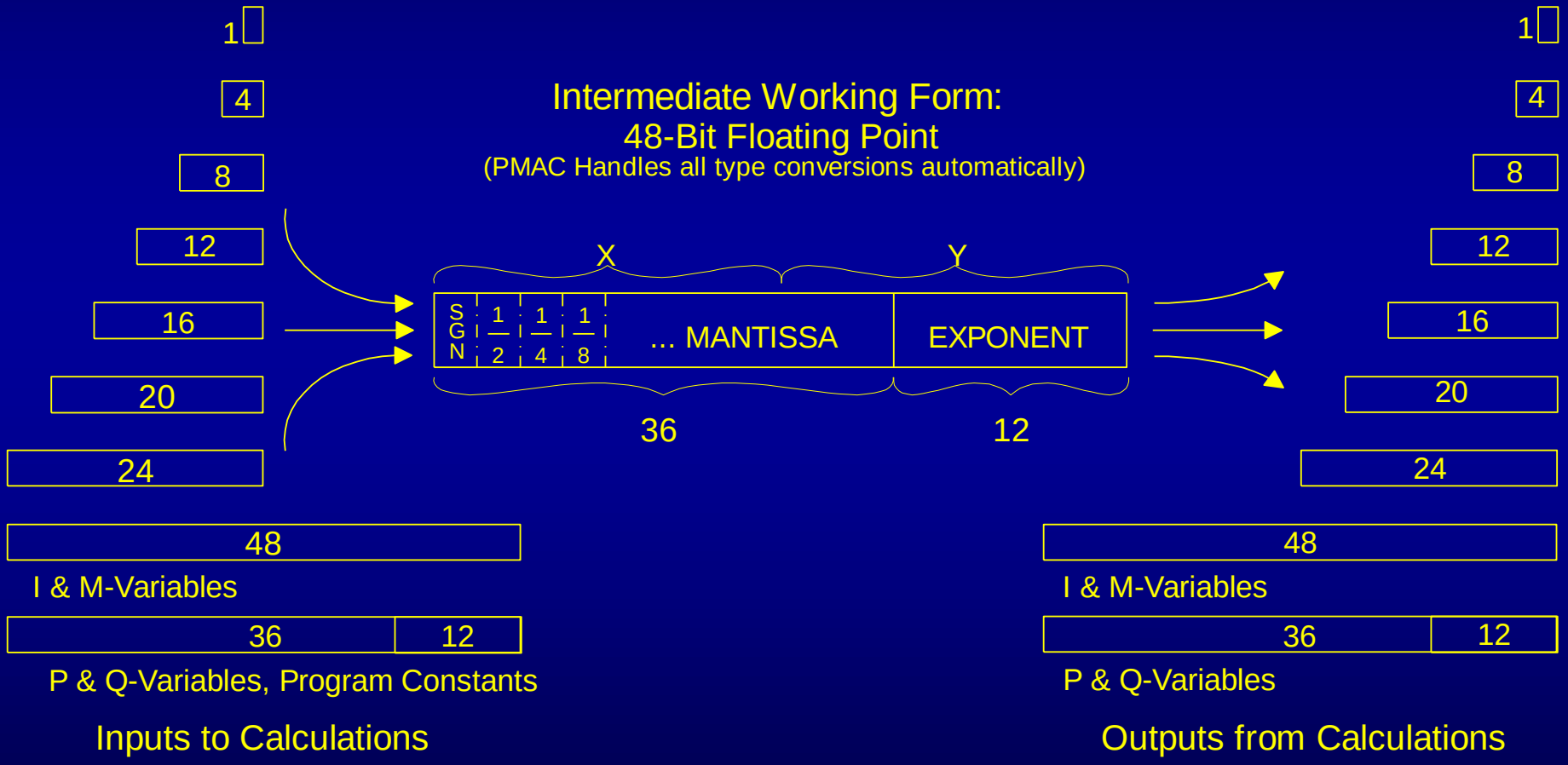
Preparing Compiled PLC's

1. PLC's often debugged in interpreted form
2. All references to compiled PLC's (OPEN, ENABLE, DISABLE) must be "PLCCn"
3. For integer arithmetic, define and substitute "L-variables" for other variable names
4. Make sure "compile on download" option in the Executive program is enabled
5. "OPEN PLCCn" is signal that program text is to be compiled
6. All PLCC's must be compiled together - same download file, or INCLUDE files



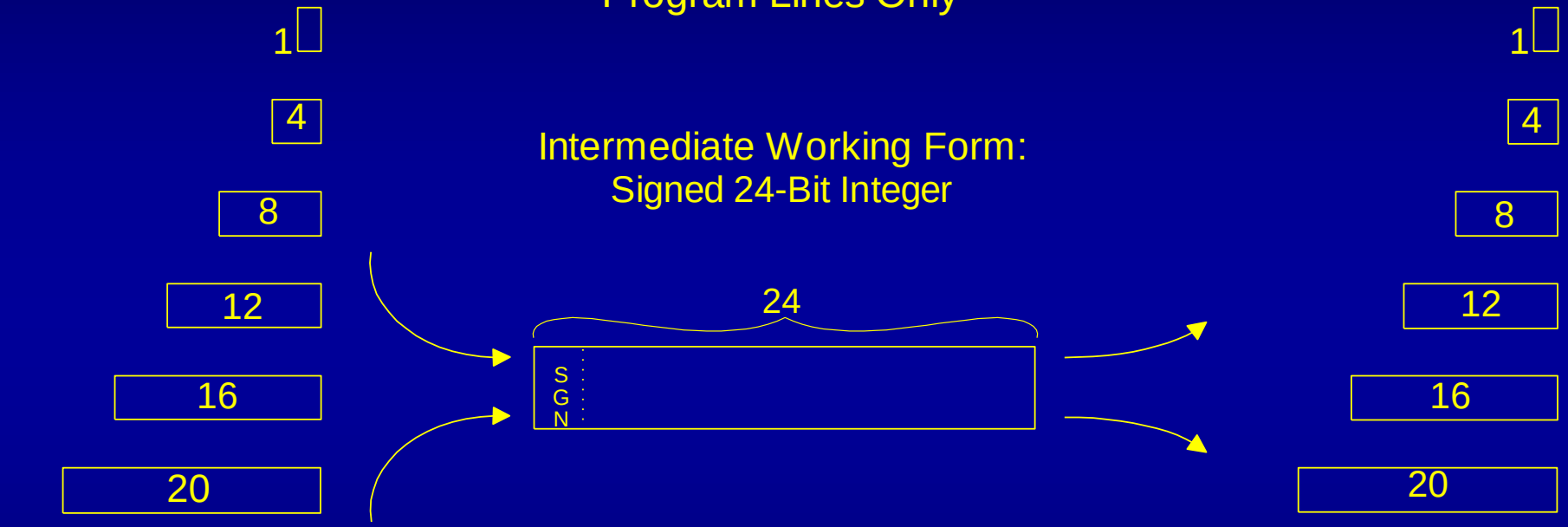
PMAC Program Mathematical Calculations

For All Motion and PLC Programs
Except Fixed-Point Compiled PLC Lines



PMAC Program Mathematical Calculations

Fixed Point Compiled PLC
Program Lines Only



- Program line may only contain L-Variables and integer constants in range
- All values - in,out, and intermediate - must stay in range - 8,388,608 to 8,388,607
- Calculations are about 10 times faster than equivalent calculations in floation point

Inputs to Calculations

Outputs from Calculations

Compiled PLC L-Variables

Uncompiled

```
M1->Y:$FFC2,8,1
M11->Y:$FFC2,0,1
M162->D:$002B
OPEN PLC1
CLEAR
IF (M11=0)
    M1=1
ELSE
    M1=0
ENDIF
P1=M162/I108/32
CLOSE
```

Compiled

```
L1->Y:$FFC2,8,1
L11->Y:$FFC2,0,1
M162->D:$002B
OPEN PLCC1
CLEAR
IF (L11=0)
    L1=1
ELSE
    L1=0
ENDIF
P1=M162/I108/32
CLOSE
```

Background PLC/PLCC Execution Example

Uncompiled PLC1, 2, 3
Compiled PLCC1, 2, 3, 4
Execution order:



PLC1
PLCC1, 2, 3, 4
PLC2
PLCC1, 2, 3, 4
PLC3
PLCC1,2,3,4

PLC0 and PLCC0 are at a higher priority level and can interrupt any of the background PLC's

Example 8: PLC Programming

This example shows how to use a PLC program to create customized inputs. As the PLC program repeatedly cycles in background, it checks the input lines and issues commands as appropriate. This program shows how to use a thumbwheel input line to create a jog switch dedicated to a particular motor. Note the use of a "latching flag" so that commands are only given on a change of state, making them edge triggered.



Example 8: PLC Programming (continued)

```
;********************** Set-up and Definitions*****************
CLOSE
M50->Y:$FFC1,0                ; Thumbwheel port input bit 0
M60->*                        ; Latching bit for M50
;******************** PLC Program Text ************************
OPEN PLC 16
CLEAR
IF (M50=1)                    ; Motor 1 jog plus switch on
        IF (M60=0)            ; But not on last time
          COMMAND"#1J+"       ; Issue command
          M60=1               ; Set latching flag
        ENDIF
ELSE                          ; Motor 1 jog plus switch off
        IF (M60=1)            ; But not off last time
          COMMAND"#1J/"       ; Issue stop command
          M60=0               ; Set latching flag
        ENDIF
ENDIF
CLOSE
```



Example 9: PLC Programming (continued)

```
;********************** Set up and Definitions **********************
CLOSE
#1->-100U+15000              ; Radial arm at 100 cts/mm; offset 150 mm
#2->40C                      ; Rotary axis at 40 cts/deg
M162->D:$002B                ; Motor 1 actual position
M262->D:$0067                ; Motor 2 actual position
M100->F:$D400                ; DPRAM register for X position reporting
M200->F:$D401                ; DPRAM register for Y position reporting
;********************** PLC Program Text ****************************
OPEN PLC 6 CLEAR
P162=150-M162/(I108*32*100)        ; Radial distance from center in mm
P262=M262/(I208*32*40)       ; Rotary position in degrees
M100=P162*COS(P262)          ; X position in mm => DPRAM register
M200=P162*SIN(P262)          ; Y position in mm => DPRAM register
DISPLAY 5,10.2,M100          ; Formatted display to LCD
DISPLAY 15,10.2,M200         ; Formatted display to LCD
CLOSE
```



Comppuls.pmc

These sample programs combine to provide a series of output pulses occurring when the motor using Encoder 1 reaches set positions at even increments apart. It uses the position-compare feature in the DSP-GATE IC to generate the pulses. PLC 0 is used to update the preloaded compare position (using the fast PLC 0 is only necessary if you need very fast -- near 1 kHz -- pulses).
PLC 1 is used to prepare the function; it executes only once, then disables itself.



```
; Definitions and Setup
CLOSE
M101->X:$C001,0,24,S   ; Encoder 1 actual position
M103->X:$C003,0,24,S   ; Encoder 1 compare position
M105->X:$07F0,0,24,S   ; Scratch 24-bit register
                       ; (to handle rollover)
M111->X:$C000,11,1     ; ENC1 EQU flag latch control
M112->X:$C000,12,1     ; ENC1 EQU output enable
M116->X:$C000,16,1     ; ENC1 EQU flag bit
P101=50                ; Increment in counts

OPEN PLC 1
CLEAR
M112=1                 ; Enable EQU output
M105=M101+P101         ; Put 1st EQU position in
                       ; scratch register (this can
                       ; be any starting position
                       ; you wish; here it is rel.
                       ; to starting position
M103=M105              ; Put value in compare reg
M105=M105+P101         ; Prepare next value
ENABLE PLC 0           ; To start up repetitive fn.
DISABLE PLC 1          ; This is one time only
CLOSE

OPEN PLC 0
CLEAR
IF (M116=1)            ; Reached last EQU position?
  M103=M105            ; Load next EQU position
  M105=M105+P101       ; Prepare following EQU pos
  M111=0               ; Clear and set control bit
  M111=1               ;  to clear latched flag
ENDIF
CLOSE
```



STEP2.PMC

This program performs a pure position step on Motor #1. It is done with the PLC writing directly to the command position registers, because motion programs would blend this. The step size in counts must be in variable P80, and the step time (each way) in milliseconds must be in P85. To run this on a different motor, only the definitions of M81 must be changed.



```
CLOSE              ; Ensure following are on-line
DELETE GATHER      ; Make sure there is room
M88->X1,18         ; Data gather prep bit
M89->X1,19         ; Data gather on bit
M0->X0,24          ; Servo cycle counter (timer)
M81->D$28          ; #1 cmd. pos. (double word)
M70->Y=$0700,24    ;User timer
OPEN PLC 2
CLEAR
M88=1              ; Prepare data gathering
M89=1              ; Turn on data gathering
P81=P80*I108*32    ; Compute increment to pos reg
P86=P85*8388608/I10 ; Step time in servo cycle
M81=M81+P81        ; Increment commanded position
M70=P86            ;Set count down timer
WHILE (M70>0)      ; Still waiting for spec time?
ENDWHILE
M81=M81-P81        ; Decrement commanded position
M70=P86            ;Set count down timer
WHILE (M70>0)      ;Still waiting?
ENDWHILE           ;
M89=0              ; Turn off data gathering
DISABLE PLC 2      ; Turn self off so no repeat
CLOSE

I5=3               ; Make sure PLCs can be enabled

; The step action will be triggered with an
; "ENABLE PLC 2" command ("ENAPLC2" for short)
```



PMAC Computation Priorities

1. Single character I/O

   Serial, PCbus, STDbus

2. Commutation update

   E29-E33 set freq.(9KHz default);

3. Servo update: interpolation & loopclosure

   E3-E6 set freq.(2.25KHz default);



PMAC Computation Priorities (continued)

4. Real time interrupt tasks

   I8 sets freq (750 Hz default):

   A. Motion program move planning

      Each time new programmed move starts

   B. PLC 0, PLCC0

      When not doing motion program

5. VME mailbox processing (16 chars.)

   Event driven



PMAC Computation Priorities (continued)

6. Background tasks

Done in time left over; rotates between:

A. PLC 1 - 31

One scan of one active PLC

B. PLCC 1 - 31

One scan of all active

C. Communications line processing

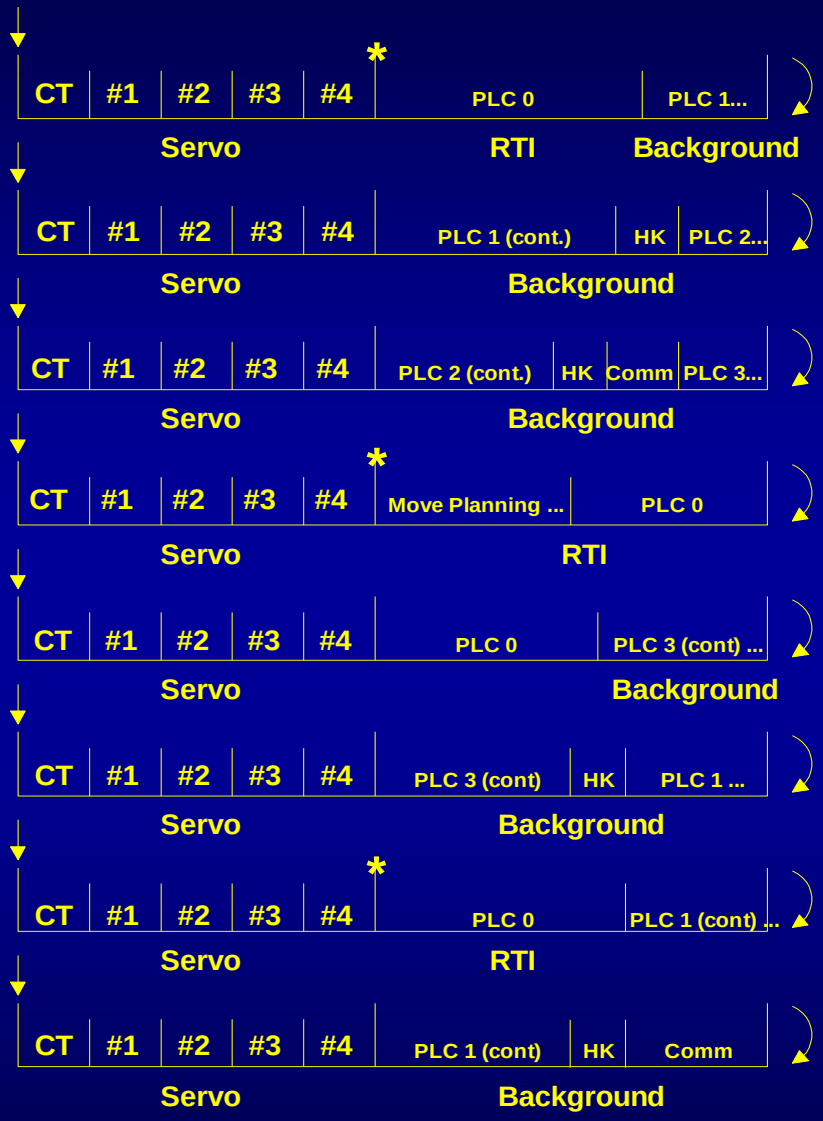
Event driven

D. General Housekeeping

Safety checks, status update



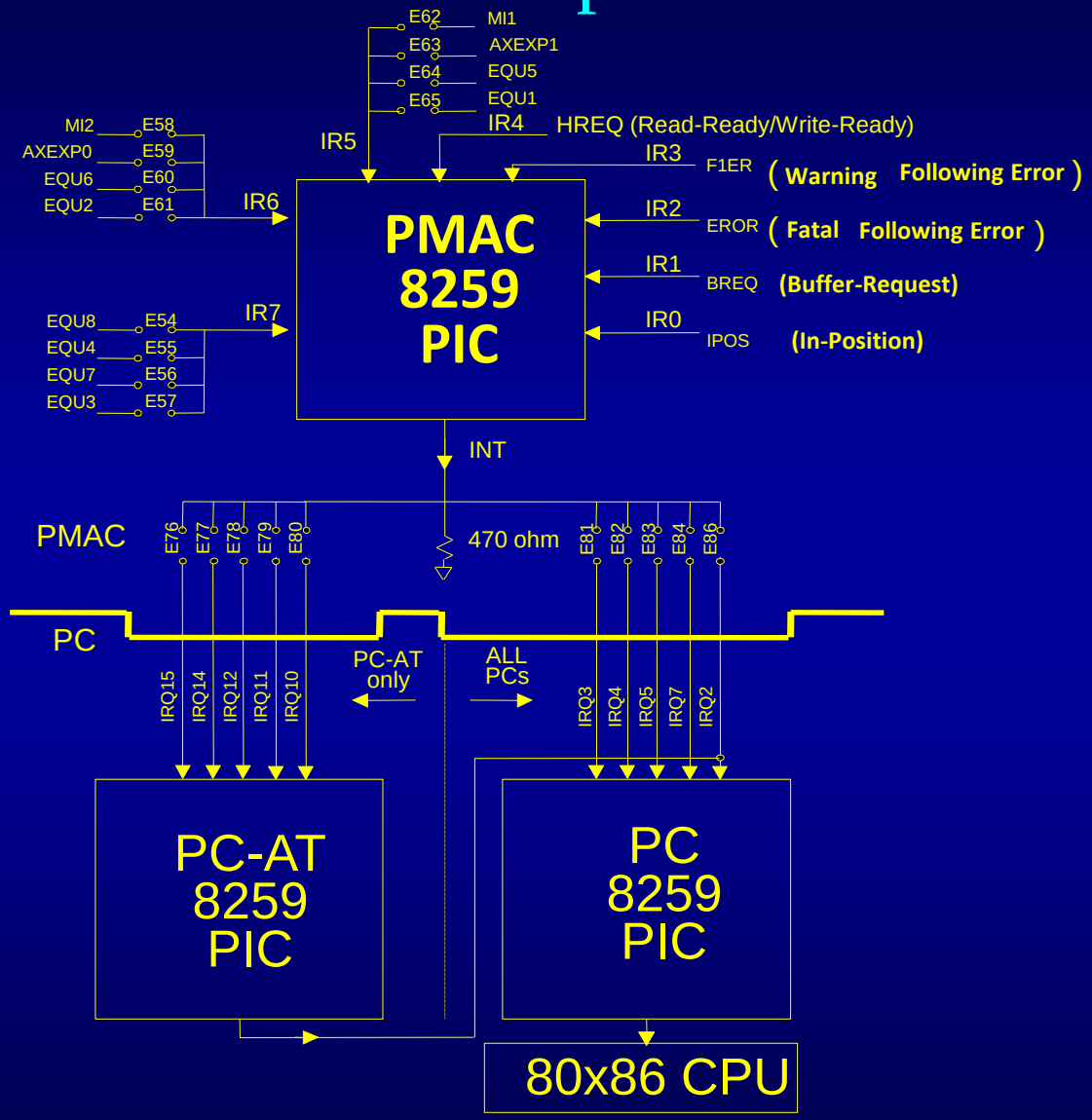
PMAC Multitasking Example



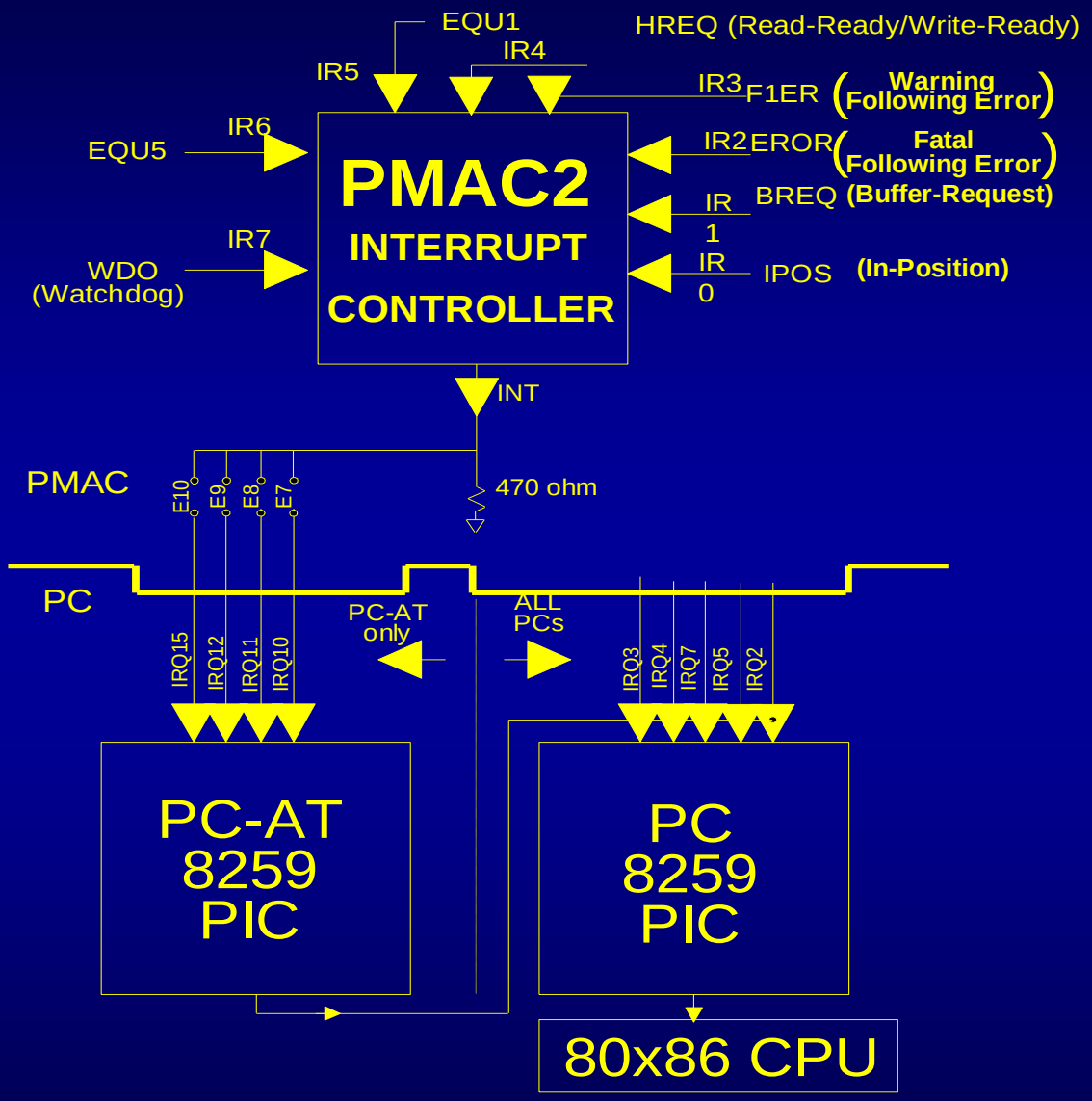
CT - Conversion Table
n - Motor n Servo Update
HK - Housekeeping

Comm - Communications Line Processin
▼ - Servo Interrupt
RTI - Real Time Interrupt Task
* - Start of RTI

PMAC PC Interrupt Structure



PMAC2-PC INTERRUPT STRUCTURE



PMAC Stop Commands

Command	Scope	Begin Immediately?	Stop on Path?	Stop at Prog. Pt.?	Decel Rate	Notes
J/	Motor	Yes	----	No	Ix19 – Ix21	Must stop motion prog. first
A	C.S.	Yes	No	No	Ix15	No easy restart
<CTRL-A>	Global	Yes	No	No	Ix15	No easy restart
Q	C.S.	No	Yes	Yes	TA, TS	Finishes calculated moves
<CTRL-Q>	Global	No	Yes	Yes	TA, TS	Finishes calculated moves
/	C.S.	No	Yes	Yes	TA, TS	Undoes blending
H	C.S.	Yes	Yes	No	Ix95	
<CTRL-O>	Global	Yes	Yes	No	Ix95	
\ (Non-Turbo)	C.S.	Yes	Yes	No	I52/Ix95	“H” if not segmented
\ (Turbo)	C.S.	Yes	Yes	No	Ix17 / Isx95	“H” outside of lookahead
K	Motor	Yes	No	No	(killed)	Must stop motion prog. first
<CTRL-K>	Global	Yes	No	No	(killed)	